

Ezairo® 8300 Firmware Reference

M-20845-018
December 2023

Table of Contents

	Page
Ezairo® 8300 Firmware Reference.....	1
Table of Contents.....	2
1. Introduction.....	48
1.1 Purpose.....	48
1.2 Intended audience.....	48
1.3 Conventions.....	48
1.4 Further Reading.....	48
2. Firmware Overview.....	50
2.1 Versions.....	50
2.2 Reserved Memory.....	50
2.2.1 System Variables.....	52
3. Hardware Definitions.....	53
3.1 Introduction.....	53
3.2 Register and Bit-field Definition.....	53
3.3 Memory Map Definition.....	55
3.4 Interrupt Vector Definition and Accessible Interrupts.....	55
4. Hardware Abstraction Layer.....	57
4.1 Usage.....	57
5. CFX System Library.....	58
5.1 Usage.....	58
6. CMSIS Library.....	59
6.1 Introduction.....	59
7. CMSIS Drivers.....	60
7.1 Introduction.....	60

7.2 Library Structure.....	60
7.3 Configuring The Driver Run-time Environment.....	61
7.3.1 Using a GUI-Based Editor.....	61
7.3.2 Using a Text-Based Editor.....	61
7.4 Including The CMSIS Drivers In Applications.....	61
8. External Drivers.....	63
8.1 The eMMC External Driver.....	63
8.2 The I3C External Driver.....	63
9. Program ROM.....	64
9.1 CFX Program ROM.....	64
9.1.1 Description.....	64
9.1.1.1 Versions.....	64
9.1.2 Boot ROM.....	64
9.1.2.1 Boot Initialization.....	64
9.1.2.2 RAM Clearing.....	65
9.1.2.3 Loading the Bootloader.....	65
9.1.2.4 Boot Sequence.....	66
9.1.2.5 Bootloader Startup.....	66
9.1.2.6 Application Startup.....	66
9.1.2.7 Security Model.....	67
9.1.2.8 Protection of Assets in the NVM image.....	67
9.1.2.9 Protection of Assets in RAM.....	67
9.1.2.10 Boot Failures.....	67
9.1.2.11 Boot Errors.....	68
9.1.3 Function Tables.....	68
9.2 Arm Cortex-M3 ROM.....	70

onsemi Confidential
Ezairo 8300 Firmware Reference

9.2.1 Description.....	70
9.2.1.1 Versions.....	70
9.2.2 Boot ROM.....	70
9.2.2.1 Boot Initialization.....	70
9.2.2.2 Boot Failures.....	71
9.2.3 Function Tables.....	71
10. NVMLIB Library.....	72
10.1 Introduction.....	72
10.2 Library Usage.....	72
10.2.1 Configuration.....	72
10.2.2 Initialization.....	73
10.2.2.1 Detection.....	73
10.2.3 Uninitialization.....	73
10.2.4 eMMC Operations.....	74
10.2.5 Unlocking Operations.....	74
10.2.6 Block Interface and Device Information.....	74
10.2.6.1 Device Information.....	74
10.2.6.2 Block Information.....	74
11. NVM Memory Layout.....	76
11.1 Volume Structure.....	76
11.1.1 FatFs Library.....	77
11.2 Reserved Area.....	77
11.2.1 Bios Parameter Block (BPB).....	78
11.2.2 Boot Information.....	79
11.2.3 Manufacturing Information.....	80
11.2.4 Bootloader.....	82

onsemi Confidential
Ezairo 8300 Firmware Reference

11.2.4.1 Bootloader Meta Contents.....	82
11.2.5 User Application Key Area.....	82
12. CFX AES Library.....	84
12.1 Usage.....	84
13. Calibration Library.....	85
13.1 Usage.....	85
13.2 Calibration References and Dependencies.....	85
14. Arm Cortex-M3 Bootloader.....	87
14.1 Bootloader API.....	87
14.2 Application Files.....	87
14.2.1 Data Sections in Application Files.....	87
14.2.1.1 Boot Sections in Application Files.....	88
14.2.1.2 Volatile Sections.....	88
14.3 Initialization of Global Variables.....	88
14.3.1 Bootloader Global Variables.....	89
14.4 Manifest Files.....	89
14.5 Bootloader File Layout.....	90
14.5.1 File Header.....	90
14.5.2 Data Section Layout.....	90
14.5.2.1 Boot Section Layout.....	91
14.5.3 Manifest File Layout.....	91
15. CFX DSP Library.....	93
15.1 Introduction.....	93
15.2 Usage.....	93
16. swmTrace Library.....	94
16.1 Usage.....	94

17. Arm CMSIS DSP Library Reference.....	95
17.1 CMSIS-Pack DSP Library.....	95
18. Hardware Abstraction Layer Reference.....	98
18.1 Summary.....	98
18.2 Detailed Description.....	98
18.3 Analog Input.....	98
18.3.1 Summary.....	98
18.3.2 Analog Input Macro Definition Documentation.....	98
18.3.2.1 ADC_CLK_UPPERLIMIT.....	99
18.3.2.2 ADC_CLK_LOWERLIMIT.....	99
18.3.2.3 ADC_SAMPLE_FREQ_UPPERLIMIT.....	99
18.3.2.4 ADC_SAMPLE_FREQ_LOWERLIMIT.....	99
18.3.2.5 SYS_SET_ADC_CTRL.....	99
18.3.2.6 SYS_GET_ADC_CTRL.....	100
18.3.2.7 SYS_SET_ADC_CFG.....	101
18.3.2.8 SYS_GET_ADC_CFG.....	102
18.3.2.9 SYS_SET_ADC_DEC_CTRL.....	103
18.3.2.10 SYS_GET_ADC_DEC_CTRL.....	103
18.3.2.11 SYS_SET_ADC_SAMPLE_FREQ_CFG.....	104
18.3.2.12 SYS_GET_ADC_SAMPLE_FREQ_CFG.....	105
18.3.2.13 SYS_SET_ADC_FDEC_CTRL.....	105
18.3.3 Analog Input Function Documentation.....	106
18.3.3.1 Sys_Calc_Sample_Freq_Val.....	106
18.3.3.2 Sys_Calc_Gain_Factor_Val.....	107
18.4 Analog Output.....	108
18.4.1 Summary.....	108

Ezairo 8300 Firmware Reference

18.4.2	Analog Output Macro Definition Documentation.....	108
18.4.2.1	SYS_SET_OUTPUT_CTRL.....	108
18.4.2.2	SYS_GET_OUTPUT_CTRL.....	109
18.4.2.3	SYS_SET_SDM_CTRL.....	110
18.4.2.4	SYS_GET_SDM_CTRL.....	110
18.4.2.5	SYS_SET_SDMCLK.....	111
18.4.2.6	SYS_GET_SDMCLK.....	112
18.4.2.7	SYS_SET_DAC_GF_CTRL.....	112
18.4.2.8	SYS_GET_O_DAC_GF_CTRL.....	113
18.4.2.9	SYS_SET_DMIC_OUTPUT_CTRL.....	114
18.4.2.10	SYS_GET_DMIC_OUTPUT_CTRL.....	114
18.4.2.11	SYS_SET_SDM_DCRM_CTRL.....	115
18.4.2.12	SYS_SET_SDM_OFFSET_CTRL.....	116
18.5	ASCC.....	116
18.5.1	Summary.....	116
18.5.2	ASCC Macro Definition Documentation.....	116
18.5.2.1	SYS_ASCC_CONFIG.....	116
18.5.3	ASCC Function Documentation.....	118
18.5.3.1	Sys_ASCC_Config.....	118
18.6	D_CRC.....	119
18.6.1	Summary.....	119
18.6.2	D_CRC Function Documentation.....	120
18.6.2.1	Sys_D_CRC_InitValue.....	120
18.6.2.2	Sys_D_CRC_GetCurrentValue.....	120
18.6.2.3	Sys_D_CRC_Add.....	121
18.7	DIO.....	121

onsemi Confidential
Ezairo 8300 Firmware Reference

18.7.1	Summary.....	122
18.7.2	DIO Macro Definition Documentation.....	122
18.7.2.1	DIO_WEAK_PULL_UP.....	122
18.7.2.2	DIO_MEDIUM_PULL_UP.....	122
18.7.2.3	DIO_STRONG_PULL_UP.....	123
18.7.2.4	DIO_WEAK_PULL_DOWN.....	123
18.7.2.5	DIO_LEVEL1_DRIVE.....	123
18.7.2.6	DIO_LEVEL2_DRIVE.....	123
18.7.2.7	DIO_LEVEL3_DRIVE.....	123
18.7.2.8	DIO_LEVEL4_DRIVE.....	124
18.7.2.9	SYS_DIO_CONFIG.....	124
18.7.3	DIO Function Documentation.....	124
18.7.3.1	Sys_DIO_Config.....	124
18.7.3.2	Sys_DIO_NMIConfig.....	125
18.7.3.3	Sys_DIO_IntConfig.....	126
18.7.3.4	Sys_DIO_INTCLKConfig.....	126
18.7.3.5	Sys_DIO_CM3JTAGConfig.....	127
18.8	EMMC_I3C.....	128
18.8.1	Summary.....	128
18.8.2	EMMC_I3C Function Documentation.....	128
18.8.2.1	Sys_EMMC_I3C_Read.....	128
18.8.2.2	Sys_EMMC_I3C_Write.....	129
18.9	Filter Engine HAL Reference.....	129
18.9.1	Summary.....	129
18.9.2	Filter Engine HAL Reference Macro Definition Documentation.....	130
18.9.2.1	SYS_FENG_PAUSE.....	130

Ezairo 8300 Firmware Reference

18.9.2.2	SYS_FENG_START.....	130
18.9.2.3	SYS_FENG_SINGLESTEP.....	131
18.9.2.4	SYS_FENG_DELAYEDPAUSE.....	131
18.9.2.5	SYS_FENG_RESET.....	132
18.9.2.6	SYS_FENG_EXECUTE.....	133
18.9.2.7	SYS_SET_FENG_MRCOUNTERCONFIG.....	133
18.9.2.8	SYS_SET_FENG_INTENABLE.....	134
18.9.2.9	SYS_FENG_INTACK.....	135
18.9.2.10	SYS_GET_FENG_INTSTATUS.....	135
18.9.2.11	SYS_SET_FENG_STATEWRITEENABLE.....	136
18.9.2.12	SYS_GET_FENG_STATEWRITEENABLE.....	137
18.9.2.13	SYS_SET_FENG_STATE.....	137
18.9.2.14	SYS_SET_FENG_STATE_SHADOW.....	138
18.9.2.15	SYS_GET_FENG_STATE.....	139
18.9.3	Filter Engine HAL Reference Function Documentation.....	140
18.9.3.1	Sys_Set_FENG_Coefficients.....	140
18.10	FIFO.....	141
18.10.1	Summary.....	141
18.10.2	FIFO Macro Definition Documentation.....	141
18.10.2.1	D_FIFO_Type.....	141
18.10.2.2	SYS_FIFO_AUTOMUTE.....	141
18.10.2.3	SYS_FIFO_CFXINTCONFIG.....	142
18.10.2.4	SYS_FIFO_HEARINTCONFIG.....	143
18.10.2.5	SYS_FIFO_CM3INTCONFIG.....	143
18.10.2.6	SYS_FIFO_CONFIGURE.....	144
18.11	G722.....	145

Ezairo 8300 Firmware Reference

18.11.1	Summary.....	145
18.11.2	G722 Macro Definition Documentation.....	146
18.11.2.1	G722_CFG_MASK.....	146
18.11.2.2	SYS_G722_CONFIG.....	146
18.11.2.3	SYS_G722_INT_CONFIG.....	147
18.11.2.4	SYS_G722_CLEAR_STATES.....	147
18.11.2.5	SYS_G722_TRUNCATE_TO_48.....	148
18.11.2.6	SYS_G722_TRUNCATE_TO_56.....	149
18.11.3	G722 Function Documentation.....	149
18.11.3.1	Sys_G722_ExtendFrom48.....	149
18.11.3.2	Sys_G722_ExtendFrom56.....	150
18.12	GPIO.....	151
18.12.1	Summary.....	151
18.12.2	GPIO Data Structures Type Documentation.....	152
18.12.2.1	GPIO_OUT0_Type.....	152
18.12.2.2	GPIO_OUT1_Type.....	152
18.12.2.3	GPIO_IN0_Type.....	153
18.12.2.4	GPIO_IN1_Type.....	153
18.12.3	GPIO Macro Definition Documentation.....	153
18.12.3.1	DIO_DATA_IN_0_BASE.....	154
18.12.3.2	DIO_DATA_IN_1_BASE.....	154
18.12.3.3	DIO_DATA_IN_0.....	154
18.12.3.4	DIO_DATA_IN_1.....	154
18.12.3.5	DIO_DATA_OUT_0_BASE.....	154
18.12.3.6	DIO_DATA_OUT_1_BASE.....	155
18.12.3.7	DIO_DATA_OUT_0.....	155

Ezairo 8300 Firmware Reference

18.12.3.8	DIO_DATA_OUT_1.....	155
18.12.4	GPIO Function Documentation.....	155
18.12.4.1	Sys_GPIO_Set_High.....	155
18.12.4.2	Sys_GPIO_Set_Low.....	156
18.12.4.3	Sys_GPIO_Toggle.....	156
18.12.4.4	Sys_GPIO_Read.....	157
18.12.4.5	Sys_GPIO_Write.....	158
18.12.4.6	Sys_GPIO_Set_Direction.....	158
18.13	HEAR.....	159
18.13.1	Summary.....	159
18.13.2	HEAR Macro Definition Documentation.....	160
18.13.2.1	HEAR_VECT_BASE_OFFSET.....	160
18.13.2.2	SYS_HEAR_PAUSE.....	160
18.13.2.3	SYS_HEAR_START.....	160
18.13.2.4	SYS_HEAR_RESET.....	161
18.13.2.5	SYS_HEAR_FUNCTIONCHAIN_ENABLE.....	161
18.13.2.6	SYS_HEAR_FUNCTIONCHAIN_PRIORITY.....	162
18.13.2.7	SYS_SET_HEAR_BFP.....	163
18.13.2.8	SYS_SET_HEAR_BFP_BLOCK_EXP.....	164
18.13.2.9	SYS_GET_HEAR_BFP_BLOCK_EXP.....	164
18.13.3	HEAR Function Documentation.....	165
18.13.3.1	Sys_HEAR_Command.....	165
18.13.3.2	Sys_Set_HEAR_FunctionChain_Vector.....	165
18.13.3.3	Sys_Hear_Runtime_Addr.....	166
18.14	I2C.....	167
18.14.1	Summary.....	167

18.14.2 I2C Macro Definition Documentation.....	168
18.14.2.1 I2C_CONFIG_MASK.....	168
18.14.2.2 I2C_PADS_NUM.....	168
18.14.2.3 SYS_I2C_DIOCONFIG.....	169
18.14.2.4 SYS_I2C_CONFIG.....	169
18.14.2.5 SYS_I2C_STARTREAD.....	170
18.14.2.6 SYS_I2C_STARTWRITE.....	171
18.14.2.7 SYS_I2C_ACK.....	171
18.14.2.8 SYS_I2C_NACK.....	172
18.14.2.9 SYS_I2C_LASTDATA.....	172
18.14.2.10 SYS_I2C_RESET.....	172
18.14.2.11 SYS_I2C_NACKANDSTOP.....	173
18.14.3 I2C Function Documentation.....	173
18.14.3.1 Sys_I2C_DIOConfig.....	173
18.14.3.2 Sys_I2C_Config.....	174
18.14.3.3 Sys_I2C_StartRead.....	175
18.14.3.4 Sys_I2C_StartWrite.....	176
18.14.3.5 Sys_I2C_ACK.....	176
18.14.3.6 Sys_I2C_NACK.....	177
18.14.3.7 Sys_I2C_LastData.....	178
18.14.3.8 Sys_I2C_Reset.....	178
18.14.3.9 Sys_I2C_NackAndStop.....	179
18.15 Interrupts.....	179
18.15.1 Summary.....	179
18.15.2 Interrupts Macro Definition Documentation.....	180
18.15.2.1 SYS_CFX_INT_MASTER_ENABLE.....	180

18.15.2.2	SYS_CFX_INT_MASTER_DISABLE.....	180
18.15.2.3	SYS_CFX_INT_DISABLEALL.....	181
18.15.2.4	SYS_CFX_INT_CLEARALLPENDING.....	182
18.15.2.5	SYS_CFX_INT_ENABLE.....	182
18.15.2.6	SYS_CFX_INT_ACK.....	183
18.15.2.7	SYS_CFX_INT_PRIORITY.....	184
18.16	IOC.....	185
18.16.1	Summary.....	185
18.16.2	IOC Macro Definition Documentation.....	186
18.16.2.1	IOC_INPUTCFG_MASK.....	186
18.16.2.2	IOC_INPUTPCMCFG_MASK.....	186
18.16.2.3	IOC_OUTPUTCFG_MASK.....	186
18.16.2.4	IOC_FIFOACCESSCFG_MASK.....	187
18.16.2.5	AUDIO_MUX_CFG_MASK.....	187
18.16.2.6	IOC_FENG_CFG_MASK.....	187
18.16.2.7	SYS_IOC_INPUTCONFIG.....	187
18.16.2.8	SYS_IOC_PCMINPUTCONFIG.....	188
18.16.2.9	SYS_IOC_OUTPUTCONFIG.....	189
18.16.2.10	SYS_IOC_FIFOACCESSCONFIG.....	189
18.16.2.11	SYS_AUDIOMUX_CONFIG.....	190
18.16.2.12	SYS_IOC_TRIGGER_CONFIG.....	191
18.17	LPDSP32.....	192
18.17.1	Summary.....	192
18.17.2	LPDSP32 Macro Definition Documentation.....	192
18.17.2.1	SYS_LPDSP32_PAUSE.....	192
18.17.2.2	SYS_LPDSP32_START.....	193

Ezairo 8300 Firmware Reference

18.17.2.3	SYS_LPDSP32_RESET.....	193
18.17.2.4	SYS_LPDSP32_COMMAND.....	194
18.17.2.5	JTAG_LPDSP32_TDI.....	194
18.17.2.6	JTAG_LPDSP32_TMS.....	195
18.17.2.7	JTAG_LPDSP32_TCK.....	195
18.17.2.8	JTAG_LPDSP32_TDO.....	195
18.17.2.9	JTAG_DIO_CONFIG.....	195
18.17.3	LPDSP32 Function Documentation.....	196
18.17.3.1	JTAG_DIO_Config.....	196
18.18	LSAD.....	196
18.18.1	Summary.....	196
18.18.2	LSAD Macro Definition Documentation.....	197
18.18.2.1	SYS_LSAD_CONFIG.....	197
18.18.3	LSAD Function Documentation.....	197
18.18.3.1	Sys_LSAD_InputConfig.....	197
18.19	Memory Instances.....	198
18.19.1	Summary.....	198
18.19.2	Memory Instances Macro Definition Documentation.....	199
18.19.2.1	SYS_ASSIGN_PRAM0.....	199
18.19.2.2	SYS_ASSIGN_PRAM1.....	199
18.19.2.3	SYS_LOCK_PRAM_ASSIGN.....	200
18.19.2.4	SYS_LOCK_PRAM_INSTANCE.....	200
18.19.3	Memory Instances Function Documentation.....	201
18.19.3.1	Sys_Memory_Enable.....	201
18.19.3.2	Sys_Memory_Disable.....	202
18.20	PCM.....	203

Ezairo 8300 Firmware Reference

18.20.1	Summary.....	203
18.20.2	PCM Macro Definition Documentation.....	203
18.20.2.1	PCM_CFG_MASK.....	203
18.20.2.2	PCM_PADS_NUM.....	204
18.20.2.3	SYS_PCM_CONFIG.....	204
18.20.2.4	SYS_PCM_DIOCONFIG.....	205
18.20.3	PCM Function Documentation.....	206
18.20.3.1	Sys_PCM_DIOConfig.....	206
18.20.3.2	Sys_PCM_Config.....	207
18.21	POWER.....	208
18.21.1	Summary.....	208
18.21.2	POWER Function Documentation.....	209
18.21.2.1	Sys_Power_AutomaticChargePumpControlEnable.....	209
18.21.2.2	Sys_Power_AutomaticChargePumpControlDisable.....	209
18.21.2.3	Sys_Power_AdaptiveVoltageScalingEnable.....	210
18.21.2.4	Sys_Power_AdaptiveVoltageScalingDisable.....	210
18.21.2.5	Sys_Clock_Power_AdaptiveVoltageScalingSwitchCCO.....	211
18.22	HAL PRAM Security Reference.....	211
18.22.1	Summary.....	211
18.22.2	HAL PRAM Security Reference Function Documentation.....	211
18.22.2.1	Sys_PRAMSec_SetLock.....	212
18.22.2.2	Sys_PRAMSec_GetLock.....	212
18.23	RS.....	213
18.23.1	Summary.....	213
18.23.2	RS Macro Definition Documentation.....	213
18.23.2.1	RS_CFG_MASK.....	213

onsemi Confidential
Ezairo 8300 Firmware Reference

18.23.2.2 RS_INT_MASK.....	213
18.23.3 RS Function Documentation.....	214
18.23.3.1 Sys_RS_Config.....	214
18.24 SPI.....	215
18.24.1 Summary.....	215
18.24.2 SPI Macro Definition Documentation.....	216
18.24.2.1 SPI_CONFIG_MASK.....	216
18.24.2.2 SPI_PADS_NUM.....	216
18.24.2.3 SYS_SPI_CONFIG.....	216
18.24.2.4 SYS_SPI_TRANSFERCONFIG.....	217
18.24.2.5 SYS_SPI_READ.....	218
18.24.2.6 SYS_SPI_WRITE.....	218
18.24.2.7 SYS_SPI_DIOCONFIG.....	219
18.24.2.8 SYS_DSPI_DIOCONFIG.....	220
18.24.2.9 SYS_QSPI_DIOCONFIG.....	220
18.24.3 SPI Function Documentation.....	221
18.24.3.1 Sys_SPI_Config.....	221
18.24.3.2 Sys_SPI_TransferConfig.....	222
18.24.3.3 Sys_SPI_Read.....	223
18.24.3.4 Sys_SPI_Write.....	224
18.24.3.5 Sys_SPI_DIOConfig.....	224
18.24.3.6 Sys_DSPI_DIOConfig.....	225
18.24.3.7 Sys_QSPI_DIOConfig.....	226
18.25 STANDBY_MODE.....	227
18.25.1 Summary.....	227
18.25.2 STANDBY_MODE Macro Definition Documentation.....	227

onsemi Confidential
Ezairo 8300 Firmware Reference

18.25.2.1	CM3_ROMJUMPTABLE_BASEADDR.....	227
18.25.2.2	CM3_ROMJUMPTABLE_SLEEP.....	227
18.25.3	STANDBY_MODE Function Documentation.....	228
18.25.3.1	Sys_StandbyMode.....	228
18.26	TIMER.....	229
18.26.1	Summary.....	229
18.26.2	TIMER Macro Definition Documentation.....	229
18.26.2.1	SYS_TIMER_CONFIG.....	229
18.26.2.2	SYS_TIMER_START.....	229
18.26.2.3	SYS_TIMER_STOP.....	229
18.26.3	TIMER Function Documentation.....	230
18.26.3.1	Sys_Timer_Config.....	230
18.26.3.2	Sys_Timer_Start.....	230
18.26.3.3	Sys_Timer_Stop.....	231
18.27	TRIMS.....	231
18.27.1	Summary.....	232
18.27.2	TRIMS Variable Documentation.....	233
18.27.2.1	manuTable.....	233
18.27.3	TRIMS Macro Definition Documentation.....	233
18.27.3.1	SYS_ERRNO_NO_ERROR.....	233
18.27.3.2	SYS_ERRNO_NO_MATCH.....	233
18.27.3.3	SYS_ERROR_SEARCH_FAILURE.....	233
18.27.3.4	VREG_0P9V.....	234
18.27.3.5	OSC_MUL_LIMIT1.....	234
18.27.3.6	OSC_MUL_LIMIT2.....	234
18.27.4	TRIMS Function Documentation.....	234

Ezairo 8300 Firmware Reference

18.27.4.1	Sys_Trims_LoadManuTable.....	234
18.27.4.2	Sys_Trims_SetVREGAndLSAD.....	235
18.27.4.3	Sys_Trims_SetVDDIF.....	236
18.27.4.4	Sys_Trims_SetVDDA.....	236
18.27.4.5	Sys_Trims_SetVDDA_PMURef.....	237
18.27.4.6	Sys_Trims_SetVDDC.....	238
18.27.4.7	Sys_Trims_SetVDDC_PMURef.....	238
18.27.4.8	Sys_Trims_SetVDDC_CP.....	239
18.27.4.9	Sys_Trims_SetAutomaticVDDC.....	240
18.27.4.10	Sys_Trims_SetVDDM.....	241
18.27.4.11	Sys_Trims_SetVDDM_PMURef.....	241
18.27.4.12	Sys_Trims_SetVDDM_CP.....	242
18.27.4.13	Sys_Trims_SetVDDOD.....	243
18.27.4.14	Sys_Trims_SetVMIC.....	243
18.27.4.15	Sys_Trims_SetOperatingFrequency.....	244
18.27.4.16	Sys_Trims_SetOperatingFrequencyMult.....	245
18.27.4.17	Sys_Trims_SetOperatingFrequencyThrottle.....	246
18.27.4.18	Sys_Trims_SetOperatingFrequencyThrottleMult.....	247
18.27.4.19	Sys_Trims_SetADCOffsets.....	248
18.28	UART.....	249
18.28.1	Summary.....	249
18.28.2	UART Macro Definition Documentation.....	249
18.28.2.1	SYS_UART_DIOCONFIG.....	249
18.28.2.2	SYS_UART_CONFIG.....	250
18.28.3	UART Function Documentation.....	251
18.28.3.1	Sys_UART_DIOConfig.....	251

onsemi Confidential
Ezairo 8300 Firmware Reference

18.28.3.2 Sys_UART_Config.....	252
18.29 Versions.....	253
18.29.1 Summary.....	253
18.29.2 Versions Variable Documentation.....	253
18.29.2.1 Sys_Version.....	253
18.29.3 Versions Macro Definition Documentation.....	253
18.29.3.1 SYS_VER_MAJOR.....	253
18.29.3.2 SYS_VER_MINOR.....	254
18.29.3.3 SYS_VER_REVISION.....	254
18.29.3.4 SYS_VER.....	254
18.30 Watchdog Timer.....	254
18.30.1 Summary.....	254
18.30.2 Watchdog Timer Macro Definition Documentation.....	255
18.30.2.1 SYS_WATCHDOG_REFRESH.....	255
18.30.2.2 SYS_WATCHDOG_RESET.....	255
19. CFX System Library Reference.....	256
19.1 Summary.....	256
19.2 CFX System Library Reference Macro Definition Documentation.....	256
19.2.1 SYSLIB_ERRNO_NO_ERROR.....	256
19.2.2 SYSLIB_ERRNO_FREQ_NO_MATCH.....	256
19.2.3 SYSLIB_OSC_MUL_LIMIT1.....	257
19.2.4 STSLIB_OSC_MUL_LIMIT2.....	257
19.3 CFX System Library Reference Function Documentation.....	257
19.3.1 SYSLIB_Reset.....	257
19.3.2 SYSLIB_Delay.....	258
19.3.3 SYSLIB_DelayBase.....	259

onsemi Confidential
Ezairo 8300 Firmware Reference

19.3.4	SYSLIB_SetOperatingFrequency.....	260
19.3.5	SYSLIB_SetOperatingFrequency_Throttle.....	260
20.	CMSIS Reference.....	261
20.1	Summary.....	261
20.2	CMSIS Reference Variable Documentation.....	264
20.2.1	__Heap_Begin__.....	264
20.2.2	__Heap_Limit__.....	264
20.2.3	__bss_start__.....	264
20.2.4	__bss_end__.....	264
20.2.5	__preinit_array_start__.....	265
20.2.6	__preinit_array_end__.....	265
20.2.7	__init_array_start__.....	265
20.2.8	__init_array_end__.....	265
20.3	CMSIS Reference Macro Definition Documentation.....	265
20.3.1	DIO0_IRQHandler.....	266
20.3.2	DIO1_IRQHandler.....	266
20.3.3	DIO2_IRQHandler.....	266
20.3.4	DIO3_IRQHandler.....	266
20.3.5	DIO0_IRQn.....	266
20.3.6	DIO1_IRQn.....	267
20.3.7	DIO2_IRQn.....	267
20.3.8	DIO3_IRQn.....	267
20.3.9	DMA0_IRQHandler.....	267
20.3.10	DMA1_IRQHandler.....	267
20.3.11	DMA2_IRQHandler.....	268
20.3.12	DMA3_IRQHandler.....	268

onsemi Confidential
Ezairo 8300 Firmware Reference

20.3.13	DMA0_IRQn.....	268
20.3.14	DMA1_IRQn.....	268
20.3.15	DMA2_IRQn.....	268
20.3.16	DMA3_IRQn.....	269
20.3.17	I2C0_IRQHandler.....	269
20.3.18	I2C1_IRQHandler.....	269
20.3.19	I2C0_IRQn.....	269
20.3.20	I2C1_IRQn.....	270
20.3.21	SPI0_RX_IRQHandler.....	270
20.3.22	SPI0_TX_IRQHandler.....	270
20.3.23	SPI0_COM_IRQHandler.....	270
20.3.24	SPI1_RX_IRQHandler.....	270
20.3.25	SPI1_TX_IRQHandler.....	271
20.3.26	SPI1_COM_IRQHandler.....	271
20.3.27	SPI0_RX_IRQn.....	271
20.3.28	SPI0_TX_IRQn.....	271
20.3.29	SPI0_COM_IRQn.....	271
20.3.30	SPI1_RX_IRQn.....	272
20.3.31	SPI1_TX_IRQn.....	272
20.3.32	SPI1_COM_IRQn.....	272
20.3.33	TIMER0_IRQHandler.....	272
20.3.34	TIMER1_IRQHandler.....	272
20.3.35	TIMER2_IRQHandler.....	273
20.3.36	TIMER3_IRQHandler.....	273
20.3.37	TIMER0_IRQn.....	273
20.3.38	TIMER1_IRQn.....	273

Ezairo 8300 Firmware Reference

20.3.39	TIMER2_IRQn.....	274
20.3.40	TIMER3_IRQn.....	274
20.3.41	UART0_RX_IRQHandler.....	274
20.3.42	UART0_TX_IRQHandler.....	274
20.3.43	UART0_ERROR_IRQHandler.....	274
20.3.44	UART0_RX_IRQn.....	275
20.3.45	UART0_TX_IRQn.....	275
20.3.46	UART0_ERROR_IRQn.....	275
20.3.47	__CM3_REV.....	275
20.3.48	__NVIC_PRIO_BITS.....	275
20.3.49	__MPU_PRESENT.....	276
20.3.50	__FPU_PRESENT.....	276
20.3.51	__Vendor_SysTickConfig.....	276
20.3.52	ASCC_REF_VALID.....	276
20.3.53	CRC_REF_VALID.....	276
20.3.54	DMA_REF_VALID.....	277
20.3.55	DIO_PAD_COUNT.....	277
20.3.56	DIO_GROUP_LOW_PAD_RANGE.....	277
20.3.57	DIO_EVENT_CHANNEL_COUNT.....	277
20.3.58	DIO_CLK_DIV_COUNT.....	278
20.3.59	DIO0.....	278
20.3.60	DIO1.....	278
20.3.61	DIO2.....	278
20.3.62	DIO3.....	279
20.3.63	DIO4.....	279
20.3.64	DIO5.....	279

onsemi Confidential
Ezairo 8300 Firmware Reference

20.3.65	DIO6.....	279
20.3.66	DIO7.....	279
20.3.67	DIO8.....	280
20.3.68	DIO9.....	280
20.3.69	DIO10.....	280
20.3.70	DIO11.....	280
20.3.71	DIO12.....	280
20.3.72	DIO13.....	281
20.3.73	DIO14.....	281
20.3.74	DIO15.....	281
20.3.75	DIO16.....	281
20.3.76	DIO17.....	281
20.3.77	DIO18.....	282
20.3.78	DIO19.....	282
20.3.79	DIO20.....	282
20.3.80	DIO21.....	282
20.3.81	DIO22.....	283
20.3.82	DIO23.....	283
20.3.83	DIO24.....	283
20.3.84	DIO25.....	283
20.3.85	DIO26.....	283
20.3.86	DIO27.....	284
20.3.87	DIO28.....	284
20.3.88	DIO29.....	284
20.3.89	DIO30.....	284
20.3.90	DIO31.....	284

onsemi Confidential
Ezairo 8300 Firmware Reference

20.3.91	DIO32.....	285
20.3.92	DIO33.....	285
20.3.93	DIO34.....	285
20.3.94	DIO35.....	285
20.3.95	I2C_REF_VALID.....	285
20.3.96	PCM_REF_VALID.....	286
20.3.97	SPI_REF_VALID.....	286
20.3.98	TIMER_REF_VALID.....	286
20.3.99	UART_REF_VALID.....	286
20.3.100	SYS_DUMMY_READ.....	287
20.3.101	SYS_DUMMY_WRITE.....	287
20.3.102	ERRNO_NO_ERROR.....	287
20.3.103	ERRNO_GENERAL_FAILURE.....	287
20.3.104	SYS_DELAY_CALL_OVERHEAD.....	287
20.3.105	SYS_DELAY_LOOP_OVERHEAD.....	288
20.3.106	SYS_RESET.....	288
20.3.107	SYSVAR_FREQ_FORMAT.....	288
20.3.108	SYSVAR_SET_FREQ.....	289
20.3.109	SYS_BOOT_ALTERNATE.....	289
20.3.110	DEFAULT_FREQ.....	290
20.3.111	DEFAULT_FREQ_IDX.....	290
20.3.112	SystemCoreClock.....	290
20.4	CMSIS Reference Function Documentation.....	290
20.4.1	_start.....	290
20.4.2	_sbrk.....	291
20.4.3	Sys_Delay.....	292

Ezairo 8300 Firmware Reference

20.4.4	SystemInit.....	292
20.4.5	SystemCoreClockUpdate.....	293
20.4.6	System_SetOperatingFrequency.....	293
20.4.7	System_SetOperatingFrequency_Throttle.....	294
21.	Non-Volatile Memory (NVM) Library Reference.....	295
21.1	Summary.....	295
21.2	Detailed Description.....	298
21.3	Non-Volatile Memory (NVM) Library Reference Typedef Documentation.....	298
21.3.1	ARM_STORAGE_BLOCK_ATTRIBUTES.....	298
21.3.2	ARM_STORAGE_BLOCK.....	298
21.3.3	ARM_STORAGE_SECURITY_FEATURES.....	299
21.3.4	ARM_STORAGE_INFO.....	299
21.3.5	ARM_STORAGE_CAPABILITIES.....	299
21.3.6	SPI_Storage_Commands.....	299
21.3.7	STORAGE_DEVICE_CHARACTERISTIC.....	299
21.3.8	STORAGE_DEVICE_DESCRIPTOR.....	300
21.3.9	NVMLIB_DRIVER.....	300
21.3.10	NVMLIB_DEVICE.....	300
21.4	Non-Volatile Memory (NVM) Library Reference Data Structures Type Documentation.....	300
21.4.1	_ARM_STORAGE_BLOCK_ATTRIBUTES.....	301
21.4.2	_ARM_STORAGE_BLOCK.....	302
21.4.3	_ARM_STORAGE_SECURITY_FEATURES.....	303
21.4.4	_ARM_STORAGE_INFO.....	303
21.4.5	_ARM_STORAGE_CAPABILITIES.....	305
21.4.6	_SPI_Storage_Commands.....	305
21.4.7	_STORAGE_DEVICE_CHARACTERISTIC.....	306

Ezairo 8300 Firmware Reference

21.4.8	_STORAGE_DEVICE_DESCRIPTOR.....	306
21.4.9	_NVMLIB_DRIVER.....	307
21.4.10	_NVMLIB_DEVICE.....	309
21.5	Non-Volatile Memory (NVM) Library Reference Macro Definition Documentation.....	309
21.5.1	ARM_STORAGE_INVALID_OFFSET.....	309
21.5.2	ARM_STORAGE_INVALID_ADDRESS.....	310
21.5.3	ARM_STORAGE_ERROR_NOT_ERASABLE.....	310
21.5.4	ARM_STORAGE_ERROR_NOT_PROGRAMMABLE.....	310
21.5.5	ARM_STORAGE_ERROR_PROTECTED.....	310
21.5.6	ARM_STORAGE_VALID_BLOCK.....	311
21.5.7	ARM_STORAGE_PROGRAMMABILITY_RAM.....	311
21.5.8	ARM_STORAGE_PROGRAMMABILITY_ROM.....	311
21.5.9	ARM_STORAGE_PROGRAMMABILITY_WORM.....	311
21.5.10	ARM_STORAGE_PROGRAMMABILITY_ERASABLE.....	312
21.5.11	ARM_RETENTION_WHILE_DEVICE_ACTIVE.....	312
21.5.12	ARM_RETENTION_ACROSS_SLEEP.....	312
21.5.13	ARM_RETENTION_ACROSS_DEEP_SLEEP.....	312
21.5.14	ARM_RETENTION_BATTERY_BACKED.....	313
21.5.15	ARM_RETENTION_NVM.....	313
21.5.16	ARM_POWER_OFF.....	313
21.5.17	ARM_POWER_LOW.....	313
21.5.18	ARM_POWER_FULL.....	314
21.5.19	ARM_STORAGE_PROGRAM_CYCLES_INFINITE.....	314
21.5.20	SPI_CHIP_ID_GENERIC.....	314
21.5.21	SPI_CHIP_ID_EMMC.....	314
21.5.22	SPI_CHIP_ID_SST26WF016B.....	315

Ezairo 8300 Firmware Reference

21.5.23	SPI_CHIP_ID_LE25S161.....	315
21.5.24	SPI_CHIP_ID_MB85AS8MT.....	315
21.5.25	SPI_MFG_ID_W25Q64JW.....	315
21.5.26	SPI_JEDEC_ID_W25Q64JW.....	315
21.5.27	SPI_CHIP_ID_W25Q64JW.....	316
21.5.28	SPI_JEDEC_ID_W25Q80EW.....	316
21.5.29	SPI_CHIP_ID_W25Q80EW.....	316
21.5.30	SPI_CHIP_ID_GT25Q16A.....	316
21.5.31	SPI_CHIP_ID_MX25.....	316
21.5.32	NVMLIB_STORAGE_UNKNOWN.....	317
21.5.33	NVMLIB_STORAGE_CLEARED.....	317
21.5.34	NVMLIB_STORAGE_AUTO.....	317
21.5.35	NVMLIB_STORAGE_GENERIC.....	317
21.5.36	NVMLIB_STORAGE_EMMC.....	318
21.5.37	NVMLIB_STORAGE_CAT25M01.....	318
21.5.38	NVMLIB_STORAGE_CAT25M02.....	318
21.5.39	NVMLIB_STORAGE_LE25S161.....	318
21.5.40	NVMLIB_STORAGE_MB85AS8MT.....	318
21.5.41	NVMLIB_STORAGE_MB85AS8MT.....	319
21.5.42	NVMLIB_STORAGE_EA2M.....	319
21.5.43	NVMLIB_STORAGE_W25Q64JW.....	319
21.5.44	NVMLIB_STORAGE_W25Q80EW.....	319
21.5.45	NVMLIB_STORAGE_GT25Q16A.....	319
21.5.46	NVMLIB_STORAGE_MX25.....	320
21.5.47	ARM_DRIVER_OK.....	320
21.5.48	ARM_DRIVER_ERROR.....	320

Ezairo 8300 Firmware Reference

21.5.49	ARM_DRIVER_ERROR_BUSY.....	320
21.5.50	ARM_DRIVER_ERROR_TIMEOUT.....	321
21.5.51	ARM_DRIVER_ERROR_UNSUPPORTED.....	321
21.5.52	ARM_DRIVER_ERROR_PARAMETER.....	321
21.5.53	ARM_DRIVER_ERROR_SPECIFIC.....	321
21.5.54	ON_STORAGE_LIB_WRITE_ENABLE_DISABLE.....	321
21.5.55	DRIVER_STORAGE_UNLOCK_CODE.....	322
21.6	Non-Volatile Memory (NVM) Library Reference Function Documentation.....	322
21.6.1	NVMLIB_GlobalWriteEnable.....	322
21.6.2	NVMLIB_GlobalWriteDisable.....	323
21.6.3	NVMLIB_DeviceWriteEnable.....	323
21.6.4	NVMLIB_DeviceWriteDisable.....	324
21.6.5	NVMLIB_StatusRegister_Read.....	325
21.6.6	NVMLIB_StatusRegister_Write.....	325
21.6.7	NVMLIB_EnableWrite.....	326
21.6.8	NVMLIB_Initialize.....	327
21.6.9	NVMLIB_Uninitialize.....	329
21.6.10	NVMLIB_PowerControl.....	330
21.6.11	NVMLIB_ReadData.....	331
21.6.12	NVMLIB_ProgramData.....	332
21.6.13	NVMLIB_Erase.....	333
21.6.14	NVMLIB_EraseAll.....	334
21.6.15	NVMLIB_GetBlock.....	335
21.6.16	NVMLIB_GetNextBlock.....	336
21.6.17	NVMLIB_GetInfo.....	337
21.6.18	NVMLIB_Command.....	338

Ezairo 8300 Firmware Reference

21.6.19 NVMLIB_Interface_Configure.....	339
21.6.20 NVMLIB_InterfaceQSPI_Configure.....	341
22. CFX AES Library Reference.....	344
22.1 Summary.....	344
22.2 CFX AES Library Reference Typedef Documentation.....	344
22.2.1 AES_KEY_EXPANSION.....	344
22.3 CFX AES Library Reference Data Structures Type Documentation.....	345
22.3.1 _AES_KEY_EXPANSION.....	345
22.4 CFX AES Library Reference Function Documentation.....	345
22.4.1 AESInitialize.....	345
22.4.2 AESExpandKey.....	346
22.4.3 AESDecrypt.....	346
22.4.4 AESCleanUp.....	347
23. Supplemental Calibration Library Reference.....	349
23.1 Summary.....	349
23.2 Detailed Description.....	351
23.3 Supplemental Calibration Library Reference Macro Definition Documentation.....	351
23.3.1 ERRNO_POWER_CAL_MARKER.....	351
23.3.2 ERRNO_VBG_CAL_ERROR.....	351
23.3.3 ERRNO_VBG_CAL_BADREF_ERROR.....	352
23.3.4 ERRNO_VDDIF_CAL_ERROR.....	352
23.3.5 ERRNO_VDDA_CAL_ERROR.....	352
23.3.6 ERRNO_VDDC_CAL_ERROR.....	352
23.3.7 ERRNO_VDDM_CAL_ERROR.....	352
23.3.8 ERRNO_VMIC_CAL_ERROR.....	353
23.3.9 ERRNO_VMIC_EBL_ERROR.....	353

Ezairo 8300 Firmware Reference

23.3.10	ERRNO_VDDOD_CAL_ERROR.....	353
23.3.11	ERRNO_VDDOD_EBL_ERROR.....	353
23.3.12	ERRNO_CLK_CAL_MARKER.....	354
23.3.13	ERRNO_INTERNAL_OSC_CAL_ERROR.....	354
23.3.14	ERRNO_SUPPORT_CAL_MARKER.....	354
23.3.15	ERRNO_MANU_CRC_CAL_ERROR.....	354
23.3.16	ERRNO_NVMLIB_READ_CAL_ERROR.....	354
23.3.17	ERRNO_NVMLIB_WRITE_CAL_ERROR.....	355
23.3.18	ERRNO_PARAMETER_CAL_ERROR.....	355
23.3.19	ERRNO_BOOTINFO_CRC_CAL_ERROR.....	355
23.3.20	ERRNO_LEN_MISMATCH_CAL_ERROR.....	355
23.3.21	ERRNO_UNALIGNED_CAL_ERROR.....	355
23.3.22	ERRNO_ADC_CAL_MARKER.....	356
23.3.23	ERRNO_ADC_OFFSET_CAL_ERROR.....	356
23.3.24	MIN_ADC_STABILIZATION_DELAY.....	356
23.3.25	MIN_ADC_AVERAGES.....	356
23.3.26	VREG_TARGET.....	356
23.3.27	VDDCM_MIN_CP_DELTA_TARGET.....	357
23.3.28	VDDC_RETENTION_TARGET.....	357
23.3.29	VDDC_LIMIT_TARGET.....	357
23.3.30	VDDC_ACTIVE_TARGET.....	357
23.3.31	VDDM_RETENTION_TARGET.....	358
23.3.32	VDDM_STANDBY_TARGET.....	358
23.3.33	VDDM_ACTIVE_TARGET.....	358
23.3.34	VDDA_ACTIVE_TARGET.....	358
23.3.35	VDDIF_ACTIVE_TARGET.....	358

Ezairo 8300 Firmware Reference

23.3.36	VMIC_ACTIVE_TARGET.....	359
23.3.37	VDDOD_ACTIVE_TARGET.....	359
23.3.38	VSSA_EXPECTED.....	359
23.4	Supplemental Calibration Library Reference Function Documentation.....	359
23.4.1	Calibrate_ADC_Offset.....	359
23.4.2	Calibrate_Clock_Initialize.....	360
23.4.3	Calibrate_Clock_Internal_OSC.....	361
23.4.4	Calibrate_Clock_Internal_OSC_Mult.....	361
23.4.5	Calibrate_Power_VREGAndLSAD.....	362
23.4.6	Calibrate_Power_VDDIF.....	363
23.4.7	Calibrate_Power_VDDA.....	365
23.4.8	Calibrate_Power_VDDA_PMURef.....	366
23.4.9	Calibrate_Power_VDDC.....	367
23.4.10	Calibrate_Power_VDDC_PMURef.....	368
23.4.11	Calibrate_Power_VDDM.....	369
23.4.12	Calibrate_Power_VDDM_PMURef.....	370
23.4.13	Calibrate_Power_VMIC.....	372
23.4.14	Calibrate_Power_VDDOD.....	373
23.4.15	Calibrate_Power_VoltageLoaded.....	374
23.4.16	Calibrate_Support_Initialize.....	374
23.4.17	Calibrate_Support_ClearManuTable.....	375
23.4.18	Calibrate_Support_LoadManuTable.....	376
23.4.19	Calibrate_Support_WriteManuTable.....	377
23.4.20	Calibrate_Support_UpdateVDDIF.....	377
23.4.21	Calibrate_Support_UpdateVDDA.....	378
23.4.22	Calibrate_Support_UpdateVDDC.....	379

Ezairo 8300 Firmware Reference

23.4.23	Calibrate_Support_UpdateVDDCByValue.....	380
23.4.24	Calibrate_Support_UpdateVDDC_CP.....	381
23.4.25	Calibrate_Support_UpdateVDDM.....	382
23.4.26	Calibrate_Support_UpdateVDDMByValue.....	383
23.4.27	Calibrate_Support_UpdateVDDM_CP.....	384
23.4.28	Calibrate_Support_UpdateVDDM_CPByValue.....	385
23.4.29	Calibrate_Support_UpdateVDDOD.....	386
23.4.30	Calibrate_Support_UpdateVMIC.....	387
23.4.31	Calibrate_Support_UpdateVREG.....	387
23.4.32	Calibrate_Support_UpdateCLK.....	388
23.4.33	Calibrate_Support_UpdateLSAD_GAIN.....	389
23.4.34	Calibrate_Support_UpdateLSAD_OFFSET.....	389
23.4.35	Calibrate_Support_UpdateADC_OFFSET.....	390
23.4.36	Calibrate_Support_Delay_Cycles.....	391
24.	Arm Cortex-M3 Core Bootloader Reference.....	392
24.1	Summary.....	392
24.2	Arm Cortex-M3 Core Bootloader Reference Data Structures Type Documentation.....	393
24.2.1	application_file_header.....	393
24.2.2	data_section_header.....	393
24.2.3	boot_section_data.....	395
24.3	Arm Cortex-M3 Core Bootloader Reference Macro Definition Documentation.....	395
24.3.1	APPFILE_IDENT_APP.....	395
24.3.2	APPFILE_IDENT_APPENCRYPTED.....	396
24.3.3	APPFILE_IDENT_MANIFEST.....	396
24.3.4	APPFILE_IDENT_MANIFESTENCRYPTED.....	396
24.3.5	SEC_CFX_X.....	396

Ezairo 8300 Firmware Reference

24.3.6	SEC_CFX_Y.....	396
24.3.7	SEC_CFX_P.....	397
24.3.8	SEC_CM3_P.....	397
24.3.9	SEC_IOMEM.....	397
24.3.10	SEC_BOOT_CFX.....	397
24.3.11	SEC_BOOT_CM3.....	397
24.3.12	APPDATA_FLAGS_CLEARTEXT.....	398
24.3.13	APPDATA_FLAGS_ENCRYPTED.....	398
24.3.14	APPDATA_FLAGS_CRC.....	399
24.3.15	APPDATA_FLAGS_VOLATILE.....	399
24.3.16	APPDATA_FLAGS_UNLOCKED.....	399
24.3.17	APPDATA_FLAGS_LOCKED.....	399
24.3.18	APPDATA_FLAGS_EMPTY.....	399
24.3.19	MANIFEST_MAXIMUM_SIZE.....	400
24.3.20	APP_HEADER_MAXIMUM_SIZE.....	400
24.4	Arm Cortex-M3 Core Bootloader Reference Function Documentation.....	400
24.4.1	bootloader_load_manu_table.....	400
24.4.2	bootloader_load_app_file.....	401
24.4.3	bootloader_boot.....	402
24.4.4	bootloader_main.....	402
25	CFX DSP Library Reference.....	404
25.1	Summary.....	404
25.2	Detailed Description.....	404
25.3	Basic Math.....	404
25.3.1	Summary.....	404
25.3.2	Basic Math Function Documentation.....	405

Ezairo 8300 Firmware Reference

25.3.2.1	cfx_abs.....	405
25.3.2.2	cfx_abs.....	405
25.3.2.3	cfx_abs.....	406
25.3.2.4	cfx_abs.....	406
25.3.2.5	cfx_abs.....	407
25.3.2.6	cfx_abs.....	407
25.3.2.7	cfx_min.....	408
25.3.2.8	cfx_min.....	408
25.3.2.9	cfx_min.....	409
25.3.2.10	cfx_min.....	409
25.3.2.11	cfx_min.....	410
25.3.2.12	cfx_min.....	410
25.3.2.13	cfx_max.....	411
25.3.2.14	cfx_max.....	412
25.3.2.15	cfx_max.....	412
25.3.2.16	cfx_max.....	413
25.3.2.17	cfx_max.....	413
25.3.2.18	cfx_max.....	414
25.4	Data Conversion.....	414
25.4.1	Summary.....	414
25.4.2	Data Conversion Function Documentation.....	415
25.4.2.1	cfx_mag_to_dB.....	415
25.4.2.2	cfx_pwr_to_dB.....	415
25.4.2.3	cfx_dB_to_mag.....	416
25.4.2.4	cfx_dB_to_pwr.....	416
25.4.2.5	cfx_mag_to_dB_vec.....	417

Ezairo 8300 Firmware Reference

25.4.2.6	cfx_pwr_to_dB_vec.....	418
25.4.2.7	cfx_dB_to_mag_vec.....	418
25.4.2.8	cfx_dB_to_pwr_vec.....	419
25.5	Math Accelerator Functions.....	419
25.5.1	Summary.....	419
25.5.2	Detailed Description.....	419
25.6	Log2 Functions.....	419
25.6.1	Summary.....	420
25.6.2	Detailed Description.....	420
25.6.3	Log2 Functions Function Documentation.....	420
25.6.3.1	cfx_log2_m8p16.....	420
25.6.3.2	cfx_log2_m8p16.....	421
25.6.3.3	cfx_log2_m8p40.....	421
25.6.3.4	cfx_log2_m8p40.....	422
25.6.3.5	cfx_log2_vec_m8p16.....	422
25.6.3.6	cfx_log2_vec_m8p16.....	423
25.6.3.7	cfx_log2_vec_m8p40.....	423
25.6.3.8	cfx_log2_vec_m8p40.....	424
25.7	Exp2 Functions.....	424
25.7.1	Summary.....	424
25.7.2	Detailed Description.....	424
25.7.3	Exp2 Functions Function Documentation.....	425
25.7.3.1	cfx_exp2_frac24.....	425
25.7.3.2	cfx_exp2_frac48.....	425
25.7.3.3	cfx_exp2_frac24.....	426
25.7.3.4	cfx_exp2_frac48.....	426

Ezairo 8300 Firmware Reference

25.7.3.5	cfx_exp2_vec_frac24.....	427
25.7.3.6	cfx_exp2_vec_frac48.....	427
25.7.3.7	cfx_exp2_vec_frac24.....	428
25.7.3.8	cfx_exp2_vec_frac48.....	428
25.8	Root Functions.....	429
25.8.1	Summary.....	429
25.8.2	Detailed Description.....	429
25.8.3	Root Functions Function Documentation.....	430
25.8.3.1	cfx_root2_frac24.....	430
25.8.3.2	cfx_root2_frac24.....	430
25.8.3.3	cfx_root2_frac48.....	431
25.8.3.4	cfx_root2_frac48.....	431
25.8.3.5	cfx_root3_frac24.....	432
25.8.3.6	cfx_root3_frac24.....	432
25.8.3.7	cfx_root3_frac48.....	433
25.8.3.8	cfx_root3_frac48.....	433
25.8.3.9	cfx_root4_frac24.....	434
25.8.3.10	cfx_root4_frac24.....	434
25.8.3.11	cfx_root4_frac48.....	435
25.8.3.12	cfx_root2_vec_frac24.....	435
25.8.3.13	cfx_root2_vec_frac24.....	436
25.8.3.14	cfx_root2_vec_frac48.....	436
25.8.3.15	cfx_root2_vec_frac48.....	437
25.8.3.16	cfx_root3_vec_frac24.....	437
25.8.3.17	cfx_root3_vec_frac24.....	438
25.8.3.18	cfx_root3_vec_frac48.....	438

Ezairo 8300 Firmware Reference

25.8.3.19	cfx_root3_vec_frac48.....	439
25.8.3.20	cfx_root4_vec_frac24.....	439
25.8.3.21	cfx_root4_vec_frac24.....	440
25.8.3.22	cfx_root4_vec_frac48.....	440
25.8.3.23	cfx_root4_vec_frac48.....	441
25.8.3.24	cfx_root4_frac48.....	441
25.9	Division Functions.....	442
25.9.1	Summary.....	442
25.9.2	Division Functions Function Documentation.....	442
25.9.2.1	cfx_div_m24p24.....	442
25.9.2.2	cfx_div_m24p24.....	443
25.9.2.3	cfx_div_vec_m24p24.....	443
25.9.2.4	cfx_div_vec_m24p24.....	444
25.10	Assignment, Arithmetic, and Relational Operators.....	444
25.10.1	Summary.....	444
25.10.2	Detailed Description.....	444
25.11	Arithmetic Operators.....	446
25.11.1	Summary.....	446
25.11.2	Arithmetic Operators Function Documentation.....	448
25.11.2.1	operator-.....	448
25.11.2.2	operator-.....	448
25.11.2.3	operator-.....	449
25.11.2.4	operator-.....	450
25.11.2.5	operator-.....	450
25.11.2.6	operator-.....	451
25.11.2.7	operator+.....	451

Ezairo 8300 Firmware Reference

25.11.2.8 operator+.....	452
25.11.2.9 operator+.....	452
25.11.2.10 operator+.....	453
25.11.2.11 operator+.....	453
25.11.2.12 operator+.....	454
25.11.2.13 operator+.....	454
25.11.2.14 operator+.....	455
25.11.2.15 operator+.....	455
25.11.2.16 operator+.....	456
25.11.2.17 operator+.....	456
25.11.2.18 operator+.....	457
25.11.2.19 operator+.....	458
25.11.2.20 operator+.....	458
25.11.2.21 operator+.....	459
25.11.2.22 operator+.....	459
25.11.2.23 operator+.....	460
25.11.2.24 operator+.....	460
25.11.2.25 operator+.....	461
25.11.2.26 operator+.....	461
25.11.2.27 operator+.....	462
25.11.2.28 operator+.....	462
25.11.2.29 operator+.....	463
25.11.2.30 operator+.....	464
25.11.2.31 operator+.....	464
25.11.2.32 operator+.....	465
25.11.2.33 operator+.....	465

Ezairo 8300 Firmware Reference

25.11.2.34	operator+	466
25.11.2.35	operator+	466
25.11.2.36	operator+	467
25.11.2.37	operator+	467
25.11.2.38	operator+	468
25.11.2.39	operator-	468
25.11.2.40	operator-	469
25.11.2.41	operator-	470
25.11.2.42	operator-	470
25.11.2.43	operator-	471
25.11.2.44	operator-	471
25.11.2.45	operator-	472
25.11.2.46	operator-	472
25.11.2.47	operator-	473
25.11.2.48	operator-	473
25.11.2.49	operator-	474
25.11.2.50	operator-	474
25.11.2.51	operator-	475
25.11.2.52	operator-	476
25.11.2.53	operator-	476
25.11.2.54	operator-	477
25.11.2.55	operator-	477
25.11.2.56	operator-	478
25.11.2.57	operator-	478
25.11.2.58	operator-	479
25.11.2.59	operator-	479

Ezairo 8300 Firmware Reference

25.11.2.60	operator-	480
25.11.2.61	operator-	480
25.11.2.62	operator-	481
25.11.2.63	operator-	482
25.11.2.64	operator-	482
25.11.2.65	operator-	483
25.11.2.66	operator-	483
25.11.2.67	operator-	484
25.11.2.68	operator-	484
25.11.2.69	operator-	485
25.11.2.70	operator-	485
25.11.2.71	operator*	486
25.11.2.72	operator*	486
25.11.2.73	operator*	487
25.11.2.74	operator*	488
25.11.2.75	operator*	488
25.11.2.76	operator*	489
25.11.2.77	operator*	489
25.11.2.78	operator*	490
25.11.2.79	operator*	490
25.11.2.80	operator/	491
25.11.2.81	operator/	491
25.11.2.82	operator<<<	492
25.11.2.83	operator<<<	492
25.11.2.84	operator<<<	493
25.11.2.85	operator<<<	494

Ezairo 8300 Firmware Reference

25.11.2.86	operator<<<.....	494
25.11.2.87	operator<<<.....	495
25.11.2.88	operator>>>.....	495
25.11.2.89	operator>>>.....	496
25.11.2.90	operator>>>.....	496
25.11.2.91	operator>>>.....	497
25.11.2.92	operator>>>.....	497
25.11.2.93	operator>>>.....	498
25.11.2.94	operator*.....	498
25.11.2.95	operator*.....	499
25.12	Assignment Operators.....	500
25.12.1	Summary.....	500
25.12.2	Assignment Operators Function Documentation.....	500
25.12.2.1	operator=.....	500
25.12.2.2	operator=.....	501
25.12.2.3	operator=.....	501
25.12.2.4	operator=.....	501
25.12.2.5	operator=.....	502
25.12.2.6	operator=.....	502
25.12.2.7	operator=.....	503
25.12.2.8	operator=.....	503
25.12.2.9	operator=.....	503
25.12.2.10	operator=.....	504
25.12.2.11	operator=.....	504
25.12.2.12	operator=.....	505
25.12.2.13	operator=.....	505

onsemi Confidential
Ezairo 8300 Firmware Reference

25.12.2.14	operator=	505
25.12.2.15	operator=	506
25.12.2.16	operator=	506
25.12.2.17	operator=	507
25.12.2.18	operator=	507
25.12.2.19	operator=	507
25.12.2.20	operator=	508
25.12.2.21	operator+=	508
25.12.2.22	operator+=	509
25.12.2.23	operator-=	509
25.12.2.24	operator-=	509
25.12.2.25	operator*=	510
25.13	Relational Operators	510
25.13.1	Summary	510
25.13.2	Relational Operators Function Documentation	511
25.13.2.1	operator>=	511
25.13.2.2	operator>=	512
25.13.2.3	operator>=	512
25.13.2.4	operator>=	513
25.13.2.5	operator>=	513
25.13.2.6	operator>=	514
25.13.2.7	operator>;	515
25.13.2.8	operator>;	515
25.13.2.9	operator>;	516
25.13.2.10	operator>;	516
25.13.2.11	operator>;	517

Ezairo 8300 Firmware Reference

25.13.2.12	operator>.....	517
25.13.2.13	operator<=.....	518
25.13.2.14	operator<=.....	518
25.13.2.15	operator<=.....	519
25.13.2.16	operator<=.....	519
25.13.2.17	operator<=.....	520
25.13.2.18	operator<=.....	521
25.13.2.19	operator<.....	521
25.13.2.20	operator<.....	522
25.13.2.21	operator<.....	522
25.13.2.22	operator<.....	523
25.13.2.23	operator<.....	523
25.13.2.24	operator<.....	524
25.13.2.25	operator==.....	524
25.13.2.26	operator==.....	525
25.13.2.27	operator==.....	525
25.13.2.28	operator==.....	526
25.13.2.29	operator==.....	527
25.13.2.30	operator==.....	527
25.13.2.31	operator!=.....	528
25.13.2.32	operator!=.....	528
25.13.2.33	operator!=.....	529
25.13.2.34	operator!=.....	529
25.13.2.35	operator!=.....	530
25.13.2.36	operator!=.....	530
25.14	Signal Processing.....	531

25.14.1 Summary.....	531
25.15 Filtering Functions.....	531
25.15.1 Summary.....	531
25.15.2 Filtering Functions Data Structures Type Documentation.....	532
25.15.2.1 AttackReleaseFilter.....	532
25.15.2.2 FIRFilter.....	532
25.15.2.3 Biquad.....	533
25.15.3 Filtering Functions Function Documentation.....	533
25.15.3.1 cfx_attack_release_init.....	533
25.15.3.2 cfx_attack_release.....	534
25.15.3.3 cfx_attack_release_vec.....	535
25.15.3.4 cfx_fir_init.....	535
25.15.3.5 cfx_fir.....	536
25.15.3.6 cfx_fir.....	536
25.15.3.7 cfx_biquad_init.....	537
25.15.3.8 cfx_biquad.....	537
25.16 Trigonometric Functions.....	537
25.16.1 Summary.....	537
25.16.2 Trigonometric Functions Function Documentation.....	538
25.16.2.1 cfx_sin.....	538
25.16.2.2 cfx_sin_vec.....	538
25.16.2.3 cfx_cos.....	539
25.16.2.4 cfx_cos_vec.....	539
25.17 Signal Generation Functions.....	540
25.17.1 Summary.....	540
25.17.2 Signal Generation Functions Data Structures Type Documentation.....	540

Ezairo 8300 Firmware Reference

25.17.2.1	ToneGenerator.....	540
25.17.3	Signal Generation Functions Function Documentation.....	541
25.17.3.1	cfx_tone_generator_init.....	541
25.17.3.2	cfx_tone_generator.....	541
25.18	Interpolation Functions.....	542
25.18.1	Summary.....	542
25.18.2	Interpolation Functions Function Documentation.....	542
25.18.2.1	cfx_linear_interpolation.....	542
25.19	Fixed-Point Type Conversion.....	543
25.19.1	Summary.....	543
25.19.2	Detailed Description.....	544
25.19.3	Fixed-Point Type Conversion Macro Definition Documentation.....	545
25.19.3.1	to_frac28.....	545
25.19.3.2	to_m2p26.....	545
25.19.3.3	to_frac24.....	546
25.19.3.4	to_m2p22.....	546
25.19.3.5	to_m8p16.....	547
25.19.3.6	to_m4p50.....	547
25.19.3.7	to_m1lp43.....	548
25.19.4	Fixed-Point Type Conversion Function Documentation.....	548
25.19.4.1	cfx_to_frac24.....	549
25.19.4.2	cfx_to_frac24.....	549
25.19.4.3	cfx_to_frac24.....	550
25.19.4.4	cfx_to_frac24.....	550
25.19.4.5	cfx_to_frac24.....	551
25.19.4.6	cfx_to_frac24.....	551

Ezairo 8300 Firmware Reference

25.19.4.7 cfx_to_frac48.....	552
25.19.4.8 cfx_to_frac48.....	552
25.19.4.9 cfx_to_frac48.....	553
25.19.4.10 cfx_to_frac48.....	553
25.19.4.11 cfx_to_frac48.....	554
25.19.4.12 cfx_to_frac48.....	554
25.19.4.13 cfx_to_m9p47.....	555
25.19.4.14 cfx_to_m9p47.....	555
25.19.4.15 cfx_to_m9p47.....	556
25.19.4.16 cfx_to_m9p47.....	556
25.19.4.17 cfx_to_m9p47.....	557
25.19.4.18 cfx_to_m9p47.....	557
25.19.4.19 cfx_to_m8p16.....	558
25.19.4.20 cfx_to_m8p16.....	558
25.19.4.21 cfx_to_m8p16.....	559
25.19.4.22 cfx_to_m8p16.....	559
25.19.4.23 cfx_to_m8p16.....	560
25.19.4.24 cfx_to_m8p16.....	560
25.19.4.25 cfx_to_m8p40.....	561
25.19.4.26 cfx_to_m8p40.....	561
25.19.4.27 cfx_to_m8p40.....	562
25.19.4.28 cfx_to_m8p40.....	562
25.19.4.29 cfx_to_m8p40.....	563
25.19.4.30 cfx_to_m8p40.....	563
25.19.4.31 cfx_to_m16p40.....	564
25.19.4.32 cfx_to_m16p40.....	564

Ezairo 8300 Firmware Reference

25.19.4.33	cfx_to_m16p40.....	565
25.19.4.34	cfx_to_m16p40.....	565
25.19.4.35	cfx_to_m16p40.....	566
25.19.4.36	cfx_to_m16p40.....	566
25.19.4.37	cfx_as_frac24.....	567
25.19.4.38	cfx_as_frac48.....	567
25.19.4.39	cfx_as_int.....	568
25.19.4.40	cfx_as_long.....	568
25.20	Fixed-Point Types.....	569
25.20.1	Summary.....	569
25.20.2	Detailed Description.....	569
25.20.3	Fixed-Point Types Data Structures Type Documentation.....	570
25.20.3.1	frac24_t.....	570
25.20.3.2	frac48_t.....	571
25.20.3.3	m9p47_t.....	571
25.20.3.4	m8p16_t.....	572
25.20.3.5	m8p40_t.....	572
25.20.3.6	m16p40_t.....	573
25.20.3.7	m24p24_t.....	573
A.	Glossary.....	575

CHAPTER 1

Introduction

1.1 PURPOSE

IMPORTANT: onsemi acknowledges that this document might contain the inappropriate terms “white list”, “master” and “slave”. We have a plan to work with other companies to identify an industry wide solution that can eradicate non-inclusive terminology but maintains the technical relationship of the original wording. Once new terminologies are agreed upon, future products will contain new terminology.

This group of topics describes the Ezairo® 8300 firmware. It includes descriptions, function listings, and usage examples to help you understand the firmware and its parts.

The firmware described in this group of topics provides developers with a convenient software layer on which to build their applications. It is also responsible for system-level tasks such as booting the system and implementing portions of the security layer. The firmware consists of include files, macros, libraries, ROM code, and executables.

1.2 INTENDED AUDIENCE

This group of topics is intended for use by developers who are designing and implementing applications for the Ezairo 8300 series. Both novice and experienced developers can benefit. The descriptions and code examples can help novice users to learn the system, while experienced developers can go straight to the reference chapters that describe the available functions, macros, libraries and executables.

The topic assume that the reader has a basic understanding of:

- The architecture of the Ezairo 8300 chip
- The C programming language
- The CFX assembly language and fundamentals of the Arm® Thumb-2 assembly language
- The integrated development environment and toolchains that form the development tools for the Ezairo 8300 series

For sources of information on these topics, see [Section 1.4 “Further Reading” on page 48](#).

1.3 CONVENTIONS

`monospace font`

Assembly code, macros, functions, defines and addresses.

italics

File and path names, or any portion of them.

<angle brackets>

Optional parameters and placeholders for specific information. To use an optional parameter or replace a placeholder, specify the information within the brackets; do not include the brackets themselves.

1.4 FURTHER READING

For more information, refer to the following documents installed with the SDK:

Ezairo 8300 Firmware Reference

- In C:/Program Files (x86)/ON Semiconductor/Ezairo 8300 SDK/documentation:
 - *Introduction to Ezairo 8300 Programming*
 - *Communication Protocols Manual for Ezairo 8300*
 - *Ezairo 8300 Hardware Reference*
 - *Integrated Development Environment User's Guide for Ezairo 8300*
- ~~In C:/Program Files (x86)/ON Semiconductor/Ezairo 8300 SDK/documentation/cadence_3rd_party_docs:~~
 - ~~*SD-Host Firmware Driver User Guide*~~
 - ~~*SD-Host Firmware Driver Quick Start Guide*~~
 - ~~*Cadence I3C Master Controller Interface Firmware Driver User Guide*~~
 - ~~*Cadence I3C Master Controller Core Driver Quick Start Guide*~~

CHAPTER 2

Firmware Overview

The firmware provides a layer of support between the application developer and the underlying hardware. It is a collection of files, macros, functions and libraries designed to make application development simpler, quicker and more reliable.

2.1 VERSIONS

Version symbols are provided for each major firmware component. The version symbols can be used directly or indirectly to verify the version of the components used to build an application. There are three types of version symbols available:

Define

A preprocessor define containing the version information

Symbolic

A compiled symbol contained within a binary library

Global Variable

A global variable in memory containing the symbol

The available version information for each firmware component is listed in the ["Firmware Versions" figure \(Table 1\)](#).

Table 1. Firmware Versions

Component	Type	Version Symbolic
CFX Firmware includes	Define	SYS_FW_VER
CFX System Library	Symbolic	SYSLIB_Version
CFX NVM Library	Symbolic	NVMLIB_Version
CFX AES Library	Symbolic	AES_Version
CFX Program ROM	Global Variable	SYSVAR_PROGROM_VER
Bootloader	Global Variable	SYSVAR_BOOTLOADER_VER

2.2 RESERVED MEMORY

The Ezairo 8300 firmware and related resources use a few areas of memory that must be considered reserved when using those firmware components. This memory usage includes the items listed in the ["Reserved Memory Items" table \(Table 2\)](#).

Ezairo 8300 Firmware Reference

Table 2. Reserved Memory Items

Firmware Element	CFX Address	Arm Cortex-M3 Address	Description
System Variables (Program ROM)	XMEM 0x4000 to 0x401F	0x20410000 to 0x2041007F and mirrors	Described in Section 2.2.1 "System Variables" , the system variables are used by the program ROM and other firmware to communicate with user applications.
Program ROM	XMEM 0x4020 to 0x402B	0x20410080 to 0x204100AF and mirrors	Program ROM working memory.
NVM library	XMEM 0x402C to 0x404B	0x204100B0 to 0x2041012F and mirrors	NVM library internal variables applies to the ROM-linked instance only. For more information on the NVM library, see Section 10 "NVMLIB Library" on page 72
Arm Cortex-M3 core bootloader	IOMEM 0x46000 to 0x47FFF	0x20018000 to 0x2001FFFF	Arm Cortex-M3 core bootloader working memory (DRAM2, DRAM3) NOTE: Applications being loaded must not place any data (any member of the <code>.data</code> section) at those memory locations. If an application has initialized data at those locations, the bootloader might overwrite its variables, leading to unpredictable behavior. The memory usage restriction does not apply to uninitialized global data (<code>.bss</code>), as this section is initialized by each application's <code>_start</code> function. This restriction also does not apply to placing the stack in DRAM2 or DRAM3. The firmware does not reserve any specific memory locations for a bootloader. If your NVM image contains multiple applications that share a bootloader, the bootloader needs to be assigned to a standard location that can be used by all of the applications.
HAL, Arm Cortex-M3 core bootloader	IOMEM 0x18000 to 0x1802E	0x30000 to 0x300B8	Manufacturing Information area copy mirror; loaded using <code>Sys_Trims_LoadManuTable()</code>, used by all other <code>Sys_Trims_*</code> functions. For more information on HAL, see Section 4 "Hardware Abstraction Layer" on page 57 The Arm Cortex-M3 core bootloader uses these functions, and thus has the same reserved memory usage.

Ezairo 8300 Firmware Reference

2.2.1 System Variables

The Ezairo 8300 firmware uses some locations in XMEM to store the system variables. The variables are shown in the "System Variables in XMEM" table (Table 3) with their CFX XMEM addresses and descriptions.

Table 3. System Variables in XMEM

Variable	CFX Address (XMEM)	Arm Cortex-M3 Address	Description
SYSVAR_BOOTLOADER_VER	0x4000	0x20410000	Bootloader version
SYSVAR_PROGROM_VER	0x4001	0x20410004	Program ROM version
SYSVAR_BOOTLOAD_VECT	0x4002	0x20410008	Not used
SYSVAR_OPERATING_FREQ	0x4003	0x2041000C	Current operating frequency
SYSVAR_KROM_VERIFY	0x4006	0x20410018	AES-key-related error code
SYSVAR_KEY	0x4007	0x2041001C	Selected AES key index
SYSVAR_SYS_ERRNO	0x4008	0x20410020	System error code
SYSVAR_BOOT_LINE	0x4009	0x20410024	Boot ROM error code
SYSVAR_BOOT_APPN	0x400A	0x20410028	Bootloader error code
SYSVAR_OPERATING_FREQ_HZ	0x400B	0x2061002C	Current operating frequency (Hz)
Reserved	0x400C-0x401F		Reserved for future use

The SYSVAR_BOOT_LINE variable is divided in two fields. The 8 least-significant bits are set with the boot error code. The SYSVAR_OPERATING_FREQ_HZ variable is mapped to the MSB-aligned CFX XMEM address so that the frequency can be read and updated as a 32-bit number from the Arm Cortex-M3 processor. The CFX sees the 24 most significant bits of the operating frequency at this memory location. CMSIS variable `SystemCoreClock` is mapped to this system variable.

CHAPTER 3

Hardware Definitions

Hardware definition files are integral to the system firmware. The hardware definitions apply a layer of data structures and address mappings to the underlying hardware, so that every control register, bit field in the system, memory, and interrupt vector is easily accessible from C and assembly code.

3.1 INTRODUCTION

The CFX DSP, Arm® Cortex®-M3 processor, and LPDSP32 DSP on the Ezairo 8300 chip are each supported by a set of header files that provide a system definition for of the Ezairo 8300 SoC, as defined from that processors perspective. For each processor this includes:

- Register and bit descriptions for control and configuration registers accessible to the processor
- Memory maps for all of the memories and memory-mapped elements that are accessible to the processor
- Interrupt vector table descriptions for the processor
- Macros that support the processors' basic core functionality

The format and configuration of all of the Arm Cortex-M3 processor's support files conform to CMSIS compatibility requirements. The header files for the CFX DSP and LPDSP32 DSP share similar structures wherever possible.

Each processor is supported by a top-level include file, that links in the hardware definitions and any relevant Hardware Abstraction Layer (HAL) elements that are defined to support that processor. For more information on the HAL support files linked into the top-level include files, see [Chapter 4.1: Usage](#). The top-level include files for the processors are:

- CFX DSP: *sk5.h* from *include\cfx*
- Arm Cortex-M3 core: *sk5.h* from *include\cm3*
- LPDSP32 DSP: *sk5.lpdsp32.h*

If an application includes the appropriate top-level file and defines `SK5_CID` to match the chip identifier of Ezairo 8300, then all of the support macros and HAL functions that are available to support that processor development on the Ezairo 8300 chip are made accessible to that application.

3.2 REGISTER AND BIT-FIELD DEFINITION

Using the hardware definition files allows you to refer to system components by C structures, assembly code, and preprocessor symbols instead of by addresses and bit fields. This greatly improves the readability, reliability and maintainability of your application code. The use of hardware definitions in an application also means that some hardware changes, such as changes to addresses or bit field values, are transparent to your application code.

Hardware register descriptions for all standard components for each processor are available in the following files, installed with the SDK in the *C:/Program Files (x86)/ON Semiconductor/Ezairo 8300 SDK/include* folder:

- CFX DSP: */cfx/sk5_cfx_hw.h* and *sk5_cfx_hw_cid*.h*
- Arm Cortex-M3 core: */cm3/sk5_hw.h* and *sk5_hw_cid*.h*
- LPDSP32 DSP: */lpdsp32/sk5_lpdsp32_hw.h*

Additionally, register descriptions for standard Arm Cortex-M3 processor peripherals, such as NVIC and SysTick, are defined in the core CMSIS header file, *core_cm3.h*.

Ezairo 8300 Firmware Reference

Hardware descriptions in the register include files provide definitions supporting the SoC components using the defines and C objects listed in the "Hardware Register Components" table (Table 4). Support for the Arm Cortex-M3 processor, which supports sub-register and bit accesses, are supplemented by the defines and C objects in the "Hardware Register Components Applicable Only to the Arm Cortex-M3 Core" table (Table 5).

Table 4. Hardware Register Components

Item	Example	Description
Component Register Structure	DIO_Type	Provides a list of all of the registers that support a specified component, and the read/write types for those registers
Component Register Instance	DIO	Link the component register structure to the underlying hardware or sets of hardware
Bit-Field Positions	DIO_CFG_DRIVE_Pos	Defines the base position for any bit-field within a register
Bit-Field Mask	DIO_CFG_DRIVE_Mask	Defines a bit mask for any bit-field of more than one bit within a register
Register Structure	DIO_CFG_Type	Provides a list of all of the sub-registers and alias structures - Sub-registers are defined byte (8-bit) or short (16-bit) access interfaces to part of a register that includes all elements belonging to the same configuration area. - Aliases are Arm Cortex-M3 processor bitband aliases that provide bit access to individual single-bit bit-fields where the underlying hardware supports this single-bit access.
Register Instance	DIO_CFG	Links the register structure to the underlying hardware or sets of hardware for sub-registers
Bit-Setting	DIO_MODE_GPIO_IN_0	Defines providing human-readable equivalents to settings that can be applied to a register bit-field to obtain the desired behavior
Bit-Setting (bitband)	DIO_LPF_DISABLE_BITBAND	Alternate forms of a bit-setting that apply when using bitband aliases to read/write single register bits
Bit-Field Sub-Register Positions	DIO_CFG_IO_MODE_BYTE_Pos	Defines the base position for any bit-field within a register's sub-register
Bit-Field Sub-Register Mask	DIO_CFG_IO_MODE_BYTE_Pos	Defines a bit mask for any bit-field of more than one bit within a register's sub-register
Sub-Register Bit-Setting	DIO_MODE_GPIO_IN_0_BYTE	Defines providing human-readable equivalents to settings that can be applied to a register's sub-register bit-field to obtain the desired behavior

Ezairo 8300 Firmware Reference

Table 5. Hardware Register Components Applicable Only to the Arm Cortex-M3 Core

Item	Example	Description
Bit-Setting (bitband)	DIO_LPF_DISABLE_BITBAND	Alternate forms of a bit-setting that apply when using bitband aliases to read/write single register bits
Bit-Field Sub-Register Positions	DIO_CFG_IO_MODE_BYTE_Pos	Defines the base position for any bit-field within a register's sub-register
Bit-Field Sub-Register Mask	DIO_CFG_IO_MODE_BYTE_Pos	Defines a bit mask for any bit-field of more than one bit within a register's sub-register
Sub-Register Bit-Setting	DIO_MODE_GPIO_IN_0_BYTE	Defines providing human-readable equivalents to settings that can be applied to a register's sub-register bit-field to obtain the desired behavior

3.3 MEMORY MAP DEFINITION

For each processor, there exist specific memory map definitions which provide the locations of the following structures within the memory maps of each processor:

- Instruction and data bus memory structures
- System bus memory structures
- Peripheral bus memory-mapped control registers (including the base of control register groups for each system component)
- Private peripheral bus internal memory-mapped control registers
- System variables

These memory map definitions are provided for each processor in the following files:

- CFX DSP: *sk5_cfx_map.h*
- Arm Cortex-M3 core: *sk5_map.h*
- LPDSP32 DSP: *sk5_lpdsp32_map.h*

For more information on the memory maps of the CFX DSP, Arm Cortex-M3 processor, and LPDSP32, see the *Ezairo 8300 Hardware Reference*.

3.4 INTERRUPT VECTOR DEFINITION AND ACCESSIBLE INTERRUPTS

Interrupt vectors provide you with access to interrupts facilitating orderly processing operations, for optimal application processing speed and minimal delays. Interrupt vector definitions are provided both either alongside the memory map or independently for each processor in the following files:

- CFX DSP: *sk5_cfx_map.h*
- Arm Cortex-M3 core: *sk5_vectors.h*
- LPDSP32 DSP: *sk5_lpdsp32_vectors.h*

These definitions typically have the form `<interrupt_name>_IRQn`. For a complete list of interrupts accessible to each core, see the *Ezairo 8300 Hardware Reference* chapters for each processor.

Ezairo 8300 Firmware Reference

The interrupt controller linked to the CFX DSP, which serves all of the Ezairo 8300 interrupt sources, is directly connected to any components capable of triggering an interrupt. Each interrupt's associated interrupt vector, mapped into the base of program RAM memory, is associated with an interrupt source and an index. This interrupt controller is supported by the Hardware Abstraction Layer library, described in [Chapter 4.1 "Usage" on page 57](#). For more information and a complete list of interrupt sources and vectors, see the CFX Digital Signal Processor chapter of the *Ezairo 8300 Hardware Reference*.

Interrupt handling functionality in the Arm Cortex-M3 processor is provided by a nested vector interrupt controller (NVIC) implemented with the processor. It handles predefined interrupts, a non-maskable interrupt (NMI), and various other types including some external to the processor that are linked to interfaces and peripherals. The NVIC is supported by standard firmware as part of the CMSIS library, as described in [Chapter 6.1 "Introduction" on page 59](#), and by additional supporting functions the Hardware Abstraction Layer library, described in [Chapter 4.1 "Usage" on page 57](#). For more information and a complete list of interrupt vectors, along with enumeration defines and values, see the Arm Cortex-M3 Processor chapter of the *Ezairo 8300 Hardware Reference*.

The interrupt controller linked to the LPDSP32 DSP serves three sets of interrupts that can trigger execution on this processor: a set from the DMA channels, a set that can be issued by the CFX or Arm Cortex-M3 core, and one interrupt that can be issued by the LPDSP32's integrated neural-network accelerator. As interrupt accesses are limited, no additional firmware is provided for the LPDSP32 DSP. For more information and a complete list of vectors and descriptions, see the LPDSP32 Processor chapter of the *Ezairo 8300 Hardware Reference*.

CHAPTER 4

Hardware Abstraction Layer

The Hardware Abstraction Layer (HAL) library provides a set of multiple register access functions for the majority of the system components included in the Ezairo 8300 SoC. These functions and the provided function headers can be used to easily configure the hardware blocks through safe register accesses without needing to refer to the register tables and the corresponding settings in the *Ezairo 8300 Hardware Reference*.

The HAL library is implemented for both the CFX and the Arm Cortex-M3 cores, and is distributed in both source-code and pre-compiled form in the Ezairo 8300 SDK. All the library functions are blocking and interruptible. The library does not use any interrupts or DMA channels.

4.1 USAGE

To use the HAL library functions, include the top-level include file *sk5.h* in the application source, and link against the *libHAL.a* library object.

The HAL functions follow `Sys_<hardware block>_<function>` name syntax with:

- **<hardware block>** indicating the system component being configured or acted upon
 - In some cases, the hardware block is prefixed by a standard operation such as `Set_`, `Get_`
- **<function>** describing the functionality implemented by the HAL firmware

Simple wrapper macros with the same but capitalized names are also provided for many functions. These macros do not require a user to provide a pointer to the block to configure or operate upon, and default to the first instance available. An example of this naming convention is the `Sys_I2C_DIOConfig` function: it begins with the `Sys` convention, `I2C` shows that it is a I²C function, and `DIOConfig` means that the function configures the two DIOs for the specified I²C interface.

For the complete HAL library API, refer to [Chapter 18 "Hardware Abstraction Layer Reference"](#) on page 98.

CHAPTER 5

CFX System Library

The CFX DSP library provides a common set of 24, 48, and 56-bit fixed-point data types, and supports a number of operators and functions that use these data types. The use of this library to implement fixed-point math and signal processing functions on the CFX is recommended, and allows for simpler, more maintainable fixed-point code by managing the necessary bit shifts and data manipulation.

To achieve this, the library includes a number of functions to convert between fixed-point data types using appropriate rounding and saturation where the binary point is preserved, as well as re-interpretation functions where the underlying memory is not changed (i.e., `frac24_t` to `int`, and `int` to `frac24_t`).

5.1 USAGE

To use the CFX system library functions, include `sk5_syslib.h` in the application source and the application should link against the `libsyslib.a` library object.

For the complete API, refer to [Chapter 19 "CFX System Library Reference" on page 256](#).

CHAPTER 6

CMSIS Library

The Arm Cortex-M3 processor is supported by a standards-compliant CMSIS library and extensions to the CMSIS requirements. The CMSIS library:

- Provides access to the generic functions provided by the standard Arm CMSIS implementation.
 - The functions are included in the CMSIS header files, and reference documentation is provided by the standard Arm CMSIS documentation (http://arm-software.github.io/CMSIS_5/Core/html/modules.html).
- Implements the CMSIS-required device-specific functions with an appropriate implementation for Ezairo 8300.
- Adds extensions to the generic and required CMSIS functions to provide common support functions, including:
 - Weak definitions for all interrupt handlers for the Arm Cortex-M3 processor and associated NVIC
 - A start-up routine for ensuring that applications reach main in a safe state
 - Dynamic memory allocation (through an implementation of sbrk)
 - Hardware-safe functions for updating the system clock source and operating frequency

6.1 INTRODUCTION

The weakly defined interrupt handlers included in the CMSIS library provide empty implementation for each of the interrupt vectors, which simply returns from the interrupt. These interrupt handlers have the form `<interrupt_name>_IRQHandler()`. If a user application defines a function with this same name, the user application's definition of the interrupt handler will replace the default (empty) handlers.

The CMSIS library is supported by the top-level include file for Ezairo 8300, *sk5.h*, which includes all CMSIS, "Introduction" on page 53, and "Usage" on page 57 elements. To use the CMSIS library, include *sk5.h* and link against the *libcmsis.a* library archive.

For the complete API, refer to Chapter 20 "CMSIS Reference" on page 261.

CHAPTER 7

CMSIS Drivers

The CMSIS drivers are generic device-independent APIs for peripherals such as I²C and UART. They are created to allow easy setup and use of peripherals and interfaces by handling most of the manual configuration work in the back-end. The drivers can be used in the sample applications by including the appropriate interface header files and the CMSIS drivers library.

7.1 INTRODUCTION

These reference topics present the description of all the available CMSIS drivers and the instructions on how to use the configuration file (*RTE_Device.h*). The specifications and headers for all of the functions used in these CMSIS drivers have been provided by Arm, and the corresponding documentation can be found on the Arm website, keil.com, on this page: <http://www.keil.com/pack/doc/CMSIS/Driver/html/index.html>.

This chapter assumes that the user is familiar with importing projects into the IDE, and also with the location of the sample applications. For more information about these topics, see the *Integrated Development Environment User's Guide for Ezairo 8300*.

The "Interfaces for Which a CMSIS Driver is Available" table (Table 6) shows the CMSIS driver interfaces and sample apps for use with Ezairo 8300.

Table 6. Interfaces for Which a CMSIS Driver is Available

CMSIS Driver	Version	Sample Application
UART	UART0	uart_cmsis
I2C	I2C_0, I2C1, and D_I2C	i2c_cmsis

7.2 LIBRARY STRUCTURE

A drivers library is organized as follows:

- drivers - root folder
 - code: Interface and utility source files
 - include: Header files for the interfaces and the utility functions, and the specifications provided for the interfaces by Arm
 - *RTE_Device.h*: Run-Time Environment device configuration file for all available interfaces and instances. See Section 7.3 "Configuring The Driver Run-time Environment" on page 61 for configuration instructions.

The "Example Application Structure" table (Table 7) shows an example of the structure of an application.

Table 7. Example Application Structure

Component	Include Files	Library	Dependencies
i2c_cmsis (application)	app.h i2c_driver.h (included from the library)	N/A	CMSIS drivers library
CMSIS drivers library	<i>Driver_Common.h Driver_I2C.h Driver_USART.h i2c_driver.h uart_driver.h utility.h</i>	<i>libdrivers.a</i>	System library

Ezairo 8300 Firmware Reference

7.3 CONFIGURING THE DRIVER RUN-TIME ENVIRONMENT

The driver run-time environment can be configured using the *RTE_Device.h* file. A local instance of this file needs to be maintained as part of a local workspace drivers library instance. Every time this file is changed, the drivers library needs to be compiled again to include the latest configuration. The resulting *libdrivers.a* file must also be built into any application project that wants to use this new configuration. The *RTE_Device.h* file uses configuration wizard annotations, allowing the configuration to be completed in a GUI or text-based editor.

After configuring all the settings using either of these methods, build the library project to generate the required *.a* file to be included in the projects that use the CMSIS drivers. The CMSIS sample applications mentioned in Section 7.2 “Library Structure” can be imported for use as reference projects.

7.3.1 Using a GUI-Based Editor

An Eclipse plug-in to edit the configuration wizard annotations is included in the SDK. To configure the CMSIS drivers using the editor with a graphical user interface:

1. Import the drivers library from the *source/Cortex-M3* folder in the installation directory, right-click on *RTE_Device.h*, and choose to **Open With CMSIS Configuration Wizard**.
2. All interfaces are enabled by default. However, each individual instance can be disabled by unchecking the check-box in the **Value** column.
3. After enabling an interface, the adjustable parameters can be configured by typing in the **Value** column or by choosing one of the options from the pull-down menus.
4. After configuring the required interfaces, the file can be saved by choosing **Save** in the **File** menu.

7.3.2 Using a Text-Based Editor

To configure the CMSIS drivers using a text editor:

1. Open the *RTE_Device.h* file in any text editor of preference.
2. Enable the required interfaces by setting the enable (*RTE_interface_ENABLED*) define statements to 1.
3. After enabling the interfaces, configure their parameters by changing the preset values to any values in the range or options specified in the comments preceding the define statements. For example, to change the I2C0 interface's drive strength to 4X, update the *RTE_I2C0_DIO_DRIVE_OPTION* define to 2 as mentioned in the comments that precede it.
4. Save the file after all the parameters have been configured.

7.4 INCLUDING THE CMSIS DRIVERS IN APPLICATIONS

To use the CMSIS drivers library, you have to include it in an application. To do this, follow these steps:

1. In your application, add include statements for each interface's header file (*<interface>_driver.h*), where **interface** is the name of the interface you are using. For example, to include the I²C CMSIS driver, include the *i2c_driver.h* file.
2. After including the header files, open project **Properties**.
3. Under project **Includes**, add the path to the driver's library include folder ("*\${workspace_loc:/drivers/include}*").
4. Under project **Libraries**, add library drivers and its search path ("*\${workspace_loc:/drivers/Release}*").

Ezairo 8300 Firmware Reference

NOTE: To use the above-mentioned search paths, the *drivers* library has to be imported into the workspace containing the application project. The path variable *workspace_loc* also needs to be replaced by the actual workspace path, if the variable does not exist by default in the SDK.

CHAPTER 8

External Drivers

This topic describes the external drivers used with Ezairo 8300 through the eMMC and I³C interfaces.

8.1 THE EMMC EXTERNAL DRIVER

Besides increasing content capacity and bandwidth, the eMMC interface adds faster switching and improved operating system performance by including a command queuing engine, which optimizes the performance of frequent small data transfers and reduces overhead.

~~For information about using the eMMC interface external driver, refer to the *SD Host Firmware Driver User Guide* and the *SD Host Firmware Driver Quick Start Guide*, included with your Ezairo 8300 documentation in C:/Program Files (x86)/ON Semiconductor/Ezairo 8300 SDK/documentation/cadence_3rd_party_docs.~~

8.2 THE I³C EXTERNAL DRIVER

The I³C interface can be implemented to support multiple instances and provide minimal name space pollution when using Ezairo 8300. The I³C-Master controller core driver is implemented as a structure containing a function pointer table. The interface can perform event and interrupt handling, and makes use of dynamic data structures to provide hardware information, modification, and dynamic driver operation.

~~For information about using the I³C interface external driver, refer to the *Cadence I³C Master Controller Interface Firmware Driver User Guide*, included with your Ezairo 8300 documentation.~~

IMPORTANT: The define `SD_REG_BASE` listed on page 2 of the *SD Host Firmware Driver Quick Start Guide* is listed as `0x50000000`, but this is not correct for use with Ezairo 8300. `SD_REG_BASE` needs to be the same as the register define address for `EMMC_HRS00`, which is `0x40001C00`. This address can also be found in the Interfaces chapter of the *Ezairo 8300 Hardware Reference*, in the eMMC Registers section.

IMPORTANT: The define `I3C_REGS_BASE` listed on page 3 of the ~~*Cadence I³C Master Controller Core Driver Quick Start Guide*~~ is listed as `0xFFFFB0000`, but this is not correct for use with Ezairo 8300. `I3C_REGS_BASE` needs to be the same as the register define address for `I3C0_DEV_IND` (`0x40001000` for channel 0), and `I3C1_DEV_IND` (`0x40001800` for channel 0). These addresses can also be found in the Interfaces chapter of the *Ezairo 8300 Hardware Reference*, in the I³C Registers section.

CHAPTER 9

Program ROM

The CFX Program ROM is a ROM mapped into program memory. It contains various elements of firmware code. The primary purpose of the CFX Program ROM is to facilitate booting the system, but it also contains firmware code required for security. In addition, many of the firmware libraries are duplicated in the CFX Program ROM so that ROM-based functions can be used instead of RAM-based versions, reducing the firmware footprint in the RAM and NVM memory spaces.

9.1 CFX PROGRAM ROM

9.1.1 Description

The CFX Program ROM provides access to the ROM instances of the firmware code through function tables and through the debug port's JumpROM facility. For more information about the JumpROM facility and file system security, see the *Communication Protocols Manual for Ezairo 8300*.

9.1.1.1 Versions

The CFX Program ROM version is a 16-bit symbol that can be read from the `SYSVAR_PROGROM_VER` variable. This value can be used directly in the application code to identify the version of the ROM available on your device. An external application or device can also read the ROM version from the `SYSVAR_PROGROM_VER` variable or through the JumpROM command. See [Section 2.1 “Versions” on page 50](#) for more information about version symbols. The "CFX Program ROM Versions" table (Table 8) lists the versions of the CFX Program ROM referenced by this topic.

Table 8. CFX Program ROM Versions

Device	Version
Ezairo 8300 version 1.01.xx	8.22

9.1.2 Boot ROM

9.1.2.1 Boot Initialization

The CFX Program ROM performs basic system initialization required for booting the system. The system initialization consists of the following:

- Setting the CFX status register to disable rounding, saturation, flag updates, and interrupts; enable biasing
- Resetting the CFX loop stack
- Initializing the CFX stack pointer
- Pausing the HEAR Configurable Accelerator, the LPDSP32 DSP, and the Filter Engine
- Setting the Watchdog timeout to maximum (nominally 3.2 seconds)
- Restoring the memory component configuration to the default state
- All DIO pins are left in the default reset state
 - Except for DIO0 to DIO3, which are not in the default state during boot
- Enabling the power supply to analog components (if disabled)
- Enabling other power supply components
- Enabling the interface charge pump and configure it to a high-power default configuration
- Resetting the internal oscillator to its default configuration
- Resetting the DMA channels
- Configuring clocking to select the internal oscillator
- Clearing unprotected RAM areas
- Initializing firmware global variables

Ezairo 8300 Firmware Reference

The system initialization verifies the state of the `BOOT_STATUS:STATUS` and `SYSCTRL_BOOT_STATUS:STATUS` bits. If either bit is set, indicating that the system has already been booted, the CFX Program ROM issues a system-wide watchdog reset.

9.1.2.2 RAM Clearing

To ensure that protected IP components and sensitive data are erased between boots, the CFX Program ROM clears several areas of RAM, as shown in the "CFX vs Arm Cortex-M3 Core Address Ranges" table (Table 9):

Table 9. CFX vs Arm Cortex-M3 Core Address Ranges

Memory Region	CFX Address Range	Arm Cortex-M3 Core Address Range
LPDSP BRAM0 to BRAM2	0x40000-0x40FFF	0x20000000-0x20007FFF
Arm Cortex-M3 DRAM0 to DRAM3	0x42000-0x47FFF	0x20008000-0x2001FFFF
LPDSP32 ARAM5 to ARAM0	0x48000-0x54FFF	0x20020000-0x20053FFF
eMMC Buffer	0x55000-0x551FF	0x20054000-0x200547FF
CFX XMEM ¹	0x04020-0x13BFF	0x20410080-0x2041FC3F
Arm Cortex-M3 PRAM0 to PRAM6	0x10000-0x13FFF	0x00010000-0x0006FFFF
LPSP32 PRAM5 to PRAM0	0x28000-0x2CFFF	0x00070000-0x00083FFF
CFX PRAM0 to PRAM1	0x08000-0x0FFFF	0x20200000-0x203FFFFF
CFX YMEM	0x04000-0x07FFF	0x20C10000-0x20C1BFFF
HEAR PMEM	0x02000-0x03FFF	0x20208000-0x2020FFFF
Filter Engine PMEM	0x01C00-0x01D00	0x20207000-0x202077FF

¹The first 32 and last 1024 words of the CFX XMEM have their contents preserved.

9.1.2.3 Loading the Bootloader

The CFX Program ROM loads the provided bootloader code stored in a non-volatile memory (NVM) device connected to the appropriate DIO pins through the NVMLIB system library. For the full list of supported NVM devices, see [Chapter 10.1 "Introduction" on page 72](#).

The Program ROM supports bootloaders for the CFX and the Arm Cortex-M3 cores.

When loading the bootloader, the Program ROM checks various file system parameters and the bootloader CRC to verify data integrity and decrypts the bootloader if it is encrypted. The Program ROM also loads and decrypts the user application key to be used by the bootloader. Finally, the Program ROM releases control of the system to the loaded bootloader:

- If the bootloader target architecture is the CFX core, execution passes to the bootloader. The Arm Cortex-M3 core is not affected.
- If the bootloader target architecture is the Arm Cortex-M3 core, the CFX issues a `CFX_0` interrupt to the Arm Cortex-M3 core and go into a loop waiting to be configured.

During loading, the following pins are configured as:

- DIO 0 as `SPI_CLK/EMMC_CLK`
- DIO 1 as `SPI_CS/EMMC_CMD`

Ezairo 8300 Firmware Reference

- DIO 2 as SPI_SERO/EMMC_DAT0
- DIO 3 as SPI_SERI/disabled

All the above pins are configured with eight times drive strength, the low-pass filter is disabled, and a 250 kΩ pull-up is enabled. After the loading process is complete, the pins are restored to their reset configuration.

For the required NVM memory layout, refer to [Chapter 10 "NVMLIB Library" on page 72](#).

The Program ROM uses the last 512 words of the CFX PRAM1 (0x0FE00 to 0x0FFFF) as a scratchpad area. The [User Application Key Area](#) is loaded in CFX PRAM1 at address 0x0FF80.

9.1.2.4 Boot Sequence

The Ezairo 8300 boot sequence consists of two steps. In the first step the CFX core loads either a CFX or an Arm Cortex-M3 bootloader. This bootloader in turn can load applications on either or both cores.

If an Arm Cortex-M3 bootloader is loaded, the startup procedure for loading an application on the CFX core is described in [Section 9.1.2.6 "Application Startup" on page 66](#).

The bootloader startup sequence is described in the following section.

9.1.2.5 Bootloader Startup

Upon power-up, the CFX Program ROM verifies the integrity of the master boot record (MBR) by reading the bytes at the NVM addresses 0x000, 0x1FE, and 0x1FF and checking that their values are 0xE9, 0xAA, and 0x55 respectively. If a match is found, the boot ROM calculates the size of the reserved area using information available in the bios parameter block (BPB) structure.

The boot ROM application checks the integrity of manufacturing and calibration data and runs initialization of power supply and clocks. After initialization, the Program ROM loads the bootloader into the program memory space of one of the cores for execution.

The bootloader itself can be optionally encrypted by a shared key. For more details on encryption, see [Section 9.1.2.7 "Security Model" on page 67](#).

The Program ROM application finishes execution at this stage and surrenders control to the bootloader. Before exiting, the Program ROM application clears any keys used for decryption of the bootloader.

The bootloader application calculates the address of the root directory using the extended BPB information. Within the root directory, one or more application files exist, which are optionally loaded into memory by the bootloader.

9.1.2.6 Application Startup

After loading an Arm Cortex-M3 bootloader, the CFX core will wait for a CM3_0 interrupt to be set by the Cortex-M3 bootloader. Upon acknowledging the interrupt, the CFX verifies that the value of 0x4B6579 (PCKEY) has been set at the D_APP_START_RESULT address (IOMEM: 0x805F in the CFX DSP address space or 0x2022017C in the Arm Cortex-M3 core address space). If this verification fails, the CFX core will return to waiting for a CM3_0 interrupt.

If the verification passes, the CFX core verifies if the value present at the D_APP_START_ADDR address (IOMEM: 0x0805E in the CFX DSP address space or 0x20220178 in the Arm Cortex-M3 core address space) is less than D_PRAM0_BASE (IOMEM: 0x08000) or greater than D_CM3_PRAM1_TOP (IOMEM: 0x17FFF) it will fail and go back to waiting for a CM3_0 interrupt.

Ezairo 8300 Firmware Reference

If both PCKEY and a valid address are found, the CFX ROM sets the value of 0x526571 (PCREQ) at the D_APP_START_RESULT address, so the Arm Cortex-M3 bootloader can have a feedback if the CFX accepted the address and value. after setting D_APP_START_RESULT, the CFX core calls the address in D_APP_START_ADDR.

9.1.2.7 Security Model

The Ezairo 8300 security model takes into account that the device package might not physically prevent direct access to the NVM by supporting two different security levels. Level 1 security is recommended when the device package does not physically prevent direct access to the the NVM storage device. In this configuration, an external agent might read the user key data directly. Level 1 security uses a two-stage encryption process to secure the application code and the user key. The Level 2 security model is suitable when the device package physically restricts direct access to the NVM. In this configuration, the user key can be stored in the NVM in plain text.

All security features are optional and can be controlled by flags in the NVM image regardless of the security level adopted. See [Chapter 11.1 "Volume Structure" on page 76](#) for the full details about the NVM image layout.

9.1.2.8 Protection of Assets in the NVM image

User asset protection with Level 1 security is provided by adopting a shared key model and the use of a two-stage encryption process. A boot key is assigned by onsemi and is unique for each customer. The index to the boot key in the CFX ROM key area is set in the BOOT_SEC variable in the boot information area of the NVM image. The boot key is not accessible by user applications. The boot key is used by the CFX ROM to decrypt the bootloader and the Application Key Area, which stores the user key to be used by the bootloader to load and decrypt user applications and other protected user assets.

9.1.2.9 Protection of Assets in RAM

To prevent the debug ports from being used to read the system memory that might contain protected assets that have been decrypted, the debug ports start in restricted mode. To unlock the debug ports, the CFX ROM wipes the system RAM and erases the Application Key Area. Once the CFX ROM verifies that the Application Key Area is empty, it unlocks the device. Once the user key has been wiped, the remaining data cannot be deciphered since the user key is unavailable. In this mode, the system cannot boot properly and the user key must be rewritten to the NVM to boot the NVM application.

9.1.2.10 Boot Failures

A boot failure occurs if the file system does not contain a valid bootloader image. In the case of a failed boot, the Program ROM implements a delay and proceeds to reboot the system. The system error code indicates the particular error. The delay before resetting the system is nominally 3200 ms (minimum 2240 ms, maximum 4480 ms). See [Section 10.2 "Library Usage" on page 72](#) for more information on boot errors, and [Section 2.1 "Versions" on page 50](#) for more information on the variables SYSVAR_SYS_ERRNO (for all boot failures) and SYSVAR_BOOT_LINE (for boot failures within the Program ROM); these variables hold the reported error codes for any errors that result in a boot failure.

For security, the Ezairo 8300 debug port is initially in restricted mode. CFX Program ROM functions are provided to wipe the NVM key area and then unlock the device. However, for convenience, the Program ROM automatically attempts to unlock the device in the special case when the value of the first word of the NVM device does not contain an application signature. In that case, the Program ROM checks the contents of the key area. If the key area has been wiped by writing it with all zeros or all ones, the device is automatically unlocked. This feature allows developers to easily connect to an un-initialized device for development.

Ezairo 8300 Firmware Reference

9.1.2.11 Boot Errors

In the case of a failure during the boot process, the ROM can generate a series of error codes to help debug the problem with said process. The error codes listed in the following table are set to `SYSVAR_SYS_ERRNO` and to `SYSVAR_BOOT_LINE`. The values are shown in the order that the ROM checks them. The "CFX Program ROM Error Codes" table (Table 10) shows the ROM error codes.

Table 10. CFX Program ROM Error Codes

Error Code	Symbol	Description
0	BOOTSTRAP_RET_OK	No error.
1	BOOTSTRAP_RET_ERROR	General error.
4	BOOTSTRAP_RET_NVMLIB_CONFIGURE_ERROR	Error when calling NVMLIB_Interface_Configure during the boot process.
16	BOOTSTRAP_RET_NVMLIB_INITIALIZE_ERROR	Error when initializing the NVM device. Most likely due to a hardware error.
17	BOOTSTRAP_RET_BS_READ_ERROR	Error when reading the Bios Parameter Block (BPB)
19	BOOTSTRAP_RET_BS_OPCODE_ERROR	Bios Parameter Block (BPB) sector 0xE9 opcode not found in BS_jmpBoot[0].
18	BOOTSTRAP_RET_BS_SIGNATURE_ERROR	Boot Sector signature not found.
26	BOOTSTRAP_RET_BOOTINFO_READ_ERROR	Error reading the Boot information from the NVM device.
20	BOOTSTRAP_RET_BOOTINFO_CRC_ERROR	Boot information CRC check failed.
9	BOOTSTRAP_RET_BOOTLOADER_READ_ERROR	Error when reading the Bootloader from the NVM device.
21	BOOTSTRAP_RET_BOOTINFO_BOOT_ADDR_ERROR	BOOT_ADDR value outside the valid address range in Boot information.
7	BOOTSTRAP_RET_BOOT_BTL_ADDR_ERROR	BTL_ADDR value outside the valid address range in Bootloader Meta Contents
10	BOOTSTRAP_RET_BOOTLOADER_CRC_ERROR	Bootloader CRC (BTL_CRC) check failed.
25	BOOTSTRAP_RET_DECRYPT_KEYROM_ERROR	Error decrypting the bootloader data using the Key ROM. CLR_CRC verification failed.
22 - 24	Reserved	–

Boot errors with a value of 16 or greater cause the device to unlock and allow debugging.

9.1.3 Function Tables

The "CFX Program ROM Versions" table (Table 8) describes the functions associated with the CFX program ROM.

Ezairo 8300 Firmware Reference

Table 11. CFX Program ROM Functions

Function Name	Function Number	Description
_PrgROM_PopLR_Return	0	
_PrgROM_SystemInitialize	1	Basic system initialization, as described in Boot Initialization
_BootCode_JumpROM_BootAppEntry	2	Used for JumpROM BootApp
_PrgROM_SystemInitialize_Boot	3	Clears system variables
_PrgROM_RetentionSleep	4	Puts all memories in retention mode at supplied VDDM power supply trimming settings
_Security_WipeSecureRAM	5	Wipes RAM as described in RAM Clearing
_Security_CheckWipeNVMApKey	6	Verifies that the User Application Key Area is erased in the NVM memory.
_Security_WipeNVMApKey	7	Erases the User Application Key Area in the NVM memory.
PushLoopsToStack	8	Push all the hardware loops to the stack.
PopLoopsFromStack	9	Pop all the hardware loops from the stack
AESInitialize	10	Initializes the AES encryption code.
AESExpandKey	11	Expand key that has already been loaded into memory.
AESDecrypt	12	Decrypts in place an AES-encrypted buffer in ECB mode.
AESCleanUp	13	Clear out all the key data (including subkeys) used by AES.
SYSLIB_Reset	14	Reset the system with a watchdog timeout.
SYSLIB_Delay	15	Delay the CFX core for the specified period of time.
SYSLIB_DelayBase	16	Delay the core for the specified amount of time.
NVMLIB_GlobalWriteEnable	17	Write enable the NVM library.
NVMLIB_GlobalWriteDisable	18	Clear the library write enable.
NVMLIB_DeviceWriteEnable	19	Write enable the NVM device.
NVMLIB_DeviceWriteDisable	20	Clear the write enable.
NVMLIB_StatusRegister_Read	21	Reads the Status Register.
NVMLIB_StatusRegister_Write	22	Writes to the Status Register.
NVMLIB_EnableWrite	23	Sets or resets the WEL bit in the Status Register.
NVMLIB_Initialize	24	Initializes an NVM device.
NVMLIB_Uninitialize	25	Implements ARM_Storage_Uninitialize : De-initialize the Storage Interface.

Ezairo 8300 Firmware Reference

Table 11. CFX Program ROM Functions (Continued)

Function Name	Function Number	Description
NVMLIB_PowerControl	26	Implements ARM_Storage_PowerControl : Control the Storage interface power.
NVMLIB_ReadData	27	Implements ARM_Storage_ReadData : Read data from Storage.
NVMLIB_ProgramData	28	Implements ARM_Storage_ProgramData : Program data to Storage.
NVMLIB_Erase	29	Implements ARM_Storage_Erase : Erase Storage range.
NVMLIB_EraseAll	30	Implements ARM_Storage_EraseAll : Erase complete Storage.
NVMLIB_GetBlock	31	Implements ARM_Storage_GetBlock : Find the storage block (iterator) encompassing a given storage address.
NVMLIB_GetNextBlock	32	Implements ARM_Storage_GetNextBlock : Advance to the successor of the current block (iterator).
NVMLIB_GetInfo	33	Implements ARM_Storage_GetInfo : Get Storage information.
NVMLIB_Command	34	Sends an arbitrary command to the underlying device.
NVMLIB_Interface_Configure	35	Configures the hardware interface pins and prescaler.

9.2 ARM CORTEX-M3 ROM

9.2.1 Description

The Arm Cortex-M3 core Program ROM is a ROM mapped into program memory.

9.2.1.1 Versions

The Arm Cortex-M3 ROM version is a 16-bit symbol present at the base of the `romjumptable` code section. (See the "Arm Cortex-M3 ROM Versions" table (Table 12).) This value can be used directly in the application code to identify the version of the ROM available on your device.

Table 12. Arm Cortex-M3 ROM Versions

Device	Version
Ezairo 8300 version 1.01.xx	1.2.1

9.2.2 Boot ROM

9.2.2.1 Boot Initialization

The Arm Cortex-M3 ROM performs basic system initialization required for booting the system. The system initialization consists of the following:

- Configures the NVIC to only allow NMI and hard fault exceptions
- Clears the error pointer
- Clears the system handler control and state register
- Waits for CFX_0 interrupt

Ezairo 8300 Firmware Reference

Upon receiving a CFX_0 interrupt, the Arm Cortex-M3 core verifies that the Arm Cortex-M3 start address (see the BTL_ADDR value in [Section 11.2.4.1 “Bootloader Meta Contents” on page 82](#)) present in the D_APP_START_ADDR address is aligned to a 512-byte boundary. The core also verifies that the application’s stack pointer is aligned to an 8 byte boundary. If this basic verification passes, the Arm Cortex-M3 ROM sets the SYSCTRL_BOOT_STATUS:STATUS bit to indicate booting of an application. The control then passes to the loaded application.

9.2.2.2 Boot Failures

If the basic verification described in [Section 9.2.2.1 “Boot Initialization”](#) fails, the Arm Cortex-M3 ROM sets the ERROR_POINTER memory location with a numeric code indicating which failure occurred. ERROR_POINTER is located at IOMEM:0x805F in the CFX DSP address space, or 0x2022017C in the Arm Cortex-M3 address space.

Failure	Value
Application alignment error	0x03
Stack alignment error	0x04

9.2.3 Function Tables

Function Name	Function Number	Notes
version	0	Not a function, the value of the Arm Cortex-M3 ROM version
__ProgramROM_RetentionStandby	1	Retention standby function

CHAPTER 10

NVMLIB Library

The Non-Volatile Memory Library (NVMLIB) is a software library that provides high-level functions to interact with non-volatile SPI (EEPROM, NVM or flash) or eMMC memories. The library provides a coherent API that abstracts the specifics of the underlying device as much as possible.

10.1 INTRODUCTION

The NVMLIB supports any memory device compatible with the eMMC interface standard, as well as most 24-bit addressable SPI memory devices, including:

- [onsemi CAT25M01 SPI EEPROM](#)
- [onsemi EA2M Audiology SPI EEPROM](#)
- [onsemi LE25S161 SPI Flash](#)
- [Microchip 25AA1024 SPI EEPROM](#)
- [Winbond W25Q16DW](#)
- [Fujitsu MB85AS8MT ReRam](#)

In addition to the devices listed above, any other SPI EEPROM or Flash device that supports the commands shown in the "General SPI Commands Supported by the NVM Library" table (Table 13) is also supported:

Table 13. General SPI Commands Supported by the NVM Library

Command	Code	Description
RDSR	0x05	Read the status register
WRSR	0x01	Write the status register
WREN	0x06	Enable write (set the <code>WEL</code> bit)
WRDI	0x04	Disable write (clear the <code>WEL</code> bit)
READ	0x03	Read data
WRITE	0x02	Write data

All library functions are blocking and interruptible. The library does not use any interrupts or DMA channels.

For the complete API, refer to [Chapter 21: Non-Volatile Memory \(NVM\) Library Reference](#)

10.2 LIBRARY USAGE

The NVM library is available in two forms:

1. As a static library under the SDK folder. It is available for the CFX core and the Arm Cortex-M3 core
 - a. The CFX core version is available in the *C:\Program Files (x86)\ON Semiconductor\Ezairo 8300 SDK\lib\cfx* folder.
 - b. The Arm Cortex-M3 core version is available in the *Ezairo 8300 SDK\lib\cm3* folder.
2. As a [CFX Program ROM](#) component, implemented for the CFX core.

Applications that need to make use of the static library should include the `nvmlib.h` header file and be linked against the `libnvmlib.a` file. Applications that need to link against the ROM version should include the `nvmlib_rom.h` header file and should not be linked against the static library.

10.2.1 Configuration

The first step to use the NVMLIB is to configure it through the `NVMLIB_Interface_Configure` function.

Ezairo 8300 Firmware Reference

An example of the NVMLIB_Interface_Configure used by the Program ROM code is

```
NVMLIB_Interface_Configure(&bootstrap_nvmlib_device,
                           (void*) SPI0,
                           SPI_PRESCALE_8,
                           3840U,
                           (DIO_8X_DRIVE | DIO_LPF_DISABLE | DIO_250K_PULL_UP),
                           0, 1, 2, 3);
```

10.2.2 Initialization

After the NVMLIB has been configured, it needs to be initialized. The functions to configure and initialize the library are separate as the user only has to configure the library once, whereas the library can be initialized more than once. The initialization is done through the NVMLIB_Initialize function.

The library initialization function initializes all the hardware resources used by the library. If initialization fails, the hardware resources are automatically released.

10.2.2.1 Detection

When using autodetection, the return code of ARM_DRIVER_OK (0) shows whether it has been successful or not. The dev_id member of NVMLIB_DEVICE is set according to the type of device that has been detected. For LE25S101 and 25AA1024 devices, the library is able to verify that a particular device is present. For other devices, as long as the RDSR command reads a value that is not 0xFF, the library is able to detect the device as a generic device.

Device	dev_id Value
onsemi CAT25M01 SPI EEPROM	NVMLIB_STORAGE_GENERIC
onsemi EA2M Audiology SPI EEPROM	NVMLIB_STORAGE_GENERIC
onsemi LE25S161 SPI Flash	NVMLIB_STORAGE_LE25S161
Microchip 25AA1024 SPI EEPROM	NVMLIB_STORAGE_25AA1024
Winbond W25Q16DW	NVMLIB_STORAGE_GENERIC
Fujitsu MB85AS8MT ReRam	NVMLIB_STORAGE_GENERIC
eMMC	NVMLIB_STORAGE_EMMC
Other	NVMLIB_STORAGE_GENERIC
Failed	NVMLIB_STORAGE_UNKNOWN

10.2.3 Uninitialization

To release the hardware resources and clear all internal structures, the library provides the NVMLIB_Uninitialize function.

The NVMLIB_Initialize function can be called to put the library into a initialized state if required.

Ezairo 8300 Firmware Reference

10.2.4 eMMC Operations

As addressing changes from bytes to blocks when eMMC devices are being handled, the `dev_id` member of `NVMLIB_DEVICE` can be used to indicate whether byte or block addressing needs to be used. The following code snippet shows an example of how to write 512 bytes of data to address `0x200`:

```
if ( nvmlib_device.dev_id == NVMLIB_STORAGE_EMMC )
{
    /* eMMC device */
    NVMLIB_ProgramData(&nvmlib_device, 0x200>>9, data_buf, 512);
}
else
{
    /* SPI device */
    NVMLIB_ProgramData(&nvmlib_device, 0x200, data_buf, 512);
}
```

10.2.5 Unlocking Operations

To write or erase a device, both the NVM library and the device must first be unlocked. Unlocking the library is achieved through the `NVMLIB_GlobalWriteEnable` function. The next step is to individually unlock the devices using the `NVMLIB_DeviceWriteEnable` function.

It is recommended that as soon as the write/erase operations complete, the device is locked again to avoid accidental data loss. Locking a device is accomplished through the `NVMLIB_DeviceWriteDisable` function. To lock the library, use the `NVMLIB_GlobalWriteDisable` function.

10.2.6 Block Interface and Device Information

Information about the underlying NVM device can be obtained through the `NVMLIB_GetInfo`, `GetBlock`, and `GetNextBlock` functions.

10.2.6.1 Device Information

The `NVMLIB_GetInfo` function allows the user to discover information about the underlying NVM device. The `ARM_STORAGE_INFO` structure provides:

- How much storage data a device can hold
- Which is the minimal programming unit
- How many erase cycles the device endures
- The type of retention the device provides

10.2.6.2 Block Information

The block information is normally used when erasing a device. The erase start address must be aligned to an erase unit. As this changes from device to device, the information can be obtained using the `NVMLIB_GetBlock` function.

Ezairo 8300 Firmware Reference

The `addr` argument can be any address in the device's address space. The `ARM_STORAGE_BLOCK.addr` member is set with the start address of the block that holds the `addr` address. For instance, if using a 4096-byte block, if `addr` is less than 4095, `ARM_STORAGE_BLOCK.addr` is set to 0x00. If `addr` is 0x1001, `ARM_STORAGE_BLOCK.addr` is set to 0x1000, and so on.

The `GetNextBlock` function provides a way to iterate through the consecutive blocks that make up the device.

The `next_block.addr` member wraps around after the address reaches the end of the device. The following code snippet shows the use of the `NVMLIB_GetNextBlock` erase the whole device:

```
ARM_STORAGE_BLOCK prev_block, next_block;

/* Get the first block */
NVMLIB_GetBlock(&nvmllib_device, 0x00, &prev_block);

do
{
    /* Erase the block */
    NVMLIB_Erase(&nvmllib_device, prev_block.addr,
                prev_block.attributes.erase_unit);

    /* Get the block after the one we just erased */
    NVMLIB_GetNextBlock(&nvmllib_device, &prev_block, &next_block);

    prev_block = next_block;
} while( next_block.addr != 0 );
```

CHAPTER 11

NVM Memory Layout

The non-volatile memory (NVM) file system supports storage of:

- Specific calibration settings, manufacturing information, and system boot information
- Bootloaders for the cores on the Ezairo 8300 SOC
- One or more applications

A FAT implementation is provided for low latency and efficient storage of data. This implementation supports FAT12 and FAT16 file system storage, with the FAT format being defined by the storage needs of the NVM used in the system. (FAT12 is used for memories 8 MB and smaller; FAT16 is used for memories larger than 8 MB but less than 2 GB; and FAT32 is used for memories larger than 2 GB.)

IMPORTANT: Regardless of the FAT format selected, the signature stored to 0x1FE of the bios parameter block (BPB) is required.

This type of implementation allows a memory device to be easily mounted as a file system on modern operating systems.

Application code and data can be stored securely using AES encryption. See ["CFX AES Library Reference" on page 344](#) for more details.

11.1 VOLUME STRUCTURE

A FAT file system volume is composed of four basic regions, which are laid out in this order on the volume:

1. Reserved Area
2. FAT Area
3. Root Directory
4. Data Area or File and Directory Data Region

An overview of the file system is shown in the ["File System Volume Overview Diagram" figure \(Figure 1\)](#):

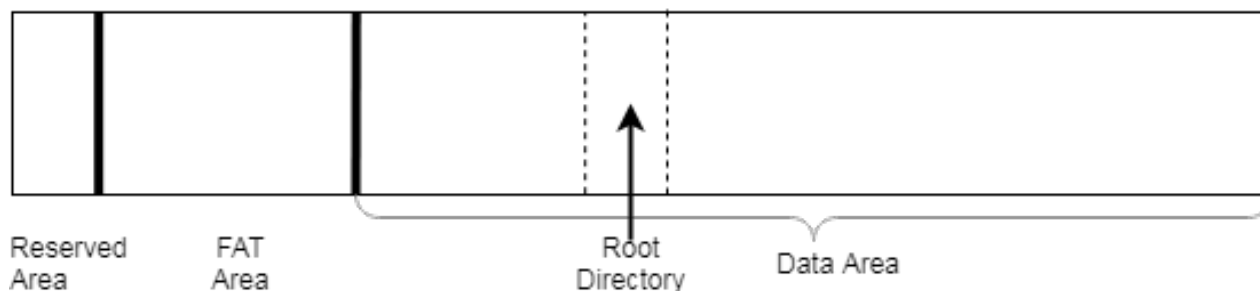


Figure 1. File System Volume Overview Diagram

All the data structures listed below are used by the ROM during the boot process. The NVM layout has been designed to facilitate the use of the FAT filesystem. If the customer does not plan on using a FAT file system, all the NVM area after the last reserved sector can be used by a custom file system with no restrictions.

Ezairo 8300 Firmware Reference

11.1.1 FatFs Library

FatFs is a filesystem library distributed with the Ezairo 8300 SDK. FatFs supports sector sizes up to 4096 bytes, and is significantly economical in its memory use when running such applications as the bootloader.

In previous versions of the SDK, the EvFat library was used for filesystem operations. EvFat only supports a 512-byte FAT sector size, while FatFs supports sector sizes of 512, 1024, 2048 and 4096 bytes. NVMI now uses the physical erase sector size of the NVM attached as the FAT sector size. This prevents filesystem corruption on unlock, improves efficiency of filesystem operations, and reduces code size.

NOTE: When using an NVM that has an erase sector size other than 512, 1024, 2048 or 4096 bytes, NVMI sets the FAT sector size to 512 bytes. The FAT filesystem specification allows only the use of the above sector sizes. Changes to *diskio.c* and *nvmctrl.c* are needed to support a non-standard erase sector size, such as backing up and updating only a 512-byte section of a larger erase sector.

When using a device with one-byte erase pages, such as an EEPROM, set the erase page setting in NVMI to 512 bytes to use a 512-byte FAT sector. Other sizes can also be used, but the sector size must be set correctly in *nvmctrl.c* (the default is 512 bytes for EEPROM devices).

For more information on the FatFs library, see http://elm-chan.org/fsw/ff/00index_e.html.

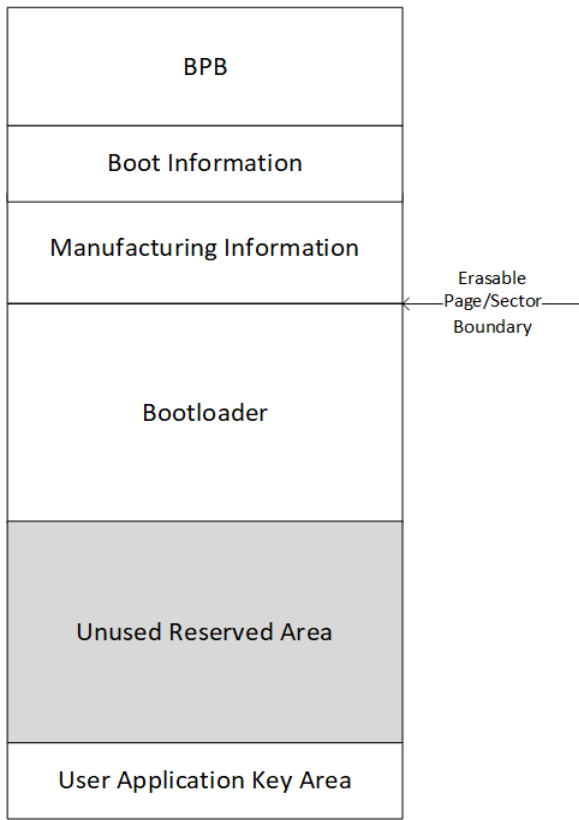
11.2 RESERVED AREA

The reserved area includes the following five regions, starting with the FAT defined bios parameter block (BPB) and followed by four special reserved areas:

1. Boot Information
2. Manufacturing information
3. Bootloader info and bootloader
4. User Application Key Area

See the "Reserved Areas" figure (Figure 2).

Ezairo 8300 Firmware Reference

**Figure 2. Reserved Areas****11.2.1 Bios Parameter Block (BPB)**

The Bios Parameter Block (BPB) is stored at the base of the NVM, and contains a 512-byte block of information that is defined by the FAT standards. Key items that must be stored to this pre-defined block are described in the "BPB Information Block" table (Table 14); all other fields are optional, and are unused by the boot ROM.

Table 14. BPB Information Block

Byte Offset	Size	Variable	Values
0 (0x00)	3	BS_jmpBoot	Byte 0: 0xE9, Bytes (2:1): Address of bootloader; this <i>must be</i> block-aligned if the NVM device is MMC. BS_jmpBoot[0] is used by the ROM to verify the BPB integrity, BS_jmpBoot[1,2] are used by the ROM during the boot process.
3 (0x03)	8	BS_OEMName	"onsemi" (this is 8 bytes long, including the trailing spaces.)
11 (0x0B)	2	BPB_BytsPerSec	--erasePage option value in NVMI (NVM project erase page size). 512, 1024, 2048 or 4096 depending on the physical sector erase size of the NVM attached. 512 for devices not requiring erase before write, such as EEPROM devices. Used by the ROM to calculate the NVM address of the User Application Key area for SPI devices.
13 (0x0D)	1	BPB_SecPerClus	Number of sectors per allocation unit.

Ezairo 8300 Firmware Reference

Table 14. BPB Information Block (Continued)

Byte Offset	Size	Variable	Values
14 (0x0E)	2	BPB_RsvdSecCnt	3+; this must reserve the following sectors: 1 sector for the BPB (this table), 1+ sectors for the Boot Information, Manufacturing Information, and Boot-loader, 0+ sectors to ensure erasing the User Application Key Area does not erase the boot or manufacturing information, 1 sector for the User Application Key Area (assumed to be held in the last reserved section). Used by the ROM to calculate the NVM address of the User Application Key area.
16 (0x10)	1	BPB_NumFATs	2
21 (0x15)	1	BPB_Media	0xF8 (non-removable media)
40 (0x28)	2	BPB_ExtFlags	0x0000 (data is mirrored into all FATs)
510 (0x1FE)	2	-	0x55AA; this data is the required signature that validates this as a bootable volume. The boot ROM verifies that this signature is present before continuing the boot process.

11.2.2 Boot Information

The boot-information block provides additional information about the underlying memory, file system, and communication interface. This information starts immediately following the BPB at byte 512 in memory.

The structure of the boot information block is shown in the ["Boot Information Variables" table \(Table 15\)](#)

Table 15. Boot Information Variables

Byte Offset	Size	Variable	Description
0 (0x0)	2	BOOT_FS_VER	NVM File System version
2 (0x2)	10	BOOT_APP_LOAD	Name of application to load from root directory. Set to "APP000" by default. Used by the bootloader. Application is loaded by the bootloader. Maximum length of string: 8 characters. Last two bytes padded with zero.
12 (0xC)	4	BOOT_APP_EXT	Application extension name. Set to "fez" by default. Example APP000.fez. Used by the bootloader. Maximum length of string 3 - characters. Last byte padded with zero.
16 (0x10)	1	BOOT_MEDIA_ID	ID of the boot media: Bits [3:0] Reserved Bits [7:4] Data bus width 0: Automatic width selection 1-8: Width of the data bus

Ezairo 8300 Firmware Reference

Table 15. Boot Information Variables (Continued)

Byte Offset	Size	Variable	Description
17 (0x11)	1	BOOT_SPEED	Boot speed for reading of boot media. Selected SPI mode is used to read from the NVM when booting. 0 : Automatic 1 : Single SPI 2 : Dual SPI 3 : Quad SPI 4-255 : Reserved
18 (0x12)	2	BOOT_SEC	Used by the ROM during the boot process. • Bit [15] 1 : Bootloader encrypted • 0 : Bootloader un-encrypted • Bits [6:0] Key ROM key index. Must match the key index issued by onsemi. • Bit [7] Preferred key. Used for encrypted images. • 0 : Use OTP key • 1 : Use Key ROM
20 (0x14)	4	BOOT_ADDR	Absolute address in PRAM (as addressed on the CFX core) where the booted image will be loaded. For a CFX bootloader, $0x8000 \leq \text{BOOT_ADDR} \leq 0xFFFF$. For a Cortex-M3 bootloader, $0x10000 \leq \text{BOOT_ADDR} \leq 0x27FFF$.
24 (0x18)	2	BOOT_MANU_OFF	Byte offset from the beginning of reserved area for Manufacturing Information
26 (0x1A)	2	BOOT_MANU_LEN	Length of the manufacturing area information in words
28 (0x1C)	2	<i>Reserved</i>	
30 (0x1E)	2	BOOT_INFO_CRC	CRC-CCITT of boot information area. Used by the ROM to verify the Boot Information area integrity.

11.2.3 Manufacturing Information

The manufacturing information area is a section within the reserved area of the NVM FAT file system that contains calibration records for the Ezairo 8300 device. The manufacturing area is defined by the boot information record variable `BOOT_MANU_OFF`, which defines the byte offset for the manufacturing area from the base of the reserved area (which is also the offset from the base of NVM memory), and the `BOOT_MANU_LEN` variable, which defines the length of the manufacturing information area in words.

The structure of the manufacturing information table is described in the "Reserved Area" table (11.2).

Table 16. Manufacturing Information

Offset	Variable Element	Elements	Size per Element	Description
0x00	MANU_VDDIF	4	2	Voltage trim settings for VDDIF
0x08	MANU_VDDA	4	2	Voltage trim settings for VDDA
0x10	MANU_VDDA_PMURef	4	2	Voltage trim settings for VDDA when using the PMU reference

Ezairo 8300 Firmware Reference

Table 16. Manufacturing Information (Continued)

Offset	Variable Element	Elements	Size per Element	Description
0x18	MANU_VDDC	8	2	Voltage trim settings for VDDC
0x28	MANU_VDDC_PMURef	8	2	Voltage trim settings for VDDC when using the PMU reference
0x38	MANU_VDDC_CP	2	2	Charge pump trim settings for VDDC
0x3C	MANU_VDDM	4	2	Voltage trim settings for VDDM
0x44	MANU_VDDM_PMURef	4	2	Voltage trim settings for VDDM when using the PMU reference
0x4C	MANU_VDDM_CP	2	2	Charge pump trim settings for VDDM
0x50	MANU_VDDOD	2	2	Voltage trim settings for VDDOD
0x54	MANU_VMIC	2	2	Voltage trim settings for VMIC
0x58	MANU_VREG	2	2	Voltage trim settings for VREG (only 1 setting is used)
0x5C	MANU_CLK	10	4	Frequency trim settings for the internal CCO
0x84	MANU_LSAD_COMP	1	4	Gain and offset trim settings for the LSAD
0x88	MANU_ADC_TRIM0	4, 4	2	ADC offset trim values for drive 2, 4
0x98	MANU_ADC_TRIM1	4, 4	2	ADC offset trim values for drive 8, 16
0xA8	MANU_ADC_TRIM2	4, 4	2	ADC offset trim values for drive 32, 64
	<i>Reserved</i>			<i>Reserved for future use</i>
0xFE	MANU_CRC_CCITT	1	2	CRC calculated over the rest of the manufacturing information area

The power supply calibration elements stored to this table are formatted as a packed data structure containing:

- The trim setting for the relevant trim register (byte 1)
- The target voltage specified in 10s of mV (byte 0)

The clock calibration elements stored to this table similarly are formatted as a packed data structure containing:

- The trim setting for the relevant trim register (bytes 3:1)
- The target voltage specified using the SYS_FREQ_* definitions (byte 0)

The LSAD element is stored to this table as a packed data structure containing:

- Trim setting for the offset (byte 1)
- Trim setting for the gain (byte 0)

An ADC offset trim is calculated for each of the four ADCs at each of the specified six drive levels. The array of data stored at MANU_ADC_TRIM* is indexed by the index of the ADC, with trims for two different drives packed into each MANU_ADC_TRIM* set.

Ezairo 8300 Firmware Reference

IMPORTANT: If the manufacturing information area is empty with 0s or 0xFFs, the CRC value is also set to the same 0s or 0xFFs. This invalidates the empty manufacturing information.

11.2.4 Bootloader

The bootloader is a special code section stored in the reserved area of the NVM FAT file system. The bootloader code is automatically loaded from NVM to RAM on reset by the boot ROM. The bootloader address in reserved memory is defined by the `BS_jumpBoot` field of the BPB. For more information on the BPB, see [Section 11.2.1 “Bios Parameter Block \(BPB\)”](#).

The bootloader meta section provides information, such as the size of the bootloader and the processor ID, which executes the bootloader itself.

NOTE: Ideally the bootloader section is placed in a separate erasable page/sector from the BPS, boot information, and manufacturing information to allow separate upgrading of the bootloader from the device specific information.

11.2.4.1 Bootloader Meta Contents

Information for the bootloader is found in the ["Bootloader Meta Contents Layout" table \(Table 17\)](#).

Table 17. Bootloader Meta Contents Layout

Byte Offset	Size	Variable	Description
0 (0x0)	2	BTL_SIZE	Size of bootloader in words (32 bits). Should include the unencrypted CRC.
2 (0x2)	1	BTL_DEST	Processor ID number 0 - CFX (Default) 1 - CM3
3 (0x3)	1	Reserved	
4 (0x4)	4	BTL_ADDR	PRAM address in the specified processor's program memory space where execution starts (start address for CFX DSP, base of vector table for Arm Cortex-M3 processor). For a CFX bootloader, $0x8000 \leq \text{BOOT_ADDR} \leq 0xFFFF$. For an Arm Cortex-M3 core boot loader, $0x10000 \leq \text{BOOT_ADDR} \leq 0x6FFFF$.
8 (0x8)	BTL_SIZE-2	data	Bootloader data.
6 + 4*BTL_SIZE	2	CLR_CRC	CRC-CCITT of the bootloader unencrypted bootloader data. This CRC is used to verify that decryption was successful. Ignored by the ROM if the bootloader is not encrypted.
8 + 4*BTL_SIZE	2	BTL_CRC	CRC-CCITT of the bootloader. This CRC is used by the ROM to verify the integrity of the data loaded from the NVM device.

11.2.5 User Application Key Area

The User Application Key Area stores a CRC protected key used to encrypt the information in an application file. If the bootloader is encrypted, the same key must be used to decrypt the user key. The boot ROM can clear the key area by setting the key values to all zeros or all ones. In this case, the application cannot be decoded from the external media.

Ezairo 8300 Firmware Reference

The User Application Key Area is stored to the last sector of the reserved area, as defined by the BPB present in the NVM device. If the BPB is considered invalid, the User Application Key Area is assumed to be located at an NVM address of 0x10000 for SPI devices and in sector 128 for eMMC devices. For more information on the BPB, see [Section 11.2.1 “Bios Parameter Block \(BPB\)”](#).

IMPORTANT: The User Application Key Area is typically aligned to the last sector of the reserved area, and buffered by additional sectors if necessary, in order to ensure that the BPB, boot information, and manufacturing area are not erased if the application key is erased. This area may also be placed so that the bootloader is not erased if the application key is erased.

The structure of the tables in the user key area can be seen in the "User Application Key Area Layout" table ([Table 18](#)).

Table 18. User Application Key Area Layout

Byte Offset	Size	Variable	Description
0 (0x0)	16	KEY_DATA	User AES-128 Key
16 (0x10)	2	CRC-CCITT	CRC of Application Key Area

Please refer to [Section 9.1.2.7 “Security Model”](#) on [page 67](#) for information on the security model.

CHAPTER 12

CFX AES Library

The CFX AES library provides a set of functions to decrypt data blocks encrypted with the AES encryption standard (NIST FIPS 197) in ECB mode. The library does not provide a function to encrypt data blocks.

The library is implemented for the CFX core and is available as a [CFX Program ROM](#) component. The CFX AES library is not provided as source code or as pre-compiled object code. All library functions are blocking and interruptible. The library does not use any interrupts or DMA channels.

12.1 USAGE

To use the AES library, include the `aeslib_rom.h` header file, which uses the Program ROM linked version of this library.

Prior to using the library to decrypt data, expand the AES key by calling the [AESExpandKey](#) function. Once the key has been expanded, the expanded key can be used to decrypt as many blocks as needed using the [AESDecrypt](#) function. Once all necessary blocks are decrypted, the key can be protected by the `AESCleanUp` function.

For the complete API, refer to "[CFX AES Library Reference](#)" on page 344

CHAPTER 13

Calibration Library

The calibration library provides a set of functions and symbols to calibrate various power blocks, the internal current-controlled oscillator, and the ADC supply voltage offset. The library also provides NVM support functions to load the manufacturing information table into a supported NVM device, and to restore a table from NVM back into RAM.

The library is implemented for both the CFX and the Arm Cortex-M3 cores, and is distributed in source-code and pre-compiled form in the SDK. All library functions are blocking and interruptible. The library does not use any interrupts or DMA channels.

13.1 USAGE

To use the calibration library functions, include *calibrate.h* in the application source and the application link against the *libcalibratelib.a* library object. After the initial hardware setup and after calling the `Calibrate_Power_VREG_And_LSAD` function, the rest of the calibration functions can be called in any order.

NOTE: The voltage reference selected for VDDA, VDDC, and VDDM remain configured to match the voltage reference required for the calibration performed, as the power supply trim is only valid for the selected reference. As these voltage references are not reverted back to the original setting after calibration, the application should manually set the power supplies to the default level before changing the reference between the PMU and bandgap supply references, and setting the trim appropriately for the desired level relative to the selected supply reference.

To use the NVM support functions, initialize the NVM device and the NVMLIB library by calling the appropriate functions. Refer to [Chapter 21 "Non-Volatile Memory \(NVM\) Library Reference" on page 295](#) for more information.

For the complete calibratelib API, refer to [Chapter 23 "Supplemental Calibration Library Reference" on page 349](#).

13.2 CALIBRATION REFERENCES AND DEPENDENCIES

IMPORTANT: The calibration library assumes that the clock distribution (including in particular the SLOWCLK configuration) is configured correctly for the current SYSCLK frequency when calibrating power supplies. This is required to ensure charge pump clocks and measurements delays are reasonably accurate.

Before calibrating any other blocks, the LSAD and VREG reference must be calibrated. To calibrate these blocks, call the `Calibrate_Power_VREG_And_LSAD` function. This function calibrates against the difference between VBAT and VSS, and we recommend supplying an accurate voltage reference of at least 1.5 V at VBAT for this purpose.

NOTE: The system can be calibrated with a lower VBAT reference supply, down to 1.0 V - but the accuracy of the calibration is reduced, and does not guarantee trim accuracy specified in the datasheet.

Once the LSAD and VREG are calibrated, all of the power supplies can be calibrated. Once VDDA has been calibrated, the internal oscillator and ADC supply voltage can also be calibrated.

To calibrate the internal oscillator (CCO), provide a 128 Hz square wave on a selected DIO for reference. This signal must use an appropriate digital signal level for the VDDO used to supply the specified DIO.

ADC supply voltage offset calibration does not require an external reference.

CHAPTER 14

Arm Cortex-M3 Bootloader

The bootloader sample is loaded by the [CFX Program ROM](#), and its function is to load application files. Some of the use cases supported are:

- Loading and running an application file
- Loading one or multiple data files and an application file
- Loading an application file for each core

The bootloader supports encryption using the key stored in the [User Application Key Area](#). All elements can be individually encrypted. The bootloader file format also offers data verification for all elements using CCITT CRC. Encryption and data verification together provide data authentication.

14.1 BOOTLOADER API

The user has three basic functions to choose from:

- [bootloader_main](#)
 - Reads the application file specified in the BOOT_APP_LOAD and BOOT_APP_EXT fields from [Boot Information](#). Does not return.
- [bootloader_boot](#)
 - Reads the application file specified in the BOOT_APP_LOAD and BOOT_APP_EXT fields from [Boot Information](#). It might return if the Cortex-M3 core is not booted.
- [bootloader_load_app_file](#)
 - Reads the application file specified in the argument. It might return if the Cortex-M3 core is not booted.

The bootloader supports two file types: application files and manifest files. Application files contain data and boot instructions, while manifest files have a list of application files to be loaded.

14.2 APPLICATION FILES

The most common output format for the Ezairo 8300 SDK toolchains is the ELF format. This format, however, does not provide data verification or security facilities and has only one entry point. The Ezairo 8300 application files are designed to provide the necessary functionality and flexibility while offering a straightforward mapping between ELF files and application files.

Application files contain binary data and instructions on how to boot the cores, and are composed of a file header and zero or more data sections. Data sections can be created directly from program headers in ELF files.

The bootloader starts the loading process of an application file, reading the file header and verifying it. If the verification fails, the bootloader stops the loading process. There is no rollback mechanism provided to clear any memory regions that have been loaded. Keep in mind that the ROM clears the memory when the device is reset.

14.2.1 Data Sections in Application Files

The data sections are loaded in the same order as they appear in the application file. After all the data sections have been loaded, the CFX core is booted, and then the Arm Cortex-M3 core is booted.

The CRC information for each data section is stored in the file header. If the file header is encrypted and a data section is tampered with, the CRC verification for that data section will fail. It is recommended that the file header is always encrypted, even if the following data sections are not.

Several data section types are supported:

Ezairo 8300 Firmware Reference

Data Section Type	Description	Memory Region Range
SEC_CFX_X	Data destination is CFX core X memory	CFX_XMEM_LSB_UNSIGNED_BASE:HEAR_BD1MEM_LSB_UNSIGNED_TOP
SEC_CFX_Y	Data destination is CFX core Y memory	CFX_YRAM0_LSB_UNSIGNED_BASE : CFX_YRAM2_LSB_UNSIGNED_TOP
SEC_CFX_P	Data destination is CFX core P memory	CFX_PRAM_BASE : CFX_PRAM_TOP
SEC_CM3_P	Data destination is Arm Cortex-M3 core P memory	PRAM_BASE : PRAM_TOP
SEC_IOMEM	Data destination is IO memory	No verification is performed; full support within the address spaces is provided for all IOMEM-mapped memory instances.
SEC_BOOT_CFX	CFX core boot section	-
SEC_BOOT_CM3	Arm Cortex-M3 core boot section	-

NOTE: Filter Engine and LPDSP32 data is loaded as SEC_IOMEM data sections.

The data sections in an application file can be individually encrypted. Encryption is applied to both the section header and the section data. For more information on memory locations for data sections, see [Section 2.2 “Reserved Memory” on page 50](#).

14.2.1.1 Boot Sections in Application Files

The boot sections are a special case of the data sections in that their data layout is fixed. The data layout for boot sections is explained in [Section 14.5.2.1 “Boot Section Layout” on page 91](#). Boot sections provide entry points for booting the cores present in Ezairo 8300.

14.2.1.2 Volatile Sections

A volatile section will have its CRC verified, but if the verification fails the loading process will not be stopped.

14.3 INITIALIZATION OF GLOBAL VARIABLES

The different program headers from the ELF file that have been generated by the build tools are converted into data sections in the application files. This includes the `.data` section of the ELF file, which holds the initialization data for the global variables.

The default startup routine for both cores does not explicitly initialize the `.data` section in memory. This is done by the bootloader when it loads the section in the application file that corresponds to a `.data` section in an ELF file.

If the initialization needs to be performed again, the `.data` section can be moved to a separate application file that contains only this section. The entire application can be loaded using a manifest file.

The Arm Cortex-M3 core bootloader uses the DRAM2 and DRAM3 sections as its working memory. Arm Cortex-M3 core applications being loaded must not place any data (any member of the `.data` section) at those memory locations. If an application has initialized data at those locations, the bootloader overwrites its local variables, leading to unpredictable behavior. The memory usage restriction does not apply to uninitialized global data (`.bss`), as this section is initialized by each application's `_start` function. This restriction also does not apply to placing the stack in DRAM2 or DRAM3.

Ezairo 8300 Firmware Reference

The regular flash image builder tool behavior is to create a data section for each program header that the tool finds in the input ELF files. The restriction explained above is enforced by ignoring the program headers that point to `.bss` sections in Arm ELF files. To prevent the bootloader memory areas or application memory areas being overwritten by other applications, the flash image builder tool raises an error if it detects a memory collision, and refuses to build the NVM image. If a collision is detected, the user can check the "Operate in verbose mode (-v)" option when building the image. The flash tool will print a detailed report of the applications in the flash image and their data sections, so the offending sections can be identified.

In some cases, the user applications share a memory area to pass data between them, and this area needs to be declared in the application for each core, usually through the use of a global array. This raises the memory collision problem explained above. To solve this problem, the user must tell the linker to not initialize the shared buffer area, so that the flash image builder tool will not include the resulting `.bss` areas from the shared buffers. For Arm Cortex-M3 applications, the default behavior is to not include the `.bss` sections in the NVM flash image. For CFX applications, use the `_no_init_range` directive in the linker file. The following example shows how to not initialize the Arm Cortex-M3 DRAM2 and DRAM3 memory areas:

```
_no_init_range IOMEM D_CM3_DRAM2_BASE .. D_CM3_DRAM3_TOP
```

The equivalent LPDSP32 linker code is:

```
_no_init_range DM 0x808000 .. 0x80FFFF
```

14.3.1 Bootloader Global Variables

As the Ezairo 8300 ROM only loads one binary image as the bootloader from the NVM device, only elements in the `.text` section are copied to the NVM image. The elements in the `.data` sections are not copied. This means that the bootloader does not have initialized global variables. Notice that this only applies to the bootloader itself, and NOT to the application files. If the user changes the bootloader code and needs initialized global variables, they have to be manually initialized within a function by an explicit attribution.

When debugging a RAM application directly, there is no bootloader initializing the global variables, so the debug tools must perform this step. To initialize global variables for a RAM based application, the application and its debug configuration must be configured as follows:

1. The `sections.ld` linker file must provide linker definitions for `data_init`, `data_start`, and `data_end`.
2. The `init_cm3_global_vars.gdb` GDB script must be run as part of the debug configuration's startup. To configure the application's debug configuration to use this script:
 - a. In the **Edit Configuration** dialog box, choose the **Startup** tab.
 - b. In the **Run/Restart Commands** box, enter the command `source ${eclipse_home}..\utilities\Cortex-M3\init_cm3_global_vars.gdb`.

This GDB script file and the linker file changes can be used with any Arm Cortex-M3 processor application.

14.4 MANIFEST FILES

Manifest files provide a mechanism for supporting the case that a user application has been divided into more than one application file.

Manifest files contain a list of application files to be loaded. The application files will be loaded in the same order as they appear in the manifest file. Manifest files have a regular file header that is followed by a list of file names in plain ASCII. The separator between file names is the CR character ("). The file list must be null terminated. Manifest files can be encrypted.

Ezairo 8300 Firmware Reference

A typing use case for manifest files is to load a shared filter coefficient file and then an application file that uses the data previously loaded.

14.5 BOOTLOADER FILE LAYOUT

14.5.1 File Header

The file header is common to both application and manifest files. The following structure shows the layout of the file header:

```
struct application_file_header
{
    uint32_t    ident;
    uint16_t    crc_table_size;
    uint16_t    crc_table[];
};
```

The `crc_table` should be followed by a padding of a single `uint16_t` word with `0x00` as its value if the number of entries in `crc_table` is **odd**. If the number of entries is **even** then there must not be any padding. The `crc_table_size` has the number of **valid** entries in the table, the padding should not be counted as an entry. The CRC information in a single `uint16_t` word is placed after the padding. The CRC is CCITT CRC stored in big-endian order.

The `ident` member represents a magic number that identifies the file type and can have the following values:

File Type	Magic Number	Description
APPFILE_IDENT_APP	0x49415A45	Application file (not encrypted)
APPFILE_IDENT_APPENCRYPTED	0x4B415A45	Application file (encrypted)
APPFILE_IDENT_MANIFEST	0x4D415A45	Manifest file (not encrypted)
APPFILE_IDENT_MANIFESTENCRYPTED	0x4F415A45	Manifest file (encrypted)

The maximum supported size for this structure is configured by the `APP_HEADER_MAXIMUM_SIZE` preprocessor symbol. The default value is 512 bytes.

For files of the `APPFILE_IDENT_APPENCRYPTED` and `APPFILE_IDENT_MANIFESTENCRYPTED` types, the total size in bytes of the file header, not counting the `ident` field, must be a multiple of an AES block size (128 bits or 16 bytes). If the size is not a multiple of 16 bytes, then the appropriate number of padding bytes must be added **after** the CRC word. The values of these padding bytes is not important (it is important in the case of manifest files), but the recommended value is zero. They are not part of the CRC calculation. All data after the `ident` must be encrypted, while `ident` **should not** be encrypted.

14.5.2 Data Section Layout

A data section begins with the `data_section_header`:

```
struct data_section_header
{
    uint32_t    offset;
    uint32_t    size;
    uint16_t    type;
```

Ezairo 8300 Firmware Reference

```

uint16_t    flags;
uint16_t    _reserved;
uint16_t    header_crc;
uint32_t    data[];
};

```

The `header_crc` members is the CCITT CRC of the whole header, including the reserved area. The data CRC must be stored in big-endian order.

The `offset` holds the number of words between the beginning of the memory section area shown in [Section 14.2.1 “Data Sections in Application Files”](#) and the destination address. as an example, if the destination address is `CFX_XRAM0_LSB_UNSIGNED_BASE`, then `offset` should be `0x00000000`.

The possible values for the `flags` member are:

File Type	Value	Description
APPDATA_FLAGS_CLEARTEXT	0x00	Section data is not encrypted
APPDATA_FLAGS_ENCRYPTED	0x80	Section data is encrypted with the Application Key
APPDATA_FLAGS_CRC	0x00	CRC will be verified for the section
APPDATA_FLAGS_VOLATILE	0x40	CRC will not be verified (volatile section)

When a section is encrypted, it should have the `APPDATA_FLAGS_ENCRYPTED` value set. The `data_section_header` structure should be encrypted as successive AES blocks, the first block being the all elements up to and including `header_crc`. For an encrypted section, the data size must be a multiple of an AES block (16 bytes). If padding is used, it is considered part of the data and the corresponding CRC in the file header should include the padding bytes.

14.5.2.1 Boot Section Layout

A boot section has a regular `data_section_header` header, but a fixed data layout:

```

struct boot_section_data
{
    uint32_t    stack_pointer;
    uint32_t    init_pointer;
};

```

The `init_pointer` hold the address that will be set in the Program Counter. The `stack_pointer` member holds the Stack Pointer address of the application and is only used by Cortex-M3 applications.

14.5.3 Manifest File Layout

The manifest file differs from an application file as it does not have data sections, but a list of file names in pure ASCII. The individual application file names must be separated by a Line Feed (0x10 or \n) character. The list must be null-terminated. The file header will hold only one entry in the CRC table. This will be verified when the manifest file is loaded. In the case of a failure the loading process is aborted.

The maximum supported size for a manifest file, including the file header, is configured by the `MANIFEST_MAXIMUM_SIZE` preprocessor symbol. The default value is 512 bytes.

Ezairo 8300 Firmware Reference

If encryption is used, the padding requirement applies: the total size of the data part must be a multiple of 16, including the null termination. For manifest files, the value of the padding bytes **must** be zero.

CHAPTER 15

CFX DSP Library

The CFX DSP library provides a common set of 24, 48, and 56-bit fixed-point data types, and supports a number of operators and functions that use these data types. The use of this library to implement fixed-point math and signal processing functions on the CFX is recommended, and allows for simpler, more maintainable fixed-point code by managing the necessary bit shifts and data manipulation.

To achieve this, the library includes a number of functions to convert between fixed-point data types using appropriate rounding and saturation where the binary point is preserved, as well as re-interpretation functions where the underlying memory is not changed (i.e., `frac24_t` to `int`, and `int` to `frac24_t`).

15.1 INTRODUCTION

Several functions make use of the hardware optimized math accelerator, including logarithmic, exponential, root, and division functions. The hardware math accelerator is always used when these functions are called.

While the library is intended to assist in the development of fixed-point math and signal processing on the CFX, it is assumed that you have an understanding of the range and precision of each fixed-point data types.

As the name implies, this library is implemented only for the CFX core. It is distributed in both source-code and pre-compiled form in the Ezairo 8300 SDK. All the library functions are blocking and interruptible. The library does not use any interrupts or DMA channels.

15.2 USAGE

To use the CFX DSP library functions, include `sk5_cfx_dsp.h` in the application source and the application should link against the `libsk5_cfx_dsp.a` library object.

For the complete API, refer to the [Chapter 25 "CFX DSP Library Reference" on page 404](#).

CHAPTER 16

swmTrace Library

The swmTrace library is a complement to the RTT Viewer plugin described in the *Integrated Development Environment User's Guide for Ezairo 8300*. It is a logging utility, intended to provide debugging capabilities through an application running on the Arm Cortex-M3 core. When using the swmTrace library functions with the RTT viewer, logging information is color-coded depending on the level of logging used.

Although we expect that the primary use case for this library is with the SEGGER RTT technology through the SDK's RTT Viewer, you can also use the library with UART simply by linking to a different library. There are two variants of each, and in all cases you can change the output device without changing the code. RTT has both blocking and non-blocking variants, and UART has variants for DMA or user-defined pins for RX and TX (both non-blocking).

16.1 USAGE

Various levels of logging are available. See *swmTrace_api.h* for full details, but examples include `SWM_LOG_LEVEL_VERBOSE` and `SWM_LOG_TEST_PASS`. All use the `SWM_LOG_` prefix.

Sample applications are available specifically to illustrate the usage of the swmTrace library. See *swmTraceExample* under the Arm Cortex-M3 sample application folder. Also, other sample applications such as *three_core_passthrough* make use of the swmTrace library.

CHAPTER 17

Arm CMSIS DSP Library Reference

To use the Arm CMSIS-Pack DSP Library with the Arm Cortex-M3 core, you first need to download the library and associated header file. Links are provided below to download the library and associated files (based on CMSIS version 5). All files required to use the CMSIS DSP Library can be found at this location:

https://github.com/ARM-software/CMSIS_5/tree/master/CMSIS/DSP

17.1 CMSIS-PACK DSP LIBRARY

You can download the pre-built library files from the following location:

https://github.com/ARM-software/CMSIS_5/tree/master/CMSIS/DSP/Lib/GCC

You need to download the *libarm_cortexM3l_math.a* file, which at the time of writing is directly located here:

https://github.com/ARM-software/CMSIS_5/blob/master/CMSIS/DSP/Lib/GCC/libarm_cortexM3l_math.a

To use the library, include the *arm_math.h* header in your project, which can be downloaded directly from the following location:

https://github.com/ARM-software/CMSIS_5/blob/master/CMSIS/DSP/Include/arm_math.h

Detailed documentation for the CMSIS-DSP Library can be found here:

<http://www.keil.com/pack/doc/CMSIS/DSP/html/index.html>

Another way to get the DSP library is to download the latest released version of the CMSIS-Pack. Once downloaded, place the pack in a directory in which it can be unzipped. Using a compression tool such as 7-zip or Pkzip, you can unzip the pack into a directory, so that the CMSIS and Device directories are visible. Within the CMSIS directory a tree is created similar to the one shown in the "CMSIS Directory" figure (Figure 3).

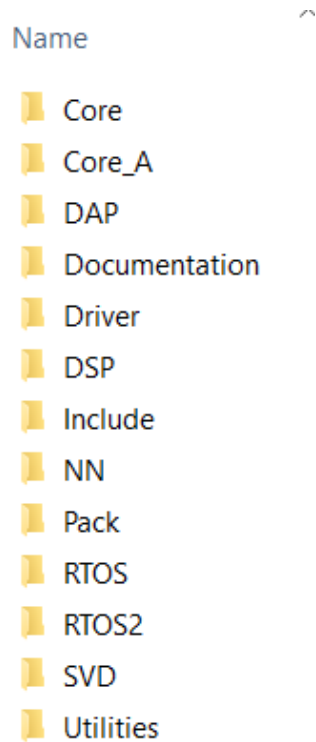


Figure 3. CMSIS Directory

Within the DSP directory is the tree shown in the "DSP Directory" figure (Figure 4):

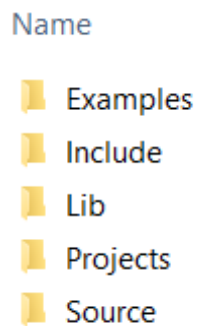


Figure 4. DSP Directory

The pre-built *libarm_cortexM3l_math.a* library is located in the *lib* directory. The *arm_math.h* file is located in the *Include* directory. Within the CMSIS-Pack, the DSP library documentation (in HTML format) can be found in the following location:

CMSIS\Documentation\DSP\html

The documentation provides detailed descriptions of all math functions.

CHAPTER 18

Hardware Abstraction Layer Reference

Simple hardware abstraction layer library reference.

18.1 SUMMARY

18.2 DETAILED DESCRIPTION

This library provides a collection of functions to access the underlying SoC hardware in a portable way.

18.3 ANALOG INPUT

Analog Input Hardware Abstraction Layer.

18.3.1 Summary

Macros

- [ADC_CLK_UPPERLIMIT](#) : ADC clock maximum frequency.
- [ADC_CLK_LOWERLIMIT](#) : ADC clock minimum frequency.
- [ADC_SAMPLE_FREQ_UPPERLIMIT](#) : ADC sample frequency setting maximum.
- [ADC_SAMPLE_FREQ_LOWERLIMIT](#) : ADC sample frequency setting minimum.
- [SYS_SET_ADC_CTRL](#) : Configure the ADC control settings for one of the input channels without calibrating the ADC DC offset.
- [SYS_GET_ADC_CTRL](#) : Return the ADC control configuration from one of the input channels.
- [SYS_SET_ADC_CFG](#) : Configure the ADC for one of the input channels.
- [SYS_GET_ADC_CFG](#) : Return the ADC configuration from one of the input channels.
- [SYS_SET_ADC_DEC_CTRL](#) : Configure the decimation filter for one of the input channels.
- [SYS_GET_ADC_DEC_CTRL](#) : Return the decimation filter configuration from one of the input channels.
- [SYS_SET_ADC_SAMPLE_FREQ_CFG](#) : Configure the sampling frequency.
- [SYS_GET_ADC_SAMPLE_FREQ_CFG](#) : Return the sampling frequency configuration.
- [SYS_SET_ADC_FDEC_CTRL](#) : ADC first decimation configuration registers.

Functions

- [Sys_Calc_Sample_Freq_Val](#) : Calculate the sampling frequency field for AUDIO_ADC_SAMPLE_FREQ_CFG.
- [Sys_Calc_Gain_Factor_Val](#) : Calculate the decimation gain factor for the nominal gain given an adc_cfg_sample_freq field.

18.3.2 Analog Input Macro Definition Documentation

18.3.2.1 ADC_CLK_UPPERLIMIT

```
#define ADC_CLK_UPPERLIMIT 5120000
```

ADC clock maximum frequency.

Location: analog_in.h:38

18.3.2.2 ADC_CLK_LOWERLIMIT

```
#define ADC_CLK_LOWERLIMIT 3840000
```

ADC clock minimum frequency.

Location: analog_in.h:39

18.3.2.3 ADC_SAMPLE_FREQ_UPPERLIMIT

```
#define ADC_SAMPLE_FREQ_UPPERLIMIT 63
```

ADC sample frequency setting maximum.

Location: analog_in.h:40

18.3.2.4 ADC_SAMPLE_FREQ_LOWERLIMIT

```
#define ADC_SAMPLE_FREQ_LOWERLIMIT 7
```

ADC sample frequency setting minimum.

Location: analog_in.h:41

18.3.2.5 SYS_SET_ADC_CTRL

```
#define SYS_SET_ADC_CTRL SYS_ASSERT(!(((channel) < 0) || ((channel) > 3)));\  
    audio->ADC_CTRL[(channel)] = (config)
```

Configure the ADC control settings for one of the input channels without calibrating the ADC DC offset.

Location: analog_in.h:156

Parameters

Direction	Name	Description
in	<i>audio</i>	Pointer to AUDIO block
in	<i>channel</i>	ADC to apply this configuration to; use a constant between 0 and 3
in	<i>config</i>	ADC control configuration; use ADC_[MODE MODE_MAX_1P6V MODE_HIPSRR MODE_HIPSRR_MAX_1P6V MODE_HIQ MODE_HIQ_MAX_1P6V MODE_HIQ_HIPSRR MODE_HIQ_HIPSRR_MAX_1P6V MODE_SDU MODE_DAC MODE_DAC_HIPSRR], ADC_FBDAC_SRC_[INT_FILT0 INT_FILT1 FENG_OUT10 FENG_OUT11], ADC_FBDAC_[INTERNAL EXTERNAL], LINEOUT_[DISABLE ENABLE], ADC_SEL_[AI* VREG VSSA], ADC_COMP_[DISABLE ENABLE] and ADC_[DISABLE ENABLE]

Example Code for SYS_SET_ADC_CTRL

```
// Configure ADC control settings for AUDIO block channel 2
SYS_SET_ADC_CTRL(AUDIO, 2, ADC_ENABLE |
    ADC_COMP_DISABLE | ADC_SEL_AI1 |
    ADC_LINEOUT_ENABLE | ADC_FBDAC_EXTERNAL);
```

18.3.2.6 SYS_GET_ADC_CTRL

```
#define SYS_GET_ADC_CTRL audio->ADC_CTRL[(channel)]
```

Return the ADC control configuration from one of the input channels.

Location: analog_in.h:169

Parameters

Ezairo 8300 Firmware Reference

Direction	Name	Description
in	<i>audio</i>	Pointer to AUDIO block
in	<i>channel</i>	ADC to load this configuration from; use a constant between 0 and 3

Return

The current ADC control configuration from AUDIO_ADC_CTRL_*

Example Code for SYS_GET_ADC_CTRL

```
//Retrieve ADC control settings for AUDIO block channel 2
config = SYS\_GET\_ADC\_CTRL(AUDIO, 2);
```

18.3.2.7 SYS_SET_ADC_CFG

```
#define SYS_SET_ADC_CFG SYS_ASSERT(!((channel) < 0) || ((channel) > 3));\
audio->ADC_CFG[(channel)] = (config)
```

Configure the ADC for one of the input channels.

Location: analog_in.h:185

Parameters

Direction	Name	Description
in	<i>audio</i>	pointer to AUDIO block
in	<i>channel</i>	ADC to apply this configuration to; use a constant between 0 and 3
in	<i>config</i>	ADC configuration; use ADC_FBDAC_DEM_[DISABLE ENABLE], ADC_FBDAC_MAX_DRIVE_[0 1 64], ADC_FBDAC_MIN_DRIVE_[0 1 64], ADC_RIN_[HIZ 4k57 2k29], ADC_COMP_CURRENT_[1UA 2UA], ADC_CURRENT_[2P5UA 14P5UA 25UA]

Example Code for SYS_SET_ADC_CFG

```
// Configure AUDIO block channel 3
SYS_SET_ADC_CFG(AUDIO, 3, ADC_FBDAC_MAX_DRIVE_64 |
                ADC_FBDAC_MIN_DRIVE_0 | ADC_RIN_4k57 |
                ADC_COMP_CURRENT_2u | ADC_CURRENT_25u);
```

18.3.2.8 SYS_GET_ADC_CFG

```
#define SYS_GET_ADC_CFG audio->ADC_CFG[(channel)]
```

Return the ADC configuration from one of the input channels.

Location: analog_in.h:197

Parameters

Direction	Name	Description
in	<i>audio</i>	Pointer to AUDIO block
in	<i>channel</i>	ADC to load this configuration from; use a constant between 0 and 3

Return

The current ADC configuration from AUDIO_ADC_CFG_*

Example Code for SYS_GET_ADC_CFG

```
// Retrieve AUDIO block channel 3 configuration
config = SYS_GET_ADC_CFG(AUDIO, 3);
```

Ezairo 8300 Firmware Reference

18.3.2.9 SYS_SET_ADC_DEC_CTRL

```
#define SYS_SET_ADC_DEC_CTRL SYS_ASSERT(!(((channel) < 0) || ((channel) > 3))); \
    audio->ADC_DEC_CTRL[(channel)] = (config)
```

Configure the decimation filter for one of the input channels.

Location: analog_in.h:216

Parameters

Direction	Name	Description
in	<i>audio</i>	Pointer to AUDIO block
in	<i>channel</i>	Decimation filter to apply this configuration to; use a constant between 0 and 3
in	<i>config</i>	Decimation filter configuration; use ADC_[MUTE UNMUTE], BAND_SELECT_ADC_*K_*K, ADC_INTEGER_DELAY_*, a constant shifted to AUDIO_ADC_DEC_CTRL_DELAY_FRACTIONAL_Pos, ADC_DEC_[ENABLE DISABLE], ADC_DC_REMOVE_CUTOFF_*HZ BYPASS] and a constant shifted to AUDIO_ADC_DEC_CTRL_ADC_GF_Pos

Example Code for SYS_SET_ADC_DEC_CTRL

```
// Configure the decimation filter for AUDIO block channel 1
SYS_SET_ADC_DEC_CTRL(AUDIO, 1, BAND_SELECT_ADC_8K_16K |
    ADC_INTEGER_DELAY_2 | ADC_MUTE |
    ADC_DEC_ENABLE |
    ADC_DC_REMOVE_CUTOFF_5HZ );
```

18.3.2.10 SYS_GET_ADC_DEC_CTRL

```
#define SYS_GET_ADC_DEC_CTRL audio->ADC_DEC_CTRL[(channel)]
```

Return the decimation filter configuration from one of the input channels.

Location: analog_in.h:230

Ezairo 8300 Firmware Reference

Parameters

Direction	Name	Description
in	<i>audio</i>	Pointer to AUDIO block
in	<i>channel</i>	Decimation filter to load this control configuration from; use a constant between 0 and 3

Return

The current decimation filter configuration

Example Code for SYS_GET_ADC_DEC_CTRL

```
// Retrieve AUDIO block channel 1 decimation filter configuration
reg = SYS\_GET\_ADC\_DEC\_CTRL(AUDIO, 1);
```

18.3.2.11 SYS_SET_ADC_SAMPLE_FREQ_CFG

```
#define SYS_SET_ADC_SAMPLE_FREQ_CFG audio->ADC_SAMPLE_FREQ_CFG = (config)
```

Configure the sampling frequency.

Location: analog_in.h:241

Parameters

Direction	Name	Description
in	<i>audio</i>	Pointer to AUDIO block
in	<i>config</i>	Sampling frequency configuration; use a constant shifted to AUDIO_ADC_SAMPLE_FREQ_CFG_SFCR_Pos

Example Code for SYS_SET_ADC_SAMPLE_FREQ_CFG

```
// Configure the AUDIO block sampling frequency  
SYS\_SET\_ADC\_SAMPLE\_FREQ\_CFG(AUDIO, ADC_MODDIV_BY8);
```

18.3.2.12 SYS_GET_ADC_SAMPLE_FREQ_CFG

```
#define SYS_GET_ADC_SAMPLE_FREQ_CFG audio->ADC_SAMPLE_FREQ_CFG
```

Return the sampling frequency configuration.

Location: analog_in.h:250

Parameters

Direction	Name	Description
in	<i>audio</i>	Pointer to AUDIO block

Return

The sampling frequency configuration

Example Code for SYS_GET_ADC_SAMPLE_FREQ_CFG

```
// Retrieve current AUDIO block sample frequency  
reg = SYS\_GET\_ADC\_SAMPLE\_FREQ\_CFG(AUDIO);
```

18.3.2.13 SYS_SET_ADC_FDEC_CTRL

```
#define SYS_SET_ADC_FDEC_CTRL SYS_ASSERT(!(((channel) < 0) || ((channel) > 3)));\  
audio->ADC_FDEC_CTRL[(channel)] = (config)
```

ADC first decimation configuration registers.

Location: analog_in.h:258

Ezairo 8300 Firmware Reference

Parameters

Direction	Name	Description
in	<i>audio</i>	Pointer to AUDIO block
in	<i>channel</i>	ADC channel to apply this configuration to; use 0 to 3
in	<i>config</i>	ADC first decimation configuration registers

18.3.3 Analog Input Function Documentation

18.3.3.1 Sys_Calc_Sample_Freq_Val

```
uint32_t Sys_Calc_Sample_Freq_Val(unsigned int adcclk, unsigned int sampling_freq)
```

Calculate the sampling frequency field for AUDIO_ADC_SAMPLE_FREQ_CFG.

Location: analog_in.h:53

Parameters

Direction	Name	Description
in	<i>adcclk</i>	ADC clock frequency in Hz
in	<i>sampling_freq</i>	Desired sampling frequency in Hz

Return

sys_calc_sf Define containing the rounded value for the sampling frequency field in AUDIO_ADC_SAMPLE_FREQ_CFG

Example Code for Sys_Calc_Sample_Freq_Val

```
// Calculate sample frequency for AUDIO_ADC_SAMPLE_FREQ_CFG
// using 5.12 MHz ADC clock and 40 kHz sampling frequency
samp_freq = Sys\_Calc\_Sample\_Freq\_Val(5120000, 40000);
```

18.3.3.2 Sys_Calc_Gain_Factor_Val

```
int32_t Sys_Calc_Gain_Factor_Val(unsigned int adc_cfg_sample_freq)
```

Calculate the decimation gain factor for the nominal gain given an adc_cfg_sample_freq field.

The resulting gain factor produces the optimal gain for the provided SFCR value.

Location: analog_in.h:107

Parameters

Direction	Name	Description
in	<i>adc_cfg_sample_freq</i>	Sampling frequency field in AUDIO_ADC_SAMPLE_FREQ_CFG

Return

sys_calc_gf Define containing the rounded value for the desired gain factor; use for the ADC_GF field of the AUDIO_ADC_DEC_CTRL[] register

Assumptions

adc_cfg_sample_freq, gain are within valid ranges

Example Code for Sys_Calc_Gain_Factor_Val

```
// Calculate gain factor needed for ADC_GF field of
// AUDIO_ADC_DEC_CTRL[] register using predefined
// AUDIO_ADC_SAMPLE_FREQ_CFG setting
GF = Sys\_Calc\_Gain\_Factor\_Val(ADC_MODDIV_BY8);
```

18.4 ANALOG OUTPUT

Analog Output Hardware Abstraction Layer.

18.4.1 Summary**Macros**

- [SYS_SET_OUTPUT_CTRL](#) : Configure the output driver configuration settings.
- [SYS_GET_OUTPUT_CTRL](#) : Return the output driver configuration settings.
- [SYS_SET_SDM_CTRL](#) : Configure the sigma-delta modulator configuration settings.
- [SYS_GET_SDM_CTRL](#) : Return the output driver configuration settings.
- [SYS_SET_SDMCLK](#) : Configure the sigma-delta modulator prescaler.
- [SYS_GET_SDMCLK](#) : Return the output modulator prescaler.
- [SYS_SET_DAC_GF_CTRL](#) : Configure the output stage gain factor control settings.
- [SYS_GET_O_DAC_GF_CTRL](#) : Return the output stage gain factor control settings.
- [SYS_SET_DMIC_OUTPUT_CTRL](#) : Configure the DMIC output channels.
- [SYS_GET_DMIC_OUTPUT_CTRL](#) : Return the DMIC output configuration.
- [SYS_SET_SDM_DCRM_CTRL](#) : DC remove filters configuration.
- [SYS_SET_SDM_OFFSET_CTRL](#) : SDM offset configuration.

18.4.2 Analog Output Macro Definition Documentation**18.4.2.1 SYS_SET_OUTPUT_CTRL**

```
#define SYS_SET_OUTPUT_CTRL audio->OUTPUT_DRIVER_CFG = (config)
```

Configure the output driver configuration settings.

Location: analog_out.h:47

Parameters

Ezairo 8300 Firmware Reference

Direction	Name	Description
in	<i>audio</i>	Pointer to AUDIO block
in	<i>config</i>	Output driver configurations; use OD*_CLK_SRC_[CCO SDMCLK], and OD*_[DISABLE ENABLE]

Example Code for SYS_SET_OUTPUT_CTRL

```
// Enable output driver 0 on AUDIO block using SDMCLK as clock source
SYS_SET_OUTPUT_CTRL(AUDIO, OD0_ENABLE | OD0_CLK_SRC_SDMCLK);
```

18.4.2.2 SYS_GET_OUTPUT_CTRL

```
#define SYS_GET_OUTPUT_CTRL audio->OUTPUT_DRIVER_CFG
```

Return the output driver configuration settings.

Location: analog_out.h:56

Parameters

Direction	Name	Description
in	<i>audio</i>	Pointer to AUDIO block

Return

The current output driver configurations

Example Code for SYS_GET_OUTPUT_CTRL

```
// Retrieve AUDIO block output driver configuration
out_driver = SYS_GET_OUTPUT_CTRL(AUDIO);
```

Ezairo 8300 Firmware Reference

18.4.2.3 SYS_SET_SDM_CTRL

```
#define SYS_SET_SDM_CTRL SYS_ASSERT(!(((channel) < 0) || ((channel) > 1))); \
    audio->SDM_CTRL[(channel)] = (config)
```

Configure the sigma-delta modulator configuration settings.

Location: analog_out.h:72

Parameters

Direction	Name	Description
in	<i>audio</i>	Pointer to AUDIO block
in	<i>channel</i>	SDM channel to apply this configuration to; use 0 or 1
in	<i>config</i>	SDM configurations; use QUANT_LEVEL_[5 6], TOGGLE_OPT_[DISABLE ENABLE], SD_[2 3]_LEVEL, CLK_RATIO_[8 4], XSDM0_[DISABLE ENABLE], OD_DELAY_[DISABLE ENABLE]

Example Code for SYS_SET_SDM_CTRL

```
// Configure the sigma-delta modulator configuration settings
// for AUDIO block 0
SYS_SET_SDM_CTRL(AUDIO, 0, QUANT_LEVEL_5 | TOGGLE_OPT_DISABLE |
    SD_2_LEVEL | CLK_RATIO_8 | XSDM0_ENABLE |
    OD_DELAY_DISABLE);
```

18.4.2.4 SYS_GET_SDM_CTRL

```
#define SYS_GET_SDM_CTRL audio->SDM_CTRL[(channel)]
```

Return the output driver configuration settings.

Location: analog_out.h:84

Parameters

Ezairo 8300 Firmware Reference

Direction	Name	Description
in	<i>audio</i>	Pointer to AUDIO block
in	<i>channel</i>	SDM to load this configuration from; use a 0 or 1

Return

The current SDM configurations

Example Code for SYS_GET_SDM_CTRL

```
// Retrieve the current SDM control configuration for AUDIO block 0
config = SYS\_GET\_SDM\_CTRL(AUDIO, 0);
```

18.4.2.5 SYS_SET_SDMCLK

```
#define SYS_SET_SDMCLK d_clk->CFG_1 = (d_clk->CFG_1 & \
    ~(D_CLK_CFG_1_SDMCLK_PRESCALE_Mask)) | \
    ((config) & D_CLK_CFG_1_SDMCLK_PRESCALE_Mask)
```

Configure the sigma-delta modulator prescaler.

Location: analog_out.h:93

Parameters

Direction	Name	Description
in	<i>d_clk</i>	Pointer to D_CLK block
in	<i>config</i>	Output modulator prescaler; use SDMCLK_PRESCALE_*

Ezairo 8300 Firmware Reference

Example Code for SYS_SET_SDMCLK

```
// Set the SDM clock prescaler
SYS_SET_SDMCLK(D_CLK, SDMCLK_PRESCALE_64);
```

18.4.2.6 SYS_GET_SDMCLK

```
#define SYS_GET_SDMCLK (((d_clk->CFG_1) & \
    D_CLK_CFG_1_SDMCLK_PRESCALE_Mask) \
    >> D_CLK_CFG_1_SDMCLK_PRESCALE_Pos)
```

Return the output modulator prescaler.

Location: analog_out.h:103

Parameters

Direction	Name	Description
in	<i>d_clk</i>	Pointer to D_CLK block

Return

The output modulator prescaler

Example Code for SYS_GET_SDMCLK

```
// Retrieve the current SDM prescaler
prescale = SYS_GET_SDMCLK(D_CLK);
```

18.4.2.7 SYS_SET_DAC_GF_CTRL

```
#define SYS_SET_DAC_GF_CTRL audio->OUTPUT_GAIN = (config)
```

Configure the output stage gain factor control settings.

Ezairo 8300 Firmware Reference

Location: analog_out.h:115

Parameters

Direction	Name	Description
in	<i>audio</i>	Pointer to AUDIO block
in	<i>config</i>	Output gain configuration; use constants shifted to AUDIO_OUTPUT_GAIN_OUTPUT*_GAIN_Pos

Example Code for SYS_SET_DAC_GF_CTRL

```
// Configure the output stage gain factor control settings
// for AUDIO block
SYS_SET_DAC_GF_CTRL(AUDIO, 0X02 << AUDIO_OUTPUT_GAIN_OUTPUT1_GAIN_Pos);
```

18.4.2.8 SYS_GET_O_DAC_GF_CTRL

```
#define SYS_GET_O_DAC_GF_CTRL audio->OUTPUT_GAIN
```

Return the output stage gain factor control settings.

Location: analog_out.h:124

Parameters

Direction	Name	Description
in	<i>audio</i>	Pointer to AUDIO block

Return

Output gain configuration

Ezairo 8300 Firmware Reference

Example Code for SYS_GET_O_DAC_GF_CTRL

```
// Retrieve current output stage gain factor control settings
// for AUDIO block
reg = SYS\_GET\_O\_DAC\_GF\_CTRL(AUDIO);
```

18.4.2.9 SYS_SET_DMIC_OUTPUT_CTRL

```
#define SYS_SET_DMIC_OUTPUT_CTRL audio->DMIC_OUTPUT_CFG = (config)
```

Configure the DMIC output channels.

Location: analog_out.h:136

Parameters

Direction	Name	Description
in	<i>audio</i>	Pointer to AUDIO block
in	<i>config</i>	DMIC output configuration; use DMIC*_HIGH_SRC_[CH0 CH1], DMIC*_LOW_SRC_[CH0 CH1], DMIC*_HIGH_[ENABLE DISABLE], and DMIC*_LOW_[ENABLE DISABLE]

Example Code for SYS_SET_DMIC_OUTPUT_CTRL

```
// Enable the AUDIO block DMIC0 low output channels using
// channel 1 as the source
SYS\_SET\_DMIC\_OUTPUT\_CTRL(AUDIO, (DMIC0_LOW_ENABLE | DMIC0_LOW_SRC_CH1 |
DMIC0_HIGH_DISABLE));
```

18.4.2.10 SYS_GET_DMIC_OUTPUT_CTRL

```
#define SYS_GET_DMIC_OUTPUT_CTRL audio->DMIC_OUTPUT_CFG
```

Return the DMIC output configuration.

Location: analog_out.h:145

Parameters

Direction	Name	Description
in	<i>audio</i>	Pointer to AUDIO block

Return

DMIC output channel configuration

Example Code for SYS_GET_DMIC_OUTPUT_CTRL

```
// Retrieve the current AUDIO block DMIC output configuration
config = SYS\_GET\_DMIC\_OUTPUT\_CTRL(AUDIO);
```

18.4.2.11 SYS_SET_SDM_DCRM_CTRL

```
#define SYS_SET_SDM_DCRM_CTRL SYS_ASSERT(!(((channel) < 0) || ((channel) > 1))); \
    audio->SDM_DCRM_CTRL[(channel)] = (config)
```

DC remove filters configuration.

Location: analog_out.h:153

Parameters

Direction	Name	Description
in	<i>audio</i>	Pointer to AUDIO block
in	<i>channel</i>	SDM channel to apply this configuration to; use 0 or 1
in	<i>config</i>	SDM DC remove filters configuration

18.4.2.12 SYS_SET_SDM_OFFSET_CTRL

```
#define SYS_SET_SDM_OFFSET_CTRL SYS_ASSERT(!((channel) < 0) || ((channel) > 1));\
    audio->SDM_OFFSET_CTRL[(channel)] = (config)
```

SDM offset configuration.

Location: analog_out.h:163

Parameters

Direction	Name	Description
in	<i>audio</i>	Pointer to AUDIO block
in	<i>channel</i>	SDM channel to apply this configuration to; use 0 or 1
in	<i>config</i>	SDM offset configuration

18.5 ASCC

ASCC Hardware Abstraction Layer.

18.5.1 Summary

Macros

- [SYS_ASCC_CONFIG](#) : Macro wrapper for [Sys_ASCC_Config\(\)](#) Configure the Audio Sink Clock Counter Block, its input clock, and the DIO interrupt used for the input clock.

Functions

- [Sys_ASCC_Config](#) : Configure the Audio Sink Clock Counter Block, its input clock, and the DIO interrupt used for the input clock.

18.5.2 ASCC Macro Definition Documentation

18.5.2.1 SYS_ASCC_CONFIG

```
#define SYS_ASCC_CONFIG Sys\_ASCC\_Config(AUDIOSINK, (config), (clkdio), (diocfg), \
    (int_cfg), (dbnc_clk), (dbnc_cnt))
```

Ezairo 8300 Firmware Reference

Macro wrapper for [Sys_ASCC_Config\(\)](#) Configure the Audio Sink Clock Counter Block, its input clock, and the DIO interrupt used for the input clock.

Location: ascc.h:118

Parameters

Direction	Name	Description
in	<i>config</i>	Interface operation configuration: Input and sampling clock sources; use FRAME_SYNC_DIO_INT_*[0-7], and AUDIOSINK_[ADC0_ADC2 ADC1_ADC3 FIFO_INT_*]
in	<i>clkdio</i>	DIO pin to use for the input clock
in	<i>diocfg</i>	DIO pin configuration for the input clock; use DIO_*X_DRIVE, DIO_LPF_[ENABLE DISABLE], DIO_*_PULL, and DIO_MODE_[*CLK INPUT]
in	<i>int_cfg</i>	DIO interrupt configuration for the input clock; use DIO_INT_DEBOUNCE_[DISABLE ENABLE], DIO_INT_SRC_* (should be the same as input clock pin 'clk'), and DIO_INT_EVENT_[NONE HIGH_LEVEL LOW_LEVEL RISING_EDGE FALLING_EDGE TRANSITION]
in	<i>dbnc_clk</i>	Interrupt button debounce filter clock; use DIO_DEBOUNCE_SLOWCLK_DIV[32 1024]
in	<i>dbnc_cnt</i>	Interrupt button debounce filter count

Assumptions

The sampling clock source (FIFO interrupt or ADC decimator) has been configured.

Ezairo 8300 Firmware Reference

Example Code for SYS_ASCC_CONFIG

```
//Configure default ASCC block, using
// - Frame sync source is DIO interrupt 6
// - FIFO interrupt 1 as Audio sink clock source
// - DIO4 as the input clock pin
// - Input clock config using 8X drive strength,
//   disabled low pass filter, and DIO as an input
// - DIO input clock interrupt config as falling edge
//   with debounce disabled
// - SLOWCLK divided by 32 as the debounce filter
// - 0 interrupt debounce filter count
SYS_ASCC_CONFIG((FRAME_SYNC_DIO_INT_6 |
    AUDIOSINK_FIFO_INT_1), DIO4, (DIO_1K_PULL_UP |
    DIO_8X_DRIVE | DIO_LPF_DISABLE |
    DIO_MODE_INPUT), (DIO_INT_DEBOUNCE_DISABLE |
    DIO_INT_EVENT_RISING_EDGE | DIO_INT_SRC_DIO_4),
    DIO_DEBOUNCE_SLOWCLK_DIV32, 0);
```

18.5.3 ASCC Function Documentation**18.5.3.1 Sys_ASCC_Config**

```
void Sys_ASCC_Config(AUDIOSINK_Type * ascc, uint32_t config, uint32_t clkdio, uint32_t
diocfg, uint32_t int_cfg, uint32_t dbnc_clk, uint32_t dbnc_cnt)
```

Configure the Audio Sink Clock Counter Block, its input clock, and the DIO interrupt used for the input clock.

Location: ascc.h:67

Parameters

Direction	Name	Description
in	<i>ascc</i>	Pointer to the Audio Sink Clock Counter instance
in	<i>config</i>	Interface operation configuration: Input and sampling clock sources; use FRAME_SYNC_DIO_INT_*[0-7], and AUDIOSINK_[ADC0_ADC2 ADC1_ADC3 FIFO_INT_*]
in	<i>clkdio</i>	DIO pin to use for the input clock
in	<i>diocfg</i>	DIO pin configuration for the input clock; use DIO_*X_DRIVE, DIO_LPF_[ENABLE DISABLE], DIO_*_PULL, and DIO_MODE_*[CLK INPUT]
in	<i>int_cfg</i>	DIO interrupt configuration for the input clock; use DIO_INT_DEBOUNCE_[DISABLE ENABLE], DIO_INT_SRC_

Ezairo 8300 Firmware Reference

Direction	Name	Description
		* (should be the same as input clock pin 'clk'), and DIO_INT_EVENT_[NONE HIGH_LEVEL LOW_LEVEL RISING_EDGE FALLING_EDGE TRANSITION]
in	<i>dbnc_clk</i>	Interrupt button debounce filter clock; use DIO_DEBOUNCE_SLOWCLK_DIV[32 1024]
in	<i>dbnc_cnt</i>	Interrupt button debounce filter count

Assumptions

The sampling clock source (FIFO interrupt or ADC decimator) has been configured.

Example Code for Sys_ASCC_Config

```
//Configure AUDIOSINK0 block, using
// - Frame sync source is DIO interrupt 6
// - FIFO interrupt 1 as Audio sink clock source
// - DIO4 as the input clock pin
// - Input clock config using 8X drive strength,
//   disabled low pass filter, and DIO as an input
// - DIO input clock interrupt config as falling edge
//   with debounce disabled
// - SLOWCLK divided by 32 as the debounce filter
// - 0 interrupt debounce filter count
Sys_ASCC_Config(AUDIOSINK0, (FRAME_SYNC_DIO_INT_6 |
    AUDIOSINK_FIFO_INT_1), DIO4, (DIO_1K_PULL_UP |
    DIO_8X_DRIVE | DIO_LPF_DISABLE |
    DIO_MODE_INPUT), (DIO_INT_DEBOUNCE_DISABLE |
    DIO_INT_EVENT_RISING_EDGE | DIO_INT_SRC_DIO_4),
    DIO_DEBOUNCE_SLOWCLK_DIV32, 0);
```

18.6 D_CRC

D_CRC Hardware Abstraction Layer.

18.6.1 Summary

Functions

- [Sys D_CRC_InitValue](#) : Initialize CRC for CRC-CCITT.
- [Sys D_CRC_GetCurrentValue](#) : Get current CRC value.

- [Sys_D_CRC_Add](#) : Add data to the current CRC calculation, based on size.

18.6.2 D_CRC Function Documentation

18.6.2.1 Sys_D_CRC_InitValue

```
void Sys_D_CRC_InitValue(D_CRC_Type * crc)
```

Initialize CRC for CRC-CCITT.

Location: d_crc.h:46

Parameters

Direction	Name	Description
in	<i>crc</i>	Pointer to the crc instance

Assumptions

CRC is configured to work in CRC-CCITT mode

Example Code for Sys_D_CRC_InitValue

```
// Initialize CRC block for CRC-CCITT
Sys\_D\_CRC\_InitValue(D_CRC);
```

18.6.2.2 Sys_D_CRC_GetCurrentValue

```
uint32_t Sys_D_CRC_GetCurrentValue(const D_CRC_Type * crc)
```

Get current CRC value.

Location: d_crc.h:57

Parameters

Ezairo 8300 Firmware Reference

Direction	Name	Description
in	<i>crc</i>	Pointer to the crc instance

Example Code for Sys_D_CRC_GetCurrentValue

```
// Retrieve current value from the CRC block
Sys_D_CRC_GetCurrentValue(D_CRC);
```

18.6.2.3 Sys_D_CRC_Add

```
void Sys_D_CRC_Add(D_CRC_Type * crc, uint32_t data, uint32_t size)
```

Add data to the current CRC calculation, based on size.

Location: d_crc.h:70

Parameters

Direction	Name	Description
in	<i>crc</i>	Pointer to the crc instance
in	<i>data</i>	Data to add
in	<i>size</i>	Size of data to add, 1, 8, 16, 24, 32 are valid.

Example Code for Sys_D_CRC_Add

```
// Add 8 bits of data to the CRC block
Sys_D_CRC_Add(D_CRC, 0xF1, 8);
```

18.7 DIO

Ezairo 8300 Firmware Reference

DIO Hardware Abstraction Layer.

18.7.1 Summary

Macros

- [DIO_WEAK_PULL_UP](#) : Weak pull-up alias.
- [DIO_MEDIUM_PULL_UP](#) : Medium pull-up alias.
- [DIO_STRONG_PULL_UP](#) : Strong pull-up alias.
- [DIO_WEAK_PULL_DOWN](#) : Weak pull-down alias.
- [DIO_LEVEL1_DRIVE](#) : Lowest drive strength alias.
- [DIO_LEVEL2_DRIVE](#) : Second lowest drive strength alias.
- [DIO_LEVEL3_DRIVE](#) : Second highest drive strength alias.
- [DIO_LEVEL4_DRIVE](#) : Highest drive strength alias.
- [SYS_DIO_CONFIG](#) : Macro wrapper for Configure the specified digital I/O.

Functions

- [Sys_DIO_Config](#) : Configure the specified digital I/O.
- [Sys_DIO_NMIConfig](#) : Configure a source for NMI input selection.
- [Sys_DIO_IntConfig](#) : Configure a DIO interrupt source.
- [Sys_DIO_INTCLKConfig](#) : Configure DIO clock divider counter interrupt source.
- [Sys_DIO_CM3JTAGConfig](#) : Configure Arm Cortex-M3 core's SWJ-DP.

18.7.2 DIO Macro Definition Documentation

18.7.2.1 DIO_WEAK_PULL_UP

```
#define DIO_WEAK_PULL_UP DIO_250K_PULL_UP
```

Weak pull-up alias.

Location: dio.h:39

18.7.2.2 DIO_MEDIUM_PULL_UP

```
#define DIO_MEDIUM_PULL_UP DIO_60K_PULL_UP
```

Medium pull-up alias.

Location: dio.h:40

18.7.2.3 DIO_STRONG_PULL_UP

```
#define DIO_STRONG_PULL_UP DIO_1K_PULL_UP
```

Strong pull-up alias.

Location: dio.h:41

18.7.2.4 DIO_WEAK_PULL_DOWN

```
#define DIO_WEAK_PULL_DOWN DIO_60K_PULL_DOWN
```

Weak pull-down alias.

Location: dio.h:42

18.7.2.5 DIO_LEVEL1_DRIVE

```
#define DIO_LEVEL1_DRIVE DIO_1X_DRIVE
```

Lowest drive strength alias.

Location: dio.h:44

18.7.2.6 DIO_LEVEL2_DRIVE

```
#define DIO_LEVEL2_DRIVE DIO_2X_DRIVE
```

Second lowest drive strength alias.

Location: dio.h:45

18.7.2.7 DIO_LEVEL3_DRIVE

```
#define DIO_LEVEL3_DRIVE DIO_4X_DRIVE
```

Second highest drive strength alias.

Location: dio.h:46

18.7.2.8 DIO_LEVEL4_DRIVE

```
#define DIO_LEVEL4_DRIVE DIO_8X_DRIVE
```

Highest drive strength alias.

Location: dio.h:47

18.7.2.9 SYS_DIO_CONFIG

```
#define SYS_DIO_CONFIG Sys\_DIO\_Config(pad, config)
```

Macro wrapper for Configure the specified digital I/O.

Location: dio.h:190

Parameters

Direction	Name	Description
in	<i>pad</i>	Digital I/O pad to configure; use a constant
in	<i>config</i>	I/O configuration; use DIO_*X_DRIVE, DIO_LPF_[ENABLE DISABLE], DIO_*_PULL, and DIO_MODE_*

Example Code for SYS_DIO_CONFIG

```
// Enable DIO 5 as GPIO input using no pull-up resistor  
SYS\_DIO\_CONFIG(DIO5, DIO_MODE_GPIO_IN | DIO_NO_PULL);
```

18.7.3 DIO Function Documentation

18.7.3.1 Sys_DIO_Config

```
void Sys_DIO_Config(uint32_t pad, uint32_t config)
```

Configure the specified digital I/O.

Location: dio.h:57

Parameters

Direction	Name	Description
in	<i>pad</i>	Digital I/O pad to configure; use a constant
in	<i>config</i>	I/O configuration; use DIO_*X_DRIVE, DIO_LPF_[ENABLE DISABLE], DIO_*_PULL, and DIO_MODE_*

Example Code for Sys_DIO_Config

```
// Enable DIO 5 as GPIO input using no pull-up resistor
Sys_DIO_Config(DIO5, DIO_MODE_GPIO_IN | DIO_NO_PULL);
```

18.7.3.2 Sys_DIO_NMIConfig

```
void Sys_DIO_NMIConfig(uint32_t config, uint32_t source, uint32_t polarity)
```

Configure a source for NMI input selection.

Location: dio.h:70

Parameters

Direction	Name	Description
in	<i>config</i>	DIO pin configuration if NMI is pad
in	<i>source</i>	NMI source; use NMI_SRC_*
in	<i>polarity</i>	NMI polarity; use NMI_ACTIVE_[LOW HIGH]

Ezairo 8300 Firmware Reference

Example Code for Sys_DIO_NMIconfig

```
// Configure NMI input using DIO 13 as active-low source without low-pass filter
Sys_DIO_NMIconfig(DIO_LPF_ENABLE, NMI_SRC_DIO_13, NMI_ACTIVE_LOW);
```

18.7.3.3 Sys_DIO_IntConfig

```
void Sys_DIO_IntConfig(uint32_t index, uint32_t config, uint32_t dbnc_clk, uint32_t dbnc_cnt)
```

Configure a DIO interrupt source.

Location: dio.h:97

Parameters

Direction	Name	Description
in	<i>index</i>	DIO interrupt channel to configure; use a constant between 0 and 7
in	<i>config</i>	DIO interrupt configuration; use DIO_INT_DEBOUNCE_[DISABLE ENABLE], DIO_INT_SRC_*, and DIO_INT_EVENT_[NONE HIGH_LEVEL LOW_LEVEL RISING_EDGE FALLING_EDGE TRANSITION]
in	<i>dbnc_clk</i>	Interrupt button debounce filter clock; use DIO_DEBOUNCE_SLOWCLK_DIV[32 1024]
in	<i>dbnc_cnt</i>	Interrupt button debounce filter count

Example Code for Sys_DIO_IntConfig

```
// configure DIO channel 7 to use DIO 11 as interrupt source, with
// 1 debounce filter count.
Sys_DIO_IntConfig(7, (DIO_INT_DEBOUNCE_ENABLE | DIO_INT_SRC_DIO_11),
DIO_DEBOUNCE_SLOWCLK_DIV1024, 1);
```

18.7.3.4 Sys_DIO_INTCLKConfig

```
void Sys_DIO_INTCLKConfig(uint32_t num, uint32_t cfg, uint32_t source)
```

Configure DIO clock divider counter interrupt source.

Location: dio.h:121

Parameters

Direction	Name	Description
in	<i>num</i>	Clock div number, use 0 or 1
in	<i>cfg</i>	DIO Clock Divider Configuration; use DIO_CLKDIV*_RESET, DIO_CLKDIV*_[ENABLE DISABLE], DIO_CLKDIV*_[RISING FALLING]_EDGE, DIO_CLKDIV*_PRESCALE_*,
in	<i>source</i>	DIO interrupt Clock Divider input selection; use DIO_CLKDIV*_SRC_DIO_*

Example Code for Sys_DIO_INTCLKConfig

```
// Configure DIO 0 clock divider 0 counter interrupt source
Sys_DIO_INTCLKConfig(1, (DIO_CLKDIV0_ENABLE | DIO_CLKDIV0_RISING_EDGE |
DIO_CLKDIV0_PRESCALE_64), DIO_CLKDIV0_SRC_DIO_0);
```

18.7.3.5 Sys_DIO_CM3JTAGConfig

```
void Sys_DIO_CM3JTAGConfig(bool enable, bool jtag)
```

Configure Arm Cortex-M3 core's SWJ-DP.

Location: dio.h:159

Parameters

Direction	Name	Description
in	<i>enable</i>	Enable or disable the SWJ-DP port
in	<i>jtag</i>	Enable JTAG mode or SW mode; use true for JTAG mode, false for SW mode

Example Code for Sys_DIO_CM3JTAGConfig

```
// Enable Arm Cortex-M3 SWJ-DP port and JTAG
Sys_DIO_CM3JTAGConfig(true, true);
```

18.8 EMMC_I3C

Hardware Abstraction Layer for direct Arm Cortex-M3 processor accesses to the memory-mapped elements of the eMMC and I3C interfaces.

18.8.1 Summary**Functions**

- [Sys EMMC I3C Read](#) : Perform a safe read from an eMMC or I3C memory mapped element.
- [Sys EMMC I3C Write](#) : Perform a safe write to an eMMC or I3C memory mapped element.

18.8.2 EMMC_I3C Function Documentation**18.8.2.1 Sys_EMMC_I3C_Read**

```
uint32_t Sys_EMMC_I3C_Read()
```

Perform a safe read from an eMMC or I3C memory mapped element.

Location: emmc_i3c.h:43

Example Code for Sys_EMMC_I3C_Read

```
// Read the I3C device ID (Arm Cortex-M3 processor only)
data = Sys_EMMC_I3C_Read(&I3C0->DEV_ID);
// Read the first word of the eMMC memory buffer (Arm Cortex-M3 processor only)
data = Sys_EMMC_I3C_Read((const volatile uint32_t *)EMMC_BUFFER_BASE);
```


Ezairo 8300 Firmware Reference

18.8.2.2 Sys_EMMC_I3C_Write

```
void Sys_EMMC_I3C_Write()
```

Perform a safe write to an eMMC or I3C memory mapped element.

Location: emmc_i3c.h:57

Example Code for Sys_EMMC_I3C_Write

```
// Write a constant to the I3C[0] TX FIFO
Sys_EMMC_I3C_Write(&I3C0->TX_FIFO, 0x1234);
// Write the first word of the eMMC memory buffer (Arm Cortex-M3 processor only)
Sys_EMMC_I3C_Write((const volatile uint32_t *)EMMC_BUFFER_BASE, 0x1234);
```

18.9 FILTER ENGINE HAL REFERENCE

Filter Engine Hardware Abstraction Layer support.

18.9.1 Summary

Macros

- [SYS_FENG_PAUSE](#) : Pause the Filter Engine immediately.
- [SYS_FENG_START](#) : Start (or resume) the Filter Engine.
- [SYS_FENG_SINGLESTEP](#) : Single-step the Filter Engine.
- [SYS_FENG_DELAYEDPAUSE](#) : Pause the Filter Engine the next time it would start execution.
- [SYS_FENG_RESET](#) : Reset the Filter Engine program decoder registers and audio path registers.
- [SYS_FENG_EXECUTE](#) : Execute one phase of the current Filter Engine program.
- [SYS_SET_FENG_MRCOUNTERCONFIG](#) : Configure the multi-rate counter divisors and reset the multi-rate counters.
- [SYS_SET_FENG_INTENABLE](#) : Configure the possible sources for the Filter Engine interrupt.
- [SYS_FENG_INTACK](#) : Acknowledge a set of Filter Engine interrupt sources.
- [SYS_GET_FENG_INTSTATUS](#) : Get the status of various possible sources of the Filter Engine interrupt.
- [SYS_SET_FENG_STATEWRITEENABLE](#) : Configure the Filter Engine state code write-enable settings.
- [SYS_GET_FENG_STATEWRITEENABLE](#) : Get the current Filter Engine state code write-enable settings.
- [SYS_SET_FENG_STATE](#) : Configure the Filter Engine state codes by writing to the active register.
- [SYS_SET_FENG_STATE_SHADOW](#) : Configure the Filter Engine state codes by writing to the shadow register.
- [SYS_GET_FENG_STATE](#) : Get the current Filter Engine state codes.

Functions

- [Sys Set FENG Coefficients](#) : Configure one set of Filter Engine coefficients for use with the Filter Engine BIQ and FIR instructions.

18.9.2 Filter Engine HAL Reference Macro Definition Documentation

18.9.2.1 SYS_FENG_PAUSE

```
#define SYS_FENG_PAUSE d_feng->CTRL = FENG_CTRL_DISABLE
```

Pause the Filter Engine immediately.

Location: feng.h:43

Parameters

Direction	Name	Description
in	<i>d_feng</i>	Pointer to D_FENG block

Example Code for SYS_FENG_PAUSE

```
// Pause the Filter Engine immediately  
SYS\_FENG\_PAUSE(D_FENG);
```

18.9.2.2 SYS_FENG_START

```
#define SYS_FENG_START d_feng->CTRL = FENG_CTRL_ENABLE
```

Start (or resume) the Filter Engine.

Location: feng.h:50

Parameters

Ezairo 8300 Firmware Reference

Direction	Name	Description
in	<i>d_feng</i>	Pointer to D_FENG block

Example Code for SYS_FENG_START

```
// Start (or resume) the Filter Engine
SYS_FENG_START(D_FENG);
```

18.9.2.3 SYS_FENG_SINGLESTEP

```
#define SYS_FENG_SINGLESTEP d_feng->CTRL = (FENG_STEP | \
                                             FENG_CTRL_DISABLE)
```

Single-step the Filter Engine.

Location: feng.h:57

Parameters

Direction	Name	Description
in	<i>d_feng</i>	Pointer to D_FENG block

Example Code for SYS_FENG_SINGLESTEP

```
// Single-step the Filter Engine
SYS_FENG_SINGLESTEP(D_FENG);
```

18.9.2.4 SYS_FENG_DELAYEDPAUSE

```
#define SYS_FENG_DELAYEDPAUSE d_feng->CTRL = (FENG_PAUSE_ENABLE | \
                                             FENG_CTRL_ENABLE)
```

Pause the Filter Engine the next time it would start execution.

Location: feng.h:66

Parameters

Direction	Name	Description
in	<i>d_feng</i>	Pointer to D_FENG block

Example Code for SYS_FENG_DELAYEDPAUSE

```
// Pause the Filter Engine the next time it would start execution
SYS_FENG_DELAYEDPAUSE(D_FENG);
```

18.9.2.5 SYS_FENG_RESET

```
#define SYS_FENG_RESET d_feng->CMDS = (FENG_DECODER_RESET | \
                                         FENG_AUDIO_RESET)
```

Reset the Filter Engine program decoder registers and audio path registers.

Location: feng.h:76

Parameters

Direction	Name	Description
in	<i>d_feng</i>	Pointer to D_FENG block

Example Code for SYS_FENG_RESET

```
// Reset the Filter Engine program decoder registers and audio path registers
SYS_FENG_RESET(D_FENG);
```

18.9.2.6 SYS_FENG_EXECUTE

```
#define SYS_FENG_EXECUTE d_feng->CMDS = FENG_EXEC
```

Execute one phase of the current Filter Engine program.

Location: feng.h:87

Parameters

Direction	Name	Description
in	<i>d_feng</i>	Pointer to D_FENG block

Assumptions

The Filter Engine clock has been enabled (Filter Engine started)

Example Code for SYS_FENG_EXECUTE

```
// Execute one phase of the current Filter Engine program  
SYS\_FENG\_EXECUTE(D_FENG);
```

18.9.2.7 SYS_SET_FENG_MRCOUNTERCONFIG

```
#define SYS_SET_FENG_MRCOUNTERCONFIG d_feng->MRCNT = ((cfg0) << D_FENG_MRCNT_MRCNT0_DR_Pos)  
| \  
                ((cfg1) << D_FENG_MRCNT_MRCNT1_DR_Pos)
```

Configure the multi-rate counter divisors and reset the multi-rate counters.

Location: feng.h:101

Parameters

Ezairo 8300 Firmware Reference

Direction	Name	Description
in	<i>d_feng</i>	Pointer to D_FENG block
in	<i>cfg0</i>	Configuration for Filter Engine multirate counter 0; use a constant indicating the number of phases desired - 1
in	<i>cfg1</i>	Configuration for Filter Engine multirate counter 1; use a constant indicating the number of phases desired - 1

Example Code for SYS_SET_FENG_MRCOUNTERCONFIG

```
// Configure the multi-rate counter divisors to 4 and 2
// phases respectively
SYS_SET_FENG_MRCOUNTERCONFIG(D_FENG, 3, 1);
```

18.9.2.8 SYS_SET_FENG_INTENABLE

```
#define SYS_SET_FENG_INTENABLE d_feng->INT_ENABLE = (int_ebl)
```

Configure the possible sources for the Filter Engine interrupt.

Location: feng.h:114

Parameters

Direction	Name	Description
in	<i>d_feng</i>	Pointer to D_FENG block
in	<i>int_ebl</i>	Filter engine interrupt configuration; use a set of INT_DIS_FENG_* and INT_EBL_FENG_* defines

Ezairo 8300 Firmware Reference

Example Code for SYS_SET_FENG_INTENABLE

```
//Configure and enable Filter Engine interrupt sources
SYS_SET_FENG_INTENABLE(D_FENG, INT_EBL_FENG_ISHC | INT_DIS_FENG_PGE |
    INT_EBL_FENG_CMP | INT_DIS_FENG_REG_ERR |
    INT_DIS_FENG_MEM_ERR | INT_EBL_FENG_CORE_SAT |
    INT_DIS_FENG_ALU_SAT | INT_EBL_FENG_OVERLOAD |
    INT_EBL_FENG_OUT | INT_EBL_FENG_SRIN);
```

18.9.2.9 SYS_FENG_INTACK

```
#define SYS_FENG_INTACK d_feng->INT_ACK = (int_ack)
```

Acknowledge a set of Filter Engine interrupt sources.

Location: feng.h:123

Parameters

Direction	Name	Description
in	<i>d_feng</i>	Pointer to D_FENG block
in	<i>int_ack</i>	Filter engine interrupt sources to acknowledge

Example Code for SYS_FENG_INTACK

```
// Acknowledge Filter Engine PGE, REG_ERR, MEM_ERR and ALU_SAT
// interrupt sources
SYS_FENG_INTACK(D_FENG, INT_ACK_FENG_PGE | INT_ACK_FENG_REG_ERR |
    INT_ACK_FENG_MEM_ERR | INT_ACK_FENG_ALU_SAT);
```

18.9.2.10 SYS_GET_FENG_INTSTATUS

```
#define SYS_GET_FENG_INTSTATUS d_feng->INT_STATUS
```

Get the status of various possible sources of the Filter Engine interrupt.

Location: feng.h:133

Parameters

Direction	Name	Description
in	<i>d_feng</i>	Pointer to D_FENG block

Return

The current Filter Engine interrupt status bits; compare with INT_PENDING_FENG_*

Example Code for SYS_GET_FENG_INTSTATUS

```
// Get the status of Filter Engine interrupt sources
SYS_GET_FENG_INTSTATUS(D_FENG);
```

18.9.2.11 SYS_SET_FENG_STATEWRITEENABLE

```
#define SYS_SET_FENG_STATEWRITEENABLE d_feng->STATE_W_EN = (cfg)
```

Configure the Filter Engine state code write-enable settings.

Location: feng.h:141

Parameters

Direction	Name	Description
in	<i>d_feng</i>	Pointer to D_FENG block
in	<i>cfg</i>	Filter engine state code writeenable settings

Example Code for SYS_SET_FENG_STATEWRITEENABLE

```
// Enable Filter Engine write-enables 31 down to 1  
SYS\_SET\_FENG\_STATEWRITEENABLE(D_FENG, 0xFFFFFFFF);
```

18.9.2.12 SYS_GET_FENG_STATEWRITEENABLE

```
#define SYS_GET_FENG_STATEWRITEENABLE d_feng->STATE_W_EN
```

Get the current Filter Engine state code write-enable settings.

Location: feng.h:151

Parameters

Direction	Name	Description
in	<i>d_feng</i>	Pointer to D_FENG block

Return

The current Filter Engine state code write-enable settings

Example Code for SYS_GET_FENG_STATEWRITEENABLE

```
// Get the current Filter Engine state code write-enable settings  
wr_state = SYS\_GET\_FENG\_STATEWRITEENABLE(D_FENG);
```

18.9.2.13 SYS_SET_FENG_STATE

```
#define SYS_SET_FENG_STATE d_feng->STATE_ACTIVE = (state)
```

Configure the Filter Engine state codes by writing to the active register.

Location: feng.h:164

Parameters

Direction	Name	Description
in	<i>d_feng</i>	Pointer to D_FENG block
in	<i>state</i>	Filter engine state codes; use 1 (condition met) or 0 (condition not met) shifted by the state code index for each state code

Assumptions

The Filter Engine is inactive

Example Code for SYS_SET_FENG_STATE

```
// Configure the Filter Engine state codes by writing to the  
// active register  
SYS\_SET\_FENG\_STATE(D_FENG, 0xFFFFFFFF);
```

18.9.2.14 SYS_SET_FENG_STATE_SHADOW

```
#define SYS_SET_FENG_STATE_SHADOW d_feng->STATE_SHADOW = (state)
```

Configure the Filter Engine state codes by writing to the shadow register.

These state codes will take effect the next time Filter Engine execution starts.

Location: feng.h:176

Parameters

Ezairo 8300 Firmware Reference

Direction	Name	Description
in	<i>d_feng</i>	Pointer to D_FENG block
in	<i>state</i>	Filter Engine state codes; use 1 (condition met) or 0 (condition not met) shifted by the state code index for each state code

Example Code for SYS_SET_FENG_STATE_SHADOW

```
// Configure the Filter Engine state codes by writing to the
// shadow register. These state codes will take effect the next
// time Filter Engine execution starts.
SYS_SET_FENG_STATE_SHADOW(D_FENG, 0xFFFFFFFF);
```

18.9.2.15 SYS_GET_FENG_STATE

```
#define SYS_GET_FENG_STATE d_feng->STATE_ACTIVE
```

Get the current Filter Engine state codes.

Location: feng.h:186

Return

The current Filter Engine state codes

Clock Cycles

1

Ezairo 8300 Firmware Reference

Example Code for SYS_GET_FENG_STATE

```
// Get the current Filter Engine state codes
codes = SYS\_GET\_FENG\_STATE(D_FENG);
```

18.9.3 Filter Engine HAL Reference Function Documentation**18.9.3.1 Sys_Set_FENG_Coefficients**

```
void Sys_Set_FENG_Coefficients(unsigned int cset, int32_t c0, int32_t c1, int32_t c2,
int32_t c3, int32_t c4)
```

Configure one set of Filter Engine coefficients for use with the Filter Engine BIQ and FIR instructions.

Location: feng.h:202

Parameters

Direction	Name	Description
in	<i>cset</i>	Index of the coefficient set to be updated
in	<i>c0</i>	Coefficient c0 from the given coefficient set (c0 to c4 for a FIR filter; b0 to b2, a0 to a1 for a BIQ filter); use constants defined using Sys_m2p26 for BIQ filters, Sys_frac28 (m1p27) for FIR filters
in	<i>c1</i>	Coefficient c1 from the given coefficient set
in	<i>c2</i>	Coefficient c2 from the given coefficient set
in	<i>c3</i>	Coefficient c3 from the given coefficient set
in	<i>c4</i>	Coefficient c4 from the given coefficient set

Example Code for Sys_Set_FENG_Coefficients

```
/* Configure the second coefficient set for a passthrough biquad
 * filter */
Sys\_Set\_FENG\_Coefficients(2, 0x7FFFFFFF, 0, 0, 0, 0);
```

18.10 FIFO

FIFO Hardware Abstraction Layer.

18.10.1 Summary

Macros

- [D_FIFO_Type](#) : Default FIFO type for type casting.
- [SYS_FIFO_AUTOMUTE](#) : Configure the auto-mute feature of one of the system FIFOs.
- [SYS_FIFO_CFXINTCONFIG](#) : Configure source for a CFX FIFO interrupt.
- [SYS_FIFO_HEARINTCONFIG](#) : Configure source for a HEAR FIFO interrupt.
- [SYS_FIFO_CM3INTCONFIG](#) : Configure source for an ARM Cortex-M3 FIFO interrupt.
- [SYS_FIFO_CONFIGURE](#) : Compatibility macro to allow any four FIFO types to be passed in for configuration.

18.10.2 FIFO Macro Definition Documentation

18.10.2.1 D_FIFO_Type

```
#define D_FIFO_Type D_FIFO_A0_Type
```

Default FIFO type for type casting.

Location: fifo.h:39

18.10.2.2 SYS_FIFO_AUTOMUTE

```
#define SYS_FIFO_AUTOMUTE fifo->AUTO_MUTE = (config)
```

Configure the auto-mute feature of one of the system FIFOs.

Location: fifo.h:172

Parameters

Ezairo 8300 Firmware Reference

Direction	Name	Description
in	<i>fifo</i>	FIFO instance to configure; use one of the D_FIFO_** blocks
in	<i>config</i>	The FIFO automute configuration; use FIFO_**_AUTO_MUTE_[DISABLE ENABLE]

Example Code for SYS_FIFO_AUTOMUTE

```
// Disable auto-mute in the D_FIFO_A0_2 block
SYS_FIFO_AUTOMUTE(D_FIFO_A0_2, FIFO_A0_AUTO_MUTE_DISABLE);
```

18.10.2.3 SYS_FIFO_CFXINTCONFIG

```
#define SYS_FIFO_CFXINTCONFIG D_FIFO_INT->CFG[(int_index)] = (fifo_int)
```

Configure source for a CFX FIFO interrupt.

Location: fifo.h:183

Parameters

Direction	Name	Description
in	<i>int_index</i>	Interrupt index to configure; use a constant 0-7
in	<i>fifo_int</i>	FIFO to use as interrupt source; use FIFO_INT_A0_*, FIFO_INT_A1_*, FIFO_INT_B0_*, or FIFO_INT_B1_*

Example Code for SYS_FIFO_CFXINTCONFIG

```
// Configure CFX interrupt index 3 to use
// FIFO_INT_B0_0 as interrupt source
SYS_FIFO_CFXINTCONFIG(3, FIFO_INT_B0_0);
```

Ezairo 8300 Firmware Reference

18.10.2.4 SYS_FIFO_HEARINTCONFIG

```
#define SYS_FIFO_HEARINTCONFIG D_FIFO_INT->HEAR_CFG[(int_index)] = (fifo_int)
```

Configure source for a HEAR FIFO interrupt.

Location: fifo.h:195

Parameters

Direction	Name	Description
in	<i>int_index</i>	Interrupt index to configure; use a constant 0-7
in	<i>fifo_int</i>	FIFO to use as interrupt source; use FIFO_INT_A0_*, FIFO_INT_A1_*, FIFO_INT_B0_*, or FIFO_INT_B1_*

Example Code for SYS_FIFO_HEARINTCONFIG

```
// Configure HEAR interrupt index 2 to use
// FIFO_INT_A1_0 as interrupt source
SYS_FIFO_HEARINTCONFIG(2, FIFO_INT_A1_0);
```

18.10.2.5 SYS_FIFO_CM3INTCONFIG

```
#define SYS_FIFO_CM3INTCONFIG D_FIFO_INT->CM3_CFG[(int_index)] = (fifo_int)
```

Configure source for an ARM Cortex-M3 FIFO interrupt.

Location: fifo.h:207

Parameters

Direction	Name	Description
in	<i>int_index</i>	Interrupt index to configure; use a constant 0-7
in	<i>fifo_int</i>	FIFO to use as interrupt source; use FIFO_INT_A0_*, FIFO_INT_A1_*, FIFO_INT_B0_*, or FIFO_INT_B1_*

Example Code for SYS_FIFO_CM3INTCONFIG

```
// Configure Cortex-M3 interrupt index 4 to use  
// FIFO_INT_B1_0 as interrupt source  
SYS\_FIFO\_CM3INTCONFIG(4, FIFO_INT_B1_0);
```

18.10.2.6 SYS_FIFO_CONFIGURE

```
#define SYS_FIFO_CONFIGURE Sys_FIFO_Configure((D\_FIFO\_Type*) (fifo), \  
                                              (start), (fifo_size), \  
                                              (block_size), (base_ptr), \  
                                              (ioblock_ptr))
```

Compatibility macro to allow any four FIFO types to be passed in for configuration.

Location: fifo.h:233

Parameters

Direction	Name	Description
in	<i>fifo</i>	One of D_FIFO_A0_*, D_FIFO_A1_*, D_FIFO_B0_*, or D_FIFO_B1_*
in	<i>start</i>	Start offset for the FIFO
in	<i>fifo_size</i>	Size of the FIFO
in	<i>block_size</i>	Size of a block in the FIFO
in	<i>base_ptr</i>	Initial value for the BASE pointer
in	<i>ioblock_ptr</i>	Initial value for the IOBLOCK pointer

Assumptions

The FIFO start offset is a multiple of 32

Assumptions

Ezairo 8300 Firmware Reference

The FIFO size is a multiple of 32 and is not larger than 2048

Assumptions

The FIFO block size is a multiple of 2 and is not larger than 1024

Assumptions

The base and IO block pointers are multiples of 2 and point to locations between FIFO start and FIFO start + FIFO size.

Example Code for SYS_FIFO_CONFIGURE

```
// Configure D_FIFO_A0_1 for use in the application
#define FIFO_INSTANCE      D_FIFO_A0_1
#define FIFO_START         32
#define FIFO_SIZE          64
#define FIFO_BLOCK_SIZE    8
#define FIFO_BASE_PTR      2
#define FIFO_IOBLOCK_PTR   4
Sys_FIFO_Configure(FIFO_INSTANCE, FIFO_START, FIFO_SIZE, FIFO_BLOCK_SIZE,
                  FIFO_BASE_PTR, FIFO_IOBLOCK_PTR);
```

18.11 G722

G722 Hardware Abstraction Layer.

18.11.1 Summary

Macros

- [G722_CFG_MASK](#) : Mask to ensure only the configuration bits are being set.
- [SYS_G722_CONFIG](#) : Configure the G722 block.
- [SYS_G722_INT_CONFIG](#) : Configure the G722 interrupt mask register.
- [SYS_G722_CLEAR_STATES](#) : Clear all G.722 codec states.
- [SYS_G722_TRUNCATE_TO_48](#) : Convert 8-bit samples to 6-bit samples for a 48 kbps rate.
- [SYS_G722_TRUNCATE_TO_56](#) : Convert 8-bit samples to 7-bit samples for a 56 kbps rate.

Functions

Ezairo 8300 Firmware Reference

- [Sys G722 ExtendFrom48](#) : Convert 6-bit samples to 8-bit samples for a 64 kbps rate.
- [Sys G722 ExtendFrom56](#) : Convert 7-bit samples to 8-bit samples for a 64 kbps rate.

18.11.2 G722 Macro Definition Documentation

18.11.2.1 G722_CFG_MASK

```
#define G722_CFG_MASK ((1U << G722_CFG_DEC_OUT_FORMAT_Pos) | \
                    (1U << G722_CFG_DEC_FB_BYPASS_Pos) | \
                    (1U << G722_CFG_DEC_CONCL_EBL_Pos) | \
                    (1U << G722_CFG_DEC_ENABLE_Pos) | \
                    (1U << G722_CFG_ENC_IN_FORMAT_Pos) | \
                    (1U << G722_CFG_ENC_FB_BYPASS_Pos) | \
                    (1U << G722_CFG_ENC_ENABLE_Pos))
```

Mask to ensure only the configuration bits are being set.

Location: g722.h:39

18.11.2.2 SYS_G722_CONFIG

```
#define SYS_G722_CONFIG (g722->CFG = (config))
```

Configure the G722 block.

Location: g722.h:60

Parameters

Direction	Name	Description
in	<i>g722</i>	Pointer to G722 block
in	<i>config</i>	Peripheral operation configuration; use G722_ENC_ENABLE DISABLE, G722_ENC_FB_ENABLE BYPASS, G722_ENC_IN_[16BIT 15BIT], G722_DEC_ENABLE DISABLE, G722_CONCL_ENABLE DISABLE, G722_DEC_FB_ENABLE BYPASS, G722_DEC_OUT_[16BIT 15BIT],

Ezairo 8300 Firmware Reference

Example Code for SYS_G722_CONFIG

```
// Configure the G722 block:
// - Enable G722 block
// - Set input data as mlp15 format
// - Enable G722 decoder
// - Set mlp14 output data to be aligned to mlp15 format (zero padded)
SYS_G722_CONFIG(G722, G722_ENC_ENABLE | G722_ENC_IN_16BIT |
                G722_DEC_ENABLE | G722_DEC_OUT_16BIT);
```

18.11.2.3 SYS_G722_INT_CONFIG

```
#define SYS_G722_INT_CONFIG (g722->INT_ENABLE = (config))
```

Configure the G722 interrupt mask register.

Location: g722.h:71

Parameters

Direction	Name	Description
in	<i>g722</i>	Pointer to G722 block
in	<i>config</i>	Peripheral operation configuration; use G722_ENC_IN_ERR_INT_[ENABLE DISABLE], G722_DEC_IN_ERR_INT_[ENABLE DISABLE], G722_DEC_CONCEAL_RDY_INT_[ENABLE DISABLE]

Example Code for SYS_G722_INT_CONFIG

```
// Configure G722 block interrupts
SYS_G722_INT_CONFIG(G722, G722_ENC_IN_ERR_INT_ENABLE |
                    G722_DEC_IN_ERR_INT_DISABLE |
                    G722_DEC_CONCEAL_RDY_INT_ENABLE);
```

18.11.2.4 SYS_G722_CLEAR_STATES

```
#define SYS_G722_CLEAR_STATES g722->CTRL = G722_DEC_SBBND_CLEAR | \
                                           G722_DEC_FB_CLEAR      | \
                                           G722_ENC_SBBND_CLEAR   | \
                                           G722_ENC_FB_CLEAR      | \
                                           G722_DEC_LSB_CNCL_CLEAR
```

Clear all G.722 codec states.

Location: g722.h:78

Parameters

Direction	Name	Description
in	<i>g722</i>	Pointer to G722 block

Example Code for SYS_G722_CLEAR_STATES

```
// Clear all G.722 codec states  
SYS\_G722\_CLEAR\_STATES(G722);
```

18.11.2.5 SYS_G722_TRUNCATE_TO_48

```
#define SYS_G722_TRUNCATE_TO_48 (value >> 2)
```

Convert 8-bit samples to 6-bit samples for a 48 kbps rate.

Location: g722.h:90

Parameters

Direction	Name	Description
in	<i>value</i>	8-bits sample

Return

Return value 6-bits sample

Ezairo 8300 Firmware Reference

Example Code for SYS_G722_TRUNCATE_TO_48

```
// Convert 8-bit sample to 6-bit sample for a 48 kbps rate
out_data = SYS\_G722\_TRUNCATE\_TO\_48(0xAA);
```

18.11.2.6 SYS_G722_TRUNCATE_TO_56

```
#define SYS_G722_TRUNCATE_TO_56 (value >> 1)
```

Convert 8-bit samples to 7-bit samples for a 56 kbps rate.

Location: g722.h:98

Parameters

Direction	Name	Description
in	<i>value</i>	8-bits sample

Return

Return value 7-bits sample

Example Code for SYS_G722_TRUNCATE_TO_56

```
// Convert 8-bit sample to 7-bit sample for a 56 kbps rate
out_data = SYS\_G722\_TRUNCATE\_TO\_56(0xAA);
```

18.11.3 G722 Function Documentation**18.11.3.1 Sys_G722_ExtendFrom48**

```
uint8_t Sys_G722_ExtendFrom48(uint8_t value)
```

Convert 6-bit samples to 8-bit samples for a 64 kbps rate.

Location: g722.h:159

Parameters

Direction	Name	Description
in	<i>value</i>	6-bits sample

Return

Return value 8-bits sample

Assumptions

Encoder rate is 48 kbps

Example Code for Sys_G722_ExtendFrom48

```
// Convert 6-bit sample to 8-bit sample for a 64 kbps rate;  
out_data = Sys\_G722\_ExtendFrom48(0x2A);
```

18.11.3.2 Sys_G722_ExtendFrom56

```
uint8_t Sys_G722_ExtendFrom56(uint8_t value)
```

Convert 7-bit samples to 8-bit samples for a 64 kbps rate.

Location: g722.h:191

Parameters

Ezairo 8300 Firmware Reference

Direction	Name	Description
in	<i>value</i>	7-bits sample

Return

Return value 8-bits sample

Assumptions

Encoder rate is 56 kbps

Example Code for Sys_G722_ExtendFrom56

```
// Convert 7-bit sample to 8-bit sample for a 64 kbps rate;
out_data = Sys_G722_ExtendFrom56(0x55);
```

18.12 GPIO

GPIO Hardware Abstraction Layer.

18.12.1 Summary**Data Structures**

- [GPIO_OUT0_Type](#) : GPIO outputs (low range)
- [GPIO_OUT1_Type](#) : GPIO outputs (high range)
- [GPIO_IN0_Type](#) : GPIO inputs (low range)
- [GPIO_IN1_Type](#) : GPIO inputs (high range)

Macros

- [DIO_DATA_IN_0_BASE](#) : Reference to the bit-band alias address of DIO->INPUT_DATA_0.
- [DIO_DATA_IN_1_BASE](#) : Reference to the bit-band alias address of DIO->INPUT_DATA_1.
- [DIO_DATA_IN_0](#) : [GPIO_IN0_Type](#) pointer to DIO_DATA_IN_0_BASE.

Ezairo 8300 Firmware Reference

- [DIO_DATA_IN_1](#) : [GPIO_IN1_Type](#) pointer to DIO_DATA_IN_1_BASE.
- [DIO_DATA_OUT_0_BASE](#) : Reference to the bit-band alias address of DIO->GPIO_OUTPUT_DATA_0.
- [DIO_DATA_OUT_1_BASE](#) : Reference to the bit-band alias address of DIO->GPIO_OUTPUT_DATA_1.
- [DIO_DATA_OUT_0](#) : [GPIO_IN1_Type](#) pointer to DIO_DATA_OUT_0_BASE.
- [DIO_DATA_OUT_1](#) : [GPIO_IN1_Type](#) pointer to DIO_DATA_OUT_1_BASE.

Functions

- [Sys_GPIO_Set_High](#) : Set the specified GPIO output value to high.
- [Sys_GPIO_Set_Low](#) : Set the specified GPIO output value to low.
- [Sys_GPIO_Toggle](#) : Toggle the current value of the specified GPIO output.
- [Sys_GPIO_Read](#) : Read the specified GPIO value.
- [Sys_GPIO_Write](#) : Write the specified GPIO value.
- [Sys_GPIO_Set_Direction](#) : Set the input/output direction for any DIOs configured as GPIOs.

18.12.2 GPIO Data Structures Type Documentation**18.12.2.1 GPIO_OUT0_Type**

Location: gpio.h:39

GPIO outputs (low range)

Data Fields

Type	Name	Description
<code>__IO uint32_t</code>	<i>ALIAS[DIO_GROUP_LOW_PAD_RANGE]</i>	Alias.

18.12.2.2 GPIO_OUT1_Type

Location: gpio.h:45

GPIO outputs (high range)

Data Fields

Ezairo 8300 Firmware Reference

Type	Name	Description
<code>__IO uint32_t</code>	<i>ALIAS[DIO_PAD_COUNT-DIO_GROUP_LOW_PAD_RANGE]</i>	Alias.

18.12.2.3 GPIO_IN0_Type

Location: gpio.h:51

GPIO inputs (low range)

Data Fields

Type	Name	Description
<code>__IO uint32_t</code>	<i>ALIAS[DIO_GROUP_LOW_PAD_RANGE]</i>	Alias.

18.12.2.4 GPIO_IN1_Type

Location: gpio.h:57

GPIO inputs (high range)

Data Fields

Type	Name	Description
<code>__IO uint32_t</code>	<i>ALIAS[DIO_PAD_COUNT-DIO_GROUP_LOW_PAD_RANGE]</i>	Alias.

18.12.3 GPIO Macro Definition Documentation

18.12.3.1 DIO_DATA_IN_0_BASE

```
#define DIO_DATA_IN_0_BASE SYS_CALC_BITBAND((uint32_t)&DIO->INPUT_DATA_0, 0)
```

Reference to the bit-band alias address of DIO->INPUT_DATA_0.

Location: gpio.h:65

18.12.3.2 DIO_DATA_IN_1_BASE

```
#define DIO_DATA_IN_1_BASE SYS_CALC_BITBAND((uint32_t)&DIO->INPUT_DATA_1, 0)
```

Reference to the bit-band alias address of DIO->INPUT_DATA_1.

Location: gpio.h:71

18.12.3.3 DIO_DATA_IN_0

```
#define DIO_DATA_IN_0 ((GPIO\_IN0\_Type *)DIO\_DATA\_IN\_0\_BASE)
```

[GPIO_IN0_Type](#) pointer to DIO_DATA_IN_0_BASE.

Location: gpio.h:77

18.12.3.4 DIO_DATA_IN_1

```
#define DIO_DATA_IN_1 ((GPIO\_IN1\_Type *)DIO\_DATA\_IN\_1\_BASE)
```

[GPIO_IN1_Type](#) pointer to DIO_DATA_IN_1_BASE.

Location: gpio.h:83

18.12.3.5 DIO_DATA_OUT_0_BASE

```
#define DIO_DATA_OUT_0_BASE SYS_CALC_BITBAND((uint32_t)&DIO->GPIO_OUTPUT_DATA_0, 0)
```

Reference to the bit-band alias address of DIO->GPIO_OUTPUT_DATA_0.

Location: gpio.h:89

18.12.3.6 DIO_DATA_OUT_1_BASE

```
#define DIO_DATA_OUT_1_BASE SYS_CALC_BITBAND((uint32_t)&DIO->GPIO_OUTPUT_DATA_1, 0)
```

Reference to the bit-band alias address of DIO->GPIO_OUTPUT_DATA_1.

Location: gpio.h:95

18.12.3.7 DIO_DATA_OUT_0

```
#define DIO_DATA_OUT_0 ((GPIO\_OUT0\_Type *)DIO\_DATA\_OUT\_0\_BASE)
```

[GPIO_IN1_Type](#) pointer to DIO_DATA_OUT_0_BASE.

Location: gpio.h:101

18.12.3.8 DIO_DATA_OUT_1

```
#define DIO_DATA_OUT_1 ((GPIO\_OUT1\_Type *)DIO\_DATA\_OUT\_1\_BASE)
```

[GPIO_IN1_Type](#) pointer to DIO_DATA_OUT_1_BASE.

Location: gpio.h:107

18.12.4 GPIO Function Documentation

18.12.4.1 Sys_GPIO_Set_High

```
void Sys_GPIO_Set_High(uint32_t pad)
```

Set the specified GPIO output value to high.

Location: gpio.h:116

Parameters

Ezairo 8300 Firmware Reference

Direction	Name	Description
in	<i>pad</i>	GPIO pad to set high

Example Code for Sys_GPIO_Set_High

```
// Set GPIO 26 value to high
Sys_GPIO_Set_High(26);
```

18.12.4.2 Sys_GPIO_Set_Low

```
void Sys_GPIO_Set_Low(uint32_t pad)
```

Set the specified GPIO output value to low.

Location: gpio.h:144

Parameters

Direction	Name	Description
in	<i>pad</i>	GPIO pad to set low

Example Code for Sys_GPIO_Set_Low

```
// Set GPIO 7 value to low
Sys_GPIO_Set_Low(7);
```

18.12.4.3 Sys_GPIO_Toggle

```
void Sys_GPIO_Toggle(uint32_t pad)
```

Toggle the current value of the specified GPIO output.

Location: gpio.h:172

Parameters

Direction	Name	Description
in	<i>pad</i>	GPIO pad to toggle

Example Code for Sys_GPIO_Toggle

```
// Toggle value of GPIO 12  
Sys\_GPIO\_Toggle(12);
```

18.12.4.4 Sys_GPIO_Read

```
uint32_t Sys_GPIO_Read(uint32_t pad)
```

Read the specified GPIO value.

Location: gpio.h:192

Parameters

Direction	Name	Description
in	<i>pad</i>	GPIO pad to set low

Return

pin value; 0 or 1

Example Code for Sys_GPIO_Read

```
// Read the value at GPIO 26  
reg = Sys_GPIO_Read(26);
```

18.12.4.5 Sys_GPIO_Write

```
void Sys_GPIO_Write(uint32_t pad, bool value)
```

Write the specified GPIO value.

Location: gpio.h:212

Parameters

Direction	Name	Description
in	<i>pad</i>	GPIO pad to write value
	<i>value</i>	

Example Code for Sys_GPIO_Write

```
// Set GPIO 28 value to high  
Sys_GPIO_Write(28, true);
```

18.12.4.6 Sys_GPIO_Set_Direction

```
void Sys_GPIO_Set_Direction(uint32_t dir0, uint32_t dir1)
```

Set the input/output direction for any DIOs configured as GPIOs.

Location: gpio.h:239

Parameters

Ezairo 8300 Firmware Reference

Direction	Name	Description
in	<i>dir0</i>	Input/output configuration settings for any DIOs from 0 to 23 that are configured as GPIOs; use GPIO*_INPUT, and GPIO*_OUTPUT
in	<i>dir1</i>	Input/output configuration settings for any DIOs from 24 to 35 that are configured as GPIOs; use GPIO*_INPUT, and GPIO*_OUTPUT

Example Code for Sys_GPIO_Set_Direction

```
// Configure GPIO 7 as output and GPIO 26 as input
Sys_GPIO_Set_Direction(GPIO7_OUTPUT, GPIO26_INPUT);
```

18.13 HEAR

HEAR Hardware Abstraction Layer.

18.13.1 Summary**Macros**

- [HEAR_VECT_BASE_OFFSET](#) : Offset to the base of the HEAR vector table.
- [SYS_HEAR_PAUSE](#) : Pause the HEAR Configurable Accelerator.
- [SYS_HEAR_START](#) : Start (or resume) the HEAR Configurable Accelerator.
- [SYS_HEAR_RESET](#) : Reset the HEAR Configurable Accelerator and the FIFO controllers.
- [SYS_HEAR_FUNCTIONCHAIN_ENABLE](#) : Enable specified HEAR function chains.
- [SYS_HEAR_FUNCTIONCHAIN_PRIORITY](#) : Set the priority of HEAR function chains.
- [SYS_SET_HEAR_BFP](#) : Configure the HEAR block floating point guard bits.
- [SYS_SET_HEAR_BFP_BLOCK_EXP](#) : Set the value of the block floating point (BFP) block exponent.
- [SYS_GET_HEAR_BFP_BLOCK_EXP](#) : Get the value of the block floating point (BFP) block exponent.

Functions

- [Sys_HEAR_Command](#) : Issue a command to execute a HEAR function chain.
- [Sys_Set_HEAR_FunctionChain_Vector](#) : Set a function chain vector to the specified function chain entry point.
- [Sys_Hear_Runtime_Addr](#) : Calculate the equivalent HEAR address to a address.

18.13.2 HEAR Macro Definition Documentation

18.13.2.1 HEAR_VECT_BASE_OFFSET

```
#define HEAR_VECT_BASE_OFFSET 0xFF80
```

Offset to the base of the HEAR vector table.

Location: hear.h:40

18.13.2.2 SYS_HEAR_PAUSE

```
#define SYS_HEAR_PAUSE d_system->CTRL = SYSTEM_HEAR_PAUSE
```

Pause the HEAR Configurable Accelerator.

Location: hear.h:47

Parameters

Direction	Name	Description
in	<i>d_system</i>	Pointer to D_SYSTEM_Type instance

Example Code for SYS_HEAR_PAUSE

```
// Pause the HEAR Configurable Accelerator  
SYS\_HEAR\_PAUSE(D_SYSTEM);
```

18.13.2.3 SYS_HEAR_START

```
#define SYS_HEAR_START d_system->CTRL = SYSTEM_HEAR_RESUME
```

Start (or resume) the HEAR Configurable Accelerator.

Location: hear.h:54

Parameters

Direction	Name	Description
in	<i>d_system</i>	Pointer to D_SYSTEM_Type instance

Example Code for SYS_HEAR_START

```
// Start (or resume) the HEAR Configurable Accelerator  
SYS\_HEAR\_START(D_SYSTEM);
```

18.13.2.4 SYS_HEAR_RESET

```
#define SYS_HEAR_RESET d_system->CTRL = SYSTEM_HEAR_RESET
```

Reset the HEAR Configurable Accelerator and the FIFO controllers.

Location: hear.h:62

Parameters

Direction	Name	Description
in	<i>d_system</i>	Pointer to D_SYSTEM_Type instance

Example Code for SYS_HEAR_RESET

```
// Reset the HEAR Configurable Accelerator and the FIFO controllers  
SYS\_HEAR\_RESET(D_SYSTEM);
```

18.13.2.5 SYS_HEAR_FUNCTIONCHAIN_ENABLE

```
#define SYS_HEAR_FUNCTIONCHAIN_ENABLE d_hear_fc->ENABLE = (function_ebl)
```

Enable specified HEAR function chains.

Location: hear.h:93

Parameters

Direction	Name	Description
in	<i>d_hear_fc</i>	Pointer to D_HEAR_FC_Type instance
in	<i>function_ebl</i>	HEAR function chains to enable

Assumptions

Previous function chain settings are no longer needed, as they are overwritten with this macro.

Example Code for SYS_HEAR_FUNCTIONCHAIN_ENABLE

```
// Enable CMD2 and CMD4 HEAR function chains  
SYS\_HEAR\_FUNCTIONCHAIN\_ENABLE(D_HEAR_FC, HEAR_FC_EBL_CMD4 |  
HEAR_FC_EBL_CMD2);
```

18.13.2.6 SYS_HEAR_FUNCTIONCHAIN_PRIORITY

```
#define SYS_HEAR_FUNCTIONCHAIN_PRIORITY d_hear_fc->PRIORITY = function_prio;
```

Set the priority of HEAR function chains.

Location: hear.h:105

Parameters

Direction	Name	Description
in	<i>d_hear_fc</i>	Pointer to D_HEAR_FC_Type instance
in	<i>function_prio</i>	HEAR function chains priority setting

Assumptions

Previous HEAR function chain priority settings are no longer needed as they are overwritten with this macro

Example Code for SYS_HEAR_FUNCTIONCHAIN_PRIORITY

```
// Set FIFO_4 HEAR function chain to high priority,  
// and CMD0 and CFX_14 to normal priority  
SYS\_HEAR\_FUNCTIONCHAIN\_PRIORITY(D_HEAR_FC, HEAR_FC_NORMALPRIO_CMD0 |  
                                HEAR_FC_NORMALPRIO_CFX_14 |  
                                HEAR_FC_HIGHPRIO_FIFO_4);
```

18.13.2.7 SYS_SET_HEAR_BFP

```
#define SYS_SET_HEAR_BFP d_hear_bfp->CFG = bfp
```

Configure the HEAR block floating point guard bits.

Location: hear.h:195

Parameters

Direction	Name	Description
in	<i>d_hear_bfp</i>	Pointer to D_HEAR_BFP_Type instance
in	<i>bfp</i>	Twopart parameter consisting of HEAR_BFP_FMD_[ENABLE DISABLE] and HEAR_BFP_FORMAT_*BITS

Example Code for SYS_SET_HEAR_BFP

```
// Enable and configure the HEAR block floating point  
// guard to 22-bit format  
SYS\_SET\_HEAR\_BFP(D_HEAR_BFP, HEAR_BFP_FMD_ENABLE |  
                  HEAR_BFP_FORMAT_22BITS);
```

18.13.2.8 SYS_SET_HEAR_BFP_BLOCK_EXP

```
#define SYS_SET_HEAR_BFP_BLOCK_EXP d_hear_bfp->BLOCK_EXP = value
```

Set the value of the block floating point (BFP) block exponent.

Location: hear.h:309

Parameters

Direction	Name	Description
in	<i>d_hear_bfp</i>	Pointer to D_HEAR_BFP_Type instance
in	<i>value</i>	The value to be written to the BFP block exponent

Example Code for SYS_SET_HEAR_BFP_BLOCK_EXP

```
// Set the value of the block floating point (BFP) block  
// exponent to 0x05  
SYS\_SET\_HEAR\_BFP\_BLOCK\_EXP(D_HEAR_BFP, 0x05);
```

18.13.2.9 SYS_GET_HEAR_BFP_BLOCK_EXP

```
#define SYS_GET_HEAR_BFP_BLOCK_EXP d_hear_bfp->BLOCK_EXP
```

Get the value of the block floating point (BFP) block exponent.

Location: hear.h:319

Parameters

Direction	Name	Description
in	<i>d_hear_bfp</i>	Pointer to D_HEAR_BFP_Type instance

Return

HEAR BFP block exponent value

Example Code for SYS_GET_HEAR_BFP_BLOCK_EXP

```
// Get the value of the block floating point (BFP) block  
// exponent  
exp = SYS\_GET\_HEAR\_BFP\_BLOCK\_EXP(D_HEAR_BFP);
```

18.13.3 HEAR Function Documentation

18.13.3.1 Sys_HEAR_Command

```
void Sys_HEAR_Command(D_HEAR_Type * d_hear, uint32_t cmd)
```

Issue a command to execute a HEAR function chain.

Location: hear.h:71

Parameters

Direction	Name	Description
in	<i>d_hear</i>	Pointer to D_HEAR_type instance
in	<i>cmd</i>	Command to issue to the HEAR; use a constant between 0 and 15 or HEAR_FC_CMD*_POS

Example Code for Sys_HEAR_Command

```
// Issue HEAR_CMD_13 to execute a HEAR function chain  
Sys\_HEAR\_Command(D_HEAR, HEAR_CMD_13);
```

18.13.3.2 Sys_Set_HEAR_FunctionChain_Vector

```
void Sys_Set_HEAR_FunctionChain_Vector(uint32_t fc_pos, uint32_t fc_entry)
```

onsemi Confidential
Ezairo 8300 Firmware Reference

Set a function chain vector to the specified function chain entry point.

Location: hear.h:120

Parameters

Direction	Name	Description
in	<i>fc_pos</i>	HEAR function chain vector to set; use HEAR_FC_*_POS
in	<i>fc_entry</i>	HEAR function chain entry point; use the _entry symbols generated by the HCT (or the location in HEAR_PRAM_BASE)

Assumptions

Previous HEAR function chain vector is no longer needed

Example Code for Sys_Set_HEAR_FunctionChain_Vector

```
// Arbitrary function chain entry point
#define unbound_function_chain_entry ( 0x8000 )
// Set the function chain vector for command 2 to
// a previously unbound function chain.
Sys\_Set\_HEAR\_FunctionChain\_Vector(HEAR_FC_CMD2_POS,
                                   unbound_function_chain_entry );
```

18.13.3.3 Sys_Hear_Runtime_Addr

```
uint32_t Sys_Hear_Runtime_Addr(uint32_t addr)
```

Calculate the equivalent HEAR address to a address.

Location: hear.h:143

Parameters

Ezairo 8300 Firmware Reference

Direction	Name	Description
in	<i>addr</i>	CFX address for a shared memory; use D_HEAR_*MEM_BASE, HEAR_*MEM_*_BASE, or a constant mapped into one of these memory mappings

Return

HEAR address for the specified shared memory

Example Code for Sys_Hear_Runtime_Addr

```
// Calculate the equivalent HEAR address
uint32_t addr = Sys\_Hear\_Runtime\_Addr(D_HEAR_A1MEM_BASE);
```

18.14 I2C

I2C Hardware Abstraction Layer.

18.14.1 Summary**Macros**

- [I2C_CONFIG_MASK](#) : I2C configuration mask.
- [I2C_PADS_NUM](#) : Number of pads used for the I2C interface.
- [SYS_I2C_DIOCONFIG](#) : Macro wrapper for [Sys_I2C_DIOConfig\(\)](#) Configure two DIOs for the specified I2C interface.
- [SYS_I2C_CONFIG](#) : Macro wrapper for [Sys_I2C_Config\(\)](#) Apply I2C Master mode related configuration.
- [SYS_I2C_STARTREAD](#) : Macro wrapper for [Sys_I2C_StartRead\(\)](#) Send slave address on the bus with a read request.
- [SYS_I2C_STARTWRITE](#) : Macro wrapper for [Sys_I2C_StartWrite\(\)](#) Send slave address on the bus with a write request.
- [SYS_I2C_ACK](#) : Macro wrapper for [Sys_I2C_ACK\(\)](#) Issue a ACK on the I2C interface.
- [SYS_I2C_NACK](#) : Macro wrapper for [Sys_I2C_NACK\(\)](#) Issue a NACK on the I2C interface.
- [SYS_I2C_LASTDATA](#) : Macro wrapper for [Sys_I2C_LastData\(\)](#) Indicate that the current data is the last byte.
- [SYS_I2C_RESET](#) : Macro wrapper for [Sys_I2C_Reset\(\)](#) Reset the I2C interface.
- [SYS_I2C_NACKANDSTOP](#) : Macro wrapper for [Sys_I2C_NackAndStop\(\)](#) Issue a NACK followed by a Stop condition on I2C bus.

Ezairo 8300 Firmware Reference

Functions

- [Sys I2C DIOConfig](#) : Configure two DIOs for the specified I2C interface.
- [Sys I2C Config](#) : Apply I2C Master mode related configuration.
- [Sys I2C StartRead](#) : Send slave address on the bus with a read request.
- [Sys I2C StartWrite](#) : Send slave address on the bus with a write request.
- [Sys I2C ACK](#) : Issue a ACK on the I2C interface.
- [Sys I2C NACK](#) : Issue a NACK on the I2C interface.
- [Sys I2C LastData](#) : Indicate that the current data is the last byte.
- [Sys I2C Reset](#) : Reset the I2C interface.
- [Sys I2C NackAndStop](#) : Issue a NACK followed by a Stop condition on I2C bus.

18.14.2 I2C Macro Definition Documentation**18.14.2.1 I2C_CONFIG_MASK**

```
#define I2C_CONFIG_MASK (((uint32_t)1U << I2C_CFG_CONNECT_IN_STANDBY_Pos) | \
    ((uint32_t)1U << I2C_CFG_TX_DMA_ENABLE_Pos) | \
    ((uint32_t)1U << I2C_CFG_RX_DMA_ENABLE_Pos) | \
    ((uint32_t)1U << I2C_CFG_TX_INT_ENABLE_Pos) | \
    ((uint32_t)1U << I2C_CFG_RX_INT_ENABLE_Pos) | \
    ((uint32_t)1U << I2C_CFG_BUS_ERROR_INT_ENABLE_Pos) | \
    (1U << I2C_CFG_OVERRUN_INT_ENABLE_Pos) | \
    (1U << I2C_CFG_STOP_INT_ENABLE_Pos) | \
    (1U << I2C_CFG_AUTO_ACK_ENABLE_Pos) | \
    I2C_CFG_SLAVE_PRESCALE_Mask | \
    I2C_CFG_MASTER_PRESCALE_Mask | \
    I2C_CFG_SLAVE_ADDRESS_Mask | \
    (1U << I2C_CFG_SLAVE_Pos))
```

I2C configuration mask.

Location: i2c.h:39

18.14.2.2 I2C_PADS_NUM

```
#define I2C_PADS_NUM 2U
```

Number of pads used for the I2C interface.

Location: i2c.h:55

18.14.2.3 SYS_I2C_DIOCONFIG

```
#define SYS_I2C_DIOCONFIG Sys\_I2C\_DIOConfig(I2C, (config), (scl), (sda))
```

Macro wrapper for [Sys_I2C_DIOConfig\(\)](#) Configure two DIOs for the specified I2C interface.

Location: i2c.h:221

Parameters

Direction	Name	Description
in	<i>config</i>	DIO pin configuration for the I2C pads
in	<i>scl</i>	DIO to use as the I2C transmit pad; use an integer
in	<i>sda</i>	DIO to use as the I2C receive pad; use an integer

Assumptions

Note if D_I2C is passed as the I2C_Type pointer, scl and sda pads are fixed on the chip for this peripheral instance. Hence these values is ignored

Example Code for SYS_I2C_DIOCONFIG

```
// Configure DIO 3 and DIO4 as SCL and SDA, enable 1K pull-up resistors,  
// and disable low-pass filter for default I2C interface  
SYS\_I2C\_DIOCONFIG((DIO_LPF_DISABLE | DIO_1K_PULL_UP),  
                  DIO3, DIO4);
```

18.14.2.4 SYS_I2C_CONFIG

```
#define SYS_I2C_CONFIG Sys\_I2C\_Config(I2C, (config))
```

Macro wrapper for [Sys_I2C_Config\(\)](#) Apply I2C Master mode related configuration.

Location: i2c.h:244

Parameters

Ezairo 8300 Firmware Reference

Direction	Name	Description
in	<i>config</i>	I2C configurations for master mode; use I2C_[CONNECT DISCONNECT]_IN_STANDBY, I2C_TX_DMA_[ENABLE DISABLE] I2C_RX_DMA_[ENABLE DISABLE] I2C_TX_INT_[ENABLE DISABLE] I2C_RX_INT_[ENABLE DISABLE] I2C_BUS_ERROR_INT_[ENABLE DISABLE] I2C_OVERRUN_INT_[ENABLE DISABLE] I2C_STOP_INT_[ENABLE DISABLE] I2C_AUTO_ACK_[ENABLE DISABLE] I2C_MASTER_PRESCALE_*, I2C_SLAVE_PRESCALE_*, a slave address constant shifted to I2C_CTRL0_SLAVE_ADDRESS_Pos, I2C_SLAVE_[ENABLE DISABLE]

Example Code for SYS_I2C_CONFIG

```
// Apply I2C Master mode related configuration for default I2C interface
// Set up prescaler, auto acknowledge, interrupt, and slave enable
SYS_I2C_CONFIG((I2C_SLAVE_PRESCALE_4 |
                 I2C_AUTO_ACK_ENABLE |
                 I2C_RX_INT_ENABLE |
                 I2C_TX_INT_ENABLE |
                 I2C_STOP_INT_ENABLE |
                 I2C_OVERRUN_INT_ENABLE |
                 I2C_BUS_ERROR_INT_ENABLE |
                 I2C_SLAVE_ENABLE |
                 (64 << I2C_CFG_SLAVE_ADDRESS_Pos)));
```

18.14.2.5 SYS_I2C_STARTREAD

```
#define SYS_I2C_STARTREAD Sys I2C StartRead(I2C, (addr))
```

Macro wrapper for [Sys I2C StartRead\(\)](#) Send slave address on the bus with a read request.

Location: i2c.h:252

Parameters

Direction	Name	Description
in	<i>addr</i>	I2C address to use for write transaction

Ezairo 8300 Firmware Reference

Example Code for SYS_I2C_STARTREAD

```
// Send slave address 64 on the default I2C bus with a read request
SYS_I2C_STARTREAD(64);
```

18.14.2.6 SYS_I2C_STARTWRITE

```
#define SYS_I2C_STARTWRITE Sys_I2C_StartWrite(I2C, (addr))
```

Macro wrapper for [Sys_I2C_StartWrite\(\)](#) Send slave address on the bus with a write request.

Location: i2c.h:260

Parameters

Direction	Name	Description
in	<i>addr</i>	I2C address to use for write transaction

Example Code for SYS_I2C_STARTWRITE

```
// Send slave address 64 on the default I2C bus with a write request
SYS_I2C_STARTWRITE(64);
```

18.14.2.7 SYS_I2C_ACK

```
#define SYS_I2C_ACK Sys_I2C_ACK(I2C)
```

Macro wrapper for [Sys_I2C_ACK\(\)](#) Issue a ACK on the I2C interface.

Location: i2c.h:267

Example Code for SYS_I2C_ACK

```
// Issue a ACK on the default I2C interface  
SYS\_I2C\_ACK();
```

18.14.2.8 SYS_I2C_NACK

```
#define SYS_I2C_NACK Sys\_I2C\_NACK(I2C)
```

Macro wrapper for [Sys_I2C_NACK\(\)](#) Issue a NACK on the I2C interface.

Location: i2c.h:274

Example Code for SYS_I2C_NACK

```
// Issue a NACK on the default I2C interface  
SYS\_I2C\_NACK();
```

18.14.2.9 SYS_I2C_LASTDATA

```
#define SYS_I2C_LASTDATA Sys\_I2C\_LastData(I2C)
```

Macro wrapper for [Sys_I2C_LastData\(\)](#) Indicate that the current data is the last byte.

Location: i2c.h:281

Example Code for SYS_I2C_LASTDATA

```
//Indicate that the current data is the last byte  
SYS\_I2C\_LASTDATA();
```

18.14.2.10 SYS_I2C_RESET

```
#define SYS_I2C_RESET Sys\_I2C\_Reset(I2C)
```

Macro wrapper for [Sys_I2C_Reset\(\)](#) Reset the I2C interface.

Location: i2c.h:288

Example Code for SYS_I2C_RESET

```
// Reset the default I2C interface  
SYS\_I2C\_RESET();
```

18.14.2.11 SYS_I2C_NACKANDSTOP

```
#define SYS_I2C_NACKANDSTOP Sys\_I2C\_NackAndStop(I2C)
```

Macro wrapper for [Sys_I2C_NackAndStop\(\)](#) Issue a NACK followed by a Stop condition on I2C bus.

Location: i2c.h:295

Example Code for SYS_I2C_NACKANDSTOP

```
// Issue a NACK followed by a Stop condition on default I2C bus  
SYS\_I2C\_NACKANDSTOP();
```

18.14.3 I2C Function Documentation

18.14.3.1 Sys_I2C_DIOConfig

```
void Sys_I2C_DIOConfig(const I2C_Type * i2c, uint32_t config, uint32_t scl, uint32_t sda)
```

Configure two DIOs for the specified I2C interface.

Location: i2c.h:69

Ezairo 8300 Firmware Reference

Parameters

Direction	Name	Description
in	<i>i2c</i>	Pointer to the I2C instance
in	<i>config</i>	DIO pin configuration for the I2C pads
in	<i>scl</i>	DIO to use as the I2C transmit pad; use an integer
in	<i>sda</i>	DIO to use as the I2C receive pad; use an integer

Assumptions

If D_I2C is passed as the I2C_Type pointer, scl and sda pads are fixed on the chip for this peripheral instance. The scl, sda values are ignored in this use case.

Example Code for Sys_I2C_DIOConfig

```
// Configure DIO 3 and DIO4 as SCL and SDA, enable 1K pull-up resistors,
// and disable low-pass filter for I2C1 interface
Sys_I2C_DIOConfig(I2C1, (DIO_LPF_DISABLE | DIO_1K_PULL_UP),
                  DIO3, DIO4);
```

18.14.3.2 Sys_I2C_Config

```
void Sys_I2C_Config(I2C_Type * i2c, uint32_t config)
```

Apply I2C Master mode related configuration.

Location: i2c.h:121

Parameters

Ezairo 8300 Firmware Reference

Direction	Name	Description
in	<i>i2c</i>	Pointer to the I2C instance
in	<i>config</i>	I2C configurations for master mode; use I2C_[CONNECT DISCONNECT]_IN_STANDBY, I2C_TX_DMA_[ENABLE DISABLE] I2C_RX_DMA_[ENABLE DISABLE] I2C_TX_INT_[ENABLE DISABLE] I2C_RX_INT_[ENABLE DISABLE] I2C_BUS_ERROR_INT_[ENABLE DISABLE] I2C_OVERRUN_INT_[ENABLE DISABLE] I2C_STOP_INT_[ENABLE DISABLE] I2C_AUTO_ACK_[ENABLE DISABLE] I2C_MASTER_PRESCALE_*, I2C_SLAVE_PRESCALE_*, a slave address constant shifted to I2C_CTRL0_SLAVE_ADDRESS_Pos, I2C_SLAVE_[ENABLE DISABLE]

Example Code for Sys_I2C_Config

```
// Apply I2C Master mode related configuration for I2C1 interface
// Set up prescaler, auto acknowledge, interrupt, and slave enable
Sys_I2C_Config(I2C1, (I2C_SLAVE_PRESCALE_4 |
    I2C_AUTO_ACK_ENABLE |
    I2C_RX_INT_ENABLE |
    I2C_TX_INT_ENABLE |
    I2C_STOP_INT_ENABLE |
    I2C_OVERRUN_INT_ENABLE |
    I2C_BUS_ERROR_INT_ENABLE |
    I2C_SLAVE_ENABLE |
    (64 << I2C_CFG_SLAVE_ADDRESS_Pos)));
```

18.14.3.3 Sys_I2C_StartRead

```
void Sys_I2C_StartRead(I2C_Type * i2c, uint32_t addr)
```

Send slave address on the bus with a read request.

Location: i2c.h:133

Parameters

Ezairo 8300 Firmware Reference

Direction	Name	Description
in	<i>i2c</i>	Pointer to the I2C instance
in	<i>addr</i>	I2C address to use for write transaction

Example Code for Sys_I2C_StartRead

```
// Send slave address 64 on the I2C1 bus with a read request
Sys_I2C_StartRead(I2C1, 64);
```

18.14.3.4 Sys_I2C_StartWrite

```
void Sys_I2C_StartWrite(I2C_Type * i2c, uint32_t addr)
```

Send slave address on the bus with a write request.

Location: i2c.h:147

Parameters

Direction	Name	Description
in	<i>i2c</i>	I2C instance number
in	<i>addr</i>	I2C address to use for write transaction

Example Code for Sys_I2C_StartWrite

```
// Send slave address 64 on the I2C1 bus with a write request
Sys_I2C_StartWrite(I2C1, 64);
```

18.14.3.5 Sys_I2C_ACK

```
void Sys_I2C_ACK(I2C_Type * i2c)
```


Issue a ACK on the I2C interface.

Location: i2c.h:159

Parameters

Direction	Name	Description
in	<i>i2c</i>	Pointer to the I2C instance

Example Code for Sys_I2C_ACK

```
// Issue a ACK on the I2C interface  
Sys\_I2C\_ACK(I2C);
```

18.14.3.6 Sys_I2C_NACK

```
void Sys_I2C_NACK(I2C_Type * i2c)
```

Issue a NACK on the I2C interface.

Location: i2c.h:170

Parameters

Direction	Name	Description
in	<i>i2c</i>	Pointer to the I2C instance

Example Code for Sys_I2C_NACK

```
// Issue a NACK on the I2C1 interface  
Sys\_I2C\_NACK(I2C1);
```

18.14.3.7 Sys_I2C_LastData

```
void Sys_I2C_LastData(I2C_Type * i2c)
```

Indicate that the current data is the last byte.

Location: i2c.h:181

Parameters

Direction	Name	Description
in	<i>i2c</i>	Pointer to the I2C instance

Example Code for Sys_I2C_LastData

```
//Indicate that the current data is the last byte  
Sys\_I2C\_LastData(I2C);
```

18.14.3.8 Sys_I2C_Reset

```
void Sys_I2C_Reset(I2C_Type * i2c)
```

Reset the I2C interface.

Location: i2c.h:192

Parameters

Direction	Name	Description
in	<i>i2c</i>	Pointer to the I2C instance

Example Code for Sys_I2C_Reset

```
// Reset the I2C interface  
Sys\_I2C\_Reset(I2C);
```

18.14.3.9 Sys_I2C_NackAndStop

```
void Sys_I2C_NackAndStop(I2C_Type * i2c)
```

Issue a NACK followed by a Stop condition on I2C bus.

Location: i2c.h:203

Parameters

Direction	Name	Description
in	<i>i2c</i>	Pointer to the I2C instance

Example Code for Sys_I2C_NackAndStop

```
// Issue a NACK followed by a Stop condition on I2C1 bus  
Sys\_I2C\_NackAndStop(I2C1);
```

18.15 INTERRUPTS

Interrupt support Hardware Abstraction Layer.

18.15.1 Summary

Macros

- [SYS_CFX_INT_MASTER_ENABLE](#) : Enable all interrupts at the interrupt controller.
- [SYS_CFX_INT_MASTER_DISABLE](#) : Disable all interrupts at the interrupt controller.
- [SYS_CFX_INT_DISABLEALL](#) : Clear all of the interrupt enable bits.
- [SYS_CFX_INT_CLEARALLPENDING](#) : Manually clear any pending interrupts.

Ezairo 8300 Firmware Reference

- [SYS_CFX_INT_ENABLE](#) : Enable a group of interrupts from one of the three interrupt sets (vectors 0 to 31, 32 to 63, and 64 to 94)
- [SYS_CFX_INT_ACK](#) : Acknowledge group of interrupts from one of the three interrupt sets (vectors 0 to 31, 32 to 63, and 64 to 94)
- [SYS_CFX_INT_PRIORITY](#) : Configure the interrupt priority for one of the three interrupt sets (vectors 0 to 31, 32 to 63, and 64 to 94)

18.15.2 Interrupts Macro Definition Documentation

18.15.2.1 SYS_CFX_INT_MASTER_ENABLE

```
#define SYS_CFX_INT_MASTER_ENABLE d_int->MASTER_ENABLE = INT_EBL_MASTER
```

Enable all interrupts at the interrupt controller.

Location: interrupts.h:33

Parameters

Direction	Name	Description
in	<i>d_int</i>	Pointer to D_INT_Type instance

Example Code for SYS_CFX_INT_MASTER_ENABLE

```
// Enable all interrupts at the interrupt controller
SYS\_CFX\_INT\_MASTER\_ENABLE(D_INT);
```

18.15.2.2 SYS_CFX_INT_MASTER_DISABLE

```
#define SYS_CFX_INT_MASTER_DISABLE d_int->MASTER_ENABLE = INT_DIS_MASTER
```

Disable all interrupts at the interrupt controller.

Location: interrupts.h:42

Ezairo 8300 Firmware Reference

Parameters

Direction	Name	Description
in	<i>d_int</i>	Pointer to D_INT_Type instance

Example Code for SYS_CFX_INT_MASTER_DISABLE

```
// Disable all interrupts at the interrupt controller
SYS_CFX_INT_MASTER_DISABLE(D_INT);
```

18.15.2.3 SYS_CFX_INT_DISABLEALL

```
#define SYS_CFX_INT_DISABLEALL {\
    d_int->ENABLE_0 = 0;\
    d_int->ENABLE_1 = 0;\
    d_int->ENABLE_2 = 0;\
}
```

Clear all of the interrupt enable bits.

Location: interrupts.h:50

Parameters

Direction	Name	Description
in	<i>d_int</i>	Pointer to D_INT_Type instance

Example Code for SYS_CFX_INT_DISABLEALL

```
// Clear all of the interrupt enable bits
SYS_CFX_INT_DISABLEALL(D_INT);
```

18.15.2.4 SYS_CFX_INT_CLEARALLPENDING

```
#define SYS_CFX_INT_CLEARALLPENDING {\
    d_int->STATUS_0 = 0xFFFFFFFF;\
    d_int->STATUS_1 = 0xFFFFFFFF;\
    d_int->STATUS_2 = 0xFFFFFFFF;\
}
```

Manually clear any pending interrupts.

Location: interrupts.h:62

Parameters

Direction	Name	Description
in	<i>d_int</i>	Pointer to D_INT_Type instance

Example Code for SYS_CFX_INT_CLEARALLPENDING

```
// Manually clear any pending interrupts
SYS_CFX_INT_CLEARALLPENDING(D_INT);
```

18.15.2.5 SYS_CFX_INT_ENABLE

```
#define SYS_CFX_INT_ENABLE {\
    switch(int_set)\
    {\
        case 0:\
            d_int->ENABLE_0 = int_ebl;\
            break;\
        case 1:\
            d_int->ENABLE_1 = int_ebl;\
            break;\
        case 2:\
            d_int->ENABLE_2 = int_ebl;\
            break;\
        default:\
            SYS_ASSERT(0);\
    }\
}
```

Enable a group of interrupts from one of the three interrupt sets (vectors 0 to 31, 32 to 63, and 64 to 94)

Location: interrupts.h:80

Parameters

Direction	Name	Description
in	<i>d_int</i>	Pointer to D_INT_Type instance
in	<i>int_set</i>	Set of interrupts to enable; use 0, 1, or 2
in	<i>int_ebl</i>	Interrupts to enable

Assumptions

Previous interrupt settings are no longer needed, as they are overwritten with this macro

Example Code for SYS_CFX_INT_ENABLE

```
// Enable a group of interrupts in interrupt set 1
SYS_CFX_INT_ENABLE(D_INT, 1, INT_EBL_SPI_1_RX | INT_EBL_UART_0_RX |
    INT_EBL_ASRC_IN | INT_EBL_RS_DEC_IN_RDY);
```

18.15.2.6 SYS_CFX_INT_ACK

```
#define SYS_CFX_INT_ACK {\
    switch(int_set)\
    {\
        case 0:\
            d_int->STATUS_0 = int_ack;\
            break;\
        case 1:\
            d_int->STATUS_1 = int_ack;\
            break;\
        case 2:\
            d_int->STATUS_2 = int_ack;\
            break;\
        default:\
            SYS_ASSERT(0);\
    }\
}
```

Acknowledge group of interrupts from one of the three interrupt sets (vectors 0 to 31, 32 to 63, and 64 to 94)

Location: interrupts.h:109

Parameters

Direction	Name	Description
in	<i>d_int</i>	Pointer to D_INT_Type instance
in	<i>int_set</i>	Set of interrupts to enable; use 0, 1, or 2
in	<i>int_ack</i>	Interrupts to acknowledge

Assumptions

Previous interrupt settings are no longer needed, as they are overwritten with this macro

Example Code for SYS_CFX_INT_ACK

```
// Enable a group of interrupts in interrupt set 2
SYS_CFX_INT_ACK(D_INT, 2, INT_ACK_HEAR_1 |INT_ACK_DSP_0 |
                INT_ACK_CM3_4 |INT_ACK_LSAD);
```

18.15.2.7 SYS_CFX_INT_PRIORITY

```
#define SYS_CFX_INT_PRIORITY {\
    switch(int_set)\
    {\
        case 0:\
            d_int->PRIORITY_0 = int_prio;\
            break;\
        case 1:\
            d_int->PRIORITY_1 = int_prio;\
            break;\
        case 2:\
            d_int->PRIORITY_2 = int_prio;\
            break;\
        default:\
            SYS_ASSERT(0);\
    }\
}
```

Configure the interrupt priority for one of the three interrupt sets (vectors 0 to 31, 32 to 63, and 64 to 94)

Location: interrupts.h:138

Parameters

Direction	Name	Description
in	<i>d_int</i>	Pointer to D_INT_Type instance
in	<i>int_set</i>	Set of interrupts to enable; use 0, 1, or 2
in	<i>int_prio</i>	Interrupt priority configuration

Assumptions

Previous interrupt settings are no longer needed, as they are overwritten with this macro

Example Code for SYS_CFX_INT_PRIORITY

```
// Set priority of a group of interrupts in interrupt set 0
SYS_CFX_INT_PRIORITY(D_INT, 0, INT_HIGHPRIO_WATCHDOG |
    INT_NORMALPRIO_AUDIOSINK_0 |
    INT_HIGHPRIO_TIMER_3 |
    INT_HIGHPRIO_I2C_1);
```

18.16 IOC

IOC Hardware Abstraction Layer.

18.16.1 Summary

Macros

- [IOC_INPUTCFG_MASK](#) : IOC input mask.
- [IOC_INPUTPCMCFG_MASK](#) : IOC PCM input mask.
- [IOC_OUTPUTCFG_MASK](#) : IOC output mask.
- [IOC_FIFOACCESSCFG_MASK](#) : IOC FIFO access mask.
- [AUDIO_MUX_CFG_MASK](#) : Audio multiplexing mask.
- [IOC_FENG_CFG_MASK](#) : IOC filter engine configuration mask.
- [SYS_IOC_INPUTCONFIG](#) : Set up the IOC ADC and Filter Engine input configuration.
- [SYS_IOC_PCMINPUTCONFIG](#) : Set up the IOC PCM input configuration.

Ezairo 8300 Firmware Reference

- [SYS IOC OUTPUTCONFIG](#) : Set up the IOC output configuration.
- [SYS IOC FIFOACCESSCONFIG](#) : Set up the IOC FIFO access configuration for each FIFO; access is either one or two data values at a time.
- [SYS AUDIOMUX CONFIG](#) : Configure the audio multiplexers and transfer formats.
- [SYS IOC TRIGGER CONFIG](#) : Configure the trigger sources for the Filter Engine outputs to the IOCs, and Filter Engine inputs from IOC1.

18.16.2 IOC Macro Definition Documentation

18.16.2.1 IOC_INPUTCFG_MASK

```
#define IOC_INPUTCFG_MASK (D_IOC_INPUT_CFG_IN0_STORE_Mask | \
                          D_IOC_INPUT_CFG_IN1_STORE_Mask | \
                          D_IOC_INPUT_CFG_IN2_STORE_Mask | \
                          D_IOC_INPUT_CFG_IN3_STORE_Mask)
```

IOC input mask.

Location: ioc.h:39

18.16.2.2 IOC_INPUTPCMCFG_MASK

```
#define IOC_INPUTPCMCFG_MASK (D_IOC_INPUT_PCM_CFG_PCM_RX0_STORE_Mask | \
                              D_IOC_INPUT_PCM_CFG_PCM_RX1_STORE_Mask)
```

IOC PCM input mask.

Location: ioc.h:45

18.16.2.3 IOC_OUTPUTCFG_MASK

```
#define IOC_OUTPUTCFG_MASK (D_IOC_OUTPUT_CFG_OUT0_LOAD_Mask | \
                            D_IOC_OUTPUT_CFG_OUT1_LOAD_Mask | \
                            D_IOC_OUTPUT_CFG_PCM_TX0_LOAD_Mask | \
                            D_IOC_OUTPUT_CFG_PCM_TX1_LOAD_Mask)
```

IOC output mask.

Location: ioc.h:49

Ezairo 8300 Firmware Reference

18.16.2.4 IOC_FIFOACCESSCFG_MASK

```
#define IOC_FIFOACCESSCFG_MASK False
```

IOC FIFO access mask.

Location: ioc.h:55

18.16.2.5 AUDIO_MUX_CFG_MASK

```
#define AUDIO_MUX_CFG_MASK ((1U << AUDIO_MUX_CFG_SDM0_Pos) | \
    (1U << AUDIO_MUX_CFG_SDM1_Pos) | \
    (1U << AUDIO_MUX_CFG_INT_FILTER0_Pos) | \
    (1U << AUDIO_MUX_CFG_INT_FILTER1_Pos) | \
    (1U << AUDIO_MUX_CFG_IOC0_SRC0_Pos) | \
    (1U << AUDIO_MUX_CFG_IOC0_SRC1_Pos) | \
    (1U << AUDIO_MUX_CFG_IOC0_SRC2_Pos) | \
    (1U << AUDIO_MUX_CFG_IOC0_SRC3_Pos) | \
    (AUDIO_MUX_CFG_INPUT_CH0_SRC_Mask) | \
    (AUDIO_MUX_CFG_INPUT_CH1_SRC_Mask) | \
    (AUDIO_MUX_CFG_INPUT_CH2_SRC_Mask) | \
    (AUDIO_MUX_CFG_INPUT_CH3_SRC_Mask))
```

Audio multiplexing mask.

Location: ioc.h:90

18.16.2.6 IOC_FENG_CFG_MASK

```
#define IOC_FENG_CFG_MASK (AUDIO_IOC_FENG_CFG_IOC0_FENG_OUT0_TRIG_Mask | \
    AUDIO_IOC_FENG_CFG_IOC0_FENG_OUT1_TRIG_Mask | \
    AUDIO_IOC_FENG_CFG_IOC0_FENG_OUT2_TRIG_Mask | \
    AUDIO_IOC_FENG_CFG_IOC0_FENG_OUT3_TRIG_Mask | \
    AUDIO_IOC_FENG_CFG_IOC1_FENG_OUT4_TRIG_Mask | \
    AUDIO_IOC_FENG_CFG_IOC1_FENG_OUT5_TRIG_Mask | \
    AUDIO_IOC_FENG_CFG_IOC1_FENG_OUT6_TRIG_Mask | \
    AUDIO_IOC_FENG_CFG_IOC1_FENG_OUT7_TRIG_Mask | \
    AUDIO_IOC_FENG_CFG_IOC1_FENG_IN6_IN7_TRIG_Mask)
```

IOC filter engine configuration mask.

Location: ioc.h:105

18.16.2.7 SYS_IOC_INPUTCONFIG

```
#define SYS_IOC_INPUTCONFIG ioc->INPUT_CFG = (arg) & IOC\_INPUTCFG\_MASK
```

Ezairo 8300 Firmware Reference

Set up the IOC ADC and Filter Engine input configuration.

Location: ioc.h:123

Parameters

Direction	Name	Description
in	<i>ioc</i>	Pointer to the IOC instance
in	<i>arg</i>	Combination of IOC_INPUT_CFG_IN0_*, IOC_INPUT_CFG_IN1_*, IOC_INPUT_CFG_IN2_*, and IOC_INPUT_CFG_IN3_*

Example Code for SYS_IOC_INPUTCONFIG

```
// Set up the D_IOC0 ADC and Filter Engine input configuration
SYS_IOC_INPUTCONFIG(D_IOC0, (IOC_INPUT_CFG_IN0_FA0_0 |
    IOC_INPUT_CFG_IN1_FA1_1 |
    IOC_INPUT_CFG_IN2_FB0_2 |
    IOC_INPUT_CFG_IN3_FB1_3));
```

18.16.2.8 SYS_IOC_PCMINPUTCONFIG

```
#define SYS_IOC_PCMINPUTCONFIG ioc->INPUT_PCM_CFG = (arg) & IOC\_INPUTPCMCFG\_MASK
```

Set up the IOC PCM input configuration.

Location: ioc.h:133

Parameters

Direction	Name	Description
in	<i>ioc</i>	Pointer to the IOC instance
in	<i>arg</i>	Combination of IOC_INPUT_CFG_PCM_RX0_*, IOC_INPUT_CFG_PCM_RX1_*

Ezairo 8300 Firmware Reference

Example Code for SYS_IOC_PCMINPUTCONFIG

```
// Set up the D_IOC0 PCM input configuration
SYS_IOC_PCMINPUTCONFIG(D_IOC0, (IOC_INPUT_CFG_PCM_RX0_FB1_2 |
                                IOC_INPUT_CFG_PCM_RX1_FA0_6));
```

18.16.2.9 SYS_IOC_OUTPUTCONFIG

```
#define SYS_IOC_OUTPUTCONFIG ioc->OUTPUT_CFG = (arg) & IOC_OUTPUTCFG_MASK
```

Set up the IOC output configuration.

Location: ioc.h:146

Parameters

Direction	Name	Description
in	<i>ioc</i>	Pointer to the IOC instance
in	<i>arg</i>	Combination of IOC_OUTPUT_CFG_PCM_TX1_*, IOC_OUTPUT_CFG_PCM_TX0_*, IOC_OUTPUT_CFG_OUT1_*, IOC_OUTPUT_CFG_OUT0_*

Example Code for SYS_IOC_OUTPUTCONFIG

```
// Set up the D_IOC0 output configuration
SYS_IOC_OUTPUTCONFIG(D_IOC0, (IOC_OUTPUT_CFG_PCM_TX0_FA0_5 |
                                IOC_OUTPUT_CFG_PCM_TX1_FB1_3 |
                                IOC_OUTPUT_CFG_OUT0_NONE |
                                IOC_OUTPUT_CFG_OUT1_NONE));
```

18.16.2.10 SYS_IOC_FIFOACCESSCONFIG

```
#define SYS_IOC_FIFOACCESSCONFIG ioc->FIFO_ACCESS_CFG = (arg) & IOC_FIFOACCESSCFG_MASK
```

Set up the IOC FIFO access configuration for each FIFO; access is either one or two data values at a time.

Location: ioc.h:156

Parameters

Direction	Name	Description
in	<i>ioc</i>	Pointer to the IOC instance
in	<i>arg</i>	A set of FIFO_***_DBL_ACC_[DIS EN]

Example Code for SYS_IOC_FIFOACCESSCONFIG

```
// Set up the D_IOC0 FIFO access configuration for each FIFO;
// Enable simple or double access for specific FIFOs
SYS_IOC_FIFOACCESSCONFIG(D_IOC0, (FIFO_A0_5_DBL_ACC_EN |
    FIFO_A1_2_DBL_ACC_EN |
    FIFO_B0_1_DBL_ACC_DIS |
    FIFO_B0_7_DBL_ACC_EN) );
```

18.16.2.11 SYS_AUDIOMUX_CONFIG

```
#define SYS_AUDIOMUX_CONFIG audio->MUX_CFG = (config) & AUDIO\_MUX\_CFG\_MASK
```

Configure the audio multiplexers and transfer formats.

This selects the audio source for each input channel, selecting between the ADC, bypass register, and DMIC inputs. This also selects the IOC input sources, selecting between the input source and FENG. Finally this selects the output driver sources, selecting between the interpolation filter and the FENG.

Location: ioc.h:176

Parameters

Direction	Name	Description
in	<i>audio</i>	Pointer to the AUDIO block
in	<i>config</i>	Audio mux configuration; use a combination of [ADC*_OUT

Ezairo 8300 Firmware Reference

		BYPASS_DATA_REG* DMIC*_DATA_RE DMIC*_DATA_FE], IOC0_SRC*_ [DEC_FILTER FENG], INT_FILTER*_ [IOC0 FENG] and SDM*_ [INT_FILTER FENG]
--	--	---

Example Code for SYS_AUDIOMUX_CONFIG

```
// Configure the audio multiplexers and transfer formats:
// - Data is provided by the bypass register
// - The decimation filter 0 output is routed to the IOC0 audio input 0
(FENG is bypassed)
// - The IOC0 audio output 1 is routed to the interpolation filter 1 input
(FENG is bypassed)
// -The interpolation filter 0 output is routed to the SDM0 input (FENG is bypassed)
SYS\_AUDIOMUX\_CONFIG(AUDIO, (BYPASS_DATA_REG1 |
    IOC0_SRC0_DEC_FILTER |
    INT_FILTER1_IOC0 |
    SDM0_INT_FILTER));
```

18.16.2.12 SYS_IOC_TRIGGER_CONFIG

```
#define SYS_IOC_TRIGGER_CONFIG audio->IOC_FENG_CFG = (config) & IOC\_FENG\_CFG\_MASK
```

Configure the trigger sources for the Filter Engine outputs to the IOCs, and Filter Engine inputs from IOC1.

Location: ioc.h:188

Parameters

Direction	Name	Description
in	<i>audio</i>	Pointer to the AUDIO block
in	<i>config</i>	Audio mux configuration; use a set of IOC*_FENG_[OUT* IN6_IN7]_[DISABLED INT_FILTER_LDP ADC0_ADC2 ADC1_ADC3 ADC*_LDP]

Example Code for SYS_IOC_TRIGGER_CONFIG

```
// Configure Filter Engine output 1 to trigger on the ADC0/ADC2 signal,
// and Filter Engine input 6 to trigger on the ADC1 low-delay path signal.
SYS_IOC_TRIGGER_CONFIG(AUDIO, (IOC0_FENG_OUT0_ADC0_ADC2 |
                                IOC1_FENG_IN6_IN7_ADC1_LDP);
```

18.17 LPDSP32

LPDSP32 Hardware Abstraction Layer.

18.17.1 Summary**Macros**

- [SYS LPDSP32 PAUSE](#) : Pause the LPDSP32 core.
- [SYS LPDSP32 START](#) : Start (or resume) the LPDSP32.
- [SYS LPDSP32 RESET](#) : Reset the LPDSP32 core.
- [SYS LPDSP32 COMMAND](#) : Issue a command to the LPDSP32.
- [JTAG LPDSP32 TDI](#) : Default DIO for JTAG TDI.
- [JTAG LPDSP32 TMS](#) : Default DIO for JTAG TMS.
- [JTAG LPDSP32 TCK](#) : Default DIO for JTAG TCK.
- [JTAG LPDSP32 TDO](#) : Default DIO for JTAG TDO.
- [JTAG DIO CONFIG](#) : Macro wrapper for [JTAG DIO Config\(\)](#) Configure DIOs for JTAG (LPDSP32).

Functions

- [JTAG DIO Config](#) : Configure DIOs for JTAG (LPDSP32)

18.17.2 LPDSP32 Macro Definition Documentation**18.17.2.1 SYS_LPDSP32_PAUSE**

```
#define SYS_LPDSP32_PAUSE sysctrl->DSP_CTRL = DSP_PAUSE
```

Pause the LPDSP32 core.

Location: lpdsp32.h:43

Parameters

Ezairo 8300 Firmware Reference

Direction	Name	Description
in	<i>sysctrl</i>	Pointer to SYSCTRL block
Example Code for SYS_LPDSP32_PAUSE <pre>// Pause the LPDSP32 core SYS_LPDSP32_PAUSE(SYSCTRL);</pre>		

18.17.2.2 SYS_LPDSP32_START

```
#define SYS_LPDSP32_START sysctrl->DSP_CTRL = DSP_RESUME
```

Start (or resume) the LPDSP32.

Location: lpdsp32.h:50

Parameters

Direction	Name	Description
in	<i>sysctrl</i>	Pointer to SYSCTRL block
Example Code for SYS_LPDSP32_START <pre>// Start (or resume) the LPDSP32 SYS_LPDSP32_START(SYSCTRL);</pre>		

18.17.2.3 SYS_LPDSP32_RESET

```
#define SYS_LPDSP32_RESET sysctrl->DSP_CTRL = DSP_RESET
```

Reset the LPDSP32 core.

Location: lpdsp32.h:57

Parameters

Ezairo 8300 Firmware Reference

Direction	Name	Description
in	<i>sysctrl</i>	Pointer to SYSCTRL block
Example Code for SYS_LPDP32_RESET <pre>// Reset the LPDP32 core SYS_LPDP32_RESET(SYSCTRL);</pre>		

18.17.2.4 SYS_LPDP32_COMMAND

```
#define SYS_LPDP32_COMMAND sysctrl->DSP_CMD = (1 << (cmd))
```

Issue a command to the LPDP32.

Location: lpdsp32.h:66

Parameters

Direction	Name	Description
in	<i>sysctrl</i>	Pointer to SYSCTRL block
in	<i>cmd</i>	Command to issue to the LPDP32; use a constant between 0 and 5 or SYSCTRL_DSP_CMD_DSP_CMD_*_POS
Example Code for SYS_LPDP32_COMMAND <pre>// Issue DSP_CMD_1 to the LPDP32 SYS_LPDP32_COMMAND(SYSCTRL, DSP_CMD_1);</pre>		

18.17.2.5 JTAG_LPDP32_TDI

```
#define JTAG_LPDP32_TDI (DIO24)
```

Default DIO for JTAG TDI.

Location: lpdsp32.h:70

18.17.2.6 JTAG_LPDSP32_TMS

```
#define JTAG_LPDSP32_TMS (DIO25)
```

Default DIO for JTAG TMS.

Location: `lpdsp32.h:71`

18.17.2.7 JTAG_LPDSP32_TCK

```
#define JTAG_LPDSP32_TCK (DIO26)
```

Default DIO for JTAG TCK.

Location: `lpdsp32.h:72`

18.17.2.8 JTAG_LPDSP32_TDO

```
#define JTAG_LPDSP32_TDO (DIO27)
```

Default DIO for JTAG TDO.

Location: `lpdsp32.h:73`

18.17.2.9 JTAG_DIO_CONFIG

```
#define JTAG_DIO_CONFIG JTAG\_DIO\_Config(JTAG\_LPDSP32\_TDI, JTAG\_LPDSP32\_TMS, \  
                                         JTAG\_LPDSP32\_TCK, JTAG\_LPDSP32\_TDO)
```

Macro wrapper for [JTAG_DIO_Config\(\)](#) Configure DIOs for JTAG (LPDSP32).

Uses predefined default DIOs.

Location: `lpdsp32.h:113`

Example Code for JTAG_DIO_CONFIG

```
// Configure the LPDSP32 debug port to use the default pads */
JTAG_DIO_CONFIG();
```

18.17.3 LPDSP32 Function Documentation

18.17.3.1 JTAG_DIO_Config

```
void JTAG_DIO_Config(uint32_t tddio, uint32_t tmsdio, uint32_t tckdio, uint32_t tdodio)
```

Configure DIOs for JTAG (LPDSP32)

Location: lpdsp32.h:83

Parameters

Direction	Name	Description
in	<i>tdidio</i>	TDI DIO
in	<i>tmsdio</i>	TMS DIO
in	<i>tckdio</i>	TCK DIO
in	<i>tdodio</i>	TDO DIO

Example Code for JTAG_DIO_Config

```
// Configure the LPDSP32 debug port */
JTAG_DIO_CONFIG(JTAG_LPDSP32_TDI, JTAG_LPDSP32_TMS,
                JTAG_LPDSP32_TCK, JTAG_LPDSP32_TDO);
```

18.18 LSAD

LSAD Hardware Abstraction Layer.

18.18.1 Summary

Macros

- [SYS_LSAD_CONFIG](#) : Configure the LSAD operation and interrupts.

Functions

- [Sys_LSAD_InputConfig](#) : Configure LSAD input channel.

18.18.2 LSAD Macro Definition Documentation

18.18.2.1 SYS_LSAD_CONFIG

```
#define SYS_LSAD_CONFIG LSAD->CFG = config
```

Configure the LSAD operation and interrupts.

Location: lsad.h:46

Parameters

Direction	Name	Description
in	<i>config</i>	Interface operation configuration; use LSAD_INT_* LSAD_INT_[ENABLE DISABLE], LSAD_OFFSET_[AUTO MANUAL], LSAD_PRESCALE_*,

Example Code for SYS_LSAD_CONFIG

```
// Configure LSAD interrupt such that the LSAD interrupt is triggered
// when the LSAD CH0 data registers are updated, using auto offset
// and prescale of 160
SYS\_LSAD\_CONFIG(LSAD_INT_CH0 | LSAD_INT_ENABLE |
    LSAD_OFFSET_AUTO | LSAD_PRESCALE_160);
```

18.18.3 LSAD Function Documentation

18.18.3.1 Sys_LSAD_InputConfig

```
void Sys_LSAD_InputConfig(uint32_t num, uint32_t source)
```

Configure LSAD input channel.

Location: lsad.h:54

Parameters

Direction	Name	Description
in	<i>num</i>	Channel number; use a integer
in	<i>source</i>	Select ADC channel input source

Example Code for Sys_LSAD_InputConfig

```
// Configure LSAD input channel 0 to use  
// LSAD_INPUT_DIO8 as ADC channel input source  
Sys\_LSAD\_InputConfig(0, LSAD_INPUT_DIO8);
```

18.19 MEMORY INSTANCES

Memory instances Hardware Abstraction Layer.

18.19.1 Summary

Macros

- [SYS_ASSIGN_PRAM0](#) : Assign PRAM0 to either the Arm Cortex M3 or CFX processor.
- [SYS_ASSIGN_PRAM1](#) : Assign PRAM1 to either the Arm Cortex M3 or CFX processor.
- [SYS_LOCK_PRAM_ASSIGN](#) : Lock the assignments of PRAM0 and PRAM1 until system reset.
- [SYS_LOCK_PRAM_INSTANCE](#) : Lock a PRAM instance against all access except the IBus until system reset.

Functions

- [Sys_Memory_Enable](#) : Enable memory instance(s)
- [Sys_Memory_Disable](#) : Disable memory instance(s)

18.19.2 Memory Instances Macro Definition Documentation

18.19.2.1 SYS_ASSIGN_PRAM0

```
#define SYS_ASSIGN_PRAM0 SYSCTRL->PRAM_CFG = (SYSCTRL->PRAM_CFG & ~PRAM0_CFX) | processor
```

Assign PRAM0 to either the Arm Cortex M3 or CFX processor.

Location: mem_cfg.h:46

Parameters

Direction	Name	Description
in	<i>processor</i>	Which processor to assign the PRAM0 instance to; use PRAM0_[CM3 CFX]

Assumptions

The assignments have not been locked already

Example Code for SYS_ASSIGN_PRAM0

```
// Assign PRAM0 to the Arm Cortex-M3 processor  
SYS\_ASSIGN\_PRAM0(PRAM0_CM3);
```

18.19.2.2 SYS_ASSIGN_PRAM1

```
#define SYS_ASSIGN_PRAM1 SYSCTRL->PRAM_CFG = (SYSCTRL->PRAM_CFG & ~PRAM1_CFX) | processor
```

Assign PRAM1 to either the Arm Cortex M3 or CFX processor.

Location: mem_cfg.h:55

Parameters

Direction	Name	Description
in	<i>processor</i>	Which processor to assign the PRAM1 instance to; use PRAM1_[CM3 CFX]

Example Code for SYS_ASSIGN_PRAM1

```
// Assign PRAM1 to the CFX processor  
SYS\_ASSIGN\_PRAM1 (PRAM1_CFX);
```

18.19.2.3 SYS_LOCK_PRAM_ASSIGN

```
#define SYS_LOCK_PRAM_ASSIGN SYSCTRL->PRAM_CFG |= PRAM_ASSIGN_LOCK
```

Lock the assignments of PRAM0 and PRAM1 until system reset.

Location: mem_cfg.h:62

Example Code for SYS_LOCK_PRAM_ASSIGN

```
// Lock the assignments of PRAM0 and PRAM1 until system reset  
SYS\_LOCK\_PRAM\_ASSIGN ();
```

18.19.2.4 SYS_LOCK_PRAM_INSTANCE

```
#define SYS_LOCK_PRAM_INSTANCE SYSCTRL->PRAM_CFG |= PRAM_LOCK
```

Lock a PRAM instance against all access except the IBus until system reset.

Location: mem_cfg.h:71

Parameters

Ezairo 8300 Firmware Reference

Direction	Name	Description
in	<i>PRAM_LOCK</i>	

Example Code for SYS_LOCK_PRAM_INSTANCE

```
// Lock the PRAM1 instance against all access except the IBus
// until system reset
SYS_LOCK_PRAM_INSTANCE (PRAM1_LOCK);
```

18.19.3 Memory Instances Function Documentation**18.19.3.1 Sys_Memory_Enable**

```
void Sys_Memory_Enable(uint32_t mem_cfg_0, uint32_t mem_cfg_1)
```

Enable memory instance(s)

Location: mem_cfg.h:104

Parameters

Direction	Name	Description
in	<i>mem_cfg_0</i>	Bit field corresponding to the Arm Cortex M3 processor (CM3) and DSP memories to be enabled. Use *_POWER_ENABLE, where * is one of the following: CM3_PRAM[0:6] CM3_DRAM[0:3] CM3_EMMC_RAM DSP_PRAM[0:5] DSP_ARAM[0:5] DSP_BRAM[0:2]
in	<i>mem_cfg_1</i>	Bit field corresponding to the CFX, HEAR, and FENG memories to be enabled. Use *_POWER_ENABLE, where * is one of the following: CFX_PROM CFX_PRAM[0:1] CFX_XRAM[0:3] CFX_YRAM[0:2] HEAR_PRAM[0:1] HEAR_HMEM HEAR_ABCD_PART[0:3] FENG_STATE FENG_COEF FENG_PRAM

Ezairo 8300 Firmware Reference

Example Code for Sys_Memory_Enable

```

// Enable specified Cortex-M3 and CFX memory instances
/*Initialize a bitfield corresponding to CM3_PRAM0 and CM3_PRAM2 */
mem_cfg_0 = CM3_PRAM0_POWER_ENABLE |
            CM3_PRAM2_POWER_ENABLE;
/* Initialize a bitfield corresponding to CFX_XRAM0 and CFX_YRAM0 */
mem_cfg_1 = CFX_XRAM0_POWER_ENABLE |
            CFX_YRAM0_POWER_ENABLE;
/* Enable the memory instances */
Sys\_Memory\_Enable(mem_cfg_0, mem_cfg_1);

```

18.19.3.2 Sys_Memory_Disable

```
void Sys_Memory_Disable(uint32_t mem_cfg_0, uint32_t mem_cfg_1)
```

Disable memory instance(s)

Location: mem_cfg.h:143

Parameters

Direction	Name	Description
in	<i>mem_cfg_0</i>	Bit field corresponding to the Arm Cortex M3 processor (CM3) and DSP memories to be enabled. Using *_POWER_ENABLE will give the correct configuration, where * is one of the following: CM3_PRAM[0:6] CM3_DRAM[0:3] CM3_EMMC_RAM DSP_PRAM[0:5] DSP_ARAM[0:5] DSP_BRAM[0:2]
in	<i>mem_cfg_1</i>	Bit field corresponding to the CFX, HEAR, and FENG memories to be disabled. Using *_POWER_ENABLE will give the correct configuration, where * is one of the following: CFX_PROM CFX_PRAM[0:1] CFX_XRAM[0:3] CFX_YRAM[0:2] HEAR_PRAM[0:1] HEAR_HMEM HEAR_ABCD_PART[0:3] FENG_STATE FENG_COEF FENG_PRAM

Ezairo 8300 Firmware Reference

Example Code for Sys_Memory_Disable

```

// Disable specified memory instances
/* Initialize a bitfield corresponding to CM3_PRAM0 and CM3_PRAM2 */
mem_cfg_0 = CM3_PRAM0_POWER_ENABLE |
            CM3_PRAM2_POWER_ENABLE;
/* Initialize a bitfield corresponding to CFX_XRAM0 and CFX_YRAM0 */
mem_cfg_1 = CFX_XRAM0_POWER_ENABLE |
            CFX_YRAM0_POWER_ENABLE;
/* Disable the memory instances */
Sys_Memory_Disable(mem_cfg_0, mem_cfg_1);

```

18.20 PCM

PCM Hardware Abstraction Layer.

18.20.1 Summary**Macros**

- [PCM_CFG_MASK](#) : Mask to ensure only the configuration bits are being set.
- [PCM_PADS_NUM](#) : Number of output pads for the PCM interface.
- [SYS_PCM_CONFIG](#) : Macro wrapper for [Sys_PCM_Config\(\)](#) Configure the PCM interface.
- [SYS_PCM_DIOCONFIG](#) : Macro wrapper for [Sys_PCM_DIOConfig\(\)](#) Configure four DIOs for the PCM interface.

Functions

- [Sys_PCM_DIOConfig](#) : Configure four DIOs for the PCM interface.
- [Sys_PCM_Config](#) : Configure the PCM interface.

18.20.2 PCM Macro Definition Documentation**18.20.2.1 PCM_CFG_MASK**

```

#define PCM_CFG_MASK ((1 << PCM_CFG_TX_DMA_ENABLE_Pos)          | \
                    (1 << PCM_CFG_RX_DMA_ENABLE_Pos)            | \
                    (1 << PCM_CFG_TX_IOC_ENABLE_Pos)            | \
                    (1 << PCM_CFG_RX_IOC_ENABLE_Pos)            | \
                    (1 << PCM_CFG_RX_TX_INT_ENABLE_Pos)         | \
                    (1 << PCM_CFG_OVERRUN_INT_EN_Pos)           | \
                    (1 << PCM_CFG_UNDERRUN_INT_EN_Pos)          | \
                    (1 << PCM_CFG_CLK_POLARITY_Pos)             | \
                    (1 << PCM_CFG_TX_DATA_ALIGN_Pos)            | \
                    (1 << PCM_CFG_RX_DATA_ALIGN_Pos)            | \
                    (PCM_CFG_WORD_SIZE_Mask)                    | \

```

Ezairo 8300 Firmware Reference

```

(1 << PCM_CFG_BIT_ORDER_Pos)          | \
(1 << PCM_CFG_FRAME_ALIGN_Pos)        | \
(1 << PCM_CFG_FRAME_WIDTH_Pos)        | \
(PCM_CFG_FRAME_LENGTH_Mask)           | \
(1 << PCM_CFG_SUBFRAME_Pos)           | \
(1 << PCM_CFG_SLAVE_Pos) )

```

Mask to ensure only the configuration bits are being set.

Location: pcm.h:39

18.20.2.2 PCM_PADS_NUM

```
#define PCM_PADS_NUM 2
```

Number of output pads for the PCM interface.

Location: pcm.h:58

18.20.2.3 SYS_PCM_CONFIG

```
#define SYS_PCM_CONFIG Sys PCM Config(PCM, (config))
```

Macro wrapper for [Sys PCM Config\(\)](#) Configure the PCM interface.

Location: pcm.h:170

Parameters

Ezairo 8300 Firmware Reference

Direction	Name	Description
in	<i>config</i>	Interface operation configuration; use PCM_TX_ACK_[DATA0 DATA1], PCM_RX_ACK_[DATA0 DATA1], PCM_TX_DMA_[ENABLE DISABLE], PCM_RX_DMA_[ENABLE DISABLE], PCM_TX_IOC_[ENABLE DISABLE], PCM_RX_IOC_[ENABLE DISABLE], PCM_RX_TX_INT_[ENABLE DISABLE], PCM_OVERRUN_INT_[ENABLE DISABLE], PCM_UNDERRUN_INT_[ENABLE DISABLE], PCM_SAMPLE_[FALLING RISING]_EDGE, PCM_TX_DATA_ALIGN_[MSB LSB], PCM_RX_DATA_ALIGN_[MSB LSB], PCM_WORD_SIZE_[8-32], PCM_BIT_ORDER_[MSB LSB]_FIRST, PCM_FRAME_ALIGN_[LAST FIRST], PCM_FRAME_WIDTH_[SHORT LONG], PCM_MULTIWORD_[2, 4, 6..16], PCM_SUBFRAME_[ENABLE DISABLE], and PCM_SELECT_[MASTER SLAVE]

Example Code for SYS_PCM_CONFIG

```
// Configure the default PCM interface such that:
// - TX data0 acknowledges the TX request
// - RX data0 acknowledges the RX request
// - A frame contains 6 words
// - The PCM interface generates the PCM_FRAME.
SYS_PCM_CONFIG(PCM_TX_ACK_DATA0 |
               PCM_RX_ACK_DATA0 |
               PCM_MULTIWORD_6 |
               PCM_SELECT_MASTER);
```

18.20.2.4 SYS_PCM_DIOCONFIG

```
#define SYS_PCM_DIOCONFIG Sys\_PCM\_DIOConfig(PCM, (slave), (cfg), (clk), \
                                           (frame), (seri), (sero), (clksrc))
```

Macro wrapper for [Sys_PCM_DIOConfig\(\)](#) Configure four DIOs for the PCM interface.

Location: pcm.h:186

Parameters

Ezairo 8300 Firmware Reference

Direction	Name	Description
in	<i>slave</i>	PCM master/slave configuration; use PCM_SELECT_[MASTER SLAVE]
in	<i>cfg</i>	DIO pin configuration for the PCM pads
in	<i>clk</i>	DIO to use as the PCM clock pad
in	<i>frame</i>	DIO to use as the PCM frame pad
in	<i>seri</i>	DIO to use as the PCM serial input pad
in	<i>sero</i>	DIO to use as the PCM serial output pad
in	<i>clksrc</i>	Clock source for PCM; use DIO_MODE_[*CLK INPUT]

Example Code for SYS_PCM_DIOCONFIG

```
// Configure DIOs 4, 5, 6, and 7 for the default PCM interface:
// - DIOs use 8X drive strength, low-pass filter enabled,
//   with 1K pull-up resistor
// - Use SLOWCLK as DIO clock source
SYS_PCM_DIOCONFIG(PCM_SELECT_SLAVE, (DIO_8X_DRIVE |
DIO_LPF_ENABLE | DIO_1K_PULL_UP),
DIO4, DIO5, DIO6, DIO7, DIO_MODE_SLOWCLK);
```

18.20.3 PCM Function Documentation**18.20.3.1 Sys_PCM_DIOConfig**

```
void Sys_PCM_DIOConfig(const PCM_Type * pcm, uint32_t slave, uint32_t cfg, uint32_t clk,
uint32_t frame, uint32_t seri, uint32_t sero, uint32_t clksrc)
```

Configure four DIOs for the PCM interface.

Location: pcm.h:74

Parameters

Ezairo 8300 Firmware Reference

Direction	Name	Description
in	<i>pcm</i>	Pointer to the PCM instance
in	<i>slave</i>	PCM master/slave configuration; use PCM_SELECT_[MASTER SLAVE]
in	<i>cfg</i>	DIO pin configuration for the PCM pads
in	<i>clk</i>	DIO to use as the PCM clock pad
in	<i>frame</i>	DIO to use as the PCM frame pad
in	<i>seri</i>	DIO to use as the PCM serial input pad
in	<i>sero</i>	DIO to use as the PCM serial output pad
in	<i>clksrc</i>	Clock source for PCM; use DIO_MODE_[*CLK INPUT]

Example Code for Sys_PCM_DIOConfig

```
// Configure DIOs 4, 5, 6, and 7 for the PCM0 interface
// - The PCM interface generates the PCM_FRAME.
// - DIOs use 8X drive strength, low-pass filter enabled,
//   with 1K pull-up resistor
// - Use SLOWCLK as DIO clock source
Sys_PCM_DIOConfig(PCM0, PCM_SELECT_MASTER,
                  (DIO_8X_DRIVE | DIO_LPF_ENABLE | DIO_1K_PULL_UP),
                  DIO4, DIO5, DIO6, DIO7,
                  DIO_MODE_SLOWCLK);
```

18.20.3.2 Sys_PCM_Config

```
void Sys_PCM_Config(PCM_Type * pcm, uint32_t config)
```

Configure the PCM interface.

Location: pcm.h:138

Parameters

Ezairo 8300 Firmware Reference

Direction	Name	Description
in	<i>pcm</i>	Pointer to the PCM instance
in	<i>config</i>	Interface operation configuration; use PCM_TX_ACK_[DATA0 DATA1], PCM_RX_ACK_[DATA0 DATA1], PCM_TX_DMA_[ENABLE DISABLE], PCM_RX_DMA_[ENABLE DISABLE], PCM_TX_IOC_[ENABLE DISABLE], PCM_RX_IOC_[ENABLE DISABLE], PCM_RX_TX_INT_[ENABLE DISABLE], PCM_OVERRUN_INT_[ENABLE DISABLE], PCM_UNDERRUN_INT_[ENABLE DISABLE], PCM_SAMPLE_[FALLING RISING]_EDGE, PCM_TX_DATA_ALIGN_[MSB LSB], PCM_RX_DATA_ALIGN_[MSB LSB], PCM_WORD_SIZE_*[8-32], PCM_BIT_ORDER_[MSB LSB]_FIRST, PCM_FRAME_ALIGN_[LAST FIRST], PCM_FRAME_WIDTH_[SHORT LONG], PCM_MULTIWORD_*[2, 4, 6..16], PCM_SUBFRAME_[ENABLE DISABLE], and PCM_SELECT_[MASTER SLAVE]

Example Code for Sys_PCM_Config

```
// Configure the PCM0 interface:
// - Enable DMA RX and TX request during PCM data requests & receives
// - The PCM0 interface generates the PCM_FRAME.
Sys_PCM_Config (PCM0, PCM_TX_DMA_ENABLE |
PCM_RX_DMA_ENABLE | PCM_SELECT_MASTER);
```

18.21 POWER

PSU Hardware Abstraction Layer.

18.21.1 Summary**Functions**

- [Sys Power AutomaticChargePumpControlEnable](#) : Enable the automatic charge pump control.
- [Sys Power AutomaticChargePumpControlDisable](#) : Disable the automatic charge pump control.
- [Sys Power AdaptiveVoltageScalingEnable](#) : Enable the adaptive voltage scaling.
- [Sys Power AdaptiveVoltageScalingDisable](#) : Disable the adaptive voltage scaling.
- [Sys Clock Power AdaptiveVoltageScalingSwitchCCO](#) : Change the multiplier used for the CCO while adaptive voltage scaling may be active.

18.21.2 POWER Function Documentation

18.21.2.1 Sys_Power_AutomaticChargePumpControlEnable

```
void Sys_Power_AutomaticChargePumpControlEnable()
```

Enable the automatic charge pump control.

Location: power.h:43

Example Code for Sys_Power_AutomaticChargePumpControlEnable

```
// Enable the automatic charge pump control  
Sys\_Power\_AutomaticChargePumpControlEnable\(\) ;
```

18.21.2.2 Sys_Power_AutomaticChargePumpControlDisable

```
void Sys_Power_AutomaticChargePumpControlDisable(unsigned int vddc_mode, unsigned int vddm_mode)
```

Disable the automatic charge pump control.

Location: power.h:63

Parameters

Direction	Name	Description
in	<i>vddc_mode</i>	VDDC ratio trim setting
in	<i>vddm_mode</i>	VDDM ratio trim setting

Example Code for Sys_Power_AutomaticChargePumpControlDisable

```
// Disable the automatic charge pump control  
Sys\_Power\_AutomaticChargePumpControlDisable(VDDC_RATIO_2_3,  
                                              VDDM_RATIO_2_3);
```

18.21.2.3 Sys_Power_AdaptiveVoltageScalingEnable

```
void Sys_Power_AdaptiveVoltageScalingEnable(unsigned int min_trim, unsigned int max_trim)
```

Enable the adaptive voltage scaling.

Location: power.h:79

Parameters

Direction	Name	Description
in	<i>min_trim</i>	Minimum allowed trim for VDDC
in	<i>max_trim</i>	Maximum allowed trim for VDDC

Example Code for Sys_Power_AdaptiveVoltageScalingEnable

```
/* Setup adaptive voltage scaling to use the limit and active  
 * trim settings previously loaded. */  
Sys\_Power\_AdaptiveVoltageScalingEnable(limit_trim, active_trim);
```

18.21.2.4 Sys_Power_AdaptiveVoltageScalingDisable

```
void Sys_Power_AdaptiveVoltageScalingDisable()
```

Disable the adaptive voltage scaling.

Location: power.h:99

Example Code for Sys_Power_AdaptiveVoltageScalingDisable

```
// Disable adaptive voltage scaling  
Sys\_Power\_AdaptiveVoltageScalingDisable();
```

18.21.2.5 Sys_Clock_Power_AdaptiveVoltageScalingSwitchCCO

```
void Sys_Clock_Power_AdaptiveVoltageScalingSwitchCCO(unsigned int new_mult)
```

Change the multiplier used for the CCO while adaptive voltage scaling may be active.

Location: power.h:110

Parameters

Direction	Name	Description
in	<i>new_mult</i>	Updated CCO multiplier

Example Code for Sys_Clock_Power_AdaptiveVoltageScalingSwitchCCO

```
// Change the multiplier used for the CCO to 2  
Sys\_Clock\_Power\_AdaptiveVoltageScalingSwitchCCO(OSC_MULTIPLY_BY_2);
```

18.22 HAL PRAM SECURITY REFERENCE

HAL PRAM Security Reference.

18.22.1 Summary

Functions

- [Sys_PRAMSec_SetLock](#) : Configure the PRAM lock bits.
- [Sys_PRAMSec_GetLock](#) : Read the PRAM lock bits.

18.22.2 HAL PRAM Security Reference Function Documentation

18.22.2.1 Sys_PRAMSec_SetLock

```
void Sys_PRAMSec_SetLock(uint32_t config)
```

Configure the PRAM lock bits.

Location: pram_security.h:43

Parameters

Direction	Name	Description
in	<i>config</i>	PRAM lock configuration; use a set of [CFX CM3]_PRAM[0 1]_LOCK defines

Example Code for Sys_PRAMSec_SetLock

```
// Lock all of the PRAM sections
Sys_PRAMSec_SetLock(CFX_PRAM0_LOCK | CFX_PRAM1_LOCK |
                    CM3_PRAM0_LOCK | CM3_PRAM1_LOCK );
```

18.22.2.2 Sys_PRAMSec_GetLock

```
uint32_t Sys_PRAMSec_GetLock()
```

Read the PRAM lock bits.

Location: pram_security.h:57

Return

PRAM lock configuration; compare with a set of [CFX|CM3]_PRAM[0|1]_LOCK defines

Example Code for Sys_PRAMSec_GetLock

```
// Read the current PRAM lock configuration
config = Sys_PRAMSec_GetLock();
```

18.23 RS

Reed-Solomon Hardware Abstraction Layer.

18.23.1 Summary**Macros**

- [RS_CFG_MASK](#) : Mask to ensure that only the configuration bits are being set.
- [RS_INT_MASK](#) : Mask to ensure that only the interrupt configuration bits are being set.

Functions

- [Sys_RS_Config](#) : Configure RS encoder and decoder.

18.23.2 RS Macro Definition Documentation**18.23.2.1 RS_CFG_MASK**

```
#define RS_CFG_MASK ((1U << RS_CFG_DEC_IN_PTR_DIR_Pos) |\
                    (1U << RS_CFG_DEC_OUT_PTR_DIR_Pos) |\
                    (1U << RS_CFG_ENC_IN_PTR_DIR_Pos) |\
                    (1U << RS_CFG_ENC_OUT_PTR_DIR_Pos) |\
                    RS_CFG_DEC_IN_PTR_MODE_Mask |\
                    RS_CFG_DEC_OUT_PTR_MODE_Mask |\
                    RS_CFG_ENC_IN_PTR_MODE_Mask |\
                    RS_CFG_ENC_OUT_PTR_MODE_Mask)
```

Mask to ensure that only the configuration bits are being set.

Location: rs.h:39

18.23.2.2 RS_INT_MASK

```
#define RS_INT_MASK ((1U << RS_INT_ENABLE_DEC_MISS_ERR_Pos) |\
                    (1U << RS_INT_ENABLE_DEC_BLK_END_Pos) |\
                    (1U << RS_INT_ENABLE_DEC_IN_ERR_Pos) |\
                    )
```

Ezairo 8300 Firmware Reference

```
(1U << RS_INT_ENABLE_ENC_BLK_END_Pos) | \
(1U << RS_INT_ENABLE_ENC_IN_ERR_Pos))
```

Mask to ensure that only the interrupt configuration bits are being set.

Location: rs.h:49

18.23.3 RS Function Documentation

18.23.3.1 Sys_RS_Config

```
void Sys_RS_Config(RS_Type * rs, uint32_t cfg, uint32_t int_cfg, uint32_t enc_block_len,
uint32_t dec_block_len)
```

Configure RS encoder and decoder.

Location: rs.h:78

Parameters

Direction	Name	Description
in	<i>rs</i>	Pointer to RS block
in	<i>cfg</i>	RS encoder/decoder configuration; use RS_ENC_IN_*, RS_ENC_OUT_*, RS_ENC_IN_[INC DEC], RS_ENC_OUT_[INC DEC], RS_DEC_IN_*, RS_DEC_OUT_*, RS_DEC_IN_[INC DEC], RS_DEC_OUT_[INC DEC]
in	<i>int_cfg</i>	RS interrupt configuration; use RS_ENC_IN_ERR_INT_[ENABLE DISABLE], RS_ENC_BLK_END_INT_[ENABLE DISABLE], RS_DEC_IN_ERR_INT_[ENABLE DISABLE], RS_DEC_BLK_END_INT_[ENABLE DISABLE], RS_DEC_MISS_ERR_INT_[ENABLE DISABLE]
in	<i>enc_block_len</i>	number of blocks to encode
in	<i>dec_block_len</i>	number of blocks to decode

Example Code for Sys_RS_Config

```
// Configure RS encoder and decoder:
// - Input data uses 4-bits per word (LSB aligned)
// - The RS Encoder input interface error can set the RS_COM interrupt line
// - 2 blocks to be encoded, 0 blocks to be decoded
Sys\_RS\_Config(RS, RS_ENC_IN_4BIT, RS_ENC_IN_ERR_INT_ENABLE,
               0x2, 0);
```

18.24 SPI

SPI Hardware Abstraction Layer.

18.24.1 Summary**Macros**

- [SPI_CONFIG_MASK](#) : Mask for SPI configuration.
- [SPI_PADS_NUM](#) : Number of SPI pads.
- [SYS_SPI_CONFIG](#) : Macro wrapper for [SYS_SPI_CONFIG\(...\)](#) Configure the specified SPI interface's operation and controller information.
- [SYS_SPI_TRANSFERCONFIG](#) : Macro wrapper for [Sys_SPI_TransferConfig\(...\)](#) Configure the SPI transfer information for the specified SPI instance.
- [SYS_SPI_READ](#) : Macro wrapper for [Sys_SPI_Read\(...\)](#) Generate clock and CS to read data from default SPI interface.
- [SYS_SPI_WRITE](#) : Macro wrapper for [Sys_SPI_Write\(...\)](#) Generate clock and CS to write data to SPI interface.
- [SYS_SPI_DIOCONFIG](#) : Macro wrapper for [Sys_SPI_DIOConfig\(...\)](#) Configure four DIOs for the SPI0 interface.
- [SYS_DSPI_DIOCONFIG](#) : Macro wrapper for [Sys_DSPI_DIOConfig\(...\)](#) Configure four DIOs for the SPI0 interface for DSPI.
- [SYS_QSPI_DIOCONFIG](#) : Macro wrapper for [Sys_QSPI_DIOConfig\(...\)](#) Configure six DIOs for the SPI0 interface for QSPI.

Functions

- [Sys_SPI_Config](#) : Configure the specified SPI interface's operation and controller information.
- [Sys_SPI_TransferConfig](#) : Configure the SPI transfer information for the specified SPI instance.
- [Sys_SPI_Read](#) : Generate clock and CS to read data from SPI interface.
- [Sys_SPI_Write](#) : Generate clock and CS to write data to SPI interface.
- [Sys_SPI_DIOConfig](#) : Configure four DIOs for the specified SPI interface.
- [Sys_DSPI_DIOConfig](#) : Configure four DIOs for the specified SPI interface.
- [Sys_QSPI_DIOConfig](#) : Configure six DIOs for the specified SPI interface.

18.24.2 SPI Macro Definition Documentation

18.24.2.1 SPI_CONFIG_MASK

```
#define SPI_CONFIG_MASK ((1 << SPI_CFG_TX_DMA_ENABLE_Pos)      | \
    (1 << SPI_CFG_RX_DMA_ENABLE_Pos)                          | \
    (1 << SPI_CFG_TX_END_INT_ENABLE_Pos)                      | \
    (1 << SPI_CFG_TX_START_INT_ENABLE_Pos)                   | \
    (1 << SPI_CFG_RX_INT_ENABLE_Pos)                         | \
    (1 << SPI_CFG_CS_RISE_INT_ENABLE_Pos)                    | \
    (1 << SPI_CFG_OVERRUN_INT_ENABLE_Pos)                   | \
    (1 << SPI_CFG_UNDERRUN_INT_ENABLE_Pos)                  | \
    (1 << SPI_CFG_MODE_Pos)                                  | \
    SPI_CFG_MODE_Mask                                        | \
    (1 << SPI_CFG_WORD_SIZE_Pos)                             | \
    SPI_CFG_WORD_SIZE_Mask                                 | \
    (1 << SPI_CFG_PRESCALE_Pos)                             | \
    SPI_CFG_PRESCALE_Mask                                 | \
    (1 << SPI_CFG_CLK_POLARITY_Pos)                         | \
    (1 << SPI_CFG_SLAVE_Pos))
```

Mask for SPI configuration.

Location: spi.h:51

18.24.2.2 SPI_PADS_NUM

```
#define SPI_PADS_NUM 6
```

Number of SPI pads.

Location: spi.h:76

18.24.2.3 SYS_SPI_CONFIG

```
#define SYS_SPI_CONFIG Sys SPI Config(SPI, (config))
```

Macro wrapper for [SYS_SPI_CONFIG\(...\)](#) Configure the specified SPI interface's operation and controller information.

Location: spi.h:328

Parameters

Ezairo 8300 Firmware Reference

Direction	Name	Description
in	<i>config</i>	Interface operation configuration; use SPI_SELECT_[MASTER SLAVE], SPI_CLK_POLARITY_[NORMAL INVERSE], SPI_PRESCALE_*, SPI_WORD_SIZE_*, SPI_MODE_[SPI DSPI QSPI] SPI_UNDERRUN_INT_[ENABLE DISABLE], SPI_OVERRUN_INT_[ENABLE DISABLE], SPI_CS_RISE_INT_[ENABLE DISABLE], SPI_RX_INT_[ENABLE DISABLE] SPI_TX_INT_[ENABLE DISABLE] SPI_RX_DMA_[ENABLE DISABLE] SPI_TX_DMA_[ENABLE DISABLE]

Example Code for SYS_SPI_CONFIG

```
// Configure the default SPI interface's operation and controller information:
// - Master mode
// - Use 8-bit words
// - Select interrupts enabled
// - Prescale the SPI interface clock by 32
SYS_SPI_CONFIG((SPI_MODE_QSPI | SPI_WORD_SIZE_8 | SPI_TX_START_INT_ENABLE |
                SPI_RX_INT_ENABLE | SPI_OVERRUN_INT_ENABLE |
                SPI_PRESCALE_32 | SPI_UNDERRUN_INT_ENABLE));
```

18.24.2.4 SYS_SPI_TRANSFERCONFIG

```
#define SYS_SPI_TRANSFERCONFIG Sys\_SPI\_TransferConfig(SPI, (config))
```

Macro wrapper for Sys_SPI_TransferConfig(...) Configure the SPI transfer information for the specified SPI instance.

Location: spi.h:347

Parameters

Direction	Name	Description
in	<i>config</i>	Interface transfer configuration; use SPI_ENABLE, SPI_DISABLE, SPI_RESET, SPI_START, SPI_MODE_READ_WRITE, SPI_MODE_READ, SPI_MODE_WRITE, SPI_MODE_NOP, SPI_CS_0, SPI_CS_1,

Example Code for SYS_SPI_TRANSFERCONFIG

```
// Enable and configure default SPI interface to read operation mode  
SYS\_SPI\_TRANSFERCONFIG(SPI_ENABLE | SPI_MODE_READ);
```

18.24.2.5 SYS_SPI_READ

```
#define SYS_SPI_READ Sys\_SPI\_Read(SPI)
```

Macro wrapper for Sys_SPI_Read(...) Generate clock and CS to read data from default SPI interface.

Location: spi.h:358

Assumptions

SPI is configured as master mode and transfer operation mode is SPI_MODE_READ_WRITE(full duplex) or (SPI_MODE_READ) half duplex

Example Code for SYS_SPI_READ

```
// Generate clock and CS to read data from the default SPI interface  
SYS\_SPI\_READ();
```

18.24.2.6 SYS_SPI_WRITE

```
#define SYS_SPI_WRITE Sys\_SPI\_Write(SPI, (data))
```

Macro wrapper for Sys_SPI_Write(...) Generate clock and CS to write data to SPI interface.

Location: spi.h:369

Parameters

Direction	Name	Description
in	<i>data</i>	Data to be sent over SPI

Assumptions

SPI is configured as master mode and transfer operation mode is SPI_MODE_READ_WRITE(full duplex) or (SPI_MODE_WRITE) half duplex

Example Code for SYS_SPI_WRITE

```
// Generate clock and CS to write data to the default SPI interface  
SYS\_SPI\_WRITE(0xFF);
```

18.24.2.7 SYS_SPI_DIOCONFIG

```
#define SYS_SPI_DIOCONFIG Sys\_SPI\_DIOConfig(SPI, (slave), (cfg), (clk), (cs), (seri),  
(sero))
```

Macro wrapper for Sys_SPI_DIOConfig(...) Configure four DIOs for the SPI0 interface.

Location: spi.h:383

Parameters

Direction	Name	Description
in	<i>slave</i>	SPI master/slave configuration; use SPI*_SELECT_[MASTER SLAVE]
in	<i>cfg</i>	DIO pin configuration for the SPI pads
in	<i>clk</i>	DIO to use as the SPI clock pad
in	<i>cs</i>	DIO to use as the SPI chip select pad
in	<i>seri</i>	DIO to use as the SPI serial input pad
in	<i>sero</i>	DIO to use as the SPI serial output pad

Ezairo 8300 Firmware Reference

Example Code for SYS_SPI_DIOCONFIG

```
// Configure DIOs 0, 1, 2, and 3 for the default SPI interface with
// low-pass filter disabled, 8X drive-strength, and 1K pull-up resistors
SYS_SPI_DIOCONFIG(DIO0, (DIO_LPF_DISABLE | DIO_8X_DRIVE |
DIO_1K_PULL_UP), DIO0, DIO1, DIO2, DIO3);
```

18.24.2.8 SYS_DSPI_DIOCONFIG

```
#define SYS_DSPI_DIOCONFIG Sys_DSPI_DIOConfig(SPI, (cfg), (clk), (cs), (io0), (io1))
```

Macro wrapper for Sys_DSPI_DIOConfig(...) Configure four DIOs for the SPI0 interface for DSPI.

Location: spi.h:396

Parameters

Direction	Name	Description
in	<i>cfg</i>	DIO pin configuration for the SPI pads
in	<i>clk</i>	DIO to use as the DSPI clock pad
in	<i>cs</i>	DIO to use as the DSPI chip select pad
in	<i>io0</i>	DIO to use as the DSPI io0
in	<i>io1</i>	DIO to use as the DSPI io1

Example Code for SYS_DSPI_DIOCONFIG

```
// Configure DIOs 0, 1, 2, and 3 for the default DSPI interface with
// low-pass filter disabled, 8X drive-strength, and 1K pull-up resistors
SYS_DSPI_DIOCONFIG((DIO_LPF_DISABLE | DIO_8X_DRIVE |
DIO_1K_PULL_UP), DIO0, DIO1, DIO2, DIO3);
```

18.24.2.9 SYS_QSPI_DIOCONFIG

```
#define SYS_QSPI_DIOCONFIG Sys_QSPI_DIOConfig(SPI, (cfg), (clk), (cs), (io0), (io1), \
(io2), (io3))
```

Macro wrapper for Sys_QSPI_DIOConfig(...) Configure six DIOs for the SPI0 interface for QSPI.

Location: spi.h:411

Parameters

Direction	Name	Description
in	<i>cfg</i>	DIO pin configuration for the SPI pads
in	<i>clk</i>	DIO to use as the QSPI clock pad
in	<i>cs</i>	DIO to use as the QSPI chip select pad
in	<i>io0</i>	DIO to use as the QSPI io0
in	<i>io1</i>	DIO to use as the QSPI io1
in	<i>io2</i>	DIO to use as the QSPI io2
in	<i>io3</i>	DIO to use as the QSPI io3

Example Code for SYS_QSPI_DIOCONFIG

```
// Configure DIOs 0, 1, 2, 3, 4, and 5 for the default QSPI interface with  
// low-pass filter disabled, 8X drive-strength, and 1K pull-up resistors  
SYS\_QSPI\_DIOCONFIG((DIO_LPF_DISABLE | DIO_8X_DRIVE | DIO_1K_PULL_UP),  
                    DIO0, DIO1, DIO2, DIO3, DIO4, DIO5);
```

18.24.3 SPI Function Documentation

18.24.3.1 Sys_SPI_Config

```
void Sys_SPI_Config(SPI_Type * spi, uint32_t config)
```

Configure the specified SPI interface's operation and controller information.

Location: spi.h:105

Parameters

Ezairo 8300 Firmware Reference

Direction	Name	Description
in	<i>spi</i>	Pointer to the SPI instance
in	<i>config</i>	Interface operation configuration; use SPI_SELECT_[MASTER SLAVE], SPI_CLK_POLARITY_[NORMAL INVERSE], SPI_PRESCALE_*, SPI_WORD_SIZE_*, SPI_MODE_[SPI DSPI QSPI] SPI_UNDERRUN_INT_[ENABLE DISABLE], SPI_OVERRUN_INT_[ENABLE DISABLE], SPI_CS_RISE_INT_[ENABLE DISABLE], SPI_RX_INT_[ENABLE DISABLE] SPI_TX_INT_[ENABLE DISABLE] SPI_RX_DMA_[ENABLE DISABLE] SPI_TX_DMA_[ENABLE DISABLE]

Example Code for Sys_SPI_Config

```
// Configure the SPI0 interface's operation and controller information:
// - Master mode
// - Use 8-bit words
// - Select interrupts enabled
// - Prescale the SPI interface clock by 32
Sys_SPI_Config(SPI0, SPI_SELECT_MASTER | SPI_WORD_SIZE_8 | SPI_TX_START_INT_ENABLE |
               SPI_RX_INT_ENABLE | SPI_OVERRUN_INT_ENABLE |
               SPI_PRESCALE_32 | SPI_UNDERRUN_INT_ENABLE);
```

18.24.3.2 Sys_SPI_TransferConfig

```
void Sys_SPI_TransferConfig(SPI_Type * spi, uint32_t config)
```

Configure the SPI transfer information for the specified SPI instance.

Location: spi.h:128

Parameters

Direction	Name	Description
in	<i>spi</i>	Pointer to the SPI instance
in	<i>config</i>	Interface transfer configuration; use SPI_ENABLE, SPI_DISABLE, SPI_RESET, SPI_START, SPI_MODE_READ_WRITE, SPI_MODE_READ, SPI_MODE_WRITE, SPI_MODE_NOP, SPI_CS_0, SPI_CS_1,

Example Code for Sys_SPI_TransferConfig

```
// Enable and configure the SPI1 interface to read operation mode
Sys_SPI_TransferConfig(SPI1, SPI_ENABLE | SPI_MODE_READ);
```

18.24.3.3 Sys_SPI_Read

```
uint32_t Sys_SPI_Read(const SPI_Type * spi)
```

Generate clock and CS to read data from SPI interface.

Location: spi.h:144

Parameters

Direction	Name	Description
in	<i>spi</i>	Pointer to the SPI instance

Return

data Data read from the SPI interface

Assumptions

SPI is configured as master mode and transfer operation mode is SPI_MODE_READ_WRITE(full duplex) or (SPI_MODE_READ) half duplex

Example Code for Sys_SPI_Read

```
// Generate clock and CS to read data from SPI0 interface
Sys_SPI_Read(SPI0);
```

Ezairo 8300 Firmware Reference

18.24.3.4 Sys_SPI_Write

```
void Sys_SPI_Write(SPI_Type * spi, uint32_t data)
```

Generate clock and CS to write data to SPI interface.

Location: spi.h:160

Parameters

Direction	Name	Description
in	<i>spi</i>	Pointer to the SPI instance
out	<i>data</i>	Data to be sent over SPI

Assumptions

SPI is configured as master mode and transfer operation mode is SPI_MODE_READ_WRITE(full duplex) or (SPI_MODE_WRITE) half duplex

Example Code for Sys_SPI_Write

```
// Generate clock and CS to write data to SPI1 interface
Sys_SPI_Write(SPI1, 0xFF);
```

18.24.3.5 Sys_SPI_DIOConfig

```
void Sys_SPI_DIOConfig(const SPI_Type * spi, uint32_t slave, uint32_t cfg, uint32_t clk,
uint32_t cs, uint32_t seri, uint32_t sero)
```

Configure four DIOs for the specified SPI interface.

Location: spi.h:178

Parameters

Ezairo 8300 Firmware Reference

Direction	Name	Description
in	<i>spi</i>	Pointer to the SPI instance
in	<i>slave</i>	SPI master/slave configuration; use SPI*_SELECT_[MASTER SLAVE]
in	<i>cfg</i>	DIO pin configuration for the SPI pads
in	<i>clk</i>	DIO to use as the SPI clock pad
in	<i>cs</i>	DIO to use as the SPI chip select pad
in	<i>seri</i>	DIO to use as the SPI serial input pad
in	<i>sero</i>	DIO to use as the SPI serial output pad

Example Code for Sys_SPI_DIOConfig

```
// Configure DIOs 0, 1, 2, and 3 for the SPI0 interface with
// low-pass filter disabled, 8X drive-strength, and 1K pull-up resistors
Sys_SPI_DIOConfig(SPI0, DIO0, (DIO_LPF_DISABLE | DIO_8X_DRIVE |
DIO_1K_PULL_UP), DIO0, DIO1, DIO2, DIO3);
```

18.24.3.6 Sys_DSPI_DIOConfig

```
void Sys_DSPI_DIOConfig(const SPI_Type * spi, uint32_t cfg, uint32_t clk, uint32_t cs,
uint32_t io0, uint32_t io1)
```

Configure four DIOs for the specified SPI interface.

Location: spi.h:235

Parameters

Direction	Name	Description
in	<i>spi</i>	Pointer to the SPI instance
in	<i>cfg</i>	DIO pin configuration for the SPI pads
in	<i>clk</i>	DIO to use as the DSPI clock pad

Ezairo 8300 Firmware Reference

Direction	Name	Description
in	<i>cs</i>	DIO to use as the DSPI chip select pad
in	<i>io0</i>	DIO to use as the DSPI io0
in	<i>io1</i>	DIO to use as the DSPI io1

Example Code for Sys_DSPI_DIOConfig

```
// Configure DIOs 0, 1, 2, and 3 for the DSPI0 interface with
// low-pass filter disabled, 8X drive-strength, and 1K pull-up resistors
Sys_DSPI_DIOConfig(SPI0, (DIO_LPF_DISABLE | DIO_8X_DRIVE |
DIO_1K_PULL_UP), DIO0, DIO1, DIO2, DIO3);
```

18.24.3.7 Sys_QSPI_DIOConfig

```
void Sys_QSPI_DIOConfig(const SPI_Type * spi, uint32_t cfg, uint32_t clk, uint32_t cs,
uint32_t io0, uint32_t io1, uint32_t io2, uint32_t io3)
```

Configure six DIOs for the specified SPI interface.

Location: spi.h:274

Parameters

Direction	Name	Description
in	<i>spi</i>	Pointer to the SPI instance
in	<i>cfg</i>	DIO pin configuration for the SPI pads
in	<i>clk</i>	DIO to use as the QSPI clock pad
in	<i>cs</i>	DIO to use as the QSPI chip select pad
in	<i>io0</i>	DIO to use as the QSPI io0
in	<i>io1</i>	DIO to use as the QSPI io1
in	<i>io2</i>	DIO to use as the QSPI io2
in	<i>io3</i>	DIO to use as the QSPI io3

Example Code for Sys_QSPI_DIOConfig

```
// Configure DIOs 0, 1, 2, 3, 4, and 5 for the QSPI0 interface with
// low-pass filter disabled, 8X drive-strength, and 1K pull-up resistors
Sys_QSPI_DIOConfig(SPI0, (DIO_LPF_DISABLE | DIO_8X_DRIVE | DIO_1K_PULL_UP),
    DIO0, DIO1, DIO2, DIO3, DIO4, DIO5);
```

18.25 STANDBY_MODE

PSU Hardware Abstraction Layer.

18.25.1 Summary**Macros**

- [CM3_ROMJUMPTABLE_BASEADDR](#) : Base address of the Arm Cortex-M3 core ROM's jump table.
- [CM3_ROMJUMPTABLE_SLEEP](#) : Vector within the Arm Cortex-M3 core ROM's jump table for the sleep function.

Functions

- [Sys_StandbyMode](#) : Put device into standby mode and the memories into retention mode.

18.25.2 STANDBY_MODE Macro Definition Documentation**18.25.2.1 CM3_ROMJUMPTABLE_BASEADDR**

```
#define CM3_ROMJUMPTABLE_BASEADDR 0x10
```

Base address of the Arm Cortex-M3 core ROM's jump table.

Location: standby_mode.h:40

18.25.2.2 CM3_ROMJUMPTABLE_SLEEP

```
#define CM3_ROMJUMPTABLE_SLEEP (CM3\_ROMJUMPTABLE\_BASEADDR + 0x04)
```

Vector within the Arm Cortex-M3 core ROM's jump table for the sleep function.

Ezairo 8300 Firmware Reference

Location: standby_mode.h:43

18.25.3 STANDBY_MODE Function Documentation

18.25.3.1 Sys_StandbyMode

```
void Sys_StandbyMode(unsigned int vddc_retention_trim, unsigned int vddm_retention_trim,
unsigned int vdda_standby_trim)
```

Put device into standby mode and the memories into retention mode.

Location: standby_mode.h:58

Parameters

Direction	Name	Description
	<i>vddc_retention_trim</i>	Desired VDDC retention mode trimming. Refer to device datasheet for the valid range.
	<i>vddm_retention_trim</i>	Desired VDDM retention mode trimming. Refer to device datasheet for the valid range.
	<i>vdda_standby_trim</i>	Desired VDDA retention mode trimming. Refer to device datasheet for the valid range.

Example Code for Sys_StandbyMode

```
// Put device into standby mode
Sys_StandbyMode(vddc_retention_trim, vddm_retention_trim, vdda_standby_trim);
```

Assumptions

VDDA_CONNECT bit of the ANALOG_CP_VDDIF_CTRL register is set to VDDIF_VDDA_AUTO_CONNECT if VDDIF is used in the application.

18.26 TIMER

TIMER Hardware Abstraction Layer.

18.26.1 Summary

Macros

- [SYS_TIMER_CONFIG](#) : Macro wrapper for [Sys_Timer_Config\(\)](#) Configure timer instance.
- [SYS_TIMER_START](#) : Macro wrapper for [Sys_Timer_Start\(\)](#) Start or restart Timer instance.
- [SYS_TIMER_STOP](#) : Macro wrapper for [Sys_Timer_Stop\(\)](#) Stop the timer instance.

Functions

- [Sys_Timer_Config](#) : Configure timer instance.
- [Sys_Timer_Start](#) : Start or restart Timer instance.
- [Sys_Timer_Stop](#) : Stop the timer instance.

18.26.2 TIMER Macro Definition Documentation

18.26.2.1 SYS_TIMER_CONFIG

```
#define SYS_TIMER_CONFIG Sys\_Timer\_Config(TIMER, (cfg0), \  
                                         (cfg1), (timeout))
```

Macro wrapper for [Sys_Timer_Config\(\)](#) Configure timer instance.

Location: timer.h:97

18.26.2.2 SYS_TIMER_START

```
#define SYS_TIMER_START Sys\_Timer\_Start(TIMER)
```

Macro wrapper for [Sys_Timer_Start\(\)](#) Start or restart Timer instance.

Location: timer.h:106

18.26.2.3 SYS_TIMER_STOP

```
#define SYS_TIMER_STOP Sys\_Timer\_Stop(TIMER)
```

Macro wrapper for [Sys_Timer_Stop\(\)](#) Stop the timer instance.

Location: timer.h:114

18.26.3 TIMER Function Documentation

18.26.3.1 Sys_Timer_Config

```
void Sys_Timer_Config(TIMER_Type * timer, uint32_t cfg0, uint32_t cfg1, uint32_t timeout)
```

Configure timer instance.

Location: timer.h:52

Parameters

Direction	Name	Description
in	<i>timer</i>	Pointer to the timer instance
in	<i>cfg0</i>	Timer configuration 0; use TIMER_PRESCALE_*,
in	<i>cfg1</i>	Timer configuration 1; use TIMER_MULTI_COUNT_* TIMER_SRC_DIO_INT* [TIMER_SHOT_MODE TIMER_FREE_RUN]
in	<i>timeout</i>	number of timer clock cycles before a timeout would occur

Example Code for Sys_Timer_Config

```
// Configure TIMER2 instance:
// - Divide the input clock frequency by 2
// - Stop on 2nd Time-out occurrence and issue an interrupt
// - Select DIO interrupt defined in DIO_INT_CFG3
// - DIO interrupt single capture mode
// - Free-run mode
// - Long timeout
Sys\_Timer\_Config(TIMER2, TIMER_PRESCALE_2,
    TIMER_MULTI_COUNT_2 |
    TIMER_SRC_DIO_INT3 |
    TIMER_DIO_INT_SINGLE |
    TIMER_FREE_RUN, 0xFFFF);
```

18.26.3.2 Sys_Timer_Start

```
void Sys_Timer_Start(TIMER_Type * timer)
```

Start or restart Timer instance.

Location: timer.h:66

Parameters

Direction	Name	Description
in	<i>timer</i>	Pointer to the timer instance Example Code for Sys_Timer_Start <pre>// Start or restart TIMER instance Sys_Timer_Start(TIMER);</pre>

18.26.3.3 Sys_Timer_Stop

```
void Sys_Timer_Stop(TIMER_Type * timer)
```

Stop the timer instance.

Location: timer.h:77

Parameters

Direction	Name	Description
in	<i>timer</i>	Pointer to the timer instance Example Code for Sys_Timer_Stop <pre>// Stop the TIMER3 instance Sys_Timer_Stop(TIMER3);</pre>

18.27 TRIMS

PSU Hardware Abstraction Layer.

Ezairo 8300 Firmware Reference

18.27.1 Summary

Variables

- [manuTable](#) : Manufacturing information table variables (for CFX ASM)

Macros

- [SYS_ERRNO_NO_ERROR](#) : No error.
- [SYS_ERRNO_NO_MATCH](#) : No trim match detected for the requested trim.
- [SYS_ERROR_SEARCH_FAILURE](#) : Failed during search for a trim.
- [VREG_0P9V](#) : 900 mV
- [OSC_MUL_LIMIT1](#) : Multiply oscillator by 2 if higher than this limit, and lower than OSC_MUL_LIMIT2.
- [OSC_MUL_LIMIT2](#) : Multiply oscillator by 4 if higher than this limit.

Functions

- [Sys Trims LoadManuTable](#) : Copies a short version of the manufacturing information table to the preset location (SYSVAR_PWRCAL_TABLE) in CM3 PRAM2.
- [Sys Trims SetVREGAndLSAD](#) : Sets the default trim values for the bandgap (VREG) and LSAD.
- [Sys Trims SetVDDIF](#) : Sets the requested trim value to the VDDIF register if it is found.
- [Sys Trims SetVDDA](#) : Sets the requested bandgap reference trim value to the VDDA register if it is found.
- [Sys Trims SetVDDA PMURef](#) : Sets the requested PMU reference trim value to the VDDA register if it is found.
- [Sys Trims SetVDDC](#) : Sets the requested bandgap reference trim value to the VDDC register if it is found.
- [Sys Trims SetVDDC PMURef](#) : Sets the requested PMU reference trim value to the VDDC register if it is found.
- [Sys Trims SetVDDC CP](#) : Sets the requested charge pump delta trim value to the VDDC register if it is found.
- [Sys Trims SetAutomaticVDDC](#) : Enable the adaptive voltage scaling with the requested VDDC register trim settings if they are both found.
- [Sys Trims SetVDDM](#) : Sets the requested bandgap reference trim value to the VDDM register if it is found.
- [Sys Trims SetVDDM PMURef](#) : Sets the requested PMU reference trim value to the VDDM register if it is found.
- [Sys Trims SetVDDM CP](#) : Sets the requested charge pump delta trim value to the VDDM register if it is found.
- [Sys Trims SetVDDOD](#) : Sets the requested trim value to the VDDOD register if it is found.
- [Sys Trims SetVMIC](#) : Sets the requested trim value to the VDDOD register if it is found.
- [Sys Trims SetOperatingFrequency](#) : Set the internal oscillator base operating frequency.
- [Sys Trims SetOperatingFrequencyMult](#) : Set the internal oscillator base operating frequency and a custom multiplier.
- [Sys Trims SetOperatingFrequencyThrottle](#) : Set the internal oscillator base operating frequency, using clock throttling to also configure SLOWCLK and SDMCLK.
- [Sys Trims SetOperatingFrequencyThrottleMult](#) : Set the internal oscillator base operating frequency and a

custom multiplier, using clock throttling to also configure SLOWCLK and SDMCLK.

- [Sys Trims SetADCOffsets](#) : Set the ADC offset trims for all of the ADCs.

18.27.2 TRIMS Variable Documentation

18.27.2.1 manuTable

```
MANU_TABLE_Type* manuTable
```

Location: trims.h:89

Manufacturing information table variables (for CFX ASM)

Global variable to access the short manufacturing information table stored in the Arm Cortex-M3 cores's PRAM2.

18.27.3 TRIMS Macro Definition Documentation

18.27.3.1 SYS_ERRNO_NO_ERROR

```
#define SYS_ERRNO_NO_ERROR ERRNO\_NO\_ERROR
```

No error.

Location: trims.h:50

18.27.3.2 SYS_ERRNO_NO_MATCH

```
#define SYS_ERRNO_NO_MATCH 0x2
```

No trim match detected for the requested trim.

Location: trims.h:51

18.27.3.3 SYS_ERROR_SEARCH_FAILURE

```
#define SYS_ERROR_SEARCH_FAILURE (int) (-1)
```

Failed during search for a trim.

Location: trims.h:52

18.27.3.4 VREG_0P9V

```
#define VREG_0P9V 90
```

900 mV

Location: trims.h:55

18.27.3.5 OSC_MUL_LIMIT1

```
#define OSC_MUL_LIMIT1 SYS_FREQ_29M44
```

Multiply oscillator by 2 if higher than this limit, and lower than OSC_MUL_LIMIT2.

Location: trims.h:59

18.27.3.6 OSC_MUL_LIMIT2

```
#define OSC_MUL_LIMIT2 SYS_FREQ_59M52
```

Multiply oscillator by 4 if higher than this limit.

Location: trims.h:60

18.27.4 TRIMS Function Documentation

18.27.4.1 Sys_Trims_LoadManuTable

```
void Sys_Trims_LoadManuTable(uint32_t * manutable_ptr)
```

Copies a short version of the manufacturing information table to the preset location (SYSVAR_PWRCAL_TABLE) in CM3 PRAM2.

Location: trims.h:98

Ezairo 8300 Firmware Reference

Parameters

Direction	Name	Description
in	<i>manutable_ptr</i>	Pointer to the full manufacturing information table loaded from NVM to local RAM.

Example Code for Sys_Trims_LoadManuTable

```
// manu_table_buffer contains the full manufacturing information table loaded
// from NVM to local RAM
Sys_Trims_LoadManuTable((uint32_t chess_storage(IOMEM *)manu_table_buffer);
```

18.27.4.2 Sys_Trims_SetVREGAndLSAD

```
unsigned int Sys_Trims_SetVREGAndLSAD()
```

Sets the default trim values for the bandgap (VREG) and LSAD.

Location: trims.h:106

Return

result SYS_ERRNO_NO_ERROR if no trim is found, SYS_ERRNO_NO_MATCH otherwise.

Example Code for Sys_Trims_SetVREGAndLSAD

```
// Find the default trim values for the bandgap (VREG) and LSAD in the manufacturing
// information table and set them if found
result = Sys_Trims_SetVREGAndLSAD();
```

Ezairo 8300 Firmware Reference

18.27.4.3 Sys_Trims_SetVDDIF

```
unsigned int Sys_Trims_SetVDDIF(unsigned int target)
```

Sets the requested trim value to the VDDIF register if it is found.

Location: trims.h:114

Parameters

Direction	Name	Description
in	<i>target</i>	The specified target voltage [10 mV] target

Return

result SYS_ERRNO_NO_ERROR if requested trim is found, SYS_ERRNO_NO_MATCH otherwise.

Example Code for Sys_Trims_SetVDDIF

```
// Set (VDDIF_ACTIVE_TARGET / 10) trim value to the VDDIF register
result = Sys_Trims_SetVDDIF((VDDIF_ACTIVE_TARGET / 10));
```

18.27.4.4 Sys_Trims_SetVDDA

```
unsigned int Sys_Trims_SetVDDA(unsigned int target)
```

Sets the requested bandgap reference trim value to the VDDA register if it is found.

Location: trims.h:123

Parameters

Direction	Name	Description
in	<i>target</i>	The specified target voltage [10 mV] target

Return

result SYS_ERRNO_NO_ERROR if requested trim is found, SYS_ERRNO_NO_MATCH otherwise.

Example Code for Sys_Trims_SetVDDA

```
// Set (VDDA_ACTIVE_TARGET / 10) bandgap reference trim value to the VDDA register  
result = Sys_Trims_SetVDDA((VDDA_ACTIVE_TARGET / 10));
```

18.27.4.5 Sys_Trims_SetVDDA_PMURef

```
unsigned int Sys_Trims_SetVDDA_PMURef(unsigned int target)
```

Sets the requested PMU reference trim value to the VDDA register if it is found.

Location: trims.h:132

Parameters

Direction	Name	Description
in	<i>target</i>	The specified target voltage [10 mV] target

Return

result SYS_ERRNO_NO_ERROR if requested trim is found, SYS_ERRNO_NO_MATCH otherwise.

Example Code for Sys_Trims_SetVDDA_PMURef

```
// Set VDDA_ACTIVE_TARGET PMU reference trim value to the VDDA register  
result = Sys_Trims_SetVDDA_PMURef(VDDA_ACTIVE_TARGET);
```

18.27.4.6 Sys_Trims_SetVDDC

```
unsigned int Sys_Trims_SetVDDC(unsigned int target)
```

Sets the requested bandgap reference trim value to the VDDC register if it is found.

Location: trims.h:141

Parameters

Direction	Name	Description
in	<i>target</i>	The specified target voltage [10 mV] target

Return

result SYS_ERRNO_NO_ERROR if requested trim is found, SYS_ERRNO_NO_MATCH otherwise.

Example Code for Sys_Trims_SetVDDC

```
// Set (VDDC_ACTIVE_TARGET / 10) bandgap reference trim value to the VDDC register  
result = Sys_Trims_SetVDDC((VDDC_ACTIVE_TARGET / 10));
```

18.27.4.7 Sys_Trims_SetVDDC_PMURef

```
unsigned int Sys_Trims_SetVDDC_PMURef(unsigned int target)
```

Sets the requested PMU reference trim value to the VDDC register if it is found.

Location: trims.h:150

Parameters

Direction	Name	Description
in	<i>target</i>	The specified target voltage [10 mV] target

Return

result SYS_ERRNO_NO_ERROR if requested trim is found, SYS_ERRNO_NO_MATCH otherwise.

Example Code for Sys_Trims_SetVDDC_PMURef

```
// Set VDDC_RETENTION_TARGET PMU reference trim value to the VDDC register  
result = Sys\_Trims\_SetVDDC\_PMURef\(VDDC\_RETENTION\_TARGET\);
```

18.27.4.8 Sys_Trims_SetVDDC_CP

```
unsigned int Sys_Trims_SetVDDC_CP(unsigned int target)
```

Sets the requested charge pump delta trim value to the VDDC register if it is found.

Location: trims.h:159

Parameters

Direction	Name	Description
in	<i>target</i>	The specified target voltage [10 mV] target

Return

result SYS_ERRNO_NO_ERROR if requested trim is found, SYS_ERRNO_NO_MATCH otherwise.

Example Code for Sys_Trims_SetVDDC_CP

```
// Set VDDCM_MIN_CP_DELTA_TARGET charge pump delta trim value to the VDDC register
result = Sys_Trims_SetVDDC_CP(VDDCM_MIN_CP_DELTA_TARGET);
```

18.27.4.9 Sys_Trims_SetAutomaticVDDC

```
unsigned int Sys_Trims_SetAutomaticVDDC(unsigned int targetMin, unsigned int targetMax)
```

Enable the adaptive voltage scaling with the requested VDDC register trim settings if they are both found.

Location: trims.h:169

Parameters

Direction	Name	Description
in	<i>targetMin</i>	The specified minimum target voltage [10 mV]
in	<i>targetMax</i>	The specified maximum target voltage [10 mV]

Return

result SYS_ERRNO_NO_ERROR if requested trims are found, SYS_ERRNO_NO_MATCH otherwise.

Example Code for Sys_Trims_SetAutomaticVDDC

```
// Set VDDC_LIMIT_TARGET as min and VDDC_ACTIVE_
// TARGET as max trim values to enable the
// adaptive voltage scaling on the VDDC register
result = Sys_Trims_SetAutomaticVDDC(VDDC_LIMIT_TARGET, VDDC_ACTIVE_TARGET);
```


Ezairo 8300 Firmware Reference

18.27.4.10 Sys_Trims_SetVDDM

```
unsigned int Sys_Trims_SetVDDM(unsigned int target)
```

Sets the requested bandgap reference trim value to the VDDM register if it is found.

Location: trims.h:178

Parameters

Direction	Name	Description
in	<i>target</i>	The specified target voltage [10 mV] target

Return

result SYS_ERRNO_NO_ERROR if requested trim is found, SYS_ERRNO_NO_MATCH otherwise.

Example Code for Sys_Trims_SetVDDM

```
// Set (VDDM_ACTIVE_TARGET / 10) bandgap reference trim value to the VDDM register
result = Sys_Trims_SetVDDM((VDDM_ACTIVE_TARGET / 10));
```

18.27.4.11 Sys_Trims_SetVDDM_PMURef

```
unsigned int Sys_Trims_SetVDDM_PMURef(unsigned int target)
```

Sets the requested PMU reference trim value to the VDDM register if it is found.

Location: trims.h:187

Parameters

Direction	Name	Description
in	<i>target</i>	The specified target voltage [10 mV] target

Return

result SYS_ERRNO_NO_ERROR if requested trim is found, SYS_ERRNO_NO_MATCH otherwise.

Example Code for Sys_Trims_SetVDDM_PMURef

```
// Set VDDM_RETENTION_TARGET PMU reference trim value to the VDDM register  
result = Sys_Trims_SetVDDM_PMURef(VDDM_RETENTION_TARGET);
```

18.27.4.12 Sys_Trims_SetVDDM_CP

```
unsigned int Sys_Trims_SetVDDM_CP(unsigned int target)
```

Sets the requested charge pump delta trim value to the VDDM register if it is found.

Location: trims.h:196

Parameters

Direction	Name	Description
in	<i>target</i>	The specified target voltage [10 mV] target

Return

result SYS_ERRNO_NO_ERROR if requested trim is found, SYS_ERRNO_NO_MATCH otherwise.

Ezairo 8300 Firmware Reference

Example Code for Sys_Trims_SetVDDM_CP

```
// Set VDDCM_MIN_CP_DELTA_TARGET charge pump delta trim value to the VDDM register
result = Sys_Trims_SetVDDM_CP(VDDCM_MIN_CP_DELTA_TARGET);
```

18.27.4.13 Sys_Trims_SetVDDOD

```
unsigned int Sys_Trims_SetVDDOD(unsigned int target)
```

Sets the requested trim value to the VDDOD register if it is found.

Location: trims.h:204

Parameters

Direction	Name	Description
in	<i>target</i>	The specified target voltage [10 mV] target

Return

result SYS_ERRNO_NO_ERROR if requested trim is found, SYS_ERRNO_NO_MATCH otherwise.

Example Code for Sys_Trims_SetVDDOD

```
// Set (VDDOD_ACTIVE_TARGET / 10) trim value to the VDDOD register
result = Sys_Trims_SetVDDOD((VDDOD_ACTIVE_TARGET / 10));
```

18.27.4.14 Sys_Trims_SetVMIC

```
unsigned int Sys_Trims_SetVMIC(unsigned int target)
```

Sets the requested trim value to the VDDOD register if it is found.

Location: trims.h:212

Parameters

Direction	Name	Description
in	<i>target</i>	The specified target voltage [10 mV] target

Return

result SYS_ERRNO_NO_ERROR if requested trim is found, SYS_ERRNO_NO_MATCH otherwise.

Example Code for Sys_Trims_SetVMIC

```
// Set (VMIC_ACTIVE_TARGET / 10) trim value to the VMIC register  
result = Sys_Trims_SetVMIC((VMIC_ACTIVE_TARGET / 10));
```

18.27.4.15 Sys_Trims_SetOperatingFrequency

```
unsigned int Sys_Trims_SetOperatingFrequency(unsigned int frequency_index)
```

Set the internal oscillator base operating frequency.

Location: trims.h:228

Parameters

Direction	Name	Description
in	<i>frequency_index</i>	Index corresponding to the desired trim value in the calibration table. Operating frequency index; use SYS_FREQ_*

Return

frequency_match If SYS_ERRNO_NO_ERROR, new operating frequency set. If SYS_ERRNO_NO_MATCH, could not set frequency (no calibration found for the requested operating frequency)

Assumptions

Calibration table loaded.

Example Code for Sys_Trims_SetOperatingFrequency

```
// Load the 15.36 MHz calibrated clock setting
result = Sys_Trims_SetOperatingFrequency(SYS_FREQ_15M36);
```

18.27.4.16 Sys_Trims_SetOperatingFrequencyMult

```
unsigned int Sys_Trims_SetOperatingFrequencyMult(unsigned int frequency_index, unsigned int
multiplier)
```

Set the internal oscillator base operating frequency and a custom multiplier.

The oscillator output frequency is the base operating frequency multiplied by the custom multiplier.

Location: trims.h:248

Parameters

Direction	Name	Description
in	<i>frequency_index</i>	Index corresponding to the desired trim value in the calibration table. Operating frequency index; use SYS_FREQ_*
in	<i>multiplier</i>	Oscillator multiplication setting; use OSC_MULTIPLY_BY_*

Return

Ezairo 8300 Firmware Reference

frequency_match If SYS_ERRNO_NO_ERROR, new operating frequency set. If SYS_ERRNO_NO_MATCH, could not set frequency (no calibration found for the requested operating frequency)

Assumptions

Calibration table loaded.

Example Code for Sys_Trims_SetOperatingFrequencyMult

```
// Load the 31.36 MHz calibrated clock setting and a custom multiplier value of 2
result = Sys_Trims_SetOperatingFrequencyMult(SYS_FREQ_15M36, OSC_MULTIPLY_BY_2);
```

18.27.4.17 Sys_Trims_SetOperatingFrequencyThrottle

```
unsigned int Sys_Trims_SetOperatingFrequencyThrottle(unsigned int frequency_index, unsigned
int slowclk_prescale, unsigned int sdmclk_prescale, unsigned int adcclk_prescale)
```

Set the internal oscillator base operating frequency, using clock throttling to also configure SLOWCLK and SDMCLK.

The SLOWCLK, SDMCLK and ADCCLK prescalers are updated to match the updated operating frequency only if the base operating frequency changes.

Location: trims.h:273

Parameters

Direction	Name	Description
in	<i>frequency_index</i>	Operating frequency index; use SYS_FREQ_*
in	<i>slowclk_prescale</i>	SLOWCLK prescale setting; use SLOWCLK_PRESCALE_*
in	<i>sdmclk_prescale</i>	SDMCLK prescale setting; use SDMCLK_PRESCALE_*
in	<i>adcclk_prescale</i>	ADCCLK prescale setting; use ADCCLK_PRESCALE_*

Return

frequency_match If SYS_ERRNO_NO_ERROR, new operating frequency set. If SYS_ERRNO_NO_MATCH, could not set frequency (no calibration found for the requested operating frequency)

Assumptions

Calibration table loaded.

Example Code for Sys_Trims_SetOperatingFrequencyThrottle

```
// Load the 15.36 MHz calibrated clock setting with clock
// throttling enabled.
result = Sys\_Trims\_SetOperatingFrequencyThrottle(SYS_FREQ_15M36,
                                                SLOWCLK_PRESCALE_12,
                                                SDMCLK_PRESCALE_2,
                                                ADCCLK_PRESCALE_4);
```

18.27.4.18 Sys_Trims_SetOperatingFrequencyThrottleMult

```
unsigned int Sys_Trims_SetOperatingFrequencyThrottleMult(unsigned int frequency_index,
unsigned int multiplier, unsigned int slowclk_prescale, unsigned int sdmclk_prescale,
unsigned int adccclk_prescale)
```

Set the internal oscillator base operating frequency and a custom multiplier, using clock throttling to also configure SLOWCLK and SDMCLK.

The SLOWCLK, SDMCLK and ADCCLK prescalers are updated to match the updated operating frequency only if the base operating frequency changes. The oscillator output frequency is the base operating frequency multiplied by the custom multiplier.

Location: trims.h:304

Parameters

Ezairo 8300 Firmware Reference

Direction	Name	Description
in	<i>frequency_index</i>	Operating frequency index; use SYS_FREQ_*
in	<i>multiplier</i>	Oscillator multiplication setting; use OSC_MULTIPLY_BY_*
in	<i>slowclk_prescale</i>	SLOWCLK prescale setting; use SLOWCLK_PRESCALE_*
in	<i>sdmclk_prescale</i>	SDMCLK prescale setting; use SDMCLK_PRESCALE_*
in	<i>adcclk_prescale</i>	ADCCLK prescale setting; use ADCCLK_PRESCALE_*

Return

frequency_match If SYS_ERRNO_NO_ERROR, new operating frequency set. If SYS_ERRNO_NO_MATCH, could not set frequency (no calibration found for the requested operating frequency)

Assumptions

Calibration table loaded.

Example Code for Sys_Trims_SetOperatingFrequencyThrottleMult

```
// Load the 31.36 MHz calibrated clock setting with clock
// throttling enabled and a custom multiplier value of 2
result = Sys_Trims_SetOperatingFrequencyThrottleMult(SYS_FREQ_31M36,
                                                    OSC_MULTIPLY_BY_2,
                                                    SLOWCLK_PRESCALE_12,
                                                    SDMCLK_PRESCALE_2,
                                                    ADCCLK_PRESCALE_4);
```

18.27.4.19 Sys_Trims_SetADCOffsets

```
unsigned int Sys_Trims_SetADCOffsets()
```

Set the ADC offset trims for all of the ADCs.

Location: trims.h:315

Return

result SYS_ERRNO_NO_ERROR if all the trims exist, SYS_ERRNO_NO_MATCH otherwise.

Example Code for Sys_Trims_SetADCOffsets

```
// Find the ADC offset trim values for all the ADCs in the manufacturing
// information table and set them if found
result = Sys\_Trims\_SetADCOffsets();
```

18.28 UART

UART Hardware Abstraction Layer.

18.28.1 Summary

Macros

- [SYS_UART_DIOCONFIG](#) : Macro wrapper for Sys_UART_DIOConfig(...) Configure two DIOs for the default UART interface.
- [SYS_UART_CONFIG](#) : Macro wrapper for Sys_UART_Config(...) Enable the default UART at the specified baud rate.

Functions

- [Sys_UART_DIOConfig](#) : Configure two DIOs for the specified UART interface.
- [Sys_UART_Config](#) : Enable a UART at the specified baud rate.

18.28.2 UART Macro Definition Documentation

18.28.2.1 SYS_UART_DIOCONFIG

```
#define SYS_UART_DIOCONFIG Sys\_UART\_DIOConfig(UART, (cfg), (pad_tx), (pad_rx))
```

Macro wrapper for Sys_UART_DIOConfig(...) Configure two DIOs for the default UART interface.

Location: uart.h:90

Parameters

Direction	Name	Description
in	<i>cfg</i>	DIO pin configuration for the UART pads
in	<i>pad_tx</i>	DIO to use as the UART transmit pad
in	<i>pad_rx</i>	DIO to use as the UART receive pad

Example Code for SYS_UART_DIOCONFIG

```
// Configure DIOs 16 and 4 for the default UART interface with
// low-pass filter disabled, 8X drive-strength, and 1K pull-up resistors
SYS_UART_DIOCONFIG((DIO_LPF_DISABLE | DIO_1K_PULL_UP |
DIO_8X_DRIVE), DIO16, DIO4);
```

18.28.2.2 SYS_UART_CONFIG

```
#define SYS_UART_CONFIG Sys\_UART\_Config(UART, (uart_clk_hz), (baud), (config))
```

Macro wrapper for Sys_UART_Config(...) Enable the default UART at the specified baud rate.

Location: uart.h:106

Parameters

Direction	Name	Description
in	<i>uart_clk_hz</i>	UART clock speed in hertz
in	<i>baud</i>	Baud rate to which UART* is configured
in	<i>config</i>	DMA mode enable; use UART_TX_DMA_[ENABLE DISABLE] UART_RX_DMA_[ENABLE DISABLE] UART_TX_INT_[ENABLE DISABLE] UART_RX_INT_[ENABLE DISABLE] UART_OVERRUN_INT_[ENABLE DISABLE]

Ezairo 8300 Firmware Reference

Example Code for SYS_UART_CONFIG

```
// Enable and Configure the default UART:
// - 7.68 MHz clock speed
// - 9600 Hz baud rate
// - A TX DMA request is generated when new data is
//   requested by the UART interface
// - An RX DMA request is generated when new data is
//   received by the UART interface
// - Interrupts enabled
SYS_UART_CONFIG(7680000, 9600, (UART_TX_DMA_ENABLE |
                                UART_RX_DMA_ENABLE | UART_TX_START_INT_ENABLE |
                                UART_RX_INT_ENABLE | UART_OVERRUN_INT_ENABLE));
```

18.28.3 UART Function Documentation**18.28.3.1 Sys_UART_DIOConfig**

```
void Sys_UART_DIOConfig(const UART_Type * uart, uint32_t cfg, uint32_t pad_tx, uint32_t
pad_rx)
```

Configure two DIOs for the specified UART interface.

Location: uart.h:46

Parameters

Direction	Name	Description
in	<i>uart</i>	Pointer to the UART instance
in	<i>cfg</i>	DIO pin configuration for the UART pads
in	<i>pad_tx</i>	DIO to use as the UART transmit pad
in	<i>pad_rx</i>	DIO to use as the UART receive pad

Ezairo 8300 Firmware Reference

Example Code for Sys_UART_DIOConfig

```
// Configure DIOs 16 and 4 for the UART interface with
// low-pass filter disabled, 8X drive-strength, and 1K pull-up resistors
Sys_UART_DIOConfig(UART, (DIO_LPF_DISABLE | DIO_1K_PULL_UP |
    DIO_8X_DRIVE), DIO16, DIO4);
```

18.28.3.2 Sys_UART_Config

```
void Sys_UART_Config(UART_Type * uart, uint32_t uart_clk_hz, uint32_t baud, uint32_t
config)
```

Enable a UART at the specified baud rate.

Location: uart.h:79

Parameters

Direction	Name	Description
in	<i>uart</i>	Pointer to the UART instance
in	<i>uart_clk_hz</i>	UART clock speed in hertz
in	<i>baud</i>	Baud rate to which UART* is configured
in	<i>config</i>	DMA mode enable; use UART_TX_DMA_[ENABLE DISABLE] UART_RX_DMA_[ENABLE DISABLE] UART_TX_INT_[ENABLE DISABLE] UART_RX_INT_[ENABLE DISABLE] UART_OVERRUN_INT_[ENABLE DISABLE]

Example Code for Sys_UART_Config

```
// Enable and Configure a UART:
// - 7.68 MHz clock speed
// - 9600 Hz baud rate
// - A TX DMA request is generated when new data is
//   requested by the UART interface
// - An RX DMA request is generated when new data is
//   received by the UART interface
// - Interrupts enabled
Sys_UART_Config(UART, 7680000, 9600, (UART_TX_DMA_ENABLE |
    UART_RX_DMA_ENABLE | UART_TX_START_INT_ENABLE |
    UART_RX_INT_ENABLE | UART_OVERRUN_INT_ENABLE));
```

18.29 VERSIONS

Hardware Abstraction Layer versions.

18.29.1 Summary

Variables

- [Sys_Version](#) : Exported global variable defined in version.c.

Macros

- [SYS_VER_MAJOR](#) : Major version number.
- [SYS_VER_MINOR](#) : Minor version number.
- [SYS_VER_REVISION](#) : Revision number.
- [SYS_VER](#) : Full version number (SK5_CID = 101)

18.29.2 Versions Variable Documentation

18.29.2.1 Sys_Version

```
const short Sys_Version
```

Location: version.h:56

Exported global variable defined in version.c.

18.29.3 Versions Macro Definition Documentation

18.29.3.1 SYS_VER_MAJOR

```
#define SYS_VER_MAJOR 0x01
```

Major version number.

Location: version.h:40

18.29.3.2 SYS_VER_MINOR

```
#define SYS_VER_MINOR 0x01
```

Minor version number.

Location: version.h:41

18.29.3.3 SYS_VER_REVISION

```
#define SYS_VER_REVISION 0x00
```

Revision number.

Location: version.h:42

18.29.3.4 SYS_VER

```
#define SYS_VER ((SYS\_VER\_MAJOR << 12) | \
                (SYS\_VER\_MINOR << 8) | \
                (SYS\_VER\_REVISION))
```

Full version number (SK5_CID = 101)

Location: version.h:45

18.30 WATCHDOG TIMER

Watchdog timer Hardware Abstraction Layer.

18.30.1 Summary

Macros

- [SYS_WATCHDOG_REFRESH](#) : Refresh the watchdog timer count.
- [SYS_WATCHDOG_RESET](#) : Reset the device using the watchdog timer.

18.30.2 Watchdog Timer Macro Definition Documentation

18.30.2.1 SYS_WATCHDOG_REFRESH

```
#define SYS_WATCHDOG_REFRESH D_SYSTEM->CTRL = SYSTEM_WATCHDOG_REFRESH
```

Refresh the watchdog timer count.

Location: watchdog.h:41

Example Code for SYS_WATCHDOG_REFRESH

```
// Refresh the watchdog timer count
while(1) {
    SYS\_WATCHDOG\_REFRESH();
}
```

18.30.2.2 SYS_WATCHDOG_RESET

```
#define SYS_WATCHDOG_RESET D_WATCHDOG->RESET = WATCHDOG_FORCE_RESET
```

Reset the device using the watchdog timer.

Location: watchdog.h:47

Example Code for SYS_WATCHDOG_RESET

```
// Reset the device using the watchdog timer
SYS\_WATCHDOG\_RESET();
```

CHAPTER 19

CFX System Library Reference

System support library is implemented for the CFX core and provides a collection of functions including a software delay and system reset function.

19.1 SUMMARY

Macros

- [SYSLIB_ERRNO_NO_ERROR](#) : Error Code: Generic pass.
- [SYSLIB_ERRNO_FREQ_NO_MATCH](#) : Requested frequency not found.
- [SYSLIB_OSC_MUL_LIMIT1](#) : Multiply oscillator by 2 if higher than this limit, and lower than STSLIB_OSC_MUL_LIMIT2.
- [STSLIB_OSC_MUL_LIMIT2](#) : Multiply oscillator by 4 if higher than this limit.

Functions

- [SYSLIB_Reset](#) : Reset the system with a watchdog timeout.
- [SYSLIB_Delay](#) : Delay the CFX core for the specified period of time.
- [SYSLIB_DelayBase](#) : Delay the core for the specified amount of time.
- [SYSLIB_SetOperatingFrequency](#) : Set the internal oscillator base operating frequency; the oscillator multiplication setting is not a component of the base operating frequency configuration.
- [SYSLIB_SetOperatingFrequency_Throttle](#) : Set the internal oscillator base operating frequency, using clock throttling to also configure SLOWCLK and SDMCLK.

19.2 CFX SYSTEM LIBRARY REFERENCE MACRO DEFINITION DOCUMENTATION

19.2.1 SYSLIB_ERRNO_NO_ERROR

```
#define SYSLIB_ERRNO_NO_ERROR 0x0
```

Error Code: Generic pass.

Location: sk5_syslib.h:37

19.2.2 SYSLIB_ERRNO_FREQ_NO_MATCH

```
#define SYSLIB_ERRNO_FREQ_NO_MATCH 0x1
```

Requested frequency not found.

Location: sk5_syslib.h:38

19.2.3 SYSLIB_OSC_MUL_LIMIT1

```
#define SYSLIB_OSC_MUL_LIMIT1 SYS_FREQ_29M44
```

Multiply oscillator by 2 if higher than this limit, and lower than STSLIB_OSC_MUL_LIMIT2.

Location: sk5_syslib.h:41

19.2.4 STSLIB_OSC_MUL_LIMIT2

```
#define STSLIB_OSC_MUL_LIMIT2 SYS_FREQ_59M52
```

Multiply oscillator by 4 if higher than this limit.

Location: sk5_syslib.h:42

19.3 CFX SYSTEM LIBRARY REFERENCE FUNCTION DOCUMENTATION

19.3.1 SYSLIB_Reset

```
void SYSLIB_Reset()
```

Reset the system with a watchdog timeout.

Location: sk5_syslib.h:49

Example Code for SYSLIB_Reset

```
// Reset the system with a watchdog timeout  
SYSLIB\_Reset();
```

Example Code for SYSLIB_Reset

```
// Reset the system with a watchdog timeout  
call.1d SYSLIB\_Reset  
nop // [ds] */
```

19.3.2 SYSLIB_Delay

```
void SYSLIB_Delay(uint24_t ms)
```

Delay the CFX core for the specified period of time.

Location: sk5_syslib.h:59

Parameters

Direction	Name	Description
in	<i>ms</i>	Delay in ms

Assumptions

Valid SYSVAR_OPERATING_FREQ set by firmware.

Example Code for SYSLIB_Delay

```
// Delay the CFX core for 100 ms  
SYSLIB\_Delay(100);
```

Example Code for SYSLIB_Delay

```
// Delay the CFX core for 100 ms  
call.1d SYSLIB\_Delay  
load b1, 100 // [ds] */ Load delay value
```

Ezairo 8300 Firmware Reference

19.3.3 SYSLIB_DelayBase

```
void SYSLIB_DelayBase(uint24_t ms, uint24_t iterations)
```

Delay the core for the specified amount of time.

Location: sk5_syslib.h:71

Parameters

Direction	Name	Description
in	<i>ms</i>	Delay in ms
in	<i>iterations</i>	Inner loop iterations required for 1 ms (accounting for overhead)

Assumptions

Valid SYSVAR_OPERATING_FREQ set by firmware.

Example Code for SYSLIB_DelayBase

```
// Delay the core for 100 ms using 2 inner loop iterations
// required for 1 ms (accounting for overhead)
SYSLIB\_DelayBase(100, 2);
```

Example Code for SYSLIB_DelayBase

```
// Delay the core for 100 ms using 2 inner loop iterations
// required for 1 ms (accounting for overhead)
load b1, 100
call.l1d SYSLIB\_Delay
load y1, 2 // [ds] */ Load iterations value
```

19.3.4 SYSLIB_SetOperatingFrequency

```
uint24_t SYSLIB_SetOperatingFrequency()
```

Set the internal oscillator base operating frequency; the oscillator multiplication setting is not a component of the base operating frequency configuration.

Location: sk5_syslib.h:80

DeprecatedUse [Sys_Trims_SetOperatingFrequency\(\)](#)

19.3.5 SYSLIB_SetOperatingFrequency_Throttle

```
uint24_t SYSLIB_SetOperatingFrequency_Throttle()
```

Set the internal oscillator base operating frequency, using clock throttling to also configure SLOWCLK and SDMCLK.

The SLOWCLK, SDMCLK and ADCCLK prescalers are updated to match the updated operating frequency only if the base operating frequency changes. The oscillator multiplication setting is not a component of the base operating frequency configuration.

Location: sk5_syslib.h:93

DeprecatedUse [Sys_Trims_SetOperatingFrequencyThrottle\(\)](#)

CHAPTER 20

CMSIS Reference

Hardware register abstraction layer for the SOC.

20.1 SUMMARY

Variables

- [Heap Begin](#) : Linker-populated variable: Start of the heap.
- [Heap Limit](#) : Linker-populated variable: Limit of the heap.
- [bss start](#) : Linker-populated variable: Start of the BSS area.
- [bss end](#) : Linker-populated variable: End of the BSS area.
- [preinit array start](#) : Linker-populated variable: Start of the pre-init data.
- [preinit array end](#) : Linker-populated variable: End of the pre-init data.
- [init array start](#) : Linker-populated variable: Start of the init data.
- [init array end](#) : Linker-populated variable: End of the init data.

Macros

- [DIO0_IRQHandler](#) : Alias for DIO_0_IRQHandler.
- [DIO1_IRQHandler](#) : Alias for DIO_1_IRQHandler.
- [DIO2_IRQHandler](#) : Alias for DIO_2_IRQHandler.
- [DIO3_IRQHandler](#) : Alias for DIO_3_IRQHandler.
- [DIO0_IRQn](#) : Alias for DIO_0_IRQn.
- [DIO1_IRQn](#) : Alias for DIO_1_IRQn.
- [DIO2_IRQn](#) : Alias for DIO_2_IRQn.
- [DIO3_IRQn](#) : Alias for DIO_3_IRQn.
- [DMA0_IRQHandler](#) : Alias for DMA_0_IRQHandler.
- [DMA1_IRQHandler](#) : Alias for DMA_1_IRQHandler.
- [DMA2_IRQHandler](#) : Alias for DMA_2_IRQHandler.
- [DMA3_IRQHandler](#) : Alias for DMA_3_IRQHandler.
- [DMA0_IRQn](#) : Alias for DMA_0_IRQn.
- [DMA1_IRQn](#) : Alias for DMA_1_IRQn.
- [DMA2_IRQn](#) : Alias for DMA_2_IRQn.
- [DMA3_IRQn](#) : Alias for DMA_3_IRQn.
- [I2C0_IRQHandler](#) : Alias for I2C_0_IRQHandler.
- [I2C1_IRQHandler](#) : Alias for I2C_1_IRQHandler.
- [I2C0_IRQn](#) : Alias for I2C_0_IRQn.
- [I2C1_IRQn](#) : Alias for I2C_1_IRQn.
- [SPI0_RX_IRQHandler](#) : Alias for SPI_0_RX_IRQHandler.
- [SPI0_TX_IRQHandler](#) : Alias for SPI_0_TX_IRQHandler.
- [SPI0_COM_IRQHandler](#) : Alias for SPI_0_COM_IRQHandler.
- [SPI1_RX_IRQHandler](#) : Alias for SPI_1_RX_IRQHandler.
- [SPI1_TX_IRQHandler](#) : Alias for SPI_1_TX_IRQHandler.
- [SPI1_COM_IRQHandler](#) : Alias for SPI_1_COM_IRQHandler.
- [SPI0_RX_IRQn](#) : Alias for SPI_0_RX_IRQn.

Ezairo 8300 Firmware Reference

- [SPI0_TX_IRQn](#) : Alias for SPI_0_TX_IRQn.
- [SPI0_COM_IRQn](#) : Alias for SPI_0_COM_IRQn.
- [SPI1_RX_IRQn](#) : Alias for SPI_1_RX_IRQn.
- [SPI1_TX_IRQn](#) : Alias for SPI_1_TX_IRQn.
- [SPI1_COM_IRQn](#) : Alias for SPI_1_COM_IRQn.
- [TIMER0_IRQHandler](#) : Alias for TIMER_0_IRQHandler.
- [TIMER1_IRQHandler](#) : Alias for TIMER_1_IRQHandler.
- [TIMER2_IRQHandler](#) : Alias for TIMER_2_IRQHandler.
- [TIMER3_IRQHandler](#) : Alias for TIMER_3_IRQHandler.
- [TIMER0_IRQn](#) : Alias for TIMER_0_IRQn.
- [TIMER1_IRQn](#) : Alias for TIMER_1_IRQn.
- [TIMER2_IRQn](#) : Alias for TIMER_2_IRQn.
- [TIMER3_IRQn](#) : Alias for TIMER_3_IRQn.
- [UART0_RX_IRQHandler](#) : Alias for UART_RX_IRQHandler.
- [UART0_TX_IRQHandler](#) : Alias for UART_TX_IRQHandler.
- [UART0_ERROR_IRQHandler](#) : Alias for UART_ERROR_IRQHandler.
- [UART0_RX_IRQn](#) : Alias for UART_RX_IRQn.
- [UART0_TX_IRQn](#) : Alias for UART_TX_IRQn.
- [UART0_ERROR_IRQn](#) : Alias for UART_ERROR_IRQn.
- [CM3_REV](#) : Core revision r2p2.
- [NVIC_PRIO_BITS](#) : 3 bits used for interrupt priority levels
- [MPU_PRESENT](#) : MPU not present.
- [FPU_PRESENT](#) : FPU not present.
- [Vendor_SysTickConfig](#) : Standard SysTick configuration is used.
- [ASCC_REF_VALID](#) : Validation of Audio Sink Clock Counter register block pointer reference for assert statements.
- [CRC_REF_VALID](#) : Validation of CRC register block pointer reference for assert statements.
- [DMA_REF_VALID](#) : Validation of DMA register block pointer reference for assert statements.
- [DIO_PAD_COUNT](#) : Number of DIO pads.
- [DIO_GROUP_LOW_PAD_RANGE](#) : Number of DIO pads in the lowest group.
- [DIO_EVENT_CHANNEL_COUNT](#) : Number of available DIO interrupts.
- [DIO_CLK_DIV_COUNT](#) : Number of DIO clock dividers.
- [DIO0](#) : DIO 0.
- [DIO1](#) : DIO 1.
- [DIO2](#) : DIO 2.
- [DIO3](#) : DIO 3.
- [DIO4](#) : DIO 4.
- [DIO5](#) : DIO 5.
- [DIO6](#) : DIO 6.
- [DIO7](#) : DIO 7.
- [DIO8](#) : DIO 8.
- [DIO9](#) : DIO 9.
- [DIO10](#) : DIO 10.
- [DIO11](#) : DIO 11.
- [DIO12](#) : DIO 12.
- [DIO13](#) : DIO 13.
- [DIO14](#) : DIO 14.
- [DIO15](#) : DIO 15.
- [DIO16](#) : DIO 16.

Ezairo 8300 Firmware Reference

- [DIO17](#) : DIO 17.
- [DIO18](#) : DIO 18.
- [DIO19](#) : DIO 19.
- [DIO20](#) : DIO 20.
- [DIO21](#) : DIO 21.
- [DIO22](#) : DIO 22.
- [DIO23](#) : DIO 23.
- [DIO24](#) : DIO 24.
- [DIO25](#) : DIO 25.
- [DIO26](#) : DIO 26.
- [DIO27](#) : DIO 27.
- [DIO28](#) : DIO 28.
- [DIO29](#) : DIO 29.
- [DIO30](#) : DIO 30.
- [DIO31](#) : DIO 31.
- [DIO32](#) : DIO 32.
- [DIO33](#) : DIO 33.
- [DIO34](#) : DIO 34.
- [DIO35](#) : DIO 35.
- [I2C_REF_VALID](#) : Validation of I2C register block pointer reference for assert statements.
- [PCM_REF_VALID](#) : Validation of PCM register block pointer reference for assert statements.
- [SPI_REF_VALID](#) : Validation of SPI register block pointer reference for assert statements.
- [TIMER_REF_VALID](#) : Validation of TIMER register block pointer reference for assert statements.
- [UART_REF_VALID](#) : Validation of UART register block pointer reference for assert statements.
- [SYS_DUMMY_READ](#) : Register that always reads back as 0x00000000.
- [SYS_DUMMY_WRITE](#) : Register to which writes are ineffective.
- [ERRNO_NO_ERROR](#) : Error Code: Generic pass.
- [ERRNO_GENERAL_FAILURE](#) : Error Code: General failure.
- [SYS_DELAY_CALL_OVERHEAD](#) : SYS_DELAY call overhead is 7 cycles.
- [SYS_DELAY_LOOP_OVERHEAD](#) : SYS_DELAY loop accuracy is 4 cycles.
- [SYS_RESET](#) : Reset the system with a watchdog timeout.
- [SYSVAR_FREQ_FORMAT](#) : Convert a valid frequency index to Hz.
- [SYSVAR_SET_FREQ](#) : Updates the SYSVAR locations that keep track of frequency.
- [SYS_BOOT_ALTERNATE](#) : Set SYSVAR_BOOT_APPN to an alternate app number and jump to the bootloader's reset handler.
- [DEFAULT_FREQ](#) : The default clock frequency is 7.68 MHz.
- [DEFAULT_FREQ_IDX](#) : The default clock index is 7.68 MHz.
- [SystemCoreClock](#) : Variable that holds the current SystemCoreClock frequency.

Functions

- [_start](#) : Initialize the application data and start execution with main.
- [_sbrk](#) : Increment (or decrement) the top of the heap.
- [Sys_Delay](#) : Delay by the specified number of clock cycles.
- [SystemInit](#) : Sets up the system core clock variable; assumes the ROM has previously initialized the system.
- [SystemCoreClockUpdate](#) : Update the declared system clock (SYSCLK) frequency used by the Arm Cortex-M3 core firmware.

Ezairo 8300 Firmware Reference

- [System_SetOperatingFrequency](#) : Set the internal oscillator base operating frequency; the oscillator multiplication setting is not a component of the base operating frequency configuration.
- [System_SetOperatingFrequency_Throttle](#) : Set the internal oscillator base operating frequency, using clock throttling to also configure SLOWCLK and SDMCLK.

20.2 CMSIS REFERENCE VARIABLE DOCUMENTATION

20.2.1 __Heap_Begin__

uint8_t __Heap_Begin__

Location: sk5_start.h:40

Linker-populated variable: Start of the heap.

20.2.2 __Heap_Limit__

uint8_t __Heap_Limit__

Location: sk5_start.h:41

Linker-populated variable: Limit of the heap.

20.2.3 __bss_start__

uint32_t __bss_start__

Location: sk5_start.h:46

Linker-populated variable: Start of the BSS area.

20.2.4 __bss_end__

uint32_t __bss_end__

Location: sk5_start.h:47

Linker-populated variable: End of the BSS area.

20.2.5 __preinit_array_start__

```
void(* __preinit_array_start__[]) (void)
```

Location: sk5_start.h:52

Linker-populated variable: Start of the pre-init data.

20.2.6 __preinit_array_end__

```
void(* __preinit_array_end__[]) (void)
```

Location: sk5_start.h:53

Linker-populated variable: End of the pre-init data.

20.2.7 __init_array_start__

```
void(* __init_array_start__[]) (void)
```

Location: sk5_start.h:54

Linker-populated variable: Start of the init data.

20.2.8 __init_array_end__

```
void(* __init_array_end__[]) (void)
```

Location: sk5_start.h:55

Linker-populated variable: End of the init data.

20.3 CMSIS REFERENCE MACRO DEFINITION DOCUMENTATION

20.3.1 DIO0_IRQHandler

```
#define DIO0_IRQHandler DIO_0_IRQHandler
```

Alias for DIO_0_IRQHandler.

Location: sk5.h:54

20.3.2 DIO1_IRQHandler

```
#define DIO1_IRQHandler DIO_1_IRQHandler
```

Alias for DIO_1_IRQHandler.

Location: sk5.h:55

20.3.3 DIO2_IRQHandler

```
#define DIO2_IRQHandler DIO_2_IRQHandler
```

Alias for DIO_2_IRQHandler.

Location: sk5.h:56

20.3.4 DIO3_IRQHandler

```
#define DIO3_IRQHandler DIO_3_IRQHandler
```

Alias for DIO_3_IRQHandler.

Location: sk5.h:57

20.3.5 DIO0_IRQn

```
#define DIO0_IRQn DIO_0_IRQn
```

Alias for DIO_0_IRQn.

Location: sk5.h:58

20.3.6 DIO1_IRQn

```
#define DIO1_IRQn DIO_1_IRQn
```

Alias for DIO_1_IRQn.

Location: sk5.h:59

20.3.7 DIO2_IRQn

```
#define DIO2_IRQn DIO_2_IRQn
```

Alias for DIO_2_IRQn.

Location: sk5.h:60

20.3.8 DIO3_IRQn

```
#define DIO3_IRQn DIO_3_IRQn
```

Alias for DIO_3_IRQn.

Location: sk5.h:61

20.3.9 DMA0_IRQHandler

```
#define DMA0_IRQHandler DMA_0_IRQHandler
```

Alias for DMA_0_IRQHandler.

Location: sk5.h:62

20.3.10 DMA1_IRQHandler

```
#define DMA1_IRQHandler DMA_1_IRQHandler
```

Alias for DMA_1_IRQHandler.

Location: sk5.h:63

20.3.11 DMA2_IRQHandler

```
#define DMA2_IRQHandler DMA_2_IRQHandler
```

Alias for DMA_2_IRQHandler.

Location: sk5.h:64

20.3.12 DMA3_IRQHandler

```
#define DMA3_IRQHandler DMA_3_IRQHandler
```

Alias for DMA_3_IRQHandler.

Location: sk5.h:65

20.3.13 DMA0_IRQn

```
#define DMA0_IRQn DMA_0_IRQn
```

Alias for DMA_0_IRQn.

Location: sk5.h:66

20.3.14 DMA1_IRQn

```
#define DMA1_IRQn DMA_1_IRQn
```

Alias for DMA_1_IRQn.

Location: sk5.h:67

20.3.15 DMA2_IRQn

```
#define DMA2_IRQn DMA_2_IRQn
```

Alias for DMA_2_IRQn.

Location: sk5.h:68

20.3.16 DMA3_IRQn

```
#define DMA3_IRQn DMA_3_IRQn
```

Alias for DMA_3_IRQn.

Location: sk5.h:69

20.3.17 I2C0_IRQHandler

```
#define I2C0_IRQHandler I2C_0_IRQHandler
```

Alias for I2C_0_IRQHandler.

Location: sk5.h:70

20.3.18 I2C1_IRQHandler

```
#define I2C1_IRQHandler I2C_1_IRQHandler
```

Alias for I2C_1_IRQHandler.

Location: sk5.h:71

20.3.19 I2C0_IRQn

```
#define I2C0_IRQn I2C_0_IRQn
```

Alias for I2C_0_IRQn.

Location: sk5.h:72

20.3.20 I2C1_IRQn

```
#define I2C1_IRQn I2C_1_IRQn
```

Alias for I2C_1_IRQn.

Location: sk5.h:73

20.3.21 SPI0_RX_IRQHandler

```
#define SPI0_RX_IRQHandler SPI_0_RX_IRQHandler
```

Alias for SPI_0_RX_IRQHandler.

Location: sk5.h:74

20.3.22 SPI0_TX_IRQHandler

```
#define SPI0_TX_IRQHandler SPI_0_TX_IRQHandler
```

Alias for SPI_0_TX_IRQHandler.

Location: sk5.h:75

20.3.23 SPI0_COM_IRQHandler

```
#define SPI0_COM_IRQHandler SPI_0_COM_IRQHandler
```

Alias for SPI_0_COM_IRQHandler.

Location: sk5.h:76

20.3.24 SPI1_RX_IRQHandler

```
#define SPI1_RX_IRQHandler SPI_1_RX_IRQHandler
```

Alias for SPI_1_RX_IRQHandler.

Location: sk5.h:77

20.3.25 SPI1_TX_IRQHandler

```
#define SPI1_TX_IRQHandler SPI_1_TX_IRQHandler
```

Alias for SPI_1_TX_IRQHandler.

Location: sk5.h:78

20.3.26 SPI1_COM_IRQHandler

```
#define SPI1_COM_IRQHandler SPI_1_COM_IRQHandler
```

Alias for SPI_1_COM_IRQHandler.

Location: sk5.h:79

20.3.27 SPI0_RX_IRQn

```
#define SPI0_RX_IRQn SPI_0_RX_IRQn
```

Alias for SPI_0_RX_IRQn.

Location: sk5.h:80

20.3.28 SPI0_TX_IRQn

```
#define SPI0_TX_IRQn SPI_0_TX_IRQn
```

Alias for SPI_0_TX_IRQn.

Location: sk5.h:81

20.3.29 SPI0_COM_IRQn

```
#define SPI0_COM_IRQn SPI_0_COM_IRQn
```

Alias for SPI_0_COM_IRQn.

Location: sk5.h:82

20.3.30 SPI1_RX_IRQn

```
#define SPI1_RX_IRQn SPI_1_RX_IRQn
```

Alias for SPI_1_RX_IRQn.

Location: sk5.h:83

20.3.31 SPI1_TX_IRQn

```
#define SPI1_TX_IRQn SPI_1_TX_IRQn
```

Alias for SPI_1_TX_IRQn.

Location: sk5.h:84

20.3.32 SPI1_COM_IRQn

```
#define SPI1_COM_IRQn SPI_1_COM_IRQn
```

Alias for SPI_1_COM_IRQn.

Location: sk5.h:85

20.3.33 TIMER0_IRQHandler

```
#define TIMER0_IRQHandler TIMER_0_IRQHandler
```

Alias for TIMER_0_IRQHandler.

Location: sk5.h:86

20.3.34 TIMER1_IRQHandler

```
#define TIMER1_IRQHandler TIMER_1_IRQHandler
```


Alias for TIMER_1_IRQHandler.

Location: sk5.h:87

20.3.35 TIMER2_IRQHandler

```
#define TIMER2_IRQHandler TIMER_2_IRQHandler
```

Alias for TIMER_2_IRQHandler.

Location: sk5.h:88

20.3.36 TIMER3_IRQHandler

```
#define TIMER3_IRQHandler TIMER_3_IRQHandler
```

Alias for TIMER_3_IRQHandler.

Location: sk5.h:89

20.3.37 TIMER0_IRQn

```
#define TIMER0_IRQn TIMER_0_IRQn
```

Alias for TIMER_0_IRQn.

Location: sk5.h:90

20.3.38 TIMER1_IRQn

```
#define TIMER1_IRQn TIMER_1_IRQn
```

Alias for TIMER_1_IRQn.

Location: sk5.h:91

20.3.39 TIMER2_IRQn

```
#define TIMER2_IRQn TIMER_2_IRQn
```

Alias for TIMER_2_IRQn.

Location: sk5.h:92

20.3.40 TIMER3_IRQn

```
#define TIMER3_IRQn TIMER_3_IRQn
```

Alias for TIMER_3_IRQn.

Location: sk5.h:93

20.3.41 UART0_RX_IRQHandler

```
#define UART0_RX_IRQHandler UART_RX_IRQHandler
```

Alias for UART_RX_IRQHandler.

Location: sk5.h:94

20.3.42 UART0_TX_IRQHandler

```
#define UART0_TX_IRQHandler UART_TX_IRQHandler
```

Alias for UART_TX_IRQHandler.

Location: sk5.h:95

20.3.43 UART0_ERROR_IRQHandler

```
#define UART0_ERROR_IRQHandler UART_ERROR_IRQHandler
```

Alias for UART_ERROR_IRQHandler.

Location: sk5.h:96

20.3.44 UART0_RX_IRQn

```
#define UART0_RX_IRQn UART_RX_IRQn
```

Alias for UART_RX_IRQn.

Location: sk5.h:97

20.3.45 UART0_TX_IRQn

```
#define UART0_TX_IRQn UART_TX_IRQn
```

Alias for UART_TX_IRQn.

Location: sk5.h:98

20.3.46 UART0_ERROR_IRQn

```
#define UART0_ERROR_IRQn UART_ERROR_IRQn
```

Alias for UART_ERROR_IRQn.

Location: sk5.h:99

20.3.47 __CM3_REV

```
#define __CM3_REV 0x0202
```

Core revision r2p2.

Location: sk5.h:105

20.3.48 __NVIC_PRIO_BITS

```
#define __NVIC_PRIO_BITS 3
```

3 bits used for interrupt priority levels

Location: sk5.h:106

20.3.49 __MPU_PRESENT

```
#define __MPU_PRESENT 0
```

MPU not present.

Location: sk5.h:107

20.3.50 __FPU_PRESENT

```
#define __FPU_PRESENT 0
```

FPU not present.

Location: sk5.h:108

20.3.51 __Vendor_SysTickConfig

```
#define __Vendor_SysTickConfig 0
```

Standard SysTick configuration is used.

Location: sk5.h:109

20.3.52 ASCC_REF_VALID

```
#define ASCC_REF_VALID ((uint32_t)(ref) == (uint32_t)AUDIOSINK0 | \  
                        (uint32_t)(ref) == (uint32_t)AUDIOSINK1)
```

Validation of Audio Sink Clock Counter register block pointer reference for assert statements.

Location: sk5.h:160

20.3.53 CRC_REF_VALID

```
#define CRC_REF_VALID ((uint32_t)(ref) == (uint32_t)CRC | \  
                      (uint32_t)(ref) == (uint32_t)D_CRC)
```

Validation of CRC register block pointer reference for assert statements.

Location: sk5.h:164

20.3.54 DMA_REF_VALID

```
#define DMA_REF_VALID (((uint32_t)(ref) == (uint32_t)DMA0) | \  
    ((uint32_t)(ref) == (uint32_t)DMA1) | \  
    ((uint32_t)(ref) == (uint32_t)DMA2) | \  
    ((uint32_t)(ref) == (uint32_t)DMA3) | \  
    ((uint32_t)(ref) == (uint32_t)DMA4) | \  
    ((uint32_t)(ref) == (uint32_t)DMA5) | \  
    ((uint32_t)(ref) == (uint32_t)DMA6) | \  
    ((uint32_t)(ref) == (uint32_t)DMA7))
```

Validation of DMA register block pointer reference for assert statements.

Location: sk5.h:168

20.3.55 DIO_PAD_COUNT

```
#define DIO_PAD_COUNT 36
```

Number of DIO pads.

Location: sk5.h:179

20.3.56 DIO_GROUP_LOW_PAD_RANGE

```
#define DIO_GROUP_LOW_PAD_RANGE 24
```

Number of DIO pads in the lowest group.

Location: sk5.h:180

20.3.57 DIO_EVENT_CHANNEL_COUNT

```
#define DIO_EVENT_CHANNEL_COUNT 8
```

Number of available DIO interrupts.

Location: sk5.h:181

20.3.58 DIO_CLK_DIV_COUNT

```
#define DIO_CLK_DIV_COUNT 2
```

Number of DIO clock dividers.

Location: sk5.h:182

20.3.59 DIO0

```
#define DIO0 0
```

DIO 0.

Location: sk5.h:185

20.3.60 DIO1

```
#define DIO1 1
```

DIO 1.

Location: sk5.h:186

20.3.61 DIO2

```
#define DIO2 2
```

DIO 2.

Location: sk5.h:187

20.3.62 DIO3

```
#define DIO3 3
```

DIO 3.

Location: sk5.h:188

20.3.63 DIO4

```
#define DIO4 4
```

DIO 4.

Location: sk5.h:189

20.3.64 DIO5

```
#define DIO5 5
```

DIO 5.

Location: sk5.h:190

20.3.65 DIO6

```
#define DIO6 6
```

DIO 6.

Location: sk5.h:191

20.3.66 DIO7

```
#define DIO7 7
```

DIO 7.

Location: sk5.h:192

20.3.67 DIO8

```
#define DIO8 8
```

DIO 8.

Location: sk5.h:193

20.3.68 DIO9

```
#define DIO9 9
```

DIO 9.

Location: sk5.h:194

20.3.69 DIO10

```
#define DIO10 10
```

DIO 10.

Location: sk5.h:195

20.3.70 DIO11

```
#define DIO11 11
```

DIO 11.

Location: sk5.h:196

20.3.71 DIO12

```
#define DIO12 12
```

DIO 12.

Location: sk5.h:197

20.3.72 DIO13

```
#define DIO13 13
```

DIO 13.

Location: sk5.h:198

20.3.73 DIO14

```
#define DIO14 14
```

DIO 14.

Location: sk5.h:199

20.3.74 DIO15

```
#define DIO15 15
```

DIO 15.

Location: sk5.h:200

20.3.75 DIO16

```
#define DIO16 16
```

DIO 16.

Location: sk5.h:201

20.3.76 DIO17

```
#define DIO17 17
```

DIO 17.

Location: sk5.h:202

20.3.77 DIO18

```
#define DIO18 18
```

DIO 18.

Location: sk5.h:203

20.3.78 DIO19

```
#define DIO19 19
```

DIO 19.

Location: sk5.h:204

20.3.79 DIO20

```
#define DIO20 20
```

DIO 20.

Location: sk5.h:205

20.3.80 DIO21

```
#define DIO21 21
```

DIO 21.

Location: sk5.h:206

20.3.81 DIO22

```
#define DIO22 22
```

DIO 22.

Location: sk5.h:207

20.3.82 DIO23

```
#define DIO23 23
```

DIO 23.

Location: sk5.h:208

20.3.83 DIO24

```
#define DIO24 24
```

DIO 24.

Location: sk5.h:209

20.3.84 DIO25

```
#define DIO25 25
```

DIO 25.

Location: sk5.h:210

20.3.85 DIO26

```
#define DIO26 26
```

DIO 26.

Location: sk5.h:211

20.3.86 DIO27

```
#define DIO27 27
```

DIO 27.

Location: sk5.h:212

20.3.87 DIO28

```
#define DIO28 28
```

DIO 28.

Location: sk5.h:213

20.3.88 DIO29

```
#define DIO29 29
```

DIO 29.

Location: sk5.h:214

20.3.89 DIO30

```
#define DIO30 30
```

DIO 30.

Location: sk5.h:215

20.3.90 DIO31

```
#define DIO31 31
```

DIO 31.

Location: sk5.h:216

20.3.91 DIO32

```
#define DIO32 32
```

DIO 32.

Location: sk5.h:217

20.3.92 DIO33

```
#define DIO33 33
```

DIO 33.

Location: sk5.h:218

20.3.93 DIO34

```
#define DIO34 34
```

DIO 34.

Location: sk5.h:219

20.3.94 DIO35

```
#define DIO35 35
```

DIO 35.

Location: sk5.h:220

20.3.95 I2C_REF_VALID

```
#define I2C_REF_VALID (((uint32_t)(ref) == (uint32_t)I2C0) | \
    ((uint32_t)(ref) == (uint32_t)I2C1) | \
    ((uint32_t)(ref) == (uint32_t)D_I2C))
```

Validation of I2C register block pointer reference for assert statements.

Location: sk5.h:223

20.3.96 PCM_REF_VALID

```
#define PCM_REF_VALID (((uint32_t)(ref) == (uint32_t)PCM0) | \  
                      ((uint32_t)(ref) == (uint32_t)PCM1))
```

Validation of PCM register block pointer reference for assert statements.

Location: sk5.h:229

20.3.97 SPI_REF_VALID

```
#define SPI_REF_VALID (((uint32_t)(ref) == (uint32_t)SPI0) | \  
                      ((uint32_t)(ref) == (uint32_t)SPI1))
```

Validation of SPI register block pointer reference for assert statements.

Location: sk5.h:233

20.3.98 TIMER_REF_VALID

```
#define TIMER_REF_VALID (((uint32_t)(ref) == (uint32_t)TIMER0) | \  
                        ((uint32_t)(ref) == (uint32_t)TIMER1) | \  
                        ((uint32_t)(ref) == (uint32_t)TIMER2))
```

Validation of TIMER register block pointer reference for assert statements.

Location: sk5.h:237

20.3.99 UART_REF_VALID

```
#define UART_REF_VALID ((uint32_t)(ref) == (uint32_t)UART)
```

Validation of UART register block pointer reference for assert statements.

Location: sk5.h:243

20.3.100 SYS_DUMMY_READ

```
#define SYS_DUMMY_READ SYSCTRL->DSP_CMD
```

Register that always reads back as 0x00000000.

Location: sk5.h:253

20.3.101 SYS_DUMMY_WRITE

```
#define SYS_DUMMY_WRITE TIMER->VAL
```

Register to which writes are ineffective.

Location: sk5.h:256

20.3.102 ERRNO_NO_ERROR

```
#define ERRNO_NO_ERROR 0
```

Error Code: Generic pass.

Location: sk5.h:308

20.3.103 ERRNO_GENERAL_FAILURE

```
#define ERRNO_GENERAL_FAILURE 1
```

Error Code: General failure.

Location: sk5.h:309

20.3.104 SYS_DELAY_CALL_OVERHEAD

```
#define SYS_DELAY_CALL_OVERHEAD 7
```

SYS_DELAY call overhead is 7 cycles.

Location: sk5_sys.h:72

20.3.105 SYS_DELAY_LOOP_OVERHEAD

```
#define SYS_DELAY_LOOP_OVERHEAD 4
```

SYS_DELAY loop accuracy is 4 cycles.

Location: sk5_sys.h:73

20.3.106 SYS_RESET

```
#define SYS_RESET D_WATCHDOG->RESET = WATCHDOG_FORCE_RESET;
```

Reset the system with a watchdog timeout.

Location: sk5_sys.h:84

Example Code for SYS_RESET

```
// Reset the system with a watchdog timeout  
SYS\_RESET();
```

20.3.107 SYSVAR_FREQ_FORMAT

```
#define SYSVAR_FREQ_FORMAT ((index + 1) * 640000)
```

Convert a valid frequency index to Hz.

Location: sk5_sys.h:93

Parameters

Ezairo 8300 Firmware Reference

Direction	Name	Description
in	<i>index</i>	Number representing a frequency value such as SYS_FREQ_7M68

Return

Frequency in Hz

20.3.108 SYSVAR_SET_FREQ

```
#define SYSVAR_SET_FREQ *((uint32_t*)SYSVAR_OPERATING_FREQ) = index; \
*((uint32_t*)SYSVAR_OPERATING_FREQ_HZ) = SYSVAR\_FREQ\_FORMAT(index);
```

Updates the SYSVAR locations that keep track of frequency.

Location: sk5_sys.h:100

Parameters

Direction	Name	Description
in	<i>index</i>	Number representing a frequency value such as SYS_FREQ_7M68

20.3.109 SYS_BOOT_ALTERNATE

```
#define SYS_BOOT_ALTERNATE *((uint32_t *)SYSVAR_BOOT_APPN) = app_num; \
asm("BX    %[res]" :: [res] "r" (SYS_BOOTLOADER_START));
```

Set SYSVAR_BOOT_APPN to an alternate app number and jump to the bootloader's reset handler.

Location: sk5_sys.h:111

Parameters

onsemi Confidential
Ezairo 8300 Firmware Reference

Direction	Name	Description
in	<i>app_num</i>	Alternate app number to boot

Pre: The bootloader code is still present in memory.

20.3.110 DEFAULT_FREQ

```
#define DEFAULT_FREQ 7680000
```

The default clock frequency is 7.68 MHz.

Location: system_sk5.h:39

20.3.111 DEFAULT_FREQ_IDX

```
#define DEFAULT_FREQ_IDX SYS_FREQ_7M68
```

The default clock index is 7.68 MHz.

Location: system_sk5.h:40

20.3.112 SystemCoreClock

```
#define SystemCoreClock (*(volatile uint32_t*)(SYSVAR_OPERATING_FREQ_HZ))
```

Variable that holds the current SystemCoreClock frequency.

Location: system_sk5.h:60

20.4 CMSIS REFERENCE FUNCTION DOCUMENTATION

20.4.1 _start

```
void _start()
```

Initialize the application data and start execution with main.

Call from the reset vector for standard application initialization.

Location: sk5_start.h:68

Assumptions

The symbols **data_init**, **data_start**, **data_end**, **bss_start**, and **bss_end** are defined when the application is linked.

20.4.2 _sbrk

```
int8_t * _sbrk(int increment)
```

Increment (or decrement) the top of the heap.

Location: sk5_start.h:80

Parameters

Direction	Name	Description
in	<i>increment</i>	Increment to be applied to the top of the heap

Return

The prior value of the heap top (points to the base of the newly allocated data if the heap was incremented); returns -1 if the function is unable to allocate the requested memory

Assumptions

The symbols **Heap_Begin**, **Heap_Limit** are defined when the application is linked

20.4.3 Sys_Delay

```
__STATIC_FORCEINLINE void Sys_Delay(uint32_t const cycles)
```

Delay by the specified number of clock cycles.

Location: sk5_sys.h:127

Parameters

Direction	Name	Description
in	<i>cycles</i>	Number of clock cycles to delay

Assumptions

The requested delay is at least 7 cycles and at most 0xFFFFFFFF cycles.

Assumptions

A delay between (cycles) and (cycles + 4) provides a sufficient delay resolution

Assumptions

The requested delay does not exceed the watchdog timeout

Example Code for Sys_Delay

```
// Delay by 1000 clock cycles  
Sys\_Delay(1000);
```

20.4.4 SystemInit

```
void SystemInit()
```

Sets up the system core clock variable; assumes the ROM has previously initialized the system.

Location: system_sk5.h:51

Example Code for SystemInit

```
// Setup the system core clock variable  
SystemInit();
```

20.4.5 SystemCoreClockUpdate

```
void SystemCoreClockUpdate()
```

Update the declared system clock (SYSCLK) frequency used by the Arm Cortex-M3 core firmware.

Location: system_sk5.h:67

Example Code for SystemCoreClockUpdate

```
// Update the declared system clock (SYSCLK) frequency  
// used by the Arm Cortex-M3 firmware  
SystemCoreClockUpdate();
```

20.4.6 System_SetOperatingFrequency

```
uint32_t System_SetOperatingFrequency()
```

Set the internal oscillator base operating frequency; the oscillator multiplication setting is not a component of the base operating frequency configuration.

Location: system_sk5.h:75

DeprecatedUse [Sys_Trims_SetOperatingFrequency\(\)](#)

20.4.7 System_SetOperatingFrequency_Throttle

```
uint32_t System_SetOperatingFrequency_Throttle()
```

Set the internal oscillator base operating frequency, using clock throttling to also configure SLOWCLK and SDMCLK.

The SLOWCLK, SDMCLK and ADCCLK prescalers are updated to match the updated operating frequency only if the base operating frequency changes. The oscillator multiplication setting is not a component of the base operating frequency configuration.

Location: system_sk5.h:87

DeprecatedUse [Sys_Trims_SetOperatingFrequencyThrottle\(\)](#)

CHAPTER 21

Non-Volatile Memory (NVM) Library Reference

Non-Volatile Memory (NVM) library for accessing external NVM memory devices.

21.1 SUMMARY

Typedefs

- [ARM_STORAGE_BLOCK_ATTRIBUTES](#) : Attributes of the storage range within a storage block.
- [ARM_STORAGE_BLOCK](#) : A storage block is a range of memory with uniform attributes.
- [ARM_STORAGE_SECURITY_FEATURES](#) : Device Data Security Protection Features.
- [ARM_STORAGE_INFO](#) : Device level metadata regarding the Storage implementation.
- [ARM_STORAGE_CAPABILITIES](#) : Storage Driver API Capabilities.
- [SPI_Storage_Commands](#) : Commands for the different SPI storage devices.
- [STORAGE_DEVICE_CHARACTERISTIC](#) : NVM device characteristics structure.
- [STORAGE_DEVICE_DESCRIPTOR](#) : NVM device descriptor.
- [NVMLIB_DRIVER](#) : Structure that holds the function pointers for the different devices.
- [NVMLIB_DEVICE](#) : Structure to hold internal states and variables for the storage driver.

Data Structures

- [ARM_STORAGE_BLOCK_ATTRIBUTES](#) : Attributes of the storage range within a storage block.
- [ARM_STORAGE_BLOCK](#) : A storage block is a range of memory with uniform attributes.
- [ARM_STORAGE_SECURITY_FEATURES](#) : Device Data Security Protection Features.
- [ARM_STORAGE_INFO](#) : Device level metadata regarding the Storage implementation.
- [ARM_STORAGE_CAPABILITIES](#) : Storage Driver API Capabilities.
- [SPI_Storage_Commands](#) : Commands for the different SPI storage devices.
- [STORAGE_DEVICE_CHARACTERISTIC](#) : NVM device characteristics structure.
- [STORAGE_DEVICE_DESCRIPTOR](#) : NVM device descriptor.
- [NVMLIB_DRIVER](#) : Structure that holds the function pointers for the different devices.
- [NVMLIB_DEVICE](#) : Structure to hold internal states and variables for the storage driver.

Macros

- [ARM_STORAGE_INVALID_OFFSET](#) : Invalid address (relative to a storage controller's address space).
- [ARM_STORAGE_INVALID_ADDRESS](#) : Invalid address within the processor's memory address space.
- [ARM_STORAGE_ERROR_NOT_ERASABLE](#) : Storage specific error codes.
- [ARM_STORAGE_ERROR_NOT_PROGRAMMABLE](#) : Part (or all) of the range provided to ProgramData() isn't programmable.
- [ARM_STORAGE_ERROR_PROTECTED](#) : Part (or all) of the range to Erase() or ProgramData() is protected.
- [ARM_STORAGE_VALID_BLOCK](#) : The check for a valid ARM_STORAGE_BLOCK.
- [ARM_STORAGE_PROGRAMMABILITY_RAM](#) : Values for encoding storage memory-types with respect to programmability.
- [ARM_STORAGE_PROGRAMMABILITY_ROM](#) : Read-only memory.

Ezairo 8300 Firmware Reference

- [ARM STORAGE PROGRAMMABILITY WORM](#) : write-once-read-only-memory (WORM).
- [ARM STORAGE PROGRAMMABILITY ERASABLE](#) : re-programmable based on erase.
- [ARM RETENTION WHILE DEVICE ACTIVE](#) : Values for encoding data-retention levels for storage blocks.
- [ARM RETENTION ACROSS SLEEP](#) : Data is retained across processor sleep.
- [ARM RETENTION ACROSS DEEP SLEEP](#) : Data is retained across processor deep-sleep.
- [ARM RETENTION BATTERY BACKED](#) : Data is battery-backed.
- [ARM RETENTION NVM](#) : Data is retained in non-volatile memory.
- [ARM POWER OFF](#) : General power states.
- [ARM POWER LOW](#) : Low Power mode: retain state, detect and signal wake-up events
- [ARM POWER FULL](#) : Power on: full operation at maximum performance
- [ARM STORAGE PROGRAM CYCLES INFINITE](#) : Infinite or unknown endurance for reprogramming.
- [SPI CHIP ID GENERIC](#) : NVM device support types.
- [SPI CHIP ID EMMC](#) : eMMC device
- [SPI CHIP ID SST26WF016B](#) : SST SST26WF016B manufacturer ID
- [SPI CHIP ID LE25S161](#) : onsemi LE25S161 manufacturer ID
- [SPI CHIP ID MB85AS8MT](#) : MB85AS8MT device ID
- [SPI MFG ID W25Q64JW](#) : Winbond manufacturer ID
- [SPI JEDEC ID W25Q64JW](#) : Winbond JEDEC device ID
- [SPI CHIP ID W25Q64JW](#) : Winbond Device ID
- [SPI JEDEC ID W25Q80EW](#) : Winbond JEDEC device ID
- [SPI CHIP ID W25Q80EW](#) : Winbond Device ID
- [SPI CHIP ID GT25Q16A](#) : GT25Q126A Device ID
- [SPI CHIP ID MX25](#) : Macronix MX25 manufacturer ID
- [NVMLIB STORAGE UNKNOWN](#) :
- [NVMLIB STORAGE CLEARED](#) : Invalid value
- [NVMLIB STORAGE AUTO](#) : Autodetect
- [NVMLIB STORAGE GENERIC](#) : Generic manufacturer ID
- [NVMLIB STORAGE EMMC](#) : eMMC device
- [NVMLIB STORAGE CAT25M01](#) : onsemi CAT25M01 SPI EEPROM
- [NVMLIB STORAGE CAT25M02](#) : onsemi CAT25M02 SPI EEPROM
- [NVMLIB STORAGE LE25S161](#) : onsemi LE25S161 SPI Flash
- [NVMLIB STORAGE MB85AS8MT](#) : Fujitsu MB85AS8MT ReRAM
- [NVMLIB STORAGE MB85AS8MT](#) : Fujitsu MB85AS8MT ReRAM
- [NVMLIB STORAGE EA2M](#) : onsemi EA2M Audiology SPI EEPROM
- [NVMLIB STORAGE W25Q64JW](#) : Winbond W25Q64JW Flash
- [NVMLIB STORAGE W25Q80EW](#) : Winbond W25Q80EW Flash
- [NVMLIB STORAGE GT25Q16A](#) : GT25Q126A Flash
- [NVMLIB STORAGE MX25](#) : Macronix MX25 Flash
- [ARM DRIVER OK](#) :
- [ARM DRIVER ERROR](#) : Unspecified error
- [ARM DRIVER ERROR BUSY](#) : Driver is busy
- [ARM DRIVER ERROR TIMEOUT](#) : Timeout occurred
- [ARM DRIVER ERROR UNSUPPORTED](#) : Operation not supported
- [ARM DRIVER ERROR PARAMETER](#) : Parameter error
- [ARM DRIVER ERROR SPECIFIC](#) : Start of driver specific errors

Ezairo 8300 Firmware Reference

- [ON STORAGE LIB WRITE ENABLE DISABLE](#) : Library magic number for unlocking writing to the NV device.
- [DRIVER STORAGE UNLOCK CODE](#) : Storage Driver unlocking code.

Functions

- [NVMLIB GlobalWriteEnable](#) : Write enable the NVM library.
- [NVMLIB GlobalWriteDisable](#) : Clear the library write enable.
- [NVMLIB DeviceWriteEnable](#) : Write enable the NVM device.
- [NVMLIB DeviceWriteDisable](#) : Clear the write enable.
- [NVMLIB StatusRegister Read](#) : Reads the Status Register.
- [NVMLIB StatusRegister Write](#) : Writes to the Status Register.
- [NVMLIB EnableWrite](#) : Sets or resets the WEL bit in the Status Register.
- [NVMLIB Initialize](#) : Initializes an NVM device.
- [NVMLIB Uninitialize](#) : De-initialize the Storage Interface.
- [NVMLIB PowerControl](#) : Control the Storage interface power.
- [NVMLIB ReadData](#) : Read data from Storage.
- [NVMLIB ProgramData](#) : Program data to Storage.
- [NVMLIB Erase](#) : Erase Storage range.
- [NVMLIB EraseAll](#) : Erase complete Storage.
- [NVMLIB GetBlock](#) : Find the storage block (iterator) encompassing a given storage address.
- [NVMLIB GetNextBlock](#) : Advance to the successor of the current block (iterator).
- [NVMLIB GetInfo](#) : Get Storage information.
- [NVMLIB Command](#) : Sends an arbitrary command to the underlying device.
- [NVMLIB Interface Configure](#) : Configures the hardware interface pins and prescaler.
- [NVMLIB InterfaceQSPI Configure](#) : Configures the hardware interface pins and prescaler.

Ezairo 8300 Firmware Reference

21.2 DETAILED DESCRIPTION

This library provides support for interfacing with external NVM devices in an abstracted way. The supported devices include:

- ON Semiconductor Audiology devices CAT25M01 and CAT25M02 SPI EEPROMs
- ON Semiconductor EA2M SPI EEPROM
- ON Semi LE25S161 SPI Flash
- eMMC

In addition to the supported devices listed above, this library supports any SPI devices that support the basic command set:

Command	Command Byte	Description
RDSR	0x05	Read status register
READ	0x03	Read
WRDI	0x04	Write disable
WREN	0x06	Write enable
WRITE	0x02	Write
WRSR	0x01	Write status register

21.3 NON-VOLATILE MEMORY (NVM) LIBRARY REFERENCE TYPEDEF DOCUMENTATION**21.3.1 ARM_STORAGE_BLOCK_ATTRIBUTES**

```
typedef struct ARM\_STORAGE\_BLOCK\_ATTRIBUTES ARM_STORAGE_BLOCK_ATTRIBUTES
```

Location: nvmlib.h:127

Attributes of the storage range within a storage block.

21.3.2 ARM_STORAGE_BLOCK

```
typedef struct ARM\_STORAGE\_BLOCK ARM_STORAGE_BLOCK
```

Location: nvmlib.h:140

A storage block is a range of memory with uniform attributes.

21.3.3 ARM_STORAGE_SECURITY_FEATURES

```
typedef struct ARM\_STORAGE\_SECURITY\_FEATURES ARM_STORAGE_SECURITY_FEATURES
```

Location: nvmlib.h:204

Device Data Security Protection Features.

Applicable mostly to EXTERNAL_NVM.

21.3.4 ARM_STORAGE_INFO

```
typedef struct ARM\_STORAGE\_INFO ARM_STORAGE_INFO
```

Location: nvmlib.h:245

Device level metadata regarding the Storage implementation.

21.3.5 ARM_STORAGE_CAPABILITIES

```
typedef struct ARM\_STORAGE\_CAPABILITIES ARM_STORAGE_CAPABILITIES
```

Location: nvmlib.h:262

Storage Driver API Capabilities.

21.3.6 SPI_Storage_Commands

```
typedef struct SPI\_Storage\_Commands SPI_Storage_Commands
```

Location: nvmlib.h:305

Commands for the different SPI storage devices.

21.3.7 STORAGE_DEVICE_CHARACTERISTIC

```
typedef struct STORAGE\_DEVICE\_CHARACTERISTIC STORAGE_DEVICE_CHARACTERISTIC
```

Location: nvmlib.h:325

NVM device characteristics structure.

21.3.8 STORAGE_DEVICE_DESCRIPTOR

```
typedef struct \_STORAGE\_DEVICE\_DESCRIPTOR STORAGE_DEVICE_DESCRIPTOR
```

Location: nvmlib.h:338

NVM device descriptor.

NOTE: We need this intermediate structure as we cannot place SPI_RESOURCES in the STORAGE_DEVICE_CHARACTERISTIC because that one is const. And we need it const as we do not have code to initialize the .data section.

21.3.9 NVMLIB_DRIVER

```
typedef struct \_NVMLIB\_DRIVER NVMLIB_DRIVER
```

Location: nvmlib.h:481

Structure that holds the function pointers for the different devices.

21.3.10 NVMLIB_DEVICE

```
typedef struct \_NVMLIB\_DEVICE NVMLIB_DEVICE
```

Location: nvmlib.h:495

Structure to hold internal states and variables for the storage driver.

Application code should **NEVER** access this structure, either directly or indirectly.

21.4 NON-VOLATILE MEMORY (NVM) LIBRARY REFERENCE DATA STRUCTURES TYPE DOCUMENTATION

Ezairo 8300 Firmware Reference

21.4.1 _ARM_STORAGE_BLOCK_ATTRIBUTES

Location: nvmlib.h:94

Attributes of the storage range within a storage block.

Data Fields

Type	Name	Description
unsigned long	<i>erasable</i>	Erasing blocks is permitted with a minimum granularity of 'erase_unit'. NOTE: if 'erasable' is 0 (i.e. the 'erase' operation isn't available) then 'erase_unit' (see below) is immaterial and should be 0.
unsigned long	<i>programmable</i>	Writing to ranges is permitted with a minimum granularity of 'program_unit'. Writes are typically achieved through the ProgramData operation (following an erase); if storage isn't erasable (see 'erasable' above) but is memory-mapped (i.e. 'memory_mapped'), it can be written directly using memory-store operations.
unsigned long	<i>executable</i>	This storage block can hold program data; the processor can fetch and execute code sourced from it. Often this is accompanied with the device being 'memory_mapped' (see ARM_STORAGE_INFO).
unsigned long	<i>protectable</i>	The entire block can be protected from program and erase operations. Once protection is enabled for a block, its 'erasable' and 'programmable' bits are turned off.
unsigned long	<i>reserved</i>	Reserved.

Ezairo 8300 Firmware Reference

unsigned long	<i>reserved2</i>	Reserved.
unsigned long	<i>erase_unit</i>	Minimum erase size in bytes. The offset of the start of the erase-range needs to be aligned with this value. Applicable if the 'erasable' attribute is set for the block. NOTE: if 'erasable' (see above) is 0 (i.e. the 'erase' operation is not available) then 'erase_unit' is immaterial and must be 0.
unsigned long	<i>protection_unit</i>	Minimum protectable size in bytes. Applicable if the 'protectable' attribute is set for the block. This must be a divisor of the block's size. A block can be considered to be made up of consecutive, individually-protectable fragments.

21.4.2 _ARM_STORAGE_BLOCK

Location: nvmlib.h:132

A storage block is a range of memory with uniform attributes.

Data Fields

Type	Name	Description
uint32_t	<i>addr</i>	This is the start address of the storage block. It is expressed as an offset from the start of the storage map maintained by the owning storage controller.
uint32_t	<i>size</i>	This is the size of the storage block, in units of bytes. Together with <i>addr</i>, it describes a range [<i>addr</i>, <i>addr</i>+<i>size</i>).
ARM_STORAGE_BLOCK_ATTRIBUTES	<i>attributes</i>	Attributes for this block.

Ezairo 8300 Firmware Reference

21.4.3 _ARM_STORAGE_SECURITY_FEATURES

Location: nvmlib.h:184

Device Data Security Protection Features.

Applicable mostly to EXTERNAL_NVM.

Data Fields

Type	Name	Description
unsigned long	<i>acls</i>	Protection against internal software attacks using ACLs.
unsigned long	<i>rollback_protection</i>	Roll-back protection. Set to true if the creator of the storage can ensure that an external attacker cannot force an older firmware to run or to revert back to a previous state.
unsigned long	<i>tamper_proof</i>	Tamper-proof memory (this is deleted on tamper-attempts using board level or chip level sensors).
unsigned long	<i>internal_flash</i>	Internal flash.
unsigned long	<i>reserved1</i>	Reserved.
unsigned long	<i>software_attacks</i>	Encode support for hardening against various classes of attacks. device software (malware running on the device).
unsigned long	<i>board_level_attacks</i>	board level attacks (debug probes, copy protection fuses.)
unsigned long	<i>chip_level_attacks</i>	chip level attacks (tamper-protection).
unsigned long	<i>side_channel_attacks</i>	side channel attacks.
unsigned long	<i>reserved2</i>	Reserved.

21.4.4 _ARM_STORAGE_INFO

Location: nvmlib.h:211

Ezairo 8300 Firmware Reference

Device level metadata regarding the Storage implementation.

Data Fields

Type	Name	Description
uint32_t	<i>total_storage</i>	Total available storage, in bytes.
uint32_t	<i>program_unit</i>	Minimum programming size in bytes. The offset of the start of the program-range should also be aligned with this value. Applicable only if the 'programmable' attribute is set for a block. NOTE: setting program_unit to 0 has the effect of disabling the size and alignment restrictions (setting it to 1 also has the same effect).
uint32_t	<i>optimal_program_unit</i>	Optimal programming page-size in bytes. Some storage controllers have internal buffers into which to receive data. Writing in chunks of 'optimal_program_unit' would achieve maximum programming speed. Applicable only if the 'programmable' attribute is set for the underlying block(s).
uint32_t	<i>program_cycles</i>	A measure of endurance for reprogramming. Use ARM_STORAGE_PROGRAM_CYCLES_INFINITE for infinite or unknown endurance.
unsigned long	<i>erased_value</i>	Contents of erased memory (usually 1 to indicate erased bytes with state 0xFF).
unsigned long	<i>memory_mapped</i>	This storage device has a mapping onto the processor's memory address space. NOTE: For a memory-mapped block which is not erasable but is programmable (i.e. if 'erasable' is set to 0, but 'programmable' is 1), writes are likely possible directly to the memory-mapped storage without going through the ProgramData operation.
unsigned long	<i>programmability</i>	A value to indicate storage programmability.

Ezairo 8300 Firmware Reference

unsigned long	<i>retention_level</i>	A value to indicate storage retention type.
unsigned long	<i>reserved</i>	Reserved.
ARM_STORAGE_SECURITY_FEATURES	<i>security</i>	ARM_STORAGE_SECURITY_FEATURES

21.4.5 _ARM_STORAGE_CAPABILITIES

Location: nvmlib.h:250

Storage Driver API Capabilities.

Data Fields

Type	Name	Description
unsigned long	<i>asynchronous_ops</i>	Used to indicate if APIs like initialize, read, erase, program, etc. can operate in asynchronous mode. Setting this bit to 1 means that the driver is capable of launching asynchronous operations; command completion is signaled by the invocation of a completion callback. If set to 1, drivers may still complete asynchronous operations synchronously as necessary (in which case they return a positive error code to indicate synchronous completion).
unsigned long	<i>erase</i>	Supports EraseAll operation.
unsigned long	<i>reserved</i>	Reserved (must be zero)

21.4.6 _SPI_Storage_Commands

Location: nvmlib.h:302

Commands for the different SPI storage devices.

Data Fields

Ezairo 8300 Firmware Reference

Type	Name	Description
uint32_t	<i>erase_page</i>	Page Erase - erase one page in memory array.

21.4.7 _STORAGE_DEVICE_CHARACTERISTIC

Location: nvmlib.h:311

NVM device characteristics structure.

Data Fields

Type	Name	Description
uint32_t	<i>total_storage</i>	Size of the device's memory array
uint32_t	<i>page_size</i>	Size of the device's write page size
uint32_t	<i>addr_prot_quarter</i>	Quarter array protection address
uint32_t	<i>addr_prot_half</i>	Half array protection address
uint32_t	<i>program_cycles</i>	Number of erase/program cycles
uint32_t	<i>program_unit</i>	Minimum programming size in bytes.
uint32_t	<i>erase_unit</i>	Minimum erase size in bytes.
uint32_t	<i>has_dual</i>	Whether the device supports SPI dual mode
uint32_t	<i>has_quad</i>	Whether the device supports SPI quad mode
uint32_t	<i>device_id</i>	Device ID from the SPI_STORAGE_CMD_RDPD command
uint32_t	<i>id_dummy_cycles</i>	Number of dummy clock cycles after a RDPD command
uint32_t	<i>chip_erase_timeout</i>	Time in milliseconds to wait for chip erase to complete
SPI Storage Commands *	<i>cmds</i>	Pointer to the structure describing the commands

21.4.8 _STORAGE_DEVICE_DESCRIPTOR

Location: nvmlib.h:334

Ezairo 8300 Firmware Reference

NVM device descriptor.

NOTE: We need this intermediate structure as we cannot place SPI_RESOURCES in the STORAGE_DEVICE_CHARACTERISTIC because that one is const. And we need it const as we do not have code to initialize the .data section.

Data Fields

Type	Name	Description
STORAGE_DEVICE_CHARACTERISTIC *	<i>characteristics</i>	Pointer to the device characteristics
HARDWARE_RESOURCES *	<i>hw</i>	Pointer to the SPI driver instance
uint32_t *	<i>cmd_buffer</i>	Pointer to the scratchpad buffer

21.4.9 _NVMLIB_DRIVER

Location: nvmlib.h:345

Structure that holds the function pointers for the different devices.

Data Fields

Type	Name	Description
uint32_t (*)	<i>StatusRegister_Read)</i> (<i>STORAGE_DEVICE_DESCRIPTOR *desc</i>)	Pointer to StatusRegister_Read: Reads the Status Register.
int (*)	<i>StatusRegister_Write)</i> (<i>STORAGE_DEVICE_DESCRIPTOR *desc,</i> <i>uint32_t status</i>)	Pointer to StatusRegister_Write: Writes to the Status Register.
int (*)	<i>EnableWrite)</i> (<i>STORAGE_DEVICE_DESCRIPTOR *desc,</i> <i>unsigned int enable</i>)	Pointer to EnableWrite: Enables writing to the device. Mostly used for SPI devices for setting/resetting the WEL bit.

Ezairo 8300 Firmware Reference

int (*)	<i>Initialize</i>)(<i>STORAGE_DEVICE_DESCRIPTOR *desc</i>)	Pointer to Initialize: Initializes an NVM device.
int (*)	<i>Uninitialize</i>)(<i>STORAGE_DEVICE_DESCRIPTOR *desc</i>)	Pointer to Uninitialize: De-initialize an NVM device.
int (*)	<i>PowerControl</i>)(<i>STORAGE_DEVICE_DESCRIPTOR *desc, unsigned int state</i>)	Pointer to PowerControl: Control the Storage interface power.
int (*)	<i>ReadData</i>)(<i>STORAGE_DEVICE_DESCRIPTOR *desc, uint32_t addr, uint32_t *data, unsigned int size</i>)	Pointer to ReadData: Read data from the NVM device.
int (*)	<i>ProgramData</i>)(<i>STORAGE_DEVICE_DESCRIPTOR *desc, uint32_t addr, const uint32_t *data, unsigned int size</i>)	Pointer to ProgramData: Read data from the NVM device.
int (*)	<i>Erase</i>)(<i>STORAGE_DEVICE_DESCRIPTOR *desc, uint32_t addr, unsigned int size</i>)	Pointer to Erase: Erase Storage range.
int (*)	<i>EraseAll</i>)(<i>STORAGE_DEVICE_DESCRIPTOR *desc</i>)	Pointer to EraseAll: Erase the whole NVM device.
int (*)	<i>SPISet</i>)(<i>STORAGE_DEVICE_DESCRIPTOR *desc, unsigned int spi_num</i>)	Pointer to SPISet: Erase the whole NVM device.
int (*)	<i>Command</i>)(<i>STORAGE_DEVICE_DESCRIPTOR *desc, const uint32_t *data_out, uint32_t *data_in, unsigned int num</i>)	Sends an arbitrary command to the NVM device.

21.4.10 _NVMLIB_DEVICE

Location: nvmlib.h:488

Structure to hold internal states and variables for the storage driver.

Application code should **NEVER** access this structure, either directly or indirectly.

Data Fields

Type	Name	Description
unsigned int	<i>initialized</i>	Used to indicate if the device has been initialized.
unsigned int	<i>power_state</i>	Used to indicate the current power status of the device.
unsigned int	<i>unlocked</i>	Used to lock/unlock writes to the device
unsigned int	<i>dev_id</i>	Device id. See NVMLIB STORAGE DEVICE ID
STORAGE_DEVICE_DESCRIPTOR	<i>desc</i>	Structure describing the device
NVMLIB_DRIVER *	<i>driver</i>	Pointer to the driver structure

21.5 NON-VOLATILE MEMORY (NVM) LIBRARY REFERENCE MACRO DEFINITION DOCUMENTATION

21.5.1 ARM_STORAGE_INVALID_OFFSET

```
#define ARM_STORAGE_INVALID_OFFSET (0xFFFFFFFFFFFFFFFFFULL)
```

Invalid address (relative to a storage controller's address space).

A storage block may never start at this address.

Location: nvmlib.h:77

21.5.2 ARM_STORAGE_INVALID_ADDRESS

```
#define ARM_STORAGE_INVALID_ADDRESS (0xFFFFFFFFFUL)
```

Invalid address within the processor's memory address space.

Refer to memory-mapped storage.

Location: nvmlib.h:81

21.5.3 ARM_STORAGE_ERROR_NOT_ERASABLE

```
#define ARM_STORAGE_ERROR_NOT_ERASABLE (ARM\_DRIVER\_ERROR\_SPECIFIC - 1)
```

Storage specific error codes.

Part (or all) of the range provided to Erase() isn't erasable.

Location: nvmlib.h:85

21.5.4 ARM_STORAGE_ERROR_NOT_PROGRAMMABLE

```
#define ARM_STORAGE_ERROR_NOT_PROGRAMMABLE (ARM\_DRIVER\_ERROR\_SPECIFIC - 2)
```

Part (or all) of the range provided to ProgramData() isn't programmable.

Location: nvmlib.h:87

21.5.5 ARM_STORAGE_ERROR_PROTECTED

```
#define ARM_STORAGE_ERROR_PROTECTED (ARM\_DRIVER\_ERROR\_SPECIFIC - 3)
```

Part (or all) of the range to Erase() or ProgramData() is protected.

Location: nvmlib.h:89

21.5.6 ARM_STORAGE_VALID_BLOCK

```
#define ARM_STORAGE_VALID_BLOCK ((BLK)->addr != ARM\_STORAGE\_INVALID\_OFFSET) && ((BLK)->size != 0))
```

The check for a valid ARM_STORAGE_BLOCK.

Location: nvmlib.h:145

21.5.7 ARM_STORAGE_PROGRAMMABILITY_RAM

```
#define ARM_STORAGE_PROGRAMMABILITY_RAM (0x0)
```

Values for encoding storage memory-types with respect to programmability.

Please ensure that the maximum of the following memory types doesn't exceed 16; we encode this in a 4-bit field within

Location: nvmlib.h:154

[ARM_STORAGE_INFO::programmability](#). RAM-like memory, reprogrammable without an erase.

21.5.8 ARM_STORAGE_PROGRAMMABILITY_ROM

```
#define ARM_STORAGE_PROGRAMMABILITY_ROM (0x1)
```

Read-only memory.

Location: nvmlib.h:155

21.5.9 ARM_STORAGE_PROGRAMMABILITY_WORM

```
#define ARM_STORAGE_PROGRAMMABILITY_WORM (0x2)
```

write-once-read-only-memory (WORM).

Location: nvmlib.h:156

21.5.10 ARM_STORAGE_PROGRAMMABILITY_ERASABLE

```
#define ARM_STORAGE_PROGRAMMABILITY_ERASABLE (0x3)
```

re-programmable based on erase.

Supports multiple writes.

Location: nvmlib.h:158

21.5.11 ARM_RETENTION_WHILE_DEVICE_ACTIVE

```
#define ARM_RETENTION_WHILE_DEVICE_ACTIVE (0x0)
```

Values for encoding data-retention levels for storage blocks.

Please ensure that the maximum of the following retention types doesn't exceed 16; we encode this in a 4-bit field within

Location: nvmlib.h:166

[ARM_STORAGE_INFO::retention_level](#). Data is retained only during device activity.

21.5.12 ARM_RETENTION_ACROSS_SLEEP

```
#define ARM_RETENTION_ACROSS_SLEEP (0x1)
```

Data is retained across processor sleep.

Location: nvmlib.h:167

21.5.13 ARM_RETENTION_ACROSS_DEEP_SLEEP

```
#define ARM_RETENTION_ACROSS_DEEP_SLEEP (0x2)
```

Data is retained across processor deep-sleep.

Location: nvmlib.h:168

21.5.14 ARM_RETENTION_BATTERY_BACKED

```
#define ARM_RETENTION_BATTERY_BACKED (0x3)
```

Data is battery-backed.

Device can be powered off.

Location: nvmlib.h:170

21.5.15 ARM_RETENTION_NVM

```
#define ARM_RETENTION_NVM (0x4)
```

Data is retained in non-volatile memory.

Location: nvmlib.h:171

21.5.16 ARM_POWER_OFF

```
#define ARM_POWER_OFF 0x01
```

General power states.

Power off: no operation possible

Location: nvmlib.h:176

21.5.17 ARM_POWER_LOW

```
#define ARM_POWER_LOW 0x02
```

Low Power mode: retain state, detect and signal wake-up events

Location: nvmlib.h:178

21.5.18 ARM_POWER_FULL

```
#define ARM_POWER_FULL 0x03
```

Power on: full operation at maximum performance

Location: nvmlib.h:179

21.5.19 ARM_STORAGE_PROGRAM_CYCLES_INFINITE

```
#define ARM_STORAGE_PROGRAM_CYCLES_INFINITE (0UL)
```

Infinite or unknown endurance for reprogramming.

Location: nvmlib.h:206

21.5.20 SPI_CHIP_ID_GENERIC

```
#define SPI_CHIP_ID_GENERIC 0xFF
```

NVM device support types.

Generic manufacturer ID

Location: nvmlib.h:267

21.5.21 SPI_CHIP_ID_EMMC

```
#define SPI_CHIP_ID_EMMC 0xFE
```

eMMC device

Location: nvmlib.h:268

Ezairo 8300 Firmware Reference

21.5.22 SPI_CHIP_ID_SST26WF016B

```
#define SPI_CHIP_ID_SST26WF016B 0x51
```

SST SST26WF016B manufacturer ID

Location: nvmlib.h:269

21.5.23 SPI_CHIP_ID_LE25S161

```
#define SPI_CHIP_ID_LE25S161 0x88
```

onsemi LE25S161 manufacturer ID

Location: nvmlib.h:270

21.5.24 SPI_CHIP_ID_MB85AS8MT

```
#define SPI_CHIP_ID_MB85AS8MT 0x03
```

MB85AS8MT device ID

Location: nvmlib.h:271

21.5.25 SPI_MFG_ID_W25Q64JW

```
#define SPI_MFG_ID_W25Q64JW 0xEF
```

Winbond manufacturer ID

Location: nvmlib.h:272

21.5.26 SPI_JEDEC_ID_W25Q64JW

```
#define SPI_JEDEC_ID_W25Q64JW 0x17
```

Winbond JEDEC device ID

Location: nvmlib.h:273

21.5.27 SPI_CHIP_ID_W25Q64JW

```
#define SPI_CHIP_ID_W25Q64JW 0x16
```

Winbond Device ID

Location: nvmlib.h:274

21.5.28 SPI_JEDEC_ID_W25Q80EW

```
#define SPI_JEDEC_ID_W25Q80EW 0x14
```

Winbond JEDEC device ID

Location: nvmlib.h:275

21.5.29 SPI_CHIP_ID_W25Q80EW

```
#define SPI_CHIP_ID_W25Q80EW 0x13
```

Winbond Device ID

Location: nvmlib.h:276

21.5.30 SPI_CHIP_ID_GT25Q16A

```
#define SPI_CHIP_ID_GT25Q16A 0x12
```

GT25Q126A Device ID

Location: nvmlib.h:277

21.5.31 SPI_CHIP_ID_MX25

```
#define SPI_CHIP_ID_MX25 0xC2
```

Macronix MX25 manufacturer ID

Location: nvmlib.h:278

21.5.32 NVMLIB_STORAGE_UNKNOWN

```
#define NVMLIB_STORAGE_UNKNOWN 0xFC
```

Used in case autodetect failed

Location: nvmlib.h:283

21.5.33 NVMLIB_STORAGE_CLEARED

```
#define NVMLIB_STORAGE_CLEARED 0x00
```

Invalid value

Location: nvmlib.h:284

21.5.34 NVMLIB_STORAGE_AUTO

```
#define NVMLIB_STORAGE_AUTO 0xFD
```

Autodetect

Location: nvmlib.h:285

21.5.35 NVMLIB_STORAGE_GENERIC

```
#define NVMLIB_STORAGE_GENERIC SPI\_CHIP\_ID\_GENERIC
```

Generic manufacturer ID

Location: nvmlib.h:286

21.5.36 NVMLIB_STORAGE_EMMC

```
#define NVMLIB_STORAGE_EMMC SPI\_CHIP\_ID\_EMMC
```

eMMC device

Location: nvmlib.h:287

21.5.37 NVMLIB_STORAGE_CAT25M01

```
#define NVMLIB_STORAGE_CAT25M01 0xFC
```

onsemi CAT25M01 SPI EEPROM

Location: nvmlib.h:288

21.5.38 NVMLIB_STORAGE_CAT25M02

```
#define NVMLIB_STORAGE_CAT25M02 0xFB
```

onsemi CAT25M02 SPI EEPROM

Location: nvmlib.h:289

21.5.39 NVMLIB_STORAGE_LE25S161

```
#define NVMLIB_STORAGE_LE25S161 SPI\_CHIP\_ID\_LE25S161
```

onsemi LE25S161 SPI Flash

Location: nvmlib.h:290

21.5.40 NVMLIB_STORAGE_MB85AS8MT

```
#define NVMLIB_STORAGE_MB85AS8MT SPI\_CHIP\_ID\_MB85AS8MT
```

Fujitsu MB85AS8MT ReRAM

Location: nvmlib.h:291

21.5.41 NVMLIB_STORAGE_MB85AS8MT

#define NVMLIB_STORAGE_MB85AS8MT [SPI_CHIP_ID_MB85AS8MT](#)

Fujitsu MB85AS8MT ReRAM

Location: nvmlib.h:293

21.5.42 NVMLIB_STORAGE_EA2M

#define NVMLIB_STORAGE_EA2M 0xFA

onsemi EA2M Audiology SPI EEPROM

Location: nvmlib.h:292

21.5.43 NVMLIB_STORAGE_W25Q64JW

#define NVMLIB_STORAGE_W25Q64JW [SPI_JEDEC_ID_W25Q64JW](#)

Winbond W25Q64JW Flash

Location: nvmlib.h:294

21.5.44 NVMLIB_STORAGE_W25Q80EW

#define NVMLIB_STORAGE_W25Q80EW [SPI_CHIP_ID_W25Q80EW](#)

Winbond W25Q80EW Flash

Location: nvmlib.h:295

21.5.45 NVMLIB_STORAGE_GT25Q16A

#define NVMLIB_STORAGE_GT25Q16A [SPI_CHIP_ID_GT25Q16A](#)

GT25Q126A Flash

Location: nvmlib.h:296

21.5.46 NVMLIB_STORAGE_MX25

```
#define NVMLIB_STORAGE_MX25 SPI\_CHIP\_ID\_MX25
```

Macronix MX25 Flash

Location: nvmlib.h:297

21.5.47 ARM_DRIVER_OK

```
#define ARM_DRIVER_OK 0
```

Operation succeeded

Location: nvmlib.h:500

21.5.48 ARM_DRIVER_ERROR

```
#define ARM_DRIVER_ERROR -1
```

Unspecified error

Location: nvmlib.h:501

21.5.49 ARM_DRIVER_ERROR_BUSY

```
#define ARM_DRIVER_ERROR_BUSY -2
```

Driver is busy

Location: nvmlib.h:502

21.5.50 ARM_DRIVER_ERROR_TIMEOUT

```
#define ARM_DRIVER_ERROR_TIMEOUT -3
```

Timeout occurred

Location: nvmlib.h:503

21.5.51 ARM_DRIVER_ERROR_UNSUPPORTED

```
#define ARM_DRIVER_ERROR_UNSUPPORTED -4
```

Operation not supported

Location: nvmlib.h:504

21.5.52 ARM_DRIVER_ERROR_PARAMETER

```
#define ARM_DRIVER_ERROR_PARAMETER -5
```

Parameter error

Location: nvmlib.h:505

21.5.53 ARM_DRIVER_ERROR_SPECIFIC

```
#define ARM_DRIVER_ERROR_SPECIFIC -6
```

Start of driver specific errors

Location: nvmlib.h:506

21.5.54 ON_STORAGE_LIB_WRITE_ENABLE_DISABLE

```
#define ON_STORAGE_LIB_WRITE_ENABLE_DISABLE 0
```

Library magic number for unlocking writing to the NV device.

Location: nvmlib.h:514

NOTE: This could be a #define, I don't have a particular reason for choosing an enum, other than being consistent with the above.

21.5.55 DRIVER_STORAGE_UNLOCK_CODE

```
#define DRIVER_STORAGE_UNLOCK_CODE 0x00CAFE
```

Storage Driver unlocking code.

Location: nvmlib.h:517

21.6 NON-VOLATILE MEMORY (NVM) LIBRARY REFERENCE FUNCTION DOCUMENTATION

21.6.1 NVMLIB_GlobalWriteEnable

```
EXTERN_C int NVMLIB_GlobalWriteEnable(uint32_t unlock)
```

Write enable the NVM library.

Location: nvmlib.h:533

Parameters

Direction	Name	Description
in	<i>unlock</i>	The unlock code DRIVER_STORAGE_UNLOCK_CODE

Return

[ARM_DRIVER_OK](#) on success, [ARM_DRIVER_ERROR](#) on failure

Example Code for NVMLIB_GlobalWriteEnable

```
// Write enable the NVM library  
status = NVMLIB\_GlobalWriteEnable(DRIVER\_STORAGE\_UNLOCK\_CODE);
```

21.6.2 NVMLIB_GlobalWriteDisable

```
EXTERN_C int NVMLIB_GlobalWriteDisable()
```

Clear the library write enable.

Location: nvmlib.h:541

Return

[ARM DRIVER OK](#) on success, [ARM DRIVER ERROR](#) on failure

Example Code for NVMLIB_GlobalWriteDisable

```
// Clear the library write enable  
status = NVMLIB\_GlobalWriteDisable();
```

21.6.3 NVMLIB_DeviceWriteEnable

```
EXTERN_C int NVMLIB_DeviceWriteEnable(NVMLIB\_DEVICE * device, uint32_t unlock)
```

Write enable the NVM device.

Location: nvmlib.h:550

Parameters

Ezairo 8300 Firmware Reference

Direction	Name	Description
in	<i>device</i>	The NVM library resource structure pointer
in	<i>unlock</i>	The unlock code DRIVER_STORAGE_UNLOCK_CODE .

Return

[ARM_DRIVER_OK](#) on success, [ARM_DRIVER_ERROR](#) on failure

Example Code for NVMLIB_DeviceWriteEnable

```
// Write enable the specified device
status = NVMLIB\_DeviceWriteEnable(&nvmlib_device, DRIVER\_STORAGE\_UNLOCK\_CODE);
```

21.6.4 NVMLIB_DeviceWriteDisable

```
EXTERN_C int NVMLIB_DeviceWriteDisable(NVMLIB\_DEVICE * device)
```

Clear the write enable.

Location: nvmlib.h:558

Parameters

Direction	Name	Description
in	<i>device</i>	The NVM library resource structure pointer

Return

[ARM_DRIVER_OK](#) on success, [ARM_DRIVER_ERROR](#) on failure

Example Code for NVMLIB_DeviceWriteDisable

```
// Write disable the specified device  
status = NVMLIB_DeviceWriteDisable(&nvmlib_device);
```

21.6.5 NVMLIB_StatusRegister_Read

```
EXTERN_C uint32_t NVMLIB_StatusRegister_Read(NVMLIB_DEVICE * device)
```

Reads the Status Register.

Location: nvmlib.h:566

Parameters

Direction	Name	Description
in	<i>device</i>	The NVM library resource structure pointer

Return

Returns the Status Register

Example Code for NVMLIB_StatusRegister_Read

```
// Read status register of the specified device'  
status = NVMLIB_StatusRegister_Read(&nvmlib_device);
```

21.6.6 NVMLIB_StatusRegister_Write

```
EXTERN_C int NVMLIB_StatusRegister_Write(NVMLIB_DEVICE * device, uint32_t status)
```

Writes to the Status Register.

Location: nvmlib.h:575

Parameters

Direction	Name	Description
in	<i>device</i>	The NVM library resource structure pointer
in	<i>status</i>	Status to be written

Return

[ARM_DRIVER_OK](#) on success, [ARM_DRIVER_ERROR](#) on failure

Example Code for NVMLIB_StatusRegister_Write

```
// Writes to the status register of the specified device
status = NVMLIB\_StatusRegister\_Write(&nvmlib_device, 0x80);
```

21.6.7 NVMLIB_EnableWrite

```
EXTERN_C int NVMLIB_EnableWrite(NVMLIB\_DEVICE * device, unsigned int enable)
```

Sets or resets the WEL bit in the Status Register.

Location: nvmlib.h:586

Parameters

Direction	Name	Description
in	<i>device</i>	The NVM library resource structure pointer
in	<i>enable</i>	Sets/resets the WEL bit. This enables/disables writing to the Status Register. Use 1 set or 0 to clear.

Return

[ARM DRIVER OK](#) on success, [ARM DRIVER ERROR](#) on failure

Example Code for NVMLIB_EnableWrite

```
// Set the write-enable (WEL) bit in the status register of the specified device  
status = NVMLIB\_EnableWrite(&nvmlib_device, 0x1);
```

21.6.8 NVMLIB_Initialize

```
EXTERN_C int NVMLIB_Initialize(NVMLIB\_DEVICE * device, unsigned int dev_id, uint32_t * cmd_  
buffer)
```

Initializes an NVM device.

Location: nvmlib.h:624

Parameters

Ezairo 8300 Firmware Reference

Direction	Name	Description
in	<i>device</i>	The NVM library resource structure pointer
in	<i>dev_id</i>	Type of the device to be initialized. the value should is one of the following: • NVMLIB_STORAGE_AUTO: Attempts to automatically detect the storage device. • NVMLIB_STORAGE_EMMC: Configures the library to use an eMMC device. • NVMLIB_STORAGE_CAT25M01: Configures the library to use a CAT25M01 SPI EEPROM. • NVMLIB_STORAGE_CAT25M02: Configures the library to use a CAT25M02 SPI EEPROM. • NVMLIB_STORAGE_LE25S161: Configures the library to use a LE25S161 SPI Flash. • NVMLIB_STORAGE_MB85AS8MT: Configures the library to use a MB85AS8MT SPI ReRAM. • NVMLIB_STORAGE_W25Q64JW: Configures the library to use a W25Q64JW SPI Flash. • NVMLIB_STORAGE_W25Q80EW: Configures the library to use a W25Q80EW SPI Flash.
in	<i>cmd_buffer</i>	Scratchpad buffer needed by the library. It <i>must</i> be a a 16-word buffer. For the CFX core <i>it must be a IOMEM buffer</i> . Use chess_storage(IOMEM) in this case.

Return

[ARM DRIVER OK](#) on success, [ARM DRIVER ERROR](#) on failure

Assumptions

NVMLIB_Interface_Configure or NVMLIB_InterfaceQSPI_Configure has been called

Assumptions

The NVM library interface configuration and initialization functions are called whenever the NVM interface is configured or reconfigured due to changes in clocking or other interface configurations.

Ezairo 8300 Firmware Reference

Example Code for NVMLIB_Initialize

```
// Configures the hardware interface pins and prescaler.
// Using 3.84 MHz clock for EEMC
NVMLIB_Interface_Configure(&nvmlib_device,
    (void*) SPI0,
    SPI_PRESCALE_8,
    3840U,
    (DIO_8X_DRIVE | DIO_LPF_DISABLE | DIO_250K_PULL_UP),
    0, 1, 2, 3);
// Initializes the specified device by auto-detecting device type
status = NVMLIB_Initialize(&nvmlib_device,
    NVMLIB_STORAGE_AUTO,
    cmd_buffer);
```

NOTE: The NVMLIB_Interface_Configure function must have been called at least once before calling this function.

21.6.9 NVMLIB_Uninitialize

```
EXTERN_C int NVMLIB_Uninitialize(NVMLIB_DEVICE * device)
```

De-initialize the Storage Interface.

Location: nvmlib.h:639

Parameters

Direction	Name	Description
in	<i>device</i>	The NVM library resource structure pointer

Return

[execution status](#) If asynchronous activity is launched, an invocation returns ARM_DRIVER_OK, and the caller can expect to receive a callback in the future with a status value of ARM_DRIVER_OK or an error-code. In the case of synchronous execution, control returns after completion with a value of 1. Return values less than ARM_DRIVER_OK (0) signify errors.

Example Code for NVMLIB_Uninitialize

```
// Uninitialize the specified device  
status = NVMLIB\_Uninitialize(&nvmlib_device);
```

21.6.10 NVMLIB_PowerControl

```
EXTERN_C int NVMLIB_PowerControl(NVMLIB\_DEVICE * device, unsigned int state)
```

Control the Storage interface power.

Location: nvmlib.h:654

Parameters

Direction	Name	Description
in	<i>device</i>	The NVM library resource structure pointer
in	<i>state</i>	Power state

Return

[execution status](#) If asynchronous activity is launched, an invocation returns ARM_DRIVER_OK, and the caller can expect to receive a callback in the future with a status value of ARM_DRIVER_OK or an error-code. In the case of synchronous execution, control returns after completion with a value of 1. Return values less than ARM_DRIVER_OK (0) signify errors.

Assumptions

NVMLIB_DEVICE 'device' has been initialized using [NVMLIB_Initialize\(\)](#)

Example Code for NVMLIB_PowerControl

```
// Restore the NVM storage device to full power operation mode  
status = NVMLIB\_PowerControl(&nvmlib_device, ARM\_POWER\_FULL);
```

21.6.11 NVMLIB_ReadData

```
EXTERN_C int NVMLIB_ReadData(NVMLIB\_DEVICE * device, uint32_t addr, uint32_t * data,  
unsigned int size)
```

Read data from Storage.

Location: nvmlib.h:672

Parameters

Direction	Name	Description
in	<i>device</i>	The NVM library resource structure pointer
in	<i>addr</i>	Data address in the NVM device address space. For SPI devices, this address is in bytes.
out	<i>data</i>	Pointer to a buffer storing the data read from the NVM device.
in	<i>size</i>	Number of bytes to read. The data buffer should be at least as large as this size.

Return

Values less than ARM_DRIVER_OK (0) signify errors.

Example Code for NVMLIB_ReadData

```
// Read 1 byte of data from storage at addr 0 into data_out buffer  
status = NVMLIB\_ReadData(&nvmlib_device, 0x00000000, data_out, 1);
```

NOTE: The buffer pointed by data **must** to be allocated in IOMEM. This driver expects that the data being pointed to is 32-bits wide. Use chess_storage(IOMEM).

Ezairo 8300 Firmware Reference

21.6.12 NVMLIB_ProgramData

```
EXTERN_C int NVMLIB_ProgramData(NVMLIB\_DEVICE * device, uint32_t addr, const uint32_t *
data, unsigned int write_size)
```

Program data to Storage.

Location: nvmlib.h:696

Parameters

Direction	Name	Description
in	<i>device</i>	The NVM library resource structure pointer
in	<i>addr</i>	Data address in the NVM device address space. This address is in bytes.
out	<i>data</i>	Pointer to a buffer with the data to be written to the NVM device.
in	<i>write_size</i>	Number of bytes to write. The data buffer should be at least as large as this size.

Return

Values less than ARM_DRIVER_OK (0) signify errors.

Example Code for NVMLIB_ProgramData

```
// Initialize data_in buffer and load in some data
data_in[0] = 0x1234;
data_in[1] = 0xFFE;
// Program data to the specified device's storage
status = NVMLIB\_ProgramData(&nvmlib_device, 0, data_in, 1);
```

NOTE: The buffer pointed by data **must** to be allocated in IOMEM. This driver expects that the data being pointed to is 32-bits wide. Use chess_storage(IOMEM).

Ezairo 8300 Firmware Reference

NOTE: For SPI writes that force a page-boundary cross, the NVM destination address must be a multiple of 4. This means (on a 256-byte page device):

- Writing 256 bytes to NVM address 4 works,
- Writing 523 bytes to NVM address 1024 works,
- Writing 4 bytes to NVM address 254 does not work.

21.6.13 NVMLIB_Erase

```
EXTERN_C int NVMLIB_Erase(NVMLIB\_DEVICE * device, uint32_t addr, unsigned int size)
```

Erase Storage range.

Location: nvmlib.h:724

Parameters

Direction	Name	Description
in	<i>device</i>	The NVM library resource structure pointer
in	<i>addr</i>	This is the start-address of the range to be erased. It must start at an 'erase_unit' boundary of the underlying block.
in	<i>size</i>	Number of bytes to be erased from the device.

Return

[execution status](#) If the range to be erased doesn't align with the erase_units of the respective start and end blocks, ARM_DRIVER_ERROR_PARAMETER is returned. If any part of the range is protected, ARM_STORAGE_ERROR_PROTECTED is returned. If any part of the range is not erasable, ARM_STORAGE_ERROR_NOT_ERASABLE is returned. All such sanity-check failures result in the error code being returned synchronously and the storage bytes within the range remain unaffected. Otherwise the function executes in the following ways: If asynchronous activity is launched, an invocation returns ARM_DRIVER_OK, and the caller can expect to receive a callback in the future with the number of successfully erased bytes passed in as the 'status' parameter. In the case of synchronous execution, control returns after completion with a positive erase-count. Return values less than ARM_DRIVER_OK (0) signify errors.

Example Code for NVMLIB_Erase

```
// Erase the first block of the specified device
status = NVMLIB\_Erase(&nvmlib_device, 0x00,
                    prev_block.attributes.erase_unit);
```

21.6.14 NVMLIB_EraseAll

```
EXTERN_C int NVMLIB_EraseAll(NVMLIB\_DEVICE * device)
```

Erase complete Storage.

Location: nvmlib.h:745

Parameters

Direction	Name	Description
in	<i>device</i>	The NVM library resource structure pointer

Return

[execution status](#) If any part of the storage range is protected, ARM_STORAGE_ERROR_PROTECTED is returned. If any part of the storage range is not erasable, ARM_STORAGE_ERROR_NOT_ERASABLE is returned. All such sanity-check failures result in the error code being returned synchronously and the storage bytes within the range remain unaffected. Otherwise the function executes in the following ways: If asynchronous activity is launched, an invocation returns ARM_DRIVER_OK, and the caller can expect to receive a callback in the future with ARM_DRIVER_OK passed in as the 'status' parameter. In the case of synchronous execution, control returns after completion with a value of 1. Return values less than ARM_DRIVER_OK (0) signify errors.

Example Code for NVMLIB_EraseAll

```
// Erase complete storage of the specified device
status = NVMLIB\_EraseAll(&nvmlib_device);
```

Ezairo 8300 Firmware Reference

21.6.15 NVMLIB_GetBlock

```
EXTERN_C int NVMLIB_GetBlock(NVMLIB\_DEVICE * device, uint32_t addr, ARM\_STORAGE\_BLOCK *
block)
```

Find the storage block (iterator) encompassing a given storage address.

Location: nvmlib.h:768

Parameters

Direction	Name	Description
in	<i>device</i>	The NVM library resource structure pointer
in	<i>addr</i>	Storage address in bytes
out	<i>block</i>	A caller-owned buffer large enough to be filled in with the ARM_STORAGE_BLOCK encapsulating the given address. This value can also be passed in as NULL if the caller isn't interested in populating a buffer with the block, if the caller only wishes to establish the presence of a containing storage block.

Return

[execution status](#) ARM_DRIVER_OK if a containing storage-block is found. In this case, if block is non-NULL, the buffer pointed to by it is populated with the contents of the storage block, i.e. if block is valid and a block is found, [ARM_STORAGE_VALID_BLOCK\(block\)](#) would return true following this call. If there is no storage block containing the given offset, or in case the driver is unable to resolve an address to a storage-block, an error (negative) value is returned and an invalid StorageBlock is populated into the supplied buffer.

Example Code for NVMLIB_GetBlock

```
// Find the storage block (iterator) encompassing address 0x20
status = NVMLIB\_GetBlock(&nvmlib_device, 0x20, &prev_block);
```

Ezairo 8300 Firmware Reference

21.6.16 NVMLIB_GetNextBlock

```
EXTERN_C int NVMLIB_GetNextBlock(NVMLIB\_DEVICE * device, const ARM\_STORAGE\_BLOCK * prev_block, ARM\_STORAGE\_BLOCK * next_block)
```

Advance to the successor of the current block (iterator).

Location: nvmlib.h:798

Parameters

Direction	Name	Description
in	<i>device</i>	The NVM library resource structure pointer
in	<i>prev_block</i>	An existing block (iterator) within the same storage controller. The memory buffer holding this block is owned by the caller. This pointer may be NULL; if so, the invocation fills in the first block into the out parameter: 'next_block'.
out	<i>next_block</i>	A caller-owned buffer large enough to be filled in with the following ARM_STORAGE_BLOCK. It is legal to provide the same buffer using 'next_block' as was passed in with 'prev_block'. It is also legal to pass a NULL into this parameter if the caller isn't interested in populating a buffer with the next block, i.e. if the caller only wishes to establish the presence of a next block.

Return

[execution status](#) ARM_DRIVER_OK if a valid next block is found (or first block, if prev_block is passed as NULL); upon successful operation, the contents of the next (or first) block are filled into the buffer pointed to by the parameter 'next_block' and [ARM_STORAGE_VALID_BLOCK\(next_block\)](#) is guaranteed to be true. Upon reaching the end of the sequence of blocks (iterators), or in case the driver is unable to fetch information about the next (or first) block, an error (negative) value is returned and an invalid StorageBlock is populated into the supplied buffer. If prev_block is NULL, the first block is returned.

Example Code for NVMLIB_GetNextBlock

```
// Advance to the successor of ARM_STORAGE_BLOCK prev_block
status = NVMLIB_GetNextBlock(&nvmlib_device,
                             &prev_block,
                             &next_block);
```

21.6.17 NVMLIB_GetInfo

```
EXTERN_C int NVMLIB_GetInfo(NVMLIB_DEVICE * device, ARM_STORAGE_INFO * info)
```

Get Storage information.

Location: nvmlib.h:817

Parameters

Direction	Name	Description
in	<i>device</i>	The NVM library resource structure pointer
out	<i>info</i>	A caller-supplied buffer capable of being filled in with an ARM_STORAGE_INFO .

Return

[execution status](#) ARM_DRIVER_OK if a ARM_STORAGE_INFO structure containing top level metadata about the storage controller is filled into the supplied buffer, else an appropriate error value.

Example Code for NVMLIB_GetInfo

```
// Read back some sectors of the specified device into
// 'ARM_STORAGE_INFO storage_info' to verify that the chip has
// actually been erased.
status = NVMLIB_GetInfo(&nvmlib_device, &storage_info);
```

Ezairo 8300 Firmware Reference

NOTE: When [NVMLIB_Initialize](#) is called with [NVMLIB_STORAGE_AUTO](#), and a SPI NVM device is detected but cannot be identified, the device's storage characteristics are set to be the same as a CAT25M01 device. The filled in [ARM_STORAGE_INFO](#) buffer may not be accurate for the NVM device used.

21.6.18 NVMLIB_Command

```
EXTERN_C int NVMLIB_Command(NVMLIB\_DEVICE * device, const uint32_t * data_out, uint32_t * data_in, uint32_t num)
```

Sends an arbitrary command to the underlying device.

Location: nvmlib.h:838

Parameters

Direction	Name	Description
in	<i>device</i>	The NVM library resource structure pointer
out	<i>data_out</i>	Pointer to the buffer with data to be written to the device. If NULL, dummy data will be written.
in	<i>data_in</i>	Pointer to the buffer with data to be read from the device. If NULL, read data will be discarded.
in	<i>num</i>	Number of octets to be transferred. For eMMC devices this function only supports num == 2.

Return

[execution status](#) ARM_DRIVER_OK if successful, ARM_DRIVER_ERROR* otherwise.

Ezairo 8300 Firmware Reference

Example Code for NVMLIB_Command

```
// Initialize data buffers
// Load data into data_in buffer
data_in[0] = (CMD_SEND_STATUS | EMMC_CRCCE_CHECK |
             EMMC_CICE_CHECK | EMMC_RTS_48RESP);
data_in[1] = (0x1234 << 16);
// Write data using nvmlib_device's interface driver command function
status = NVMLIB_Command(&nvmlib_device, data_out, data_in, 2);
```

NOTE: This function does not verify anything about the command. It is the user's responsibility to issue a command that can be correctly interpreted by the device.

NOTE: When performing unidirectional data transfers via SPI, data_out or data_in can be NULL, but not both. When using the eMMC interface, neither data_out nor data_in can be NULL.

21.6.19 NVMLIB_Interface_Configure

```
EXTERN_C int NVMLIB_Interface_Configure(NVMLIB_DEVICE * device, const void * interface,
unsigned int spi_cfg, unsigned int emmc_prescaler, uint32_t cfg, unsigned int clk, unsigned
int ctrl, unsigned int io0, unsigned int io1)
```

Configures the hardware interface pins and prescaler.

This function configures the underlying hardware interface to be used by the NVM library. It has to be called

Location: nvmlib.h:884

Parameters

Direction	Name	Description
in	<i>device</i>	The NVM library resource structure pointer. This allows an application to use the library with more than one device, as all the needed context is stored in this structure.
in	<i>interface</i>	Pointer to a SPI interface (SPI0 or SPI1) if the underlying NVM device is a SPI device or a pointer to the eMMC INTERFACE instance. For the autodetection to work, this argument must be one of the SPI interfaces.
in	<i>spi_cfg</i>	value to be set in the SPI_CFG register

Ezairo 8300 Firmware Reference

Direction	Name	Description
in	<i>emmc_prescaler</i>	The eMMC interface clock rate. A typical value would be 3840.
in	<i>cfg</i>	the pull-up, low-pass filter and drive strength options for the DIO pins. This value is used as-is in the DRIVE, LPF and PULL_CTRL fields of the DIO_CFG registers.
in	<i>clk</i>	Pin number for the Clock signal. Valid values are 0 to 35.
in	<i>ctrl</i>	Pin number for the Slave Select signal in the case of the SPI interface or cmd in the case of the eMMC interface. Valid values are 0 to 35.
in	<i>io0</i>	Pin number for the Master Out, Slave In (SERO) signal for the SPI interface or dat for the eMMC interface. Valid values are 0 to 35.
in	<i>io1</i>	Pin number for the Master In, Slave Out (SERI) signal for the SPI interface. Ignored for the eMMC interface. Valid values are 0 to 35.

Return

[execution status](#) ARM_DRIVER_OK if successful, ARM_DRIVER_ERROR* otherwise.

before NVMLIB_Initialize. This function can be called more than once. To configure the SPI interface, pass a valid SPI_Type pointer as the second parameter. To configure the eMMC interface, pass a pointer to the eMMC INTERFACE instance.

Assumptions

The NVM library interface configuration and initialization functions are called whenever the NVM interface is configured or reconfigured due to changes in clocking or other interface configurations.

Ezairo 8300 Firmware Reference

Example Code for NVMLIB_Interface_Configure

```
// Configures the hardware interface pins and prescaler.
// Using 3.84 MHz clock for EEMC
NVMLIB_Interface_Configure(&nvmlib_device,
    (void*) SPI0,
    SPI_PRESCALE_8,
    3840U,
    (DIO_8X_DRIVE | DIO_LPF_DISABLE | DIO_250K_PULL_UP),
    0, 1, 2, 3);
```

NOTE: This function has to be called at least once before [NVMLIB_Initialize](#).

21.6.20 NVMLIB_InterfaceQSPI_Configure

```
EXTERN_C int NVMLIB_InterfaceQSPI_Configure(NVMLIB_DEVICE * device, const void * interface,
    unsigned int spi_cfg, unsigned int emmc_prescaler, uint32_t cfg, unsigned int clk, unsigned
    int ctrl, unsigned int io0, unsigned int io1, unsigned int io2, unsigned int io3)
```

Configures the hardware interface pins and prescaler.

This function configures the underlying hardware interface to be used by the NVM library. It has to be called

Location: nvmlib.h:935

Parameters

Direction	Name	Description
in	<i>device</i>	The NVM library resource structure pointer. This allows an application to use the library with more than one device, as all the needed context is stored in this structure.
in	<i>interface</i>	Pointer to a SPI interface (SPI0 or SPI1) if the underlying NVM device is a SPI device or a pointer to the eMMC INTERFACE instance. For the autodetection to work, this argument must be one of the SPI interfaces.
in	<i>spi_cfg</i>	value to be set in the SPI_CFG register
in	<i>emmc_prescaler</i>	The eMMC interface clock rate. A typical value would be 3840.
in	<i>cfg</i>	the pull-up, low-pass filter and drive strength options for the DIO pins. This value is used as-is in the DRIVE, LPF and

Ezairo 8300 Firmware Reference

Direction	Name	Description
		PULL_CTRL fields of the DIO_CFG registers.
in	<i>clk</i>	Pin number for the Clock signal. Valid values are 0 to 35.
in	<i>ctrl</i>	Pin number for the Slave Select signal in the case of the SPI interface or cmd in the case of the eMMC interface. Valid values are 0 to 35.
in	<i>io0</i>	Pin number for the Master Out, Slave In (SERO) signal for the SPI interface or dat for the eMMC interface. Valid values are 0 to 35.
in	<i>io1</i>	Pin number for the Master In, Slave Out (SERI) signal for the SPI interface. Ignored for the eMMC interface. Valid values are 0 to 35.
in	<i>io2</i>	Pin number for QSPI IO2. Valid values are 0 to 35.
in	<i>io3</i>	Pin number for QSPI IO3. Valid values are 0 to 35.

Return

[execution status](#) ARM_DRIVER_OK if successful, ARM_DRIVER_ERROR* otherwise.

before NVMLIB_Initialize. This function can be called more than once. To configure the SPI interface, pass a valid SPI_Type pointer as the second parameter. To configure the eMMC interface, pass a pointer to the eMMC INTERFACE instance.

Assumptions

The NVM library interface configuration and initialization functions are called whenever the NVM interface is configured or reconfigured due to changes in clocking or other interface configurations.

Example Code for NVMLIB_InterfaceQSPI_Configure

```
// Configures the hardware interface pins and prescaler.
// Using 3.84 MHz clock for EEMC
NVMLIB_Interface_Configure(&nvmllib_device,
    (void*) SPI0,
    SPI_PRESCALE_8,
    3840U,
    (DIO_8X_DRIVE | DIO_LPF_DISABLE | DIO_250K_PULL_UP),
    0, 1, 2, 3);
```

NOTE: This function has to be called at least once before [NVMLIB Initialize](#).

CHAPTER 22

CFX AES Library Reference

This collection of functions describes the AES Security Library implementation for the CFX core.

22.1 SUMMARY

Typedefs

- [AES_KEY_EXPANSION](#) : AES key memory structure.

Data Structures

- [AES_KEY_EXPANSION](#) : AES key memory structure.

Functions

- [AESInitialize](#) : Initializes the AES encryption code.
- [AESEExpandKey](#) : Expand a 128-bit AES key to the full expanded key—including both the encryption form (needed for the expansion of the decryption key), and the decryption form (needed for decrypting blocks of AES encrypted data).
- [AESDecrypt](#) : Decrypts in-place (ECB mode) one or more blocks of contiguous data using the specified expanded key.
- [AESCleanUp](#) : Clear the keys from memory—use this before the application switches the access mode to unrestricted, if it is important to maintain the security of the current key.

22.2 CFX AES LIBRARY REFERENCE TYPEDEF DOCUMENTATION

22.2.1 AES_KEY_EXPANSION

```
typedef struct AES\_KEY\_EXPANSION AES_KEY_EXPANSION
```

Location: aeslib.h:61

AES key memory structure.

Don't initialize, filled by AESEExpandKey

22.3 CFX AES LIBRARY REFERENCE DATA STRUCTURES TYPE DOCUMENTATION

22.3.1 _AES_KEY_EXPANSION

Location: aeslib.h:54

AES key memory structure.

Don't initialize, filled by AESExpandKey

Data Fields

Type	Name	Description
uint32_t	<i>decryption_subkeys</i> [40]	Don't initialize, filled by AESExpandKey.
uint32_t	<i>key[4]</i>	AES key.
uint32_t	<i>encryption_subkeys</i> [40]	Don't initialize, filled by AESExpandKey.

22.4 CFX AES LIBRARY REFERENCE FUNCTION DOCUMENTATION

22.4.1 AESInitialize

```
void AESInitialize()
```

Initializes the AES encryption code.

Location: aeslib.h:71

Example Code for AESInitialize

```
/* Initialize the AES library (included for backward  
 * compatibility with previous AES library implementations) */  
AESInitialize();
```

NOTE: See aes.S for more details on the implementation.

22.4.2 AESExpandKey

```
void AESExpandKey(uint32_t * key)
```

Expand a 128-bit AES key to the full expanded key—including both the encryption form (needed for the expansion of the decryption key), and the decryption form (needed for decrypting blocks of AES encrypted data).

Location: aeslib.h:82

Parameters

Direction	Name	Description
in	<i>key</i>	Pointer to the key member of a AES_KEY_EXPANSION structure.

Example Code for AESExpandKey

```
// Decryption key buffer
static AES\_KEY\_EXPANSION User_Key;

// Expand boot key
AESExpandKey(User_Key.key);
```

NOTE: See aes.S for more details on the implementation.

22.4.3 AESDecrypt

```
void AESDecrypt(uint32_t * data, uint32_t * key, unsigned int blocks)
```

Decrypts in-place (ECB mode) one or more blocks of contiguous data using the specified expanded key.

Location: aeslib.h:95

Ezairo 8300 Firmware Reference

Parameters

Direction	Name	Description
in	<i>data</i>	Pointer to the IOMEM buffer with the data to be decrypted.
in	<i>key</i>	Pointer to the key member of a AES_KEY_EXPANSION structure.
in	<i>blocks</i>	Number of blocks to be decrypted.

Example Code for AESDecrypt

```
// Create an 8 word (256 bit) buffer to hold encrypted data
static uint32_t chess_storage(IOMEM) Encrypted_Data_buffer[8];
// Use the correct key to decrypt 2 blocks (256 bits) of data
AESDecrypt(Encrypted_Data_buffer, User_Key.key, 2);
```

NOTE: Both data and key pointers must point to IOMEM. The key pointer must be the key member of the AES_KEY_EXPANSION structure.

NOTE: See aes.S for more details on the implementation.

22.4.4 AESCleanUp

```
void AESCleanUp(uint32_t * key)
```

Clear the keys from memory—use this before the application switches the access mode to unrestricted, if it is important to maintain the security of the current key.

Location: aeslib.h:106

Parameters

Direction	Name	Description
in	<i>key</i>	Pointer to the key member of a AES_KEY_EXPANSION structure.

Example Code for AESCleanup

```
// Clear out all the key data (including subkeys) used by AES  
AESCleanup(User_Key.key);
```

CHAPTER 23

Supplemental Calibration Library Reference

The supplemental calibration library supports handling and saving the calibration information.

23.1 SUMMARY

Macros

- [ERRNO POWER CAL MARKER](#) : Power calibration error marker.
- [ERRNO VBG CAL ERROR](#) : Error when calibrating the bandgap.
- [ERRNO VBG CAL BADREF ERROR](#) : Error when calibrating the bandgap (bad supply reference)
- [ERRNO VDDIF CAL ERROR](#) : Error when calibrating VDDIF.
- [ERRNO VDDA CAL ERROR](#) : Error when calibrating VDDA.
- [ERRNO VDDC CAL ERROR](#) : Error when calibrating VDDC.
- [ERRNO VDDM CAL ERROR](#) : Error when calibrating VDDM.
- [ERRNO VMIC CAL ERROR](#) : Error when calibrating VMIC.
- [ERRNO VMIC EBL ERROR](#) : Error when enabling VMIC.
- [ERRNO VDDOD CAL ERROR](#) : Error when calibrating VDDOD.
- [ERRNO VDDOD EBL ERROR](#) : Error when enabling VDDOD.
- [ERRNO CLK CAL MARKER](#) : Clock calibration error marker.
- [ERRNO INTERNAL OSC CAL ERROR](#) : Error when calibrating the internal oscillator.
- [ERRNO SUPPORT CAL MARKER](#) : Calibration support error marker.
- [ERRNO MANU CRC CAL ERROR](#) : Manufacturing area CRC check failed
- [ERRNO NVMLIB READ CAL ERROR](#) : Error when reading from the NVM device
- [ERRNO NVMLIB WRITE CAL ERROR](#) : Error when writing to the NVM device
- [ERRNO PARAMETER CAL ERROR](#) : Invalid input parameters
- [ERRNO BOOTINFO CRC CAL ERROR](#) : Boot Information area CRC check failed
- [ERRNO LEN MISMATCH CAL ERROR](#) : The table length in boot info != MANU_TABLE_SIZE_OCTETS
- [ERRNO UNALIGNED CAL ERROR](#) : The manufacturing table is not word aligned in NVM
- [ERRNO ADC CAL MARKER](#) : ADC calibration error marker.
- [ERRNO ADC OFFSET CAL ERROR](#) : ADC offset calibration didn't succeed for all drive settings.
- [MIN ADC STABILIZATION DELAY](#) : ADC stabilization delay.
- [MIN ADC AVERAGES](#) : Number of ADC samples to average over.
- [VREG TARGET](#) : VREG target in mV.
- [VDDCM MIN CP DELTA TARGET](#) : VDDC charge pump delta target in mV.
- [VDDC RETENTION TARGET](#) : VDDC retention target in mV.
- [VDDC LIMIT TARGET](#) : VDDC limit target in mV.
- [VDDC ACTIVE TARGET](#) : VDDC target in mV.
- [VDDM RETENTION TARGET](#) : VDDM retention target in mV.
- [VDDM STANDBY TARGET](#) : VDDM standby target in mV.
- [VDDM ACTIVE TARGET](#) : VDDM target in mV.
- [VDDA ACTIVE TARGET](#) : VDDA target in mV.
- [VDDIF ACTIVE TARGET](#) : VDDIF target in mV.
- [VMIC ACTIVE TARGET](#) : VMIC target in mV.
- [VDDOD ACTIVE TARGET](#) : VDDOD target in mV.
- [VSSA EXPECTED](#) : Expected VSSA in mV.

Ezairo 8300 Firmware Reference

Functions

- [Calibrate ADC Offset](#) : Calibrate the ADC offset trim value for a given channel number.
- [Calibrate Clock Initialize](#) : Initialize the internal oscillator and the input stage clock (ADCCLK) for the ASCC.
- [Calibrate Clock Internal OSC](#) : Used to calibrate the internal oscillator to a specified frequency.
- [Calibrate Clock Internal OSC Mult](#) : Used to calibrate the internal oscillator to a specified frequency with a specific multiplier.
- [Calibrate Power VREGAndLSAD](#) : Calibrate VREG and LSAD using a two-point calibration based on VBAT and VSSA.
- [Calibrate Power VDDIF](#) : Calibrate the interfaces charge pump voltage supply (VDDIF).
- [Calibrate Power VDDA](#) : Wrapper function to call _Calibrate_Power_VDDA with bandgap reference configuration for VDDA.
- [Calibrate Power VDDA PMURef](#) : Wrapper function to call _Calibrate_Power_VDDA with PMU reference configuration for VDDA.
- [Calibrate Power VDDC](#) : Wrapper function to call _Calibrate_Power_VDDC with bandgap reference configuration for VDDC/VDDM.
- [Calibrate Power VDDC PMURef](#) : Wrapper function to call _Calibrate_Power_VDDC with PMU reference configuration for VDDC.
- [Calibrate Power VDDM](#) : Wrapper function to call _Calibrate_Power_VDDM with bandgap reference configuration for VDDM/VDDC.
- [Calibrate Power VDDM PMURef](#) : Wrapper function to call _Calibrate_Power_VDDM with PMU reference configuration for VDDM.
- [Calibrate Power VMIC](#) : Calibrate the microphone voltage supply (VMIC).
- [Calibrate Power VDDOD](#) : Calibrate the output drivers voltage supply (VDDOD).
- [Calibrate Power VoltageLoaded](#) : Set to true if slow stabilizing voltages such as VDDOD/VMIC/VDDIF are loaded using a 1k to 2k ohm resistor to reduce stabilization delays for faster calibration.
- [Calibrate Support Initialize](#) : Initializes the local manufacturing table and configures support functions for a NVM device.
- [Calibrate Support ClearManuTable](#) : Clears the local manufacturing table.
- [Calibrate Support LoadManuTable](#) : Load the existing manufacturing information from an attached NVM device into the local RAM instance of the manufacturing information table.
- [Calibrate Support WriteManuTable](#) : Updates the CRC and writes a local RAM instance of the manufacturing table to the NVM device (overwriting any previous version)
- [Calibrate Support UpdateVDDIF](#) : Load the calibrated setting from the trim register into the local RAM instance of the manufacturing information table.
- [Calibrate Support UpdateVDDA](#) : Load the calibrated setting from the trim register into the local RAM instance of the manufacturing information table.
- [Calibrate Support UpdateVDDC](#) : Load the calibrated setting from the trim register into the local RAM instance of the manufacturing information table.
- [Calibrate Support UpdateVDDCByValue](#) : Store the supplied trim setting into the local RAM instance of the manufacturing information table.
- [Calibrate Support UpdateVDDC CP](#) : Load the calibrated setting from the trim register into the local RAM instance of the manufacturing information table.
- [Calibrate Support UpdateVDDM](#) : Load the calibrated setting from the trim register into the local RAM instance of the manufacturing information table.

Ezairo 8300 Firmware Reference

- [Calibrate Support UpdateVDDMByValue](#) : Store the supplied trim setting into the local RAM instance of the manufacturing information table.
- [Calibrate Support UpdateVDDM CP](#) : Load the calibrated setting from the trim register into the local RAM instance of the manufacturing information table.
- [Calibrate Support UpdateVDDM CPByValue](#) : Load the calibrated setting from the trim register into the local RAM instance of the manufacturing information table.
- [Calibrate Support UpdateVDDOD](#) : Load the calibrated setting from the trim register into the local RAM instance of the manufacturing information table.
- [Calibrate Support UpdateVMIC](#) : Load the calibrated setting from the trim register into the local RAM instance of the manufacturing information table.
- [Calibrate Support UpdateVREG](#) : Load the calibrated setting from the trim register into the local RAM instance of the manufacturing information table.
- [Calibrate Support UpdateCLK](#) : Load the calibrated setting from the trim register into the local RAM instance of the manufacturing information table.
- [Calibrate Support UpdateLSAD GAIN](#) : Load the calibrated setting from the trim register into the local RAM instance of the manufacturing information table.
- [Calibrate Support UpdateLSAD OFFSET](#) : Load the calibrated setting from the trim register into the local RAM instance of the manufacturing information table.
- [Calibrate Support UpdateADC OFFSET](#) : Load the calibrated offsets from the trim registers into the local RAM instance of the manufacturing information table.
- [Calibrate Support Delay Cycles](#) : Delay for the specified number of cycles.

23.2 DETAILED DESCRIPTION

This library provides functions for calibrating the SoC's power supplies, oscillators, and ADCs. This library also contains support functions for reading and writing this calibrated information in calibration tables stored to an external NVM device.

23.3 SUPPLEMENTAL CALIBRATION LIBRARY REFERENCE MACRO DEFINITION DOCUMENTATION

23.3.1 ERRNO_POWER_CAL_MARKER

```
#define ERRNO_POWER_CAL_MARKER 0x10
```

Power calibration error marker.

Location: calibrate.h:60

23.3.2 ERRNO_VBG_CAL_ERROR

```
#define ERRNO_VBG_CAL_ERROR (0x0001 | ERRNO\_POWER\_CAL\_MARKER)
```

Error when calibrating the bandgap.

Location: calibrate.h:61

23.3.3 ERRNO_VBG_CAL_BADREF_ERROR

```
#define ERRNO_VBG_CAL_BADREF_ERROR (0x0002 | ERRNO\_POWER\_CAL\_MARKER)
```

Error when calibrating the bandgap (bad supply reference)

Location: calibrate.h:62

23.3.4 ERRNO_VDDIF_CAL_ERROR

```
#define ERRNO_VDDIF_CAL_ERROR (0x0003 | ERRNO\_POWER\_CAL\_MARKER)
```

Error when calibrating VDDIF.

Location: calibrate.h:63

23.3.5 ERRNO_VDDA_CAL_ERROR

```
#define ERRNO_VDDA_CAL_ERROR (0x0004 | ERRNO\_POWER\_CAL\_MARKER)
```

Error when calibrating VDDA.

Location: calibrate.h:64

23.3.6 ERRNO_VDDC_CAL_ERROR

```
#define ERRNO_VDDC_CAL_ERROR (0x0005 | ERRNO\_POWER\_CAL\_MARKER)
```

Error when calibrating VDDC.

Location: calibrate.h:65

23.3.7 ERRNO_VDDM_CAL_ERROR

```
#define ERRNO_VDDM_CAL_ERROR (0x0006 | ERRNO\_POWER\_CAL\_MARKER)
```


Error when calibrating VDDM.

Location: calibrate.h:66

23.3.8 ERRNO_VMIC_CAL_ERROR

```
#define ERRNO_VMIC_CAL_ERROR (0x0007 | ERRNO\_POWER\_CAL\_MARKER)
```

Error when calibrating VMIC.

Location: calibrate.h:67

23.3.9 ERRNO_VMIC_EBL_ERROR

```
#define ERRNO_VMIC_EBL_ERROR (0x0008 | ERRNO\_POWER\_CAL\_MARKER)
```

Error when enabling VMIC.

Location: calibrate.h:68

23.3.10 ERRNO_VDDOD_CAL_ERROR

```
#define ERRNO_VDDOD_CAL_ERROR (0x0009 | ERRNO\_POWER\_CAL\_MARKER)
```

Error when calibrating VDDOD.

Location: calibrate.h:69

23.3.11 ERRNO_VDDOD_EBL_ERROR

```
#define ERRNO_VDDOD_EBL_ERROR (0x000A | ERRNO\_POWER\_CAL\_MARKER)
```

Error when enabling VDDOD.

Location: calibrate.h:70

23.3.12 ERRNO_CLK_CAL_MARKER

```
#define ERRNO_CLK_CAL_MARKER 0x20
```

Clock calibration error marker.

Location: calibrate.h:73

23.3.13 ERRNO_INTERNAL_OSC_CAL_ERROR

```
#define ERRNO_INTERNAL_OSC_CAL_ERROR (0x0001 | ERRNO\_CLK\_CAL\_MARKER)
```

Error when calibrating the internal oscillator.

Location: calibrate.h:74

23.3.14 ERRNO_SUPPORT_CAL_MARKER

```
#define ERRNO_SUPPORT_CAL_MARKER 0x30
```

Calibration support error marker.

Location: calibrate.h:77

23.3.15 ERRNO_MANU_CRC_CAL_ERROR

```
#define ERRNO_MANU_CRC_CAL_ERROR (0x0001 | ERRNO\_SUPPORT\_CAL\_MARKER)
```

Manufacturing area CRC check failed

Location: calibrate.h:78

23.3.16 ERRNO_NVMLIB_READ_CAL_ERROR

```
#define ERRNO_NVMLIB_READ_CAL_ERROR (0x0002 | ERRNO\_SUPPORT\_CAL\_MARKER)
```

Error when reading from the NVM device

Location: calibrate.h:79

23.3.17 ERRNO_NVMLIB_WRITE_CAL_ERROR

```
#define ERRNO_NVMLIB_WRITE_CAL_ERROR (0x0003 | ERRNO\_SUPPORT\_CAL\_MARKER)
```

Error when writing to the NVM device

Location: calibrate.h:80

23.3.18 ERRNO_PARAMETER_CAL_ERROR

```
#define ERRNO_PARAMETER_CAL_ERROR (0x0004 | ERRNO\_SUPPORT\_CAL\_MARKER)
```

Invalid input parameters

Location: calibrate.h:81

23.3.19 ERRNO_BOOTINFO_CRC_CAL_ERROR

```
#define ERRNO_BOOTINFO_CRC_CAL_ERROR (0x0005 | ERRNO\_SUPPORT\_CAL\_MARKER)
```

Boot Information area CRC check failed

Location: calibrate.h:82

23.3.20 ERRNO_LEN_MISMATCH_CAL_ERROR

```
#define ERRNO_LEN_MISMATCH_CAL_ERROR (0x0006 | ERRNO\_SUPPORT\_CAL\_MARKER)
```

The table length in boot info != MANU_TABLE_SIZE_OCTETS

Location: calibrate.h:83

23.3.21 ERRNO_UNALIGNED_CAL_ERROR

```
#define ERRNO_UNALIGNED_CAL_ERROR (0x0007 | ERRNO\_SUPPORT\_CAL\_MARKER)
```

The manufacturing table is not word aligned in NVM

Location: calibrate.h:84

23.3.22 ERRNO_ADC_CAL_MARKER

```
#define ERRNO_ADC_CAL_MARKER 0x40
```

ADC calibration error marker.

Location: calibrate.h:87

23.3.23 ERRNO_ADC_OFFSET_CAL_ERROR

```
#define ERRNO_ADC_OFFSET_CAL_ERROR (0x001 | ERRNO\_ADC\_CAL\_MARKER)
```

ADC offset calibration didn't succeed for all drive settings.

Location: calibrate.h:88

23.3.24 MIN_ADC_STABILIZATION_DELAY

```
#define MIN_ADC_STABILIZATION_DELAY 12
```

ADC stabilization delay.

Location: calibrate_adc.h:37

23.3.25 MIN_ADC_AVERAGES

```
#define MIN_ADC_AVERAGES 8
```

Number of ADC samples to average over.

Location: calibrate_adc.h:38

23.3.26 VREG_TARGET

```
#define VREG_TARGET 900
```

VREG target in mV.

Location: `calibrate_power.h:37`

23.3.27 VDDCM_MIN_CP_DELTA_TARGET

```
#define VDDCM_MIN_CP_DELTA_TARGET 7
```

VDDC charge pump delta target in mV.

Location: `calibrate_power.h:38`

23.3.28 VDDC_RETENTION_TARGET

```
#define VDDC_RETENTION_TARGET 530
```

VDDC retention target in mV.

Location: `calibrate_power.h:39`

23.3.29 VDDC_LIMIT_TARGET

```
#define VDDC_LIMIT_TARGET 510
```

VDDC limit target in mV.

Location: `calibrate_power.h:40`

23.3.30 VDDC_ACTIVE_TARGET

```
#define VDDC_ACTIVE_TARGET 780
```

VDDC target in mV.

Location: `calibrate_power.h:41`

23.3.31 VDDM_RETENTION_TARGET

```
#define VDDM_RETENTION_TARGET 580
```

VDDM retention target in mV.

Location: `calibrate_power.h`:42

23.3.32 VDDM_STANDBY_TARGET

```
#define VDDM_STANDBY_TARGET 800
```

VDDM standby target in mV.

Location: `calibrate_power.h`:43

23.3.33 VDDM_ACTIVE_TARGET

```
#define VDDM_ACTIVE_TARGET 780
```

VDDM target in mV.

Location: `calibrate_power.h`:44

23.3.34 VDDA_ACTIVE_TARGET

```
#define VDDA_ACTIVE_TARGET 1800
```

VDDA target in mV.

Location: `calibrate_power.h`:45

23.3.35 VDDIF_ACTIVE_TARGET

```
#define VDDIF_ACTIVE_TARGET 1800
```

VDDIF target in mV.

Location: `calibrate_power.h`:46

23.3.36 VMIC_ACTIVE_TARGET

```
#define VMIC_ACTIVE_TARGET 1150
```

VMIC target in mV.

Location: calibrate_power.h:47

23.3.37 VDDOD_ACTIVE_TARGET

```
#define VDDOD_ACTIVE_TARGET 1150
```

VDDOD target in mV.

Location: calibrate_power.h:48

23.3.38 VSSA_EXPECTED

```
#define VSSA_EXPECTED 0
```

Expected VSSA in mV.

Location: calibrate_power.h:53

23.4 SUPPLEMENTAL CALIBRATION LIBRARY REFERENCE FUNCTION DOCUMENTATION

23.4.1 Calibrate_ADC_Offset

```
int Calibrate_ADC_Offset(unsigned int channel_num, unsigned int stabilization_delay,  
unsigned int averages)
```

Calibrate the ADC offset trim value for a given channel number.

Location: calibrate_adc.h:59

Parameters

Ezairo 8300 Firmware Reference

Direction	Name	Description
in	<i>channel_num</i>	Channel number of the ADC to calibrate
in	<i>stabilization_delay</i>	maximum stabilization delay [ms] to use. Input a value higher than 12 ms (minimum recommended delay). NOTE: Delay is reduced by 4 ms after initial setup and halved for every time drive_setting is incremented.
in	<i>averages</i>	Number of samples to use for averaging ADC readings. Rounded up to the closest power of 2 to allow shifting.

Return

Status code indicating whether the calibration has succeeded

Example Code for Calibrate_ADC_Offset

```
// Calibrate the ADC offsets for channel number 0 with a stabilization
// delay of 12 ms and 8 averages
Calibrate\_ADC\_Offset(0, 12, 8);
```

23.4.2 Calibrate_Clock_Initialize

```
void Calibrate_Clock_Initialize()
```

Initialize the internal oscillator and the input stage clock (ADCCLK) for the ASCC.

Location: calibrate_clock.h:39

Example Code for Calibrate_Clock_Initialize

```
// Initialize the internal oscillator and the input stage clock
// (ADCCLK) for the ASCC
Calibrate\_Clock\_Initialize();
```


23.4.3 Calibrate_Clock_Internal_OSC

```
int Calibrate_Clock_Internal_OSC(unsigned int target, unsigned int ref_dio)
```

Used to calibrate the internal oscillator to a specified frequency.

Location: `calibrate_clock.h:51`

Parameters

Direction	Name	Description
in	<i>target</i>	Desired clock frequency in kHz; use a value between 2000 kHz and 61440 kHz
in	<i>ref_dio</i>	Reference clock DIO

Return

Status code indicating whether the calibration has succeeded

Assumptions

[Calibrate_Clock_Initialize\(\)](#) has been called.

Example Code for Calibrate_Clock_Internal_OSC

```
// Use DIO12 as a reference to calibrate the internal oscillator to 45 MHz  
Calibrate\_Clock\_Initialize\(\);  
result = Calibrate\_Clock\_Internal\_OSC(45000, DIO12);
```

23.4.4 Calibrate_Clock_Internal_OSC_Mult

```
int Calibrate_Clock_Internal_OSC_Mult(unsigned int target, unsigned int ref_dio, unsigned  
int multiplier)
```

Used to calibrate the internal oscillator to a specified frequency with a specific multiplier.

Location: `calibrate_clock.h:66`

Parameters

Direction	Name	Description
in	<i>target</i>	Desired clock frequency in kHz; use a value between 2000 kHz and 61440 kHz
in	<i>ref_dio</i>	Reference clock DIO
in	<i>multiplier</i>	Valid multiplier value that would work with the specified target frequency

Return

Status code indicating whether the calibration has succeeded

Assumptions

[Calibrate_Clock_Initialize\(\)](#) has been called.

Example Code for Calibrate_Clock_Internal_OSC_Mult

```
// Use DIO12 as a reference to calibrate the internal oscillator to 45 MHz
Calibrate\_Clock\_Initialize\(\);
result = Calibrate\_Clock\_Internal\_OSC(45000, DIO12);
```

23.4.5 Calibrate_Power_VREGAndLSAD

```
int Calibrate_Power_VREGAndLSAD(unsigned int lsad_num, unsigned int vbat_level)
```

Calibrate VREG and LSAD using a two-point calibration based on VBAT and VSSA.

Location: `calibrate_power.h:71`

Parameters

Direction	Name	Description
in	<i>lsad_num</i>	LSAD channel number [0-3]
in	<i>vbat_level</i>	VBAT level in mV

Return

Status code indicating whether the calibration has succeeded

Assumptions

For optimal accuracy, VBAT is sourced from an accurate power supply providing at least 1.5 V

Assumptions

All analog power blocks have been initialized to default values by the boot ROM

Example Code for Calibrate_Power_VREGAndLSAD

```
// Calibrate VREG and LSAD using a two-point calibration
// using VBAT supplied at 1.25 V, VSSA as references.
result1 = Calibrate\_Power\_VREGAndLSAD(0, 1250);
```

23.4.6 Calibrate_Power_VDDIF

```
int Calibrate_Power_VDDIF(unsigned int lsad_num, unsigned int target)
```

Calibrate the interfaces charge pump voltage supply (VDDIF).

Targets above 1774 mV are extrapolated due to LSAD saturation. Accuracy might be affected due to this especially if the target is far above 1774 mV.

Location: `calibrate_power.h:89`

Parameters

Direction	Name	Description
in	<i>lsad_num</i>	LSAD channel number [0-3]
in	<i>target</i>	Target voltage readback [mv]

Return

Status code indicating whether the calibration has succeeded

Assumptions

[Calibrate_Power_VREGAndLSAD\(\)](#) has been called

Assumptions

All analog power blocks have been initialized to default values.

Assumptions

Target doesn't exceed 1950 mV.

Example Code for Calibrate_Power_VDDIF

```
// Calibrate the interfaces charge pump voltage supply (VDDIF)
// based on VREG
result1 = Calibrate_Power_VREG_And_LSAD(0, 1250);
result2 = Calibrate\_Power\_VDDIF(0, 1775);
```

Ezairo 8300 Firmware Reference

23.4.7 Calibrate_Power_VDDA

```
int Calibrate_Power_VDDA(unsigned int lsad_num, unsigned int target)
```

Wrapper function to call `_Calibrate_Power_VDDA` with bandgap reference configuration for VDDA.

Targets above 1774 mV are extrapolated due to LSAD saturation. Accuracy might be affected due to this especially if the target is far above 1774 mV. NOTE: VDDA configuration isn't reverted back at the end of execution. It needs to be reverted back manually if needed.

Location: `calibrate_power.h:110`

Parameters

Direction	Name	Description
in	<i>lsad_num</i>	LSAD channel number [0-3]
in	<i>target</i>	Target voltage readback [mv]

Return

Status code indicating whether the calibration has succeeded

Assumptions

[Calibrate Power VREGAndLSAD\(\)](#) has been called

Assumptions

All analog power blocks have been initialized to default values

Assumptions

target doesn't exceed 1950 mV.

Ezairo 8300 Firmware Reference

Example Code for Calibrate_Power_VDDA

```
// Calibrate the analog charge pump voltage supply (VDDA)
// based on VREG
result1 = Calibrate_Power_VREG_And_LSAD(0, 1250);
result2 = Calibrate\_Power\_VDDA(0, 1800);
```

23.4.8 Calibrate_Power_VDDA_PMURef

```
int Calibrate_Power_VDDA_PMURef(unsigned int lsad_num, unsigned int target)
```

Wrapper function to call `_Calibrate_Power_VDDA` with PMU reference configuration for VDDA.

Targets above 1774 mV are extrapolated due to LSAD saturation. Accuracy might be affected due to this especially if the target is far above 1774 mV. NOTE: VDDA configuration isn't reverted back at the end of execution. It needs to be reverted back manually if needed.

Location: `calibrate_power.h:131`

Parameters

Direction	Name	Description
in	<i>lsad_num</i>	LSAD channel number [0-3]
in	<i>target</i>	Target voltage readback [mv]

Return

Status code indicating whether the calibration has succeeded

Assumptions

[Calibrate_Power_VREGAndLSAD\(\)](#) has been called

Assumptions

All analog power blocks have been initialized to default values

Assumptions

target doesn't exceed 1950 mV.

Example Code for Calibrate_Power_VDDA_PMURef

```
// Calibrate the analog charge pump voltage supply (VDDA)
// based on VREG
result1 = Calibrate_Power_VREG_And_LSAD(0, 1250);
result2 = Calibrate\_Power\_VDDA\_PMURef(0, 1800);
```

23.4.9 Calibrate_Power_VDDC

```
int Calibrate_Power_VDDC(unsigned int ldo_target, int cp_delta_target)
```

Wrapper function to call _Calibrate_Power_VDDC with bandgap reference configuration for VDDC/VDDM.

NOTE: Reference isn't reverted back at the end of execution It needs to be reverted back manually if needed.

Location: calibrate_power.h:148

Parameters

Direction	Name	Description
in	<i>ldo_target</i>	Target linear regulator voltage [mV]
in	<i>cp_delta_target</i>	Target charge pump delta voltage [mV]

Return

Status code indicating whether the calibration has succeeded

Assumptions

[Calibrate_Power_VREGAndLSAD\(\)](#) and `Calibrate_Power_VDDA` have been called

Assumptions

All analog power blocks have been initialized to default values

Example Code for `Calibrate_Power_VDDC`

```
// Calibrate the digital core voltage power supply (VDDC)
// based on VREG
result1 = Calibrate_Power_VREG_And_LSAD(0, 1250);
result2 = Calibrate\_Power\_VDDC(650, 7);
```

23.4.10 `Calibrate_Power_VDDC_PMURef`

```
int Calibrate_Power_VDDC_PMURef(unsigned int ldo_target, int cp_delta_target, uint16_t *
trim)
```

Wrapper function to call `_Calibrate_Power_VDDC` with PMU reference configuration for VDDC.

NOTE: Reference is reverted back at the end of execution.

Location: `calibrate_power.h`:167

Parameters

Ezairo 8300 Firmware Reference

Direction	Name	Description
in	<i>ldo_target</i>	Target linear regulator voltage [mV]
in	<i>cp_delta_target</i>	Target charge pump delta voltage [mV]
out	<i>trim</i>	Calculated trim code for VDDC (compatible with ANALOG_CP_VDDC_TRIM)

Return

Status code indicating whether the calibration has succeeded

Assumptions

[Calibrate_Power_VREGAndLSAD\(\)](#) and `Calibrate_Power_VDDA` have been called

Assumptions

All analog power blocks have been initialized to default values

Example Code for Calibrate_Power_VDDC_PMURef

```
// Calibrate the digital core voltage power supply (VDDC)
// based on VREG
result1 = Calibrate_Power_VREG_And_LSAD(0, 1250);
result2 = Calibrate\_Power\_VDDC\_PMURef(650, 7);
```

23.4.11 Calibrate_Power_VDDM

```
int Calibrate_Power_VDDM(unsigned int ldo_target, int cp_delta_target)
```

Wrapper function to call `_Calibrate_Power_VDDM` with bandgap reference configuration for VDDM/VDDC.

NOTE: Reference isn't reverted back at the end of execution It needs to be reverted back manually if needed.

onsemi Confidential
Ezairo 8300 Firmware Reference

Location: `calibrate_power.h:187`

Parameters

Direction	Name	Description
in	<i>ldo_target</i>	Target linear regulator voltage [mV] should be higher than current VDDC voltage
in	<i>cp_delta_target</i>	Target charge pump delta voltage [mV]

Return

Status code indicating whether the calibration has succeeded

Assumptions

[Calibrate_Power_VREGAndLSAD\(\)](#) and `Calibrate_Power_VDDA` have been called

Assumptions

All analog power blocks have been initialized to default values

Example Code for Calibrate_Power_VDDM

```
// Calibrate the digital memory voltage (VDDM)
// based on VREG and VDDC
result1 = Calibrate_Power_VREG_And_LSAD(0, 1250);
result2 = Calibrate\_Power\_VDDC(590, 7);
result3 = Calibrate\_Power\_VDDM(760, 7);
```

23.4.12 Calibrate_Power_VDDM_PMURef

```
int Calibrate_Power_VDDM_PMURef(unsigned int ldo_target, int cp_delta_target, uint16_t *
trim)
```

Wrapper function to call `_Calibrate_Power_VDDM` with PMU reference configuration for VDDM.

NOTE: Unlike other calibration functions the calculated reference is returned separately.

Location: `calibrate_power.h:212`

Parameters

Direction	Name	Description
in	<i>ldo_target</i>	Target linear regulator voltage [mV] should be higher than current VDDC voltage
in	<i>cp_delta_target</i>	Target charge pump delta voltage [mV]
out	<i>trim</i>	Calculated trim code for the VDDM (compatible with ANALOG_CP_VDDC_TRIM)

Return

Status code indicating whether the has calibration has succeeded

Assumptions

[Calibrate Power VREGAndLSAD\(\)](#) and `Calibrate_Power_VDDA` have been called

Assumptions

All analog power blocks have been initialized to default values

Assumptions

This function can use DMA channels 0-6 and all of the activity counters

Assumptions

This function is not called from within an active interrupt vector

Example Code for Calibrate_Power_VDDM_PMURef

```
// Calibrate the digital memory voltage (VDDM)
// based on VREG and VDDC
result1 = Calibrate_Power_VREG_And_LSAD(0, 1250);
result2 = Calibrate\_Power\_VDDC\_PMURef(590, 7);
result3 = Calibrate\_Power\_VDDM\_PMURef(760, 7);
```

23.4.13 Calibrate_Power_VMIC

```
int Calibrate_Power_VMIC(unsigned int lsad_num, unsigned int target)
```

Calibrate the microphone voltage supply (VMIC).

Location: `calibrate_power.h:228`

Parameters

Direction	Name	Description
in	<i>lsad_num</i>	LSAD channel number [0-3]
in	<i>target</i>	Target voltage readback [mv]

Return

Status code indicating whether the calibration has succeeded

Assumptions

[Calibrate_Power_VREGAndLSAD\(\)](#) has been called

Assumptions

All analog power blocks have been initialized to default values

Assumptions

VMIC regulator is enabled

Example Code for Calibrate_Power_VMIC

```
// Calibrate the microphone voltage supply (VMIC)
// based on VREG
result1 = Calibrate_Power_VREG_And_LSAD(0, 1250);
result2 = Calibrate\_Power\_VMIC(0, 1200);
```

23.4.14 Calibrate_Power_VDDOD

```
int Calibrate_Power_VDDOD(unsigned int lsad_num, unsigned int target)
```

Calibrate the output drivers voltage supply (VDDOD).

Location: `calibrate_power.h`:243

Parameters

Direction	Name	Description
in	<i>lsad_num</i>	LSAD channel number [0-3]
in	<i>target</i>	Target voltage readback [mv]

Return

Status code indicating whether the calibration has succeeded

Assumptions

[Calibrate_Power_VREGAndLSAD\(\)](#) has been called

Assumptions

All analog power blocks have been initialized to default values

Assumptions

VDDOD regulator is enabled

Example Code for Calibrate_Power_VDDOD

```
// Calibrate the output drivers voltage supply (VDDOD)
// based on VREG
result1 = Calibrate_Power_VREG_And_LSAD(0, 1250);
result2 = Calibrate\_Power\_VDDOD(0, 1200);
```

23.4.15 Calibrate_Power_VoltageLoaded

```
void Calibrate_Power_VoltageLoaded(bool setting)
```

Set to true if slow stabilizing voltages such as VDDOD/VMIC/VDDIF are loaded using a 1k to 2k ohm resistor to reduce stabilization delays for faster calibration.

Location: `calibrate_power.h:254`

Parameters

Direction	Name	Description
in	<i>setting</i>	true if voltages are loaded else false

Assumptions

Function is called after [Calibrate_Support_Initialize\(\)](#)

23.4.16 Calibrate_Support_Initialize

```
int Calibrate_Support_Initialize(NVMLIB\_DEVICE * device, uint32_t * manu_table, uint32_t * buffer)
```

Initializes the local manufacturing table and configures support functions for a NVM device.

Location: `calibrate_support.h:60`

Parameters

Direction	Name	Description
in	<i>device</i>	The NVM library resource structure pointer
in	<i>manu_table</i>	Manufacturing table RAM buffer pointer
in	<i>buffer</i>	Buffer used for read/write operations

Return

Values other than `ERRNO_NO_ERROR (0)` signify NVM verification and access errors. Load and write table functions shouldn't be called without NVM access.

Example Code for Calibrate_Support_Initialize

```
// Initializes the local manufacturing table and configures support functions
// for a NVM device
// Note: NVMLIB_Initialize should be called first.
result = Calibrate\_Support\_Initialize(&nvmlib_device, manu_table, buffer);
```

NOTE: The `Calibrate_Support_Initialize` function must be called first before using any other calibration support functions. `manu_table` must be a `uint32_t` array of size `MANU_TABLE_SIZE`. `manu_table` must not be modified after initialization. For EEPROM devices the buffer can be `NULL`. For Flash devices the buffer size must be `>=` erase unit size. For eMMC the buffer size must be `>=` `NVM_EMMC_READ_SIZE`. The buffer and `manu_table` **must** to be allocated in IOMEM. This expects that the data being pointed to is 32-bits wide. Use `chess_storage(IOMEM)`. The buffer may be modified between support function operations.

23.4.17 Calibrate_Support_ClearManuTable

```
void Calibrate_Support_ClearManuTable()
```

Clears the local manufacturing table.

Location: `calibrate_support.h:68`

Example Code for `Calibrate_Support_ClearManuTable`

```
// Clears the local manufacturing table  
Calibrate\_Support\_ClearManuTable();
```

23.4.18 `Calibrate_Support_LoadManuTable`

```
int Calibrate_Support_LoadManuTable()
```

Load the existing manufacturing information from an attached NVM device into the local RAM instance of the manufacturing information table.

Location: `calibrate_support.h:79`

Return

Values other than `ERRNO_NO_ERROR` (0) signify errors.

Example Code for `Calibrate_Support_LoadManuTable`

```
// Load the existing manufacturing information from an attached NVM device  
// into the local RAM instance of the manufacturing  
result = Calibrate\_Support\_LoadManuTable();
```

Assumptions

[Calibrate_Support_Initialize\(\)](#) has been called and no errors were returned.

Ezairo 8300 Firmware Reference

23.4.19 Calibrate_Support_WriteManuTable

```
int Calibrate_Support_WriteManuTable()
```

Updates the CRC and writes a local RAM instance of the manufacturing table to the NVM device (overwriting any previous version)

Location: calibrate_support.h:90

Return

Values other than ERRNO_NO_ERROR (0) signify errors.

Example Code for Calibrate_Support_WriteManuTable

```
// Writes a local RAM instance of the manufacturing table to the NVM device  
result = Calibrate\_Support\_WriteManuTable();
```

Assumptions

[Calibrate_Support_Initialize\(\)](#) has been called and no errors were returned.

23.4.20 Calibrate_Support_UpdateVDDIF

```
int Calibrate_Support_UpdateVDDIF(uint8_t target, uint8_t index)
```

Load the calibrated setting from the trim register into the local RAM instance of the manufacturing information table.

Location: calibrate_support.h:102

Parameters

onsemi Confidential
Ezairo 8300 Firmware Reference

Direction	Name	Description
in	<i>target</i>	The specified target voltage [10 mV]
in	<i>index</i>	The index in the manufacturing table where the variable will be stored.

Return

Values other than `ERRNO_NO_ERROR` (0) signify errors.

Example Code for Calibrate_Support_UpdateVDDIF

```
// Load the calibrated setting from the trim register into the local RAM
// instance of the manufacturing information table
result = Calibrate\_Support\_UpdateVDDIF(178, 0);
```

Assumptions

Calibration has already run

23.4.21 Calibrate_Support_UpdateVDDA

```
int Calibrate_Support_UpdateVDDA(uint8_t target, uint8_t index, uint8_t pmuref)
```

Load the calibrated setting from the trim register into the local RAM instance of the manufacturing information table.

Location: `calibrate_support.h:116`

Parameters

Ezairo 8300 Firmware Reference

Direction	Name	Description
in	<i>target</i>	The specified target voltage [10 mV]
in	<i>index</i>	The index in the manufacturing table where the variable will be stored.
in	<i>pmuref</i>	Use 1 for PMU reference trim value, and 0 for bandgap reference trim value.

Return

Values other than ERRNO_NO_ERROR (0) signify errors.

Example Code for Calibrate_Support_UpdateVDDA

```
// Load the bandgap reference calibrated setting from the trim register into the local RAM
// instance of the manufacturing information table
result = Calibrate_Support_UpdateVDDA(178, 0, 0);
```

Assumptions

Calibration has already run

23.4.22 Calibrate_Support_UpdateVDDC

```
int Calibrate_Support_UpdateVDDC(uint8_t target, uint8_t index, uint8_t pmuref)
```

Load the calibrated setting from the trim register into the local RAM instance of the manufacturing information table.

Location: calibrate_support.h:130

Parameters

Ezairo 8300 Firmware Reference

Direction	Name	Description
in	<i>target</i>	The specified target voltage [10 mV]
in	<i>index</i>	The index in the manufacturing table where the variable will be stored.
in	<i>pmuref</i>	Use 1 for PMU reference trim value, and 0 for bandgap reference trim value.

Return

Values other than ERRNO_NO_ERROR (0) signify errors.

Example Code for Calibrate_Support_UpdateVDDC

```
// Load the bandgap reference calibrated setting from the trim register into the local RAM
// instance of the manufacturing information table
result = Calibrate_Support_UpdateVDDC(59, 0, 0);
```

Assumptions

Calibration has already run

23.4.23 Calibrate_Support_UpdateVDDCByValue

```
int Calibrate_Support_UpdateVDDCByValue(uint8_t target, uint8_t index, uint8_t pmuref,
uint8_t value)
```

Store the supplied trim setting into the local RAM instance of the manufacturing information table.

Location: calibrate_support.h:144

Parameters

Ezairo 8300 Firmware Reference

Direction	Name	Description
in	<i>target</i>	The specified target voltage [10 mV]
in	<i>index</i>	The index in the manufacturing table where the variable will be stored.
in	<i>pmuref</i>	Use 1 for PMU reference trim value, and 0 for bandgap reference trim value.
in	<i>value</i>	Value to be stored (only LDO trim)

Return

Values other than `ERRNO_NO_ERROR` (0) signify errors.

Example Code for Calibrate_Support_UpdateVDDCByValue

```
// Load the bandgap reference calibrated setting from the trim register into the local RAM
// instance of the manufacturing information table
result = Calibrate_Support_UpdateVDDC(59, 0, 0);
```

23.4.24 Calibrate_Support_UpdateVDDC_CP

```
int Calibrate_Support_UpdateVDDC_CP(int8_t target, uint8_t index)
```

Load the calibrated setting from the trim register into the local RAM instance of the manufacturing information table.

Location: `calibrate_support.h:158`

Parameters

Direction	Name	Description
in	<i>target</i>	The specified target voltage [mV]
in	<i>index</i>	The index in the manufacturing table where the variable will be stored.

Return

Values other than `ERRNO_NO_ERROR` (0) signify errors.

Example Code for Calibrate_Support_UpdateVDDC_CP

```
// Load the calibrated setting from the trim register into the local RAM  
// instance of the manufacturing information table  
result = Calibrate\_Support\_UpdateVDDC\_CP(7, 0);
```

Assumptions

Calibration has already run

23.4.25 Calibrate_Support_UpdateVDDM

```
int Calibrate_Support_UpdateVDDM(uint8_t target, uint8_t index, uint8_t pmuref)
```

Load the calibrated setting from the trim register into the local RAM instance of the manufacturing information table.

Location: `calibrate_support.h`:172

Parameters

Direction	Name	Description
in	<i>target</i>	The specified target voltage [10 mV]
in	<i>index</i>	The index in the manufacturing table where the variable will be stored.
in	<i>pmuref</i>	Use 1 for PMU reference trim value, and 0 for bandgap reference trim value.

Return

Values other than `ERRNO_NO_ERROR` (0) signify errors.

Example Code for Calibrate_Support_UpdateVDDM

```
// Load the bandgap reference calibrated setting from the trim register into the local RAM
// instance of the manufacturing information table
result = Calibrate_Support_UpdateVDDM(76, 0, 0);
```

Assumptions

Calibration has already run

23.4.26 Calibrate_Support_UpdateVDDMByValue

```
int Calibrate_Support_UpdateVDDMByValue(uint8_t target, uint8_t index, uint8_t pmuref,
uint8_t value)
```

Store the supplied trim setting into the local RAM instance of the manufacturing information table.

Location: `calibrate_support.h`:186

Parameters

Direction	Name	Description
in	<i>target</i>	The specified target voltage [10 mV]
in	<i>index</i>	The index in the manufacturing table where the variable will be stored.
in	<i>pmuref</i>	Use 1 for PMU reference trim value, and 0 for bandgap reference trim value.
in	<i>value</i>	Value to be stored (only LDO trim)

Return

Values other than ERRNO_NO_ERROR (0) signify errors.

Example Code for Calibrate_Support_UpdateVDDMByValue

```
// Load the bandgap reference calibrated setting from the trim register into the local RAM
// instance of the manufacturing information table
result = Calibrate_Support_UpdateVDDM(76, 0, 0);
```

23.4.27 Calibrate_Support_UpdateVDDM_CP

```
int Calibrate_Support_UpdateVDDM_CP(int8_t target, uint8_t index)
```

Load the calibrated setting from the trim register into the local RAM instance of the manufacturing information table.

Location: calibrate_support.h:199

Parameters

Direction	Name	Description
in	<i>target</i>	The specified target voltage [mV]
in	<i>index</i>	The index in the manufacturing table where the variable will be stored.

Return

Values other than ERRNO_NO_ERROR (0) signify errors.

Example Code for Calibrate_Support_UpdateVDDM_CP

```
// Load the calibrated setting from the trim register into the local RAM  
// instance of the manufacturing information table  
result = Calibrate_Support_UpdateVDDM_CP(7, 0);
```

Assumptions

Calibration has already run

23.4.28 Calibrate_Support_UpdateVDDM_CPByValue

```
int Calibrate_Support_UpdateVDDM_CPByValue(int8_t target, uint8_t index, uint8_t value)
```

Load the calibrated setting from the trim register into the local RAM instance of the manufacturing information table.

Location: calibrate_support.h:211

Parameters

Direction	Name	Description
in	<i>target</i>	The specified target voltage [mV]
in	<i>index</i>	The index in the manufacturing table where the variable will be stored.
in	<i>value</i>	Value to be stored (only CP trim)

Return

Values other than ERRNO_NO_ERROR (0) signify errors.

Ezairo 8300 Firmware Reference

Example Code for Calibrate_Support_UpdateVDDM_CPByValue

```
// Load the calibrated setting from the trim register into the local RAM
// instance of the manufacturing information table
result = Calibrate\_Support\_UpdateVDDM\_CP(7, 0);
```

23.4.29 Calibrate_Support_UpdateVDDOD

```
int Calibrate_Support_UpdateVDDOD(uint8_t target, uint8_t index)
```

Load the calibrated setting from the trim register into the local RAM instance of the manufacturing information table.

Location: `calibrate_support.h:224`

Parameters

Direction	Name	Description
in	<i>target</i>	The specified target voltage [10 mV]
in	<i>index</i>	The index in the manufacturing table where the variable will be stored.

Return

Values other than `ERRNO_NO_ERROR` (0) signify errors.

Example Code for Calibrate_Support_UpdateVDDOD

```
// Load the calibrated setting from the trim register into the local RAM
// instance of the manufacturing information table
result = Calibrate\_Support\_UpdateVDDOD(90, 0);
```

Assumptions

Calibration has already run

23.4.30 Calibrate_Support_UpdateVMIC

```
int Calibrate_Support_UpdateVMIC(uint8_t target, uint8_t index)
```

Load the calibrated setting from the trim register into the local RAM instance of the manufacturing information table.

Location: calibrate_support.h:236

Parameters

Direction	Name	Description
in	<i>target</i>	The specified target voltage [10 mV]
in	<i>index</i>	The index in the manufacturing table where the variable will be stored.

Return

Values other than ERRNO_NO_ERROR (0) signify errors.

Example Code for Calibrate_Support_UpdateVMIC

```
// Load the calibrated setting from the trim register into the local RAM
// instance of the manufacturing information table
result = Calibrate\_Support\_UpdateVMIC(80, 0);
```

Assumptions

Calibration has already run

23.4.31 Calibrate_Support_UpdateVREG

```
void Calibrate_Support_UpdateVREG()
```

Load the calibrated setting from the trim register into the local RAM instance of the manufacturing information table.

Location: `calibrate_support.h:245`

Example Code for Calibrate_Support_UpdateVREG

```
// Load the calibrated setting from the trim register into the local RAM
// instance of the manufacturing information table
Calibrate_Support_UpdateVREG();
```

Assumptions

Calibration has already run Always calibrated to 900 mV

23.4.32 Calibrate_Support_UpdateCLK

```
int Calibrate_Support_UpdateCLK(uint8_t target, uint8_t index)
```

Load the calibrated setting from the trim register into the local RAM instance of the manufacturing information table.

Location: `calibrate_support.h:258`

Parameters

Direction	Name	Description
in	<i>target</i>	The specified target frequency. Use symbols <code>SYS_FREQ_*</code> defined in <code>sk5_sys_calib.h</code>
in	<i>index</i>	The index in the manufacturing table where the variable will be stored.

Return

Values other than ERRNO_NO_ERROR (0) signify errors.

Example Code for Calibrate_Support_UpdateCLK

```
// Load the calibrated setting from the trim register into the local RAM
// instance of the manufacturing information table
result = Calibrate\_Support\_UpdateCLK(SYS_FREQ_7M68, 0);
```

Assumptions

Calibration has already run

23.4.33 Calibrate_Support_UpdateLSAD_GAIN

```
void Calibrate_Support_UpdateLSAD_GAIN()
```

Load the calibrated setting from the trim register into the local RAM instance of the manufacturing information table.

Location: calibrate_support.h:266

Example Code for Calibrate_Support_UpdateLSAD_GAIN

```
// Load the calibrated setting from the trim register into the local RAM
// instance of the manufacturing information table
Calibrate\_Support\_UpdateLSAD\_GAIN();
```

Assumptions

Calibration has already run

23.4.34 Calibrate_Support_UpdateLSAD_OFFSET

```
void Calibrate_Support_UpdateLSAD_OFFSET()
```

Load the calibrated setting from the trim register into the local RAM instance of the manufacturing information table.

Location: `calibrate_support.h:274`

Example Code for Calibrate_Support_UpdateLSAD_OFFSET

```
// Load the calibrated setting from the trim register into the local RAM
// instance of the manufacturing information table
Calibrate_Support_UpdateLSAD_OFFSET();
```

Assumptions

Calibration has already run

23.4.35 Calibrate_Support_UpdateADC_OFFSET

```
int Calibrate_Support_UpdateADC_OFFSET(uint8_t channel_num)
```

Load the calibrated offsets from the trim registers into the local RAM instance of the manufacturing information table.

Location: `calibrate_support.h:285`

Parameters

Direction	Name	Description
in	<i>channel_num</i>	ADC channel number

Return

Values other than `ERRNO_NO_ERROR` (0) signify errors.

Assumptions

Calibration has already run

Example Code for Calibrate_Support_UpdateADC_OFFSET

```
// Load the calibrated setting from the trim register into the local RAM  
// instance of the manufacturing information table  
result = Calibrate\_Support\_UpdateADC\_OFFSET(0);
```

23.4.36 Calibrate_Support_Delay_Cycles

```
void Calibrate_Support_Delay_Cycles(unsigned int delay_cycles)
```

Delay for the specified number of cycles.

Exact delay may change depending on which core this function is run.

Location: calibrate_support.h:292

Parameters

Direction	Name	Description
in	<i>delay_cycles</i>	Number of cycles to delay.

CHAPTER 24

Arm Cortex-M3 Core Bootloader Reference

Arm Cortex-M3 Core Bootloader.

24.1 SUMMARY

Data Structures

- [application file header](#) : Header for an Ezairo application file.
- [data section header](#) : Defines the header for a block of data in the file.
- [boot section data](#) : Data layout for boot sections (SEC_BOOT_CFX or SEC_BOOT_CM3).

Macros

- [APPFILE IDENT APP](#) : Application file (not encrypted),
- [APPFILE IDENT APPENCRYPTED](#) : Application file (encrypted),
- [APPFILE IDENT MANIFEST](#) : Manifest file (not encrypted),
- [APPFILE IDENT MANIFESTENCRYPTED](#) : Manifest file (encrypted)
- [SEC CFX X](#) : Data destination is CFX X memory
- [SEC CFX Y](#) : Data destination is CFX Y memory
- [SEC CFX P](#) : Data destination is CFX P memory
- [SEC CM3 P](#) : Data destination is CM3 P memory
- [SEC IOMEM](#) : Data destination is IO memory.
- [SEC BOOT CFX](#) : CFX core boot section
- [SEC BOOT CM3](#) : CM3 core boot section
- [APPDATA FLAGS CLEARTEXT](#) : The following table shows the permissible range for the section types defined above.
- [APPDATA FLAGS ENCRYPTED](#) : Section data is encrypted with the Application Key
- [APPDATA FLAGS CRC](#) : CRC will be verified for the section
- [APPDATA FLAGS VOLATILE](#) : CRC will not be verified (volatile section)
- [APPDATA FLAGS UNLOCKED](#) : Do not lock the PRAM segment.
- [APPDATA FLAGS LOCKED](#) : PRAM segment will be locked if it is PRAM or PRAM1.
- [APPDATA FLAGS EMPTY](#) : This data section carries no data, it will write `size tooffset`.
- [MANIFEST MAXIMUM SIZE](#) : Maximum size of a manifest file.
- [APP HEADER MAXIMUM SIZE](#) : Maximum size of the [application file header](#).

Functions

- [bootloader load manu table](#) : Reads the manufacturing table from the NVM into RAM and then loads it to a known location in PRAM.
- [bootloader load app file](#) : Loads and processes an application file.
- [bootloader boot](#) : Loads the application file specified in the NVM image Boot Information sector.
- [bootloader main](#) : Loads the application file specified in the NVM image Boot Information sector.

Ezairo 8300 Firmware Reference

24.2 ARM CORTEX-M3 CORE BOOTLOADER REFERENCE DATA STRUCTURES TYPE DOCUMENTATION

24.2.1 application_file_header

Location: bootloader_application_file.h:109

Header for an Ezairo application file.

For encrypted files, the encryption applies to the whole file with the exception of the ident field in the file header. This provides some level of authentication as the CRC fields will also be encrypted. The crc_table member is followed by a CCITT CRC field stored in big-endian order. The CRC field is calculated over the whole structure. If the number of entries in the CRC tables is odd, there is a uint16_t word with zero as a padding between the last entry of crc_table and the CRC field.

Data Fields

Type	Name	Description
uint32_t	<i>ident</i>	Magic number indicating an Ezairo application file: 'ezai': Application file (not encrypted), 'EZAI': Application file (encrypted), 'ezam': Manifest file (not encrypted), 'EZAM': Manifest file (encrypted)
uint16_t	<i>crc_table_size</i>	The number of entries in the crc_table.
uint16_t	<i>crc_table[]</i>	The CCITT CRC for the data sections that are part of this file. If the number of sections is odd, the last entry is a padding value.

24.2.2 data_section_header

Location: bootloader_application_file.h:126

Defines the header for a block of data in the file.

Ezairo 8300 Firmware Reference

NOTE: The whole structure is 16 bytes long (an AES block).

Data Fields

Type	Name	Description
uint32_t	<i>offset</i>	Offset (in words) from the beginning of the memory instance specified by the type field.
uint32_t	<i>size</i>	Section data size. The size only counts the number of words in the data, it does not count the header size.
uint16_t	<i>type</i>	Section type. One of the following: • SEC_CFX_X • SEC_CFX_Y • SEC_CFX_P • SEC_CM3_P • SEC_IOMEM • SEC_BOOT_CFX • SEC_BOOT_CM3
uint16_t	<i>flags</i>	A bitfield for flags: • Bit 7: encrypted section • 0 : Section data is not encrypted • 1 : Section data and CRC are encrypted with the Application Key. • Bit 6: CRC check • 0: CRC will be verified for the section • 1: CRC will not be verified (volatile section) • Bit 5: Lock PRAM. Valid only if type is SEC_CFX_P or SEC_CM3_P. • 0: Do not lock the PRAM segment. • 1: PRAM segment will be locked if it is PRAM or PRAM1.
uint8_t	<i>aes_pad_count</i>	Number of AES padding bytes if the section is encrypted.

Ezairo 8300 Firmware Reference

uint8_t	<i>_reserved</i>	Zero-filled area.
uint16_t	<i>header_crc</i>	The CCITT CRC for the header section. If the encryption flag is set, it should be encrypted with the same key as the data section. This is stored in big-endian order.
uint32_t	<i>data[]</i>	The section data

24.2.3 boot_section_data

Location: `bootloader_application_file.h:166`

Data layout for boot sections (SEC_BOOT_CFX or SEC_BOOT_CM3).

NOTE: If this data is packet into an encrypted section, the data must be padded so it's a multiple of an AES block.

Data Fields

Type	Name	Description
uint32_t	<i>stack_pointer</i>	Stack pointer absolute address in the core's address space.
uint32_t	<i>init_pointer</i>	Address the control will jump to

24.3 ARM CORTEX-M3 CORE BOOTLOADER REFERENCE MACRO DEFINITION DOCUMENTATION**24.3.1 APPFILE_IDENT_APP**

```
#define APPFILE_IDENT_APP 0x657A6169
```

Application file (not encrypted),

Location: `bootloader_application_file.h:46`

24.3.2 APPFILE_IDENT_APPENCRYPTED

```
#define APPFILE_IDENT_APPENCRYPTED 0x455A4149
```

Application file (encrypted),

Location: bootloader_application_file.h:47

24.3.3 APPFILE_IDENT_MANIFEST

```
#define APPFILE_IDENT_MANIFEST 0x657A616D
```

Manifest file (not encrypted),

Location: bootloader_application_file.h:48

24.3.4 APPFILE_IDENT_MANIFESTENCRYPTED

```
#define APPFILE_IDENT_MANIFESTENCRYPTED 0x455A414D
```

Manifest file (encrypted)

Location: bootloader_application_file.h:49

24.3.5 SEC_CFX_X

```
#define SEC_CFX_X 0x01
```

Data destination is CFX X memory

Location: bootloader_application_file.h:52

24.3.6 SEC_CFX_Y

```
#define SEC_CFX_Y 0x02
```

Data destination is CFX Y memory

Location: bootloader_application_file.h:53

24.3.7 SEC_CFX_P

```
#define SEC_CFX_P 0x03
```

Data destination is CFX P memory

Location: bootloader_application_file.h:54

24.3.8 SEC_CM3_P

```
#define SEC_CM3_P 0x04
```

Data destination is CM3 P memory

Location: bootloader_application_file.h:55

24.3.9 SEC_IOMEM

```
#define SEC_IOMEM 0x05
```

Data destination is IO memory.

No boundary checks are performed here, use it at your own risk.

Location: bootloader_application_file.h:58

24.3.10 SEC_BOOT_CFX

```
#define SEC_BOOT_CFX 0x08
```

CFX core boot section

Location: bootloader_application_file.h:59

24.3.11 SEC_BOOT_CM3

```
#define SEC_BOOT_CM3 0x09
```

CM3 core boot section

Location: bootloader_application_file.h:60

24.3.12 APPDATA_FLAGS_CLEARTEXT

```
#define APPDATA_FLAGS_CLEARTEXT 0x00
```

The following table shows the permissible range for the section types defined above.

The sum of the data offset and the data size must fit within the range.

Location: bootloader_application_file.h:77

Section	Start Address	End Address
SEC_CFX_X	CFX_XRAM0_LSB_UNSIGNED_BASE	CFX_XRAM3_LSB_UNSIGNED_TOP
SEC_CFX_Y	CFX_YRAM0_LSB_UNSIGNED_BASE	CFX_YRAM2_LSB_UNSIGNED_TOP
SEC_CFX_XY	-	-
SEC_CFX_P	CFX_PRAM_BASE	CFX_PRAM_TOP
SEC_CM3_P	PRAM_BASE	PRAM_TOP
SEC_IOMEM	0	0xFFFFFFFF

24.3.13 APPDATA_FLAGS_ENCRYPTED

```
#define APPDATA_FLAGS_ENCRYPTED 0x80
```

Section data is encrypted with the Application Key

Location: bootloader_application_file.h:79

24.3.14 APPDATA_FLAGS_CRC

```
#define APPDATA_FLAGS_CRC 0x00
```

CRC will be verified for the section

Location: bootloader_application_file.h:80

24.3.15 APPDATA_FLAGS_VOLATILE

```
#define APPDATA_FLAGS_VOLATILE 0x40
```

CRC will not be verified (volatile section)

Location: bootloader_application_file.h:81

24.3.16 APPDATA_FLAGS_UNLOCKED

```
#define APPDATA_FLAGS_UNLOCKED 0x00
```

Do not lock the PRAM segment.

Location: bootloader_application_file.h:83

24.3.17 APPDATA_FLAGS_LOCKED

```
#define APPDATA_FLAGS_LOCKED 0x20
```

PRAM segment will be locked if it is PRAM or PRAM1.

Location: bootloader_application_file.h:85

24.3.18 APPDATA_FLAGS_EMPTY

```
#define APPDATA_FLAGS_EMPTY 0x10
```

This data section carries no data, it will write size tooffset.

Location: bootloader_application_file.h:88

24.3.19 MANIFEST_MAXIMUM_SIZE

```
#define MANIFEST_MAXIMUM_SIZE 512
```

Maximum size of a manifest file.

In bytes.

Location: bootloader_application_file.h:90

24.3.20 APP_HEADER_MAXIMUM_SIZE

```
#define APP_HEADER_MAXIMUM_SIZE 512
```

Maximum size of the [application file header](#).

In bytes.

Location: bootloader_application_file.h:92

24.4 ARM CORTEX-M3 CORE BOOTLOADER REFERENCE FUNCTION DOCUMENTATION

24.4.1 bootloader_load_manu_table

```
int bootloader_load_manu_table(uint16_t boot_manu_off)
```

Reads the manufacturing table from the NVM into RAM and then loads it to a known location in PRAM.

Tries to set the default trim values if found in the table. Finds and sets the fastest system clock trim value that can be used.

Location: bootloader.h:63

Parameters

Direction	Name	Description
in	<i>boot_manu_off</i>	The manufacturing table offset to use when reading the table from the NVM.

Return

result -1 on error and 0 if the table was loaded successfully.

24.4.2 bootloader_load_app_file

```
int bootloader_load_app_file(const char * file_name)
```

Loads and processes an application file.

Location: bootloader.h:71

Parameters

Direction	Name	Description
in	<i>file_name</i>	The name of the application file

Return

0 if successful, less than 0 otherwise.

Example Code for bootloader_load_app_file

```
// Boots the "APP0001.fez" image file
ret = bootloader\_load\_app\_file("APP0001.fez");
```

24.4.3 bootloader_boot

```
int bootloader_boot()
```

Loads the application file specified in the NVM image Boot Information sector.

It might return if the Arm Cortex-M3 core is not booted.

Location: bootloader.h:79

Return

0 if successful, less than 0 otherwise.

Example Code for bootloader_boot

```
// Boots the application image file  
ret = bootloader\_boot();
```

24.4.4 bootloader_main

```
void bootloader_main()
```

Loads the application file specified in the NVM image Boot Information sector.

Does not return.

Location: bootloader.h:86

Example Code for bootloader_main

```
// Boots the application image file  
ret = bootloader\_main();
```

CHAPTER 25

CFX DSP Library Reference

CFX DSP library fixed-point data types and functions reference.

25.1 SUMMARY

25.2 DETAILED DESCRIPTION

This library is provided to aid in the development of fixed-point math and signal processing using the C language on the CFX DSP. Specifically, it allows for:

- The declaration and initialization of common fixed-point data types including [frac24_t](#), [frac48_t](#), [m9p47_t](#), [m8p16_t](#), [m8p40_t](#), and [m16p40_t](#).
- Conversion between fixed-point data types with appropriate rounding and saturation where the binary point is preserved using functions like [cfx_to_frac24\(\)](#).
- The use of common mathematical and relational operators that handle fixed-point data types.
- Basic mathematical functions like [cfx_min\(\)](#), [cfx_max\(\)](#), and [cfx_abs\(\)](#).
- Common data conversion functions like [cfx_mag_to_dB\(\)](#) and [cfx_dB_to_mag\(\)](#).
- Signal processing functions like [cfx_biquad\(\)](#) and [cfx_fir\(\)](#).
- Functions that use the hardware optimized math accelerator like [cfx_log2_m8p16\(\)](#) and [cfx_exp2_frac24\(\)](#);

While the library makes it easier to write and maintain fixed-point code using C, it is still very important to select the appropriate fixed-point data types, and be aware of the tradeoffs between bit precision and cycle efficiency. For example, while the multiplication between a number of 24 bit and 48 bit data types is supported, it is less efficient than multiplication between 24 bit data types. A number of recommendations are made throughout this document to assist in developing an understanding of these tradeoffs.

25.3 BASIC MATH

CFX DSP library functions for basic math.

25.3.1 Summary

Functions

- [cfx_abs](#) : Calculate the absolute value.
- [cfx_abs](#) : Calculate the absolute value.
- [cfx_abs](#) : Calculate the absolute value.
- [cfx_abs](#) : Calculate the absolute value.
- [cfx_abs](#) : Calculate the absolute value.
- [cfx_abs](#) : Calculate the absolute value.
- [cfx_abs](#) : Calculate the absolute value.
- [cfx_min](#) : Calculate the minimum of two values.
- [cfx_min](#) : Calculate the minimum of two values.
- [cfx_min](#) : Calculate the minimum of two values.
- [cfx_min](#) : Calculate the minimum of two values.
- [cfx_min](#) : Calculate the minimum of two values.

Ezairo 8300 Firmware Reference

- [cfx_min](#) : Calculate the minimum of two values.
- [cfx_max](#) : Calculate the maximum of two values.
- [cfx_max](#) : Calculate the maximum of two values.
- [cfx_max](#) : Calculate the maximum of two values.
- [cfx_max](#) : Calculate the maximum of two values.
- [cfx_max](#) : Calculate the maximum of two values.
- [cfx_max](#) : Calculate the maximum of two values.

25.3.2 Basic Math Function Documentation

25.3.2.1 cfx_abs

[frac24 t](#) cfx_abs([frac24 t](#) a)

Calculate the absolute value.

Location: sk5_cfx_dsp_basic_math.h:35

Parameters

Direction	Name	Description
in	<i>a</i>	Input in frac24 t format

Return

Absolute value in [frac24 t](#) format

25.3.2.2 cfx_abs

[frac48 t](#) cfx_abs([frac48 t](#) a)

Calculate the absolute value.

Location: sk5_cfx_dsp_basic_math.h:47

Parameters

Direction	Name	Description
in	<i>a</i>	Input in frac48_t format

Return

Absolute value in [frac48_t](#) format

25.3.2.3 cfx_abs

[m9p47_t](#) cfx_abs([m9p47_t](#) a)

Calculate the absolute value.

Location: sk5_cfx_dsp_basic_math.h:59

Parameters

Direction	Name	Description
in	<i>a</i>	Input in m9p47_t format

Return

Absolute value in [m9p47_t](#) format

25.3.2.4 cfx_abs

[m8p16_t](#) cfx_abs([m8p16_t](#) a)

Calculate the absolute value.

Location: sk5_cfx_dsp_basic_math.h:71

Parameters

Direction	Name	Description
in	<i>a</i>	Input in m8p16 t format

Return

Absolute value in [m8p16 t](#) format

25.3.2.5 cfx_abs

[m8p40 t](#) cfx_abs ([m8p40 t](#) a)

Calculate the absolute value.

Location: sk5_cfx_dsp_basic_math.h:83

Parameters

Direction	Name	Description
in	<i>a</i>	Input in m8p40 t format

Return

Output in [m8p40 t](#) format

25.3.2.6 cfx_abs

[m16p40 t](#) cfx_abs ([m16p40 t](#) a)

Calculate the absolute value.

Location: sk5_cfx_dsp_basic_math.h:95

Parameters

Direction	Name	Description
in	<i>a</i>	Input in m16p40 t format

Return

Absolute value in [m16p40 t](#) format

25.3.2.7 cfx_min

[frac24 t](#) cfx_min([frac24 t](#) a, [frac24 t](#) b)

Calculate the minimum of two values.

Location: sk5_cfx_dsp_basic_math.h:108

Parameters

Direction	Name	Description
in	<i>a</i>	First value in frac24 t format
in	<i>b</i>	Second value in frac24 t format

Return

Minimum value in [frac24 t](#) format

25.3.2.8 cfx_min

[frac48 t](#) cfx_min([frac48 t](#) a, [frac48 t](#) b)

Calculate the minimum of two values.

Location: sk5_cfx_dsp_basic_math.h:121

Parameters

Direction	Name	Description
in	<i>a</i>	First value in frac48_t format
in	<i>b</i>	Second value in frac48_t format

Return

Minimum value in [frac48_t](#) format

25.3.2.9 cfx_min

[m9p47_t](#) cfx_min([m9p47_t](#) a, [m9p47_t](#) b)

Calculate the minimum of two values.

Location: sk5_cfx_dsp_basic_math.h:134

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m9p47_t format
in	<i>b</i>	Second value in m9p47_t format

Return

Minimum value in [m9p47_t](#) format

25.3.2.10 cfx_min

[m8p16_t](#) cfx_min([m8p16_t](#) a, [m8p16_t](#) b)

Calculate the minimum of two values.

Location: sk5_cfx_dsp_basic_math.h:147

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m8p16_t format
in	<i>b</i>	Second value in m8p16_t format

Return

Minimum value in [m8p16_t](#) format

25.3.2.11 cfx_min

[m8p40_t](#) cfx_min([m8p40_t](#) a, [m8p40_t](#) b)

Calculate the minimum of two values.

Location: sk5_cfx_dsp_basic_math.h:160

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m8p40_t format
in	<i>b</i>	Second value in m8p40_t format

Return

Minimum value in [m8p40_t](#) format

25.3.2.12 cfx_min

[m16p40_t](#) cfx_min([m16p40_t](#) a, [m16p40_t](#) b)

Calculate the minimum of two values.

Location: sk5_cfx_dsp_basic_math.h:173

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m16p40 t format
in	<i>b</i>	Second value in m16p40 t format

Return

Minimum value in [m16p40 t](#) format

25.3.2.13 cfx_max

[frac24 t](#) cfx_max([frac24 t](#) a, [frac24 t](#) b)

Calculate the maximum of two values.

Location: sk5_cfx_dsp_basic_math.h:186

Parameters

Direction	Name	Description
in	<i>a</i>	First value in frac24 t format
in	<i>b</i>	Second value in frac24 t format

Return

Maximum value in [frac24 t](#) format

25.3.2.14 cfx_max

[frac48_t](#) cfx_max([frac48_t](#) a, [frac48_t](#) b)

Calculate the maximum of two values.

Location: sk5_cfx_dsp_basic_math.h:199

Parameters

Direction	Name	Description
in	<i>a</i>	First value in frac48_t format
in	<i>b</i>	Second value in frac48_t format

Return

Maximum value in [frac48_t](#) format

25.3.2.15 cfx_max

[m9p47_t](#) cfx_max([m9p47_t](#) a, [m9p47_t](#) b)

Calculate the maximum of two values.

Location: sk5_cfx_dsp_basic_math.h:212

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m9p47_t format
in	<i>b</i>	Second value in m9p47_t format

Return

Maximum value in [m9p47 t](#) format

25.3.2.16 cfx_max

[m8p16 t](#) cfx_max([m8p16 t](#) a, [m8p16 t](#) b)

Calculate the maximum of two values.

Location: sk5_cfx_dsp_basic_math.h:225

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m8p16 t format
in	<i>b</i>	Second value in m8p16 t format

Return

Maximum value in [m8p16 t](#) format

25.3.2.17 cfx_max

[m8p40 t](#) cfx_max([m8p40 t](#) a, [m8p40 t](#) b)

Calculate the maximum of two values.

Location: sk5_cfx_dsp_basic_math.h:238

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m8p40 t format
in	<i>b</i>	Second value in m8p40 t format

Return

Maximum value in [m8p40 t](#) format

25.3.2.18 cfx_max

[m16p40 t](#) cfx_max([m16p40 t](#) a, [m16p40 t](#) b)

Calculate the maximum of two values.

Location: sk5_cfx_dsp_basic_math.h:251

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m16p40 t format
in	<i>b</i>	Second value in m16p40 t format

Return

Maximum value in [m16p40 t](#) format

25.4 DATA CONVERSION

CFX DSP library data conversion functions.

25.4.1 Summary

Functions

- [cfx_mag to dB](#) : Convert magnitude to dB.
- [cfx_pwr to dB](#) : Convert power to dB.
- [cfx_dB to mag](#) : Convert dB to magnitude.
- [cfx_dB to pwr](#) : Convert dB to power.
- [cfx_mag to dB vec](#) : Convert vector of magnitude values to dB.
- [cfx_pwr to dB vec](#) : Convert vector of power values to dB.

- [cfx dB to mag vec](#) : Convert vector of dB values to magnitude.
- [cfx dB to pwr vec](#) : Convert vector of dB values to power.

25.4.2 Data Conversion Function Documentation

25.4.2.1 cfx_mag_to_dB

[m8p16 t](#) cfx_mag_to_dB([frac48 t](#) mag)

Convert magnitude to dB.

Calculated as $20 \cdot \log_{10}(\text{abs}(\text{mag}))$.

Location: sk5_cfx_dsp_data_conversions.h:103

Parameters

Direction	Name	Description
in	<i>mag</i>	Input magnitude in frac48 t format

Return

Converted dB value in [m8p16 t](#) format

25.4.2.2 cfx_pwr_to_dB

[m8p16 t](#) cfx_pwr_to_dB([frac48 t](#) pwr)

Convert power to dB.

Calculated as $10 \cdot \log_{10}(\text{abs}(\text{pwr}))$.

Location: sk5_cfx_dsp_data_conversions.h:116

Parameters

Direction	Name	Description
in	<i>pwr</i>	Input power in frac48 t format

Return

Converted dB value in [m8p16 t](#) format

25.4.2.3 cfx_dB_to_mag

[frac48 t](#) cfx_dB_to_mag ([m8p16 t](#) dB)

Convert dB to magnitude.

Calculated as 10

Location: sk5_cfx_dsp_data_conversions.h:129

Parameters

Direction	Name	Description
in	<i>dB</i>	Input dB value in m8p16 t format

Return

Converted magnitude in [frac48 t](#) format

(dB/20).

25.4.2.4 cfx_dB_to_pwr

[frac48 t](#) cfx_dB_to_pwr ([m8p16 t](#) dB)

Convert dB to power.

Calculated as 10

Location: sk5_cfx_dsp_data_conversions.h:142

Parameters

Direction	Name	Description
in	<i>dB</i>	Input dB value in m8p16_t format

Return

Converted power in [frac48_t](#) format

(dB/10).

25.4.2.5 cfx_mag_to_dB_vec

```
void cfx_mag_to_dB_vec(frac48\_t _XYMEM * mag, m8p16\_t * dB, int size)
```

Convert vector of magnitude values to dB.

Calculated as $20 \cdot \log_{10}(\text{abs}(\text{mag}))$.

Location: sk5_cfx_dsp_data_conversions.h:156

Parameters

Direction	Name	Description
	<i>mag</i>	
	<i>dB</i>	
in	<i>size</i>	Vector length

25.4.2.6 cfx_pwr_to_dB_vec

```
void cfx_pwr_to_dB_vec(frac48 t _XYMEM * pwr, m8p16 t * dB, int size)
```

Convert vector of power values to dB.

Calculated as $10 \cdot \log_{10}(\text{abs}(\text{pwr}))$.

Location: sk5_cfx_dsp_data_conversions.h:171

Parameters

Direction	Name	Description
	<i>pwr</i>	
	<i>dB</i>	
in	<i>size</i>	Vector length

25.4.2.7 cfx_dB_to_mag_vec

```
void cfx_dB_to_mag_vec(m8p16 t * dB, frac48 t _XYMEM * mag, int size)
```

Convert vector of dB values to magnitude.

Calculated as 10

Location: sk5_cfx_dsp_data_conversions.h:186

Parameters

Direction	Name	Description
	<i>dB</i>	
	<i>mag</i>	
in	<i>size</i>	Vector length

(dB/20).

25.4.2.8 cfx_dB_to_pwr_vec

```
void cfx_dB_to_pwr_vec(m8p16\_t * dB, frac48\_t _XYMEM * pwr, int size)
```

Convert vector of dB values to power.

Calculated as 10

Location: sk5_cfx_dsp_data_conversions.h:201

Parameters

Direction	Name	Description
	<i>dB</i>	
	<i>pwr</i>	
in	<i>size</i>	Vector length

(dB/10).

25.5 MATH ACCELERATOR FUNCTIONS

CFX DSP library functions that use the hardware math accelerator.

25.5.1 Summary

25.5.2 Detailed Description

The math accelerator functions call dedicated hardware math accelerator blocks to provide fast run-time calculations. Some function variants take 48-bit vector inputs and outputs, which expect data to stored in _XYMEM to allow for faster run-time performance.

25.6 LOG2 FUNCTIONS

CFX DSP library log2 functions that use the hardware math accelerator.

25.6.1 Summary

Functions

- [cfx_log2_m8p16](#) : Calculate log2().
- [cfx_log2_m8p16](#) : Calculate log2().
- [cfx_log2_m8p40](#) : Calculate log2().
- [cfx_log2_m8p40](#) : Calculate log2().
- [cfx_log2_vec_m8p16](#) : Calculate log2() of each value in a vector.
- [cfx_log2_vec_m8p16](#) : Calculate log2() of each value in a vector.
- [cfx_log2_vec_m8p40](#) : Calculate log2() of each value in a vector.
- [cfx_log2_vec_m8p40](#) : Calculate log2() of each value in a vector.

25.6.2 Detailed Description

Log2 inputs are expected to be positive fractional values. Negative inputs are forced to 0 before calculation. Output precision is limited to m8p24 where the m8 portion holds 2 equal negative sign bits. Therefore, the smallest output is -64.

25.6.3 Log2 Functions Function Documentation

25.6.3.1 cfx_log2_m8p16

[m8p16 t](#) cfx_log2_m8p16([frac24 t](#) a)

Calculate log2().

Location: sk5_cfx_dsp_math_accelerator.h:318

Parameters

Direction	Name	Description
in	<i>a</i>	Linear value in frac24 t format

Return

Calculated log2 value in [m8p16 t](#) format

25.6.3.2 cfx_log2_m8p16

[m8p16 t](#) cfx_log2_m8p16([frac48 t](#) a)

Calculate log2().

Location: sk5_cfx_dsp_math_accelerator.h:330

Parameters

Direction	Name	Description
in	<i>a</i>	Linear value in frac48 t format

Return

Calculated log2 value in [m8p16 t](#) format

25.6.3.3 cfx_log2_m8p40

[m8p40 t](#) cfx_log2_m8p40([frac24 t](#) a)

Calculate log2().

Location: sk5_cfx_dsp_math_accelerator.h:342

Parameters

Direction	Name	Description
in	<i>a</i>	Linear value in frac24 t format

Return

Calculated log2 value in [m8p40 t](#) format

25.6.3.4 cfx_log2_m8p40

[m8p40 t](#) cfx_log2_m8p40([frac48 t](#) a)

Calculate log2().

Location: sk5_cfx_dsp_math_accelerator.h:354

Parameters

Direction	Name	Description
in	<i>a</i>	Linear value in frac48 t format

Return

Calculated log2 value in [m8p40 t](#) format

25.6.3.5 cfx_log2_vec_m8p16

void cfx_log2_vec_m8p16([frac24 t](#) * in, [m8p16 t](#) * out, int size)

Calculate log2() of each value in a vector.

See cfx_log2_frac24_m8p16.

Location: sk5_cfx_dsp_math_accelerator.h:369

Parameters

Direction	Name	Description
	<i>in</i>	
	<i>out</i>	
in	<i>size</i>	Number of values in the vector

25.6.3.6 cfx_log2_vec_m8p16

```
void cfx_log2_vec_m8p16(frac48\_t _XYMEM * in, m8p16\_t * out, int size)
```

Calculate log2() of each value in a vector.

See cfx_log2_frac48_m8p16.

Location: sk5_cfx_dsp_math_accelerator.h:379

Parameters

Direction	Name	Description
	<i>in</i>	
	<i>out</i>	
in	<i>size</i>	Number of values in the vector

25.6.3.7 cfx_log2_vec_m8p40

```
void cfx_log2_vec_m8p40(frac24\_t * in, m8p40\_t _XYMEM * out, int size)
```

Calculate log2() of each value in a vector.

See cfx_log2_frac24_m8p40.

Location: sk5_cfx_dsp_math_accelerator.h:389

Parameters

Direction	Name	Description
	<i>in</i>	
	<i>out</i>	
in	<i>size</i>	Number of values in the vector

25.6.3.8 cfx_log2_vec_m8p40

```
void cfx_log2_vec_m8p40(frac48\_t _XYMEM * in, m8p40\_t _XYMEM * out, int size)
```

Calculate log2() of each value in a vector.

See cfx_log2_frac48_m8p40.

Location: sk5_cfx_dsp_math_accelerator.h:399

Parameters

Direction	Name	Description
	<i>in</i>	
	<i>out</i>	
in	<i>size</i>	Number of values in the vector

25.7 EXP2 FUNCTIONS

CFX DSP library exp2 functions that use the hardware math accelerator.

25.7.1 Summary

Functions

- [cfx_exp2_frac24](#) : Calculate 2^x .
- [cfx_exp2_frac48](#) : Calculate 2^x .
- [cfx_exp2_frac24](#) : Calculate 2^x .
- [cfx_exp2_frac48](#) : Calculate 2^x .
- [cfx_exp2_vec_frac24](#) : Calculate 2^x of each value in a vector.
- [cfx_exp2_vec_frac48](#) : Calculate 2^x of each value in a vector.
- [cfx_exp2_vec_frac24](#) : Calculate 2^x of each value in a vector.
- [cfx_exp2_vec_frac48](#) : Calculate 2^x of each value in a vector.

25.7.2 Detailed Description

Log2 inputs are expected to be negative values. Positive inputs are forced to 0 before calculation. Input precision is limited to m8p24.

25.7.3 Exp2 Functions Function Documentation

25.7.3.1 cfx_exp2_frac24

[frac24 t](#) cfx_exp2_frac24([m8p16 t](#) a)

Calculate 2^x .

Location: sk5_cfx_dsp_math_accelerator.h:419

Parameters

Direction	Name	Description
in	<i>a</i>	Log2 value in m8p16 t format

Return

Calculated linear value in [frac24 t](#) format

25.7.3.2 cfx_exp2_frac48

[frac48 t](#) cfx_exp2_frac48([m8p16 t](#) a)

Calculate 2^x .

Location: sk5_cfx_dsp_math_accelerator.h:431

Parameters

Direction	Name	Description
in	<i>a</i>	Log2 value in m8p16 t format

Return

Calculated linear value in [frac48_t](#) format

25.7.3.3 cfx_exp2_frac24

[frac24_t](#) cfx_exp2_frac24([m8p40_t](#) a)

Calculate 2^x .

Location: sk5_cfx_dsp_math_accelerator.h:443

Parameters

Direction	Name	Description
in	<i>a</i>	Log2 value in m8p40_t format

Return

Calculated linear value in [frac24_t](#) format

25.7.3.4 cfx_exp2_frac48

[frac48_t](#) cfx_exp2_frac48([m8p40_t](#) a)

Calculate 2^x .

Location: sk5_cfx_dsp_math_accelerator.h:455

Parameters

Direction	Name	Description
in	<i>a</i>	Log2 value in m8p40_t format

Return

Calculated linear value in [frac48 t](#) format

25.7.3.5 cfx_exp2_vec_frac24

```
void cfx_exp2_vec_frac24(m8p16 t * in, frac24 t * out, int size)
```

Calculate 2^x of each value in a vector.

See cfx_exp2_m8p16_frac24.

Location: sk5_cfx_dsp_math_accelerator.h:470

Parameters

Direction	Name	Description
	<i>in</i>	
	<i>out</i>	
<i>in</i>	<i>size</i>	Number of values in the vector

25.7.3.6 cfx_exp2_vec_frac48

```
void cfx_exp2_vec_frac48(m8p16 t * in, frac48 t _XYMEM * out, int size)
```

Calculate 2^x of each value in a vector.

See cfx_exp2_m8p16_frac48.

Location: sk5_cfx_dsp_math_accelerator.h:480

Parameters

Direction	Name	Description
	<i>in</i>	
	<i>out</i>	
in	<i>size</i>	Number of values in the vector

25.7.3.7 cfx_exp2_vec_frac24

```
void cfx_exp2_vec_frac24(m8p40 t _XYMEM * in, frac24 t * out, int size)
```

Calculate 2^x of each value in a vector.

See cfx_exp2_m8p40_frac24.

Location: sk5_cfx_dsp_math_accelerator.h:490

Parameters

Direction	Name	Description
	<i>in</i>	
	<i>out</i>	
in	<i>size</i>	Number of values in the vector

25.7.3.8 cfx_exp2_vec_frac48

```
void cfx_exp2_vec_frac48(m8p40 t _XYMEM * in, frac48 t _XYMEM * out, int size)
```

Calculate 2^x of each value in a vector.

See cfx_exp2_m8p40_frac48.

Location: sk5_cfx_dsp_math_accelerator.h:500

Parameters

Ezairo 8300 Firmware Reference

Direction	Name	Description
	<i>in</i>	
	<i>out</i>	
in	<i>size</i>	Number of values in the vector

25.8 ROOT FUNCTIONS

CFX DSP library root functions that use the hardware math accelerator.

25.8.1 Summary

Functions

- [cfx_root2_frac24](#) : Calculate the square root.
- [cfx_root2_frac24](#) : Calculate the square root.
- [cfx_root2_frac48](#) : Calculate the square root.
- [cfx_root2_frac48](#) : Calculate the square root.
- [cfx_root3_frac24](#) : Calculate the cubic root.
- [cfx_root3_frac24](#) : Calculate the cubic root.
- [cfx_root3_frac48](#) : Calculate the cubic root.
- [cfx_root3_frac48](#) : Calculate the cubic root.
- [cfx_root4_frac24](#) : Calculate the fourth root.
- [cfx_root4_frac24](#) : Calculate the fourth root.
- [cfx_root4_frac48](#) : Calculate the fourth root.
- [cfx_root2_vec_frac24](#) : Calculate the square root of each value in a vector.
- [cfx_root2_vec_frac24](#) : Calculate the square root of each value in a vector.
- [cfx_root2_vec_frac48](#) : Calculate the square root of each value in a vector.
- [cfx_root2_vec_frac48](#) : Calculate the square root of each value in a vector.
- [cfx_root3_vec_frac24](#) : Calculate the cubic root of each value in a vector.
- [cfx_root3_vec_frac24](#) : Calculate the cubic root of each value in a vector.
- [cfx_root3_vec_frac48](#) : Calculate the cubic root of each value in a vector.
- [cfx_root3_vec_frac48](#) : Calculate the cubic root of each value in a vector.
- [cfx_root4_vec_frac24](#) : Calculate the fourth root of each value in a vector.
- [cfx_root4_vec_frac24](#) : Calculate the fourth root of each value in a vector.
- [cfx_root4_vec_frac48](#) : Calculate the fourth root of each value in a vector.
- [cfx_root4_vec_frac48](#) : Calculate the fourth root of each value in a vector.
- [cfx_root4_frac48](#) : Calculate the fourth root.

25.8.2 Detailed Description

Root2 and Root4 inputs are expected to be positive values. Negative inputs are forced to 0 before calculation.

25.8.3 Root Functions Function Documentation

25.8.3.1 cfx_root2_frac24

[frac24_t](#) cfx_root2_frac24([frac24_t](#) a)

Calculate the square root.

Location: sk5_cfx_dsp_math_accelerator.h:518

Parameters

Direction	Name	Description
in	<i>a</i>	Value in frac24_t format

Return

Root in [frac48_t](#) format

25.8.3.2 cfx_root2_frac24

[frac24_t](#) cfx_root2_frac24([frac48_t](#) a)

Calculate the square root.

Location: sk5_cfx_dsp_math_accelerator.h:530

Parameters

Direction	Name	Description
in	<i>a</i>	Value in frac48_t format

Return

Root in [frac48 t](#) format

25.8.3.3 cfx_root2_frac48

[frac48 t](#) cfx_root2_frac48([frac24 t](#) a)

Calculate the square root.

Location: sk5_cfx_dsp_math_accelerator.h:542

Parameters

Direction	Name	Description
in	<i>a</i>	Value in frac24 t format

Return

Root in [frac48 t](#) format

25.8.3.4 cfx_root2_frac48

[frac48 t](#) cfx_root2_frac48([frac48 t](#) a)

Calculate the square root.

Location: sk5_cfx_dsp_math_accelerator.h:554

Parameters

Direction	Name	Description
in	<i>a</i>	Value in frac48 t format

Return

Root in [frac48 t](#) format

25.8.3.5 cfx_root3_frac24

[frac24 t](#) cfx_root3_frac24([frac24 t](#) a)

Calculate the cubic root.

Location: sk5_cfx_dsp_math_accelerator.h:566

Parameters

Direction	Name	Description
in	<i>a</i>	Value in frac24 t format

Return

Cubic root in [frac24 t](#) format

25.8.3.6 cfx_root3_frac24

[frac24 t](#) cfx_root3_frac24([frac48 t](#) a)

Calculate the cubic root.

Location: sk5_cfx_dsp_math_accelerator.h:578

Parameters

Direction	Name	Description
in	<i>a</i>	Value in frac48 t format

Return

Cubic root in [frac24 t](#) format

25.8.3.7 cfx_root3_frac48

[frac48 t](#) cfx_root3_frac48([frac24 t](#) a)

Calculate the cubic root.

Location: sk5_cfx_dsp_math_accelerator.h:590

Parameters

Direction	Name	Description
in	<i>a</i>	Value in frac24 t format

Return

Cubic root in [frac24 t](#) format

25.8.3.8 cfx_root3_frac48

[frac48 t](#) cfx_root3_frac48([frac48 t](#) a)

Calculate the cubic root.

Location: sk5_cfx_dsp_math_accelerator.h:602

Parameters

Direction	Name	Description
in	<i>a</i>	Value in frac48 t format

Return

Cubic root in [frac24 t](#) format

25.8.3.9 cfx_root4_frac24

[frac24 t](#) cfx_root4_frac24([frac24 t](#) a)

Calculate the fourth root.

Location: sk5_cfx_dsp_math_accelerator.h:614

Parameters

Direction	Name	Description
in	<i>a</i>	Value in frac24 t format

Return

Fourth root in [frac48 t](#) format

25.8.3.10 cfx_root4_frac24

[frac24 t](#) cfx_root4_frac24([frac48 t](#) a)

Calculate the fourth root.

Location: sk5_cfx_dsp_math_accelerator.h:626

Parameters

Direction	Name	Description
in	<i>a</i>	Value in frac48 t format

Return

Fourth root in [frac48 t](#) format

25.8.3.11 cfx_root4_frac48

[frac48 t](#) cfx_root4_frac48([frac24 t](#) a)

Calculate the fourth root.

Location: sk5_cfx_dsp_math_accelerator.h:638

Parameters

Direction	Name	Description
in	<i>a</i>	Value in frac24 t format

Return

Fourth root in [frac48 t](#) format

25.8.3.12 cfx_root2_vec_frac24

void cfx_root2_vec_frac24([frac24 t](#) * in, [frac24 t](#) * out, int size)

Calculate the square root of each value in a vector.

See cfx_root2_frac24_frac24.

Location: sk5_cfx_dsp_math_accelerator.h:653

Parameters

Direction	Name	Description
	<i>in</i>	
	<i>out</i>	
in	<i>size</i>	Number of values in the vector

25.8.3.13 cfx_root2_vec_frac24

```
void cfx_root2_vec_frac24(frac48\_t _XYMEM * in, frac24\_t * out, int size)
```

Calculate the square root of each value in a vector.

See cfx_root2_frac48_frac24.

Location: sk5_cfx_dsp_math_accelerator.h:663

Parameters

Direction	Name	Description
	<i>in</i>	
	<i>out</i>	
in	<i>size</i>	Number of values in the vector

25.8.3.14 cfx_root2_vec_frac48

```
void cfx_root2_vec_frac48(frac24\_t * in, frac48\_t _XYMEM * out, int size)
```

Calculate the square root of each value in a vector.

See cfx_root2_frac24_frac48.

Location: sk5_cfx_dsp_math_accelerator.h:673

Parameters

Direction	Name	Description
	<i>in</i>	
	<i>out</i>	
in	<i>size</i>	Number of values in the vector

25.8.3.15 cfx_root2_vec_frac48

```
void cfx_root2_vec_frac48(frac48\_t _XYMEM * in, frac48\_t _XYMEM * out, int size)
```

Calculate the square root of each value in a vector.

See `cfx_root2_frac48_frac48`.

Location: `sk5_cfx_dsp_math_accelerator.h:683`

Parameters

Direction	Name	Description
	<i>in</i>	
	<i>out</i>	
in	<i>size</i>	Number of values in the vector

25.8.3.16 cfx_root3_vec_frac24

```
void cfx_root3_vec_frac24(frac24\_t * in, frac24\_t * out, int size)
```

Calculate the cubic root of each value in a vector.

See `cfx_root3_frac24_frac24`.

Location: `sk5_cfx_dsp_math_accelerator.h:693`

Parameters

Direction	Name	Description
	<i>in</i>	
	<i>out</i>	
in	<i>size</i>	Number of values in the vector

25.8.3.17 cfx_root3_vec_frac24

```
void cfx_root3_vec_frac24(frac48\_t _XYMEM * in, frac24\_t * out, int size)
```

Calculate the cubic root of each value in a vector.

See cfx_root3_frac48_frac24.

Location: sk5_cfx_dsp_math_accelerator.h:703

Parameters

Direction	Name	Description
	<i>in</i>	
	<i>out</i>	
in	<i>size</i>	Number of values in the vector

25.8.3.18 cfx_root3_vec_frac48

```
void cfx_root3_vec_frac48(frac24\_t * in, frac48\_t _XYMEM * out, int size)
```

Calculate the cubic root of each value in a vector.

See cfx_root3_frac24_frac48.

Location: sk5_cfx_dsp_math_accelerator.h:713

Parameters

Direction	Name	Description
	<i>in</i>	
	<i>out</i>	
in	<i>size</i>	Number of values in the vector

25.8.3.19 cfx_root3_vec_frac48

```
void cfx_root3_vec_frac48(frac48\_t _XYMEM * in, frac48\_t _XYMEM * out, int size)
```

Calculate the cubic root of each value in a vector.

See `cfx_root3_frac48_frac48`.

Location: `sk5_cfx_dsp_math_accelerator.h:723`

Parameters

Direction	Name	Description
	<i>in</i>	
	<i>out</i>	
in	<i>size</i>	Number of values in the vector

25.8.3.20 cfx_root4_vec_frac24

```
void cfx_root4_vec_frac24(frac24\_t * in, frac24\_t * out, int size)
```

Calculate the fourth root of each value in a vector.

See `cfx_root4_frac24_frac24`.

Location: `sk5_cfx_dsp_math_accelerator.h:733`

Parameters

Direction	Name	Description
	<i>in</i>	
	<i>out</i>	
in	<i>size</i>	Number of values in the vector

25.8.3.21 cfx_root4_vec_frac24

```
void cfx_root4_vec_frac24(frac48\_t _XYMEM * in, frac24\_t * out, int size)
```

Calculate the fourth root of each value in a vector.

See cfx_root4_frac48_frac24.

Location: sk5_cfx_dsp_math_accelerator.h:743

Parameters

Direction	Name	Description
	<i>in</i>	
	<i>out</i>	
in	<i>size</i>	Number of values in the vector

25.8.3.22 cfx_root4_vec_frac48

```
void cfx_root4_vec_frac48(frac24\_t * in, frac48\_t _XYMEM * out, int size)
```

Calculate the fourth root of each value in a vector.

See cfx_root4_frac24_frac48.

Location: sk5_cfx_dsp_math_accelerator.h:753

Parameters

Direction	Name	Description
	<i>in</i>	
	<i>out</i>	
in	<i>size</i>	Number of values in the vector

25.8.3.23 cfx_root4_vec_frac48

```
void cfx_root4_vec_frac48(frac48\_t _XYMEM * in, frac48\_t _XYMEM * out, int size)
```

Calculate the fourth root of each value in a vector.

See cfx_root4_frac48_frac48.

Location: sk5_cfx_dsp_math_accelerator.h:763

Parameters

Direction	Name	Description
	<i>in</i>	
	<i>out</i>	
in	<i>size</i>	Number of values in the vector

25.8.3.24 cfx_root4_frac48

```
frac48\_t cfx_root4_frac48(frac48\_t a)
```

Calculate the fourth root.

Location: sk5_cfx_dsp_math_accelerator.h:770

Parameters

Direction	Name	Description
in	<i>a</i>	Value in frac48_t format

Return

Fourth root in [frac48 t](#) format

25.9 DIVISION FUNCTIONS

CFX DSP library division functions that use the hardware math accelerator.

25.9.1 Summary

Functions

- [cfx_div_m24p24](#) : Division.
- [cfx_div_m24p24](#) : Division.
- [cfx_div_vec_m24p24](#) : Division of each value in two input vectors.
- [cfx_div_vec_m24p24](#) : Division of each value in two input vectors.

25.9.2 Division Functions Function Documentation

25.9.2.1 cfx_div_m24p24

[m24p24 t](#) cfx_div_m24p24([frac24 t](#) dividend, [frac24 t](#) divisor)

Division.

Location: sk5_cfx_dsp_math_accelerator.h:790

Parameters

Direction	Name	Description
in	<i>dividend</i>	Dividend in frac24 t format
in	<i>divisor</i>	Divisor in frac24 t format

Return

Quotient in [m24p24 t](#) format

25.9.2.2 cfx_div_m24p24

[m24p24 t](#) cfx_div_m24p24([m8p16 t](#) dividend, [m8p16 t](#) divisor)

Division.

Location: sk5_cfx_dsp_math_accelerator.h:803

Parameters

Direction	Name	Description
in	<i>dividend</i>	Dividend in m8p16 t format
in	<i>divisor</i>	Divisor in m8p16 t format

Return

Quotient in [m24p24 t](#) format

25.9.2.3 cfx_div_vec_m24p24

void cfx_div_vec_m24p24([frac24 t](#) * divisor, [frac24 t](#) * dividend, [m24p24 t](#) _XYMEM * out, int size)

Division of each value in two input vectors.

See cfx_div_m24p24.

Location: sk5_cfx_dsp_math_accelerator.h:818

Parameters

Direction	Name	Description
	<i>divisor</i>	
	<i>dividend</i>	
	<i>out</i>	
in	<i>size</i>	Number of values in the vector

25.9.2.4 cfx_div_vec_m24p24

```
void cfx_div_vec_m24p24(m8p16\_t * divisor, m8p16\_t * dividend, m24p24\_t _XYMEM * out, int size)
```

Division of each value in two input vectors.

See `cfx_div_m24p24`.

Location: `sk5_cfx_dsp_math_accelerator.h:828`

Parameters

Direction	Name	Description
	<i>divisor</i>	
	<i>dividend</i>	
	<i>out</i>	
in	<i>size</i>	Number of values in the vector

25.10 ASSIGNMENT, ARITHMETIC, AND RELATIONAL OPERATORS

CFX DSP library assignment, arithmetic, and relational operators.

25.10.1 Summary

25.10.2 Detailed Description

When operators are used with fixed-point data types, the compiler automatically substitutes the appropriate fixed-point math functions. The inclusion of these operators means that it is not necessary to call any of these functions directly. For example, it is possible to multiply two fixed-point values like this:

Ezairo 8300 Firmware Reference

- [frac24_t](#) a = 0.5;
- [frac24_t](#) b = 0.25;
- [m9p47_t](#) c = a * b;

The following operators are supported for the most common combinations of data types:

Arithmetic Operators

Most arithmetic operators return an [m9p47_t](#) or [m16p40_t](#) data type in order to minimize the chance of overflow when chaining a number of arithmetic operations together. Multiplication between 24 bit types is very efficient and should be used where possible. While multiplication between 24 bit and 48 bit types may provide some additional precision it is less efficient. Multiplication between 48 bit and 48 bit types is highly inefficient and not currently supported. It is not possible to multiply 56 bit types by any other type, and so the 56 bit data type will need to be converted to a 24 or 48 bit data type before multiplication. Addition and subtraction is most efficient when the decimal point between both types is already aligned. Types with the same number of bits are aligned if their types match. Alignment between a 24 bit and 48 bit type is the same when the number of integer bits match. Alignment between a 56 bit and 24 or 48 bit type is the same when the 56 bit type has 8 additional integer bits. For example, the addition between a [frac24_t](#) and an [frac24_t](#), or an [m8p40_t](#) and an [m8p16_t](#), or an [m16p40_t](#) and an [m8p16_t](#) are all very efficient. Addition and subtraction when the decimal point is not already aligned between both types is supported in most cases by the library, but is less efficient, requiring an additional shift to align the decimal point. For example, The addition between a [frac24_t](#) and an [m8p16_t](#) is supported but less efficient.

Operator	Symbol
Addition operation for two values	+
Subtraction operation for two values	-
Multiplication operation for two values	*
Left shift operation by a specified number of bits	<<
Right shift operation by a specified number of bits	>>

Assignment Operators

Fixed-point data types can be assigned using floating-point numbers and integer literal values (where the value is stored directly). The assignment of fixed-point values where there is no loss of bits (i.e. [frac24_t](#) to [frac48_t](#)) is also supported without the need to use a fixed-Point Data Type Conversion function. For conversion between fixed-point values where there is a loss of bits, the appropriate Fixed-Point Data Type Conversion function must be used.

Operator	Symbol
In-place assignment	=
In-place addition	+=
In-place subtraction	-=
In-place binary multiplication	*=

Relational Operators

Both relational input arguments must be of the same fixed-point data type. If they are not, an appropriate Fixed-Point Data Type Conversion Function must be used on one of those arguments.

25.11 ARITHMETIC OPERATORS

25.11.1 Summary

[illegible]

- www.onsemi.com**

Ezairo 8300 Firmware Reference

- [operator*](#) : Multiplication operation for two values.
- [operator*](#) : Multiplication operation for two values.
- [operator/](#) : Division operation for two values.
- [operator/](#) : Division operation for two values.
- [operator<<<](#) : Left shift operation by a specified number of bits.
- [operator<<<](#) : Left shift operation by a specified number of bits.
- [operator<<<](#) : Left shift operation by a specified number of bits.
- [operator<<<](#) : Left shift operation by a specified number of bits.
- [operator<<<](#) : Left shift operation by a specified number of bits.
- [operator<<<](#) : Left shift operation by a specified number of bits.
- [operator<<<](#) : Left shift operation by a specified number of bits.
- [operator>>>](#) : Right shift operation by a specified number of bits.
- [operator>>>](#) : Right shift operation by a specified number of bits.
- [operator>>>](#) : Right shift operation by a specified number of bits.
- [operator>>>](#) : Right shift operation by a specified number of bits.
- [operator>>>](#) : Right shift operation by a specified number of bits.
- [operator>>>](#) : Right shift operation by a specified number of bits.
- [operator*](#) : Multiplication operation for two values.
- [operator*](#) : Multiplication operation for two values.

25.11.2 Arithmetic Operators Function Documentation

25.11.2.1 operator-

[frac24 t](#) operator-([frac24 t](#) a)

Calculate the negative value of a type.

Location: sk5_cfx_dsp_arithmetic_operators.h:35

Parameters

Direction	Name	Description
in	<i>a</i>	Input in frac24 t format

Return

Returns the negated value in [frac24 t](#) format

25.11.2.2 operator-

[frac48 t](#) operator-([frac48 t](#) a)

Calculate the negative value of a type.

Location: sk5_cfx_dsp_arithmetic_operators.h:47

Parameters

Direction	Name	Description
in	<i>a</i>	Input in frac48_t format

Return

Returns the negated value in [frac48_t](#) format

25.11.2.3 operator-

[m9p47_t](#) operator- ([m9p47_t](#) a)

Calculate the negative value of a type.

Location: sk5_cfx_dsp_arithmetic_operators.h:59

Parameters

Direction	Name	Description
in	<i>a</i>	Input in m9p47_t format

Return

Returns the negated value in [m9p47_t](#) format

25.11.2.4 operator-

[m8p16 t](#) operator- ([m8p16 t](#) a)

Calculate the negative value of a type.

Location: sk5_cfx_dsp_arithmetic_operators.h:71

Parameters

Direction	Name	Description
in	<i>a</i>	Input in m8p16 t format

Return

Returns the negated value in [m8p16 t](#) format

25.11.2.5 operator-

[m8p40 t](#) operator- ([m8p40 t](#) a)

Calculate the negative value of a type.

Location: sk5_cfx_dsp_arithmetic_operators.h:83

Parameters

Direction	Name	Description
in	<i>a</i>	Input in m8p40 t format

Return

Returns the negated value in [m8p40 t](#) format

25.11.2.6 operator-

[m16p40 t](#) operator- ([m16p40 t](#) a)

Calculate the negative value of a type.

Location: sk5_cfx_dsp_arithmetic_operators.h:95

Parameters

Direction	Name	Description
in	<i>a</i>	Input in m16p40 t format

Return

Returns the negated value in [m16p40 t](#) format

25.11.2.7 operator+

[m9p47 t](#) operator+ ([frac24 t](#) a, [frac24 t](#) b)

Addition operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:108

Parameters

Direction	Name	Description
in	<i>a</i>	First value in frac24 t format
in	<i>b</i>	Second value in frac24 t format

Return

Sum in [m9p47 t](#) format

25.11.2.8 operator+

[m9p47_t](#) operator+ ([frac24_t](#) a, [frac48_t](#) b)

Addition operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:121

Parameters

Direction	Name	Description
in	<i>a</i>	First value in frac24_t format
in	<i>b</i>	Second value in frac48_t format

Return

Sum in [m9p47_t](#) format

25.11.2.9 operator+

[m9p47_t](#) operator+ ([frac24_t](#) a, [m9p47_t](#) b)

Addition operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:134

Parameters

Direction	Name	Description
in	<i>a</i>	First value in frac24_t format
in	<i>b</i>	Second value in m9p47_t format

Return

Sum in [m9p47 t](#) format

25.11.2.10 operator+

[m9p47 t](#) operator+ ([frac48 t](#) a, [frac24 t](#) b)

Addition operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:147

Parameters

Direction	Name	Description
in	<i>a</i>	First value in frac48 t format
in	<i>b</i>	Second value in frac24 t format

Return

Sum in [m9p47 t](#) format

25.11.2.11 operator+

[m9p47 t](#) operator+ ([frac48 t](#) a, [frac48 t](#) b)

Addition operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:160

Parameters

Direction	Name	Description
in	<i>a</i>	First value in frac48 t format
in	<i>b</i>	Second value in frac48 t format

Return

Sum in [m9p47 t](#) format

25.11.2.12 operator+

[m9p47 t](#) operator+ ([frac48 t](#) a, [m9p47 t](#) b)

Addition operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:173

Parameters

Direction	Name	Description
in	<i>a</i>	First value in frac48 t format
in	<i>b</i>	Second value in m9p47 t format

Return

Sum in [m9p47 t](#) format

25.11.2.13 operator+

[m9p47 t](#) operator+ ([frac48 t](#) a, [m8p16 t](#) b)

Addition operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:186

Parameters

Direction	Name	Description
in	<i>a</i>	First value in frac48_t format
in	<i>b</i>	Second value in m8p16_t format

Return

Sum in [m9p47_t](#) format

25.11.2.14 operator+

[m9p47_t](#) operator+ ([frac48_t](#) a, [m8p40_t](#) b)

Addition operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:199

Parameters

Direction	Name	Description
in	<i>a</i>	First value in frac48_t format
in	<i>b</i>	Second value in m8p40_t format

Return

Sum in [m9p47_t](#) format

25.11.2.15 operator+

[m9p47_t](#) operator+ ([m9p47_t](#) a, [frac24_t](#) b)

Addition operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:212

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m9p47 t format
in	<i>b</i>	Second value in frac24 t format

Return

Sum in [m9p47 t](#) format

25.11.2.16 operator+

[m9p47 t](#) operator+ ([m9p47 t](#) a, [frac48 t](#) b)

Addition operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:225

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m9p47 t format
in	<i>b</i>	Second value in frac48 t format

Return

Sum in [m9p47 t](#) format

25.11.2.17 operator+

[m9p47 t](#) operator+ ([m9p47 t](#) a, [m9p47 t](#) b)

Addition operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:238

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m9p47_t format
in	<i>b</i>	Second value in m9p47_t format

Return

Sum in [m9p47_t](#) format

25.11.2.18 operator+

[m9p47_t](#) operator+ ([m9p47_t](#) a, [m8p16_t](#) b)

Addition operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:251

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m9p47_t format
in	<i>b</i>	Second value in m8p16_t format

Return

Sum in [m9p47_t](#) format

25.11.2.19 operator+

[m9p47_t](#) operator+ ([m9p47_t](#) a, [m8p40_t](#) b)

Addition operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:264

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m9p47_t format
in	<i>b</i>	Second value in m8p40_t format

Return

Sum in [m9p47_t](#) format

25.11.2.20 operator+

[m9p47_t](#) operator+ ([m8p16_t](#) a, [frac48_t](#) b)

Addition operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:277

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m8p16_t format
in	<i>b</i>	Second value in frac48_t format

Return

Sum in [m9p47 t](#) format

25.11.2.21 operator+

[m9p47 t](#) operator+ ([m8p16 t](#) a, [m9p47 t](#) b)

Addition operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:290

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m8p16 t format
in	<i>b</i>	Second value in m9p47 t format

Return

Sum in [m9p47 t](#) format

25.11.2.22 operator+

[m9p47 t](#) operator+ ([m8p40 t](#) a, [frac48 t](#) b)

Addition operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:303

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m8p40 t format
in	<i>b</i>	Second value in frac48 t format

Return

Sum in [m9p47 t](#) format

25.11.2.23 operator+

[m9p47 t](#) operator+ ([m8p40 t](#) a, [m9p47 t](#) b)

Addition operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:316

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m8p40 t format
in	<i>b</i>	Second value in m9p47 t format

Return

Sum in [m9p47 t](#) format

25.11.2.24 operator+

[m16p40 t](#) operator+ ([frac24 t](#) a, [m8p16 t](#) b)

Addition operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:329

Parameters

Direction	Name	Description
in	<i>a</i>	First value in frac24_t format
in	<i>b</i>	Second value in m8p16_t format

Return

Sum in [m16p40_t](#) format

25.11.2.25 operator+

[m16p40_t](#) operator+ ([frac24_t](#) a, [m8p40_t](#) b)

Addition operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:342

Parameters

Direction	Name	Description
in	<i>a</i>	First value in frac24_t format
in	<i>b</i>	Second value in m8p40_t format

Return

Sum in [m16p40_t](#) format

25.11.2.26 operator+

[m16p40_t](#) operator+ ([frac24_t](#) a, [m16p40_t](#) b)

Addition operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:355

Parameters

Direction	Name	Description
in	<i>a</i>	First value in frac24_t format
in	<i>b</i>	Second value in m16p40_t format

Return

Sum in [m16p40_t](#) format

25.11.2.27 operator+

[m16p40_t](#) operator+([m8p16_t](#) a, [frac24_t](#) b)

Addition operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:368

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m8p16_t format
in	<i>b</i>	Second value in frac24_t format

Return

Sum in [m16p40_t](#) format

25.11.2.28 operator+

[m16p40_t](#) operator+([m8p16_t](#) a, [m8p16_t](#) b)

Addition operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:382

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m8p16 t format
in	<i>b</i>	Second value in m8p16 t format

Return

Sum in [m16p40 t](#) format

25.11.2.29 operator+

[m16p40 t](#) operator+ ([m8p16 t](#) a, [m8p40 t](#) b)

Addition operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:395

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m8p16 t format
in	<i>b</i>	Second value in m8p40 t format

Return

Sum in [m16p40 t](#) format

25.11.2.30 operator+

[m16p40_t](#) operator+ ([m8p16_t](#) a, [m16p40_t](#) b)

Addition operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:408

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m8p16_t format
in	<i>b</i>	Second value in m16p40_t format

Return

Sum in [m16p40_t](#) format

25.11.2.31 operator+

[m16p40_t](#) operator+ ([m8p40_t](#) a, [frac24_t](#) b)

Addition operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:421

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m8p40_t format
in	<i>b</i>	Second value in frac24_t format

Return

Sum in [m16p40 t](#) format

25.11.2.32 operator+

[m16p40 t](#) operator+ ([m8p40 t](#) a, [m8p16 t](#) b)

Addition operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:434

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m8p40 t format
in	<i>b</i>	Second value in m8p16 t format

Return

Sum in [m16p40 t](#) format

25.11.2.33 operator+

[m16p40 t](#) operator+ ([m8p40 t](#) a, [m8p40 t](#) b)

Addition operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:447

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m8p40 t format
in	<i>b</i>	Second value in m8p40 t format

Return

Sum in [m16p40 t](#) format

25.11.2.34 operator+

[m16p40 t](#) operator+ ([m8p40 t](#) a, [m16p40 t](#) b)

Addition operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:460

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m8p40 t format
in	<i>b</i>	Second value in m16p40 t format

Return

Sum in [m16p40 t](#) format

25.11.2.35 operator+

[m16p40 t](#) operator+ ([m16p40 t](#) a, [frac24 t](#) b)

Addition operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:473

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m16p40_t format
in	<i>b</i>	Second value in frac24_t format

Return

Sum in [m16p40_t](#) format

25.11.2.36 operator+

[m16p40_t](#) operator+ ([m16p40_t](#) a, [m8p16_t](#) b)

Addition operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:486

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m16p40_t format
in	<i>b</i>	Second value in m8p16_t format

Return

Sum in [m16p40_t](#) format

25.11.2.37 operator+

[m16p40_t](#) operator+ ([m16p40_t](#) a, [m8p40_t](#) b)

Addition operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:499

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m16p40 t format
in	<i>b</i>	Second value in m8p40 t format

Return

Sum in [m16p40 t](#) format

25.11.2.38 operator+

[m16p40 t](#) operator+ ([m16p40 t](#) a, [m16p40 t](#) b)

Addition operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:512

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m16p40 t format
in	<i>b</i>	Second value in m16p40 t format

Return

Sum in [m16p40 t](#) format

25.11.2.39 operator-

[m9p47 t](#) operator- ([frac24 t](#) a, [frac24 t](#) b)

Subtraction operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:525

Parameters

Direction	Name	Description
in	<i>a</i>	First value in frac24_t format
in	<i>b</i>	Second value in frac24_t format

Return

Difference $a - b$ in [m9p47_t](#) format

25.11.2.40 operator-

[m9p47_t](#) operator- ([frac24_t](#) a, [frac48_t](#) b)

Subtraction operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:538

Parameters

Direction	Name	Description
in	<i>a</i>	First value in frac24_t format
in	<i>b</i>	Second value in frac48_t format

Return

Difference $a - b$ in [m9p47_t](#) format

25.11.2.41 operator-

[m9p47_t](#) operator- ([frac24_t](#) a, [m9p47_t](#) b)

Subtraction operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:551

Parameters

Direction	Name	Description
in	<i>a</i>	First value in frac24_t format
in	<i>b</i>	Second value in m9p47_t format

Return

Difference a - b in [m9p47_t](#) format

25.11.2.42 operator-

[m9p47_t](#) operator- ([frac48_t](#) a, [frac24_t](#) b)

Subtraction operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:564

Parameters

Direction	Name	Description
in	<i>a</i>	First value in frac48_t format
in	<i>b</i>	Second value in frac24_t format

Return

Ezairo 8300 Firmware Reference

Difference $a - b$ in [m9p47 t](#) format

25.11.2.43 operator-

[m9p47 t](#) operator- ([frac48 t](#) a , [frac48 t](#) b)

Subtraction operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:577

Parameters

Direction	Name	Description
in	a	First value in frac48 t format
in	b	Second value in frac48 t format

Return

Difference $a - b$ in [m9p47 t](#) format

25.11.2.44 operator-

[m9p47 t](#) operator- ([frac48 t](#) a , [m9p47 t](#) b)

Subtraction operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:590

Parameters

Direction	Name	Description
in	a	First value in frac48 t format
in	b	Second value in m9p47 t format

Return

Difference $a - b$ in [m9p47_t](#) format

25.11.2.45 operator-

[m9p47_t](#) operator- ([frac48_t](#) a , [m8p16_t](#) b)

Subtraction operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:603

Parameters

Direction	Name	Description
in	a	First value in frac48_t format
in	b	Second value in m8p16_t format

Return

Difference $a - b$ in [m9p47_t](#) format

25.11.2.46 operator-

[m9p47_t](#) operator- ([frac48_t](#) a , [m8p40_t](#) b)

Subtraction operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:616

Parameters

Direction	Name	Description
in	<i>a</i>	First value in frac48_t format
in	<i>b</i>	Second value in m8p40_t format

Return

Difference $a - b$ in [m9p47_t](#) format

25.11.2.47 operator-

[m9p47_t](#) operator- ([m9p47_t](#) *a*, [frac24_t](#) *b*)

Subtraction operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:629

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m9p47_t format
in	<i>b</i>	Second value in frac24_t format

Return

Difference $a - b$ in [m9p47_t](#) format

25.11.2.48 operator-

[m9p47_t](#) operator- ([m9p47_t](#) *a*, [frac48_t](#) *b*)

Subtraction operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:642

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m9p47 t format
in	<i>b</i>	Second value in frac48 t format

Return

Difference $a - b$ in [m9p47 t](#) format

25.11.2.49 operator-

[m9p47 t](#) operator- ([m9p47 t](#) *a*, [m9p47 t](#) *b*)

Subtraction operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:655

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m9p47 t format
in	<i>b</i>	Second value in m9p47 t format

Return

Difference $a - b$ in [m9p47 t](#) format

25.11.2.50 operator-

[m9p47 t](#) operator- ([m9p47 t](#) *a*, [m8p16 t](#) *b*)

Subtraction operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:668

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m9p47_t format
in	<i>b</i>	Second value in m8p16_t format

Return

Difference $a - b$ in [m9p47_t](#) format

25.11.2.51 operator-

[m9p47_t](#) operator- ([m9p47_t](#) a, [m8p40_t](#) b)

Subtraction operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:681

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m9p47_t format
in	<i>b</i>	Second value in m8p40_t format

Return

Difference $a - b$ in [m9p47_t](#) format

25.11.2.52 operator-

[m9p47 t](#) operator- ([m8p16 t](#) a, [frac48 t](#) b)

Subtraction operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:694

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m8p16 t format
in	<i>b</i>	Second value in frac48 t format

Return

Difference a - b in [m9p47 t](#) format

25.11.2.53 operator-

[m9p47 t](#) operator- ([m8p16 t](#) a, [m9p47 t](#) b)

Subtraction operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:707

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m8p16 t format
in	<i>b</i>	Second value in m9p47 t format

Return

Ezairo 8300 Firmware Reference

Difference $a - b$ in [m9p47 t](#) format

25.11.2.54 operator-

[m9p47 t](#) operator- ([m8p40 t](#) a , [frac48 t](#) b)

Subtraction operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:720

Parameters

Direction	Name	Description
in	a	First value in m8p40 t format
in	b	Second value in frac48 t format

Return

Difference $a - b$ in [m9p47 t](#) format

25.11.2.55 operator-

[m9p47 t](#) operator- ([m8p40 t](#) a , [m9p47 t](#) b)

Subtraction operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:733

Parameters

Direction	Name	Description
in	a	First value in m8p40 t format
in	b	Second value in m9p47 t format

Return

Difference $a - b$ in [m9p47_t](#) format

25.11.2.56 operator-

[m16p40_t](#) operator-([frac24_t](#) a, [m8p16_t](#) b)

Subtraction operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:746

Parameters

Direction	Name	Description
in	<i>a</i>	First value in frac24_t format
in	<i>b</i>	Second value in m8p16_t format

Return

Difference $a - b$ in [m16p40_t](#) format

25.11.2.57 operator-

[m16p40_t](#) operator-([frac24_t](#) a, [m8p40_t](#) b)

Subtraction operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:759

Parameters

Direction	Name	Description
in	<i>a</i>	First value in frac24_t format
in	<i>b</i>	Second value in m8p40_t format

Return

Difference $a - b$ in [m16p40_t](#) format

25.11.2.58 operator-

[m16p40_t](#) operator-([frac24_t](#) a, [m16p40_t](#) b)

Subtraction operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:772

Parameters

Direction	Name	Description
in	<i>a</i>	First value in frac24_t format
in	<i>b</i>	Second value in m16p40_t format

Return

Difference $a - b$ in [m9p47_t](#) format

25.11.2.59 operator-

[m16p40_t](#) operator-([m8p16_t](#) a, [frac24_t](#) b)

Subtraction operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:785

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m8p16_t format
in	<i>b</i>	Second value in frac24_t format

Return

Difference $a - b$ in [m16p40_t](#) format

25.11.2.60 operator-

[m16p40_t](#) operator-([m8p16_t](#) a, [m8p16_t](#) b)

Subtraction operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:798

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m8p16_t format
in	<i>b</i>	Second value in m8p16_t format

Return

Difference $a - b$ in [m16p40_t](#) format

25.11.2.61 operator-

[m16p40_t](#) operator-([m8p16_t](#) a, [m8p40_t](#) b)

Subtraction operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:811

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m8p16 t format
in	<i>b</i>	Second value in m8p40 t format

Return

Difference $a - b$ in [m16p40 t](#) format

25.11.2.62 operator-

[m16p40 t](#) operator-([m8p16 t](#) a, [m16p40 t](#) b)

Subtraction operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:824

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m8p16 t format
in	<i>b</i>	Second value in m16p40 t format

Return

Difference $a - b$ in [m16p40 t](#) format

25.11.2.63 operator-

[m16p40 t](#) operator- ([m8p40 t](#) a, [frac24 t](#) b)

Subtraction operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:837

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m8p40 t format
in	<i>b</i>	Second value in frac24 t format

Return

Difference a - b in [m16p40 t](#) format

25.11.2.64 operator-

[m16p40 t](#) operator- ([m8p40 t](#) a, [m8p16 t](#) b)

Subtraction operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:850

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m8p40 t format
in	<i>b</i>	Second value in m8p16 t format

Return

Ezairo 8300 Firmware Reference

Difference $a - b$ in [m16p40 t](#) format

25.11.2.65 operator-

[m16p40 t](#) operator- ([m8p40 t](#) a , [m8p40 t](#) b)

Subtraction operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:863

Parameters

Direction	Name	Description
in	a	First value in m8p40 t format
in	b	Second value in m8p40 t format

Return

Difference $a - b$ in [m16p40 t](#) format

25.11.2.66 operator-

[m16p40 t](#) operator- ([m8p40 t](#) a , [m16p40 t](#) b)

Subtraction operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:876

Parameters

Direction	Name	Description
in	a	First value in m8p40 t format
in	b	Second value in m16p40 t format

Return

Difference $a - b$ in [m16p40 t](#) format

25.11.2.67 operator-

[m16p40 t](#) operator- ([m16p40 t](#) a , [frac24 t](#) b)

Subtraction operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:889

Parameters

Direction	Name	Description
in	a	First value in m16p40 t format
in	b	Second value in frac24 t format

Return

Difference $a - b$ in [m9p47 t](#) format

25.11.2.68 operator-

[m16p40 t](#) operator- ([m16p40 t](#) a , [m8p16 t](#) b)

Subtraction operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:902

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m16p40_t format
in	<i>b</i>	Second value in m8p16_t format

Return

Difference $a - b$ in [m16p40_t](#) format

25.11.2.69 operator-

[m16p40_t](#) operator- ([m16p40_t](#) a, [m8p40_t](#) b)

Subtraction operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:915

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m16p40_t format
in	<i>b</i>	Second value in m8p40_t format

Return

Difference $a - b$ in [m16p40_t](#) format

25.11.2.70 operator-

[m16p40_t](#) operator- ([m16p40_t](#) a, [m16p40_t](#) b)

Subtraction operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:928

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m16p40 t format
in	<i>b</i>	Second value in m16p40 t format

Return

Difference $a - b$ in [m16p40 t](#) format

25.11.2.71 operator*

[m9p47 t](#) operator* ([frac24 t](#) *a*, [frac24 t](#) *b*)

Multiplication operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:941

Parameters

Direction	Name	Description
in	<i>a</i>	First value in frac24 t format
in	<i>b</i>	Second value in frac24 t format

Return

Product in [m9p47 t](#) format

25.11.2.72 operator*

[m9p47 t](#) operator* ([frac24 t](#) *a*, [frac48 t](#) *b*)

Multiplication operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:954

Parameters

Direction	Name	Description
in	<i>a</i>	First value in frac24 t format
in	<i>b</i>	Second value in frac48 t format

Return

Product in [m9p47 t](#) format

25.11.2.73 operator*

[m9p47 t](#) operator* ([frac48 t](#) a, [frac24 t](#) b)

Multiplication operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:967

Parameters

Direction	Name	Description
in	<i>a</i>	First value in frac48 t format
in	<i>b</i>	Second value in frac24 t format

Return

Product in [m9p47 t](#) format

25.11.2.74 operator*

[m16p40_t](#) operator*([frac24_t](#) a, [m8p16_t](#) b)

Multiplication operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:980

Parameters

Direction	Name	Description
in	<i>a</i>	First value in frac24_t format
in	<i>b</i>	Second value in m8p16_t format

Return

Product in [m16p40_t](#) format

25.11.2.75 operator*

[m16p40_t](#) operator*([frac24_t](#) a, [m8p40_t](#) b)

Multiplication operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:993

Parameters

Direction	Name	Description
in	<i>a</i>	First value in frac24_t format
in	<i>b</i>	Second value in m8p40_t format

Return

Product in [m16p40_t](#) format

25.11.2.76 operator*

[m16p40_t](#) operator*([frac48_t](#) a, [m8p16_t](#) b)

Multiplication operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:1006

Parameters

Direction	Name	Description
in	<i>a</i>	First value in frac48_t format
in	<i>b</i>	Second value in m8p16_t format

Return

Product in [m16p40_t](#) format

25.11.2.77 operator*

[m16p40_t](#) operator*([m8p16_t](#) a, [frac24_t](#) b)

Multiplication operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:1019

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m8p16_t format
in	<i>b</i>	Second value in frac24_t format

Return

Product in [m16p40 t](#) format

25.11.2.78 operator*

[m16p40 t](#) operator* ([m8p16 t](#) a, [frac48 t](#) b)

Multiplication operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:1032

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m8p16 t format
in	<i>b</i>	Second value in frac48 t format

Return

Product in [m16p40 t](#) format

25.11.2.79 operator*

[m16p40 t](#) operator* ([m8p40 t](#) a, [frac24 t](#) b)

Multiplication operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:1045

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m8p40 t format
in	<i>b</i>	Second value in frac24 t format

Return

Product in [m16p40 t](#) format

25.11.2.80 operator/

[m24p24 t](#) operator/ ([frac24 t](#) a, [frac24 t](#) b)

Division operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:1058

Parameters

Direction	Name	Description
in	<i>a</i>	Dividend in frac24 t format
in	<i>b</i>	Divisor in frac24 t format

Return

Quotient a / b in [m24p24 t](#) format

25.11.2.81 operator/

[m24p24 t](#) operator/ ([m8p16 t](#) a, [m8p16 t](#) b)

Division operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:1071

Parameters

Direction	Name	Description
in	<i>a</i>	Dividend in m8p16 t format
in	<i>b</i>	Divisor in m8p16 t format

Return

Quotient a / b in [m24p24 t](#) format

25.11.2.82 operator<<<

[frac24 t](#) operator<<<([frac24 t](#) a, int b)

Left shift operation by a specified number of bits.

Location: sk5_cfx_dsp_arithmetic_operators.h:1084

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in frac24 t format
in	<i>b</i>	Number of bits as an integer

Return

Shifted value in [frac24 t](#) format

25.11.2.83 operator<<<

[frac48 t](#) operator<<<([frac48 t](#) a, int b)

Left shift operation by a specified number of bits.

Location: sk5_cfx_dsp_arithmetic_operators.h:1097

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in frac48 t format
in	<i>b</i>	Number of bits as an integer

Return

Shifted value in [frac48 t](#) format

25.11.2.84 operator<<<

[m9p47 t](#) operator<<<([m9p47 t](#) a, int b)

Left shift operation by a specified number of bits.

Location: sk5_cfx_dsp_arithmetic_operators.h:1110

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in m9p47 t format
in	<i>b</i>	Number of bits as an integer

Return

Shifted value in [m9p47 t](#) format

25.11.2.85 operator<<<

[m8p16_t](#) operator<<<([m8p16_t](#) a, int b)

Left shift operation by a specified number of bits.

Location: sk5_cfx_dsp_arithmetic_operators.h:1123

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in m8p16_t format
in	<i>b</i>	Number of bits as an integer

Return

Shifted value in [m8p16_t](#) format

25.11.2.86 operator<<<

[m8p40_t](#) operator<<<([m8p40_t](#) a, int b)

Left shift operation by a specified number of bits.

Location: sk5_cfx_dsp_arithmetic_operators.h:1136

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in m8p40_t format
in	<i>b</i>	Number of bits as an integer

Return

Shifted value in [m8p40 t](#) format

25.11.2.87 operator<<<

[m16p40 t](#) operator<<< ([m16p40 t](#) a, int b)

Left shift operation by a specified number of bits.

Location: sk5_cfx_dsp_arithmetic_operators.h:1149

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in m16p40 t format
in	<i>b</i>	Number of bits as an integer

Return

Shifted value in [m16p40 t](#) format

25.11.2.88 operator>>>

[frac24 t](#) operator>>> ([frac24 t](#) a, int b)

Right shift operation by a specified number of bits.

Location: sk5_cfx_dsp_arithmetic_operators.h:1162

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in frac24 t format
in	<i>b</i>	Number of bits as an integer

Return

Shifted value in [frac24_t](#) format

25.11.2.89 operator>>

[frac48_t](#) operator>>([frac48_t](#) a, int b)

Right shift operation by a specified number of bits.

Location: sk5_cfx_dsp_arithmetic_operators.h:1175

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in frac48_t format
in	<i>b</i>	Number of bits as an integer

Return

Shifted value in [frac48_t](#) format

25.11.2.90 operator>>

[m9p47_t](#) operator>>([m9p47_t](#) a, int b)

Right shift operation by a specified number of bits.

Location: sk5_cfx_dsp_arithmetic_operators.h:1188

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in m9p47_t format
in	<i>b</i>	Number of bits as an integer

Return

Shifted value in [m9p47_t](#) format

25.11.2.91 operator<>>>

[m8p16_t](#) operator<>>> ([m8p16_t](#) a, int b)

Right shift operation by a specified number of bits.

Location: sk5_cfx_dsp_arithmetic_operators.h:1201

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in m8p16_t format
in	<i>b</i>	Number of bits as an integer

Return

Shifted value in [m8p16_t](#) format

25.11.2.92 operator<>>>

[m8p40_t](#) operator<>>> ([m8p40_t](#) a, int b)

Right shift operation by a specified number of bits.

Location: sk5_cfx_dsp_arithmetic_operators.h:1214

Parameters

Direction	Name	Description
in	<i>a</i>	a Input value in m8p40 t format
in	<i>b</i>	Number of bits as an integer

Return

Shifted value in [m8p40 t](#) format

25.11.2.93 operator>>

[m16p40 t](#) operator>>([m16p40 t](#) a, int b)

Right shift operation by a specified number of bits.

Location: sk5_cfx_dsp_arithmetic_operators.h:1227

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in m16p40 t format
in	<i>b</i>	Number of bits as an integer

Return

Shifted value in [m16p40 t](#) format

25.11.2.94 operator*

[m24p24 t](#) operator*([frac24 t](#) a, int b)

Multiplication operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:1249

Parameters

Direction	Name	Description
in	<i>a</i>	First value in frac24_t format
in	<i>b</i>	Second value in int format

Return

Product in [m24p24_t](#) format

25.11.2.95 operator*

[m24p24_t](#) operator*(int a, [frac24_t](#) b)

Multiplication operation for two values.

Location: sk5_cfx_dsp_arithmetic_operators.h:1264

Parameters

Direction	Name	Description
in	<i>a</i>	First value in int format
in	<i>b</i>	Second value in frac24_t format

Return

Product in [m24p24_t](#) format

Ezairo 8300 Firmware Reference

25.12 ASSIGNMENT OPERATORS

CFX DSP library assignment operators.

25.12.1 Summary

Functions

- [`operator=`](#) : Assignment of a [`frac24\_t`](#) type.
- [`operator=`](#) : Assignment of a [`frac24\_t`](#) type.
- [`operator=`](#) : Assignment of a [`frac48\_t`](#) type.
- [`operator=`](#) : Assignment of a [`frac48\_t`](#) type.
- [`operator=`](#) : Assignment of a [`frac48\_t`](#) type.
- [`operator=`](#) : Assignment of a [`m9p47\_t`](#) type.
- [`operator=`](#) : Assignment of a [`m9p47\_t`](#) type.
- [`operator=`](#) : Assignment of a [`m9p47\_t`](#) type.
- [`operator=`](#) : Assignment of a [`m9p47\_t`](#) type.
- [`operator=`](#) : Assignment of a [`m8p16\_t`](#) type.
- [`operator=`](#) : Assignment of a [`m8p16\_t`](#) type.
- [`operator=`](#) : Assignment of a [`m8p40\_t`](#) type.
- [`operator=`](#) : Assignment of a [`m8p40\_t`](#) type.
- [`operator=`](#) : Assignment of a [`m8p40\_t`](#) type.
- [`operator=`](#) : Assignment of a [`m8p40\_t`](#) type.
- [`operator=`](#) : Assignment of a [`m16p40\_t`](#) type.
- [`operator=`](#) : Assignment of a [`m16p40\_t`](#) type.
- [`operator=`](#) : Assignment of a [`m16p40\_t`](#) type.
- [`operator=`](#) : Assignment of a [`m16p40\_t`](#) type.
- [`operator=`](#) : Assignment of a [`m16p40\_t`](#) type.
- [`operator+=`](#) : In-place incrementation of a [`frac24\_t`](#) type by another [`frac24\_t`](#) type.
- [`operator+=`](#) : In-place incrementation of a [`frac48\_t`](#) type by another [`frac48\_t`](#) type.
- [`operator-=`](#) : In-place decrementation of a [`frac24\_t`](#) type by another [`frac24\_t`](#) type.
- [`operator-=`](#) : In-place decrementation of a [`frac48\_t`](#) type by another [`frac48\_t`](#) type.
- [`operator\*=`](#) : In-place multiplication assignment of a [`frac24\_t`](#) type by another [`frac24\_t`](#) type.

25.12.2 Assignment Operators Function Documentation

25.12.2.1 `operator=`

```
void operator=(frac24\_t & a, double b)
```

Assignment of a [`frac24\_t`](#) type.

Location: `sk5_cfx_dsp_assignment_operators.h:35`

Parameters

Direction	Name	Description
	<i>a</i>	
in	<i>b</i>	double value to be assigned

25.12.2.2 operator=

```
void operator=(frac24 t & a, int b)
```

Assignment of a [frac24 t](#) type.

Location: sk5_cfx_dsp_assignment_operators.h:45

Parameters

Direction	Name	Description
	<i>a</i>	
in	<i>b</i>	int value to be assigned

25.12.2.3 operator=

```
void operator=(frac48 t & a, frac24 t b)
```

Assignment of a [frac48 t](#) type.

Location: sk5_cfx_dsp_assignment_operators.h:55

Parameters

Direction	Name	Description
	<i>a</i>	
in	<i>b</i>	frac24 t value to be assigned

25.12.2.4 operator=

```
void operator=(frac48 t & a, double b)
```

Assignment of a [frac48_t](#) type.

Location: sk5_cfx_dsp_assignment_operators.h:65

Parameters

Direction	Name	Description
	<i>a</i>	
in	<i>b</i>	double value to be assigned

25.12.2.5 operator=

```
void operator=(frac48\_t & a, long b)
```

Assignment of a [frac48_t](#) type.

Location: sk5_cfx_dsp_assignment_operators.h:75

Parameters

Direction	Name	Description
	<i>a</i>	
in	<i>b</i>	long value to be assigned

25.12.2.6 operator=

```
void operator=(m9p47\_t & a, frac24\_t b)
```

Assignment of a [m9p47_t](#) type.

Location: sk5_cfx_dsp_assignment_operators.h:85

Parameters

Direction	Name	Description
	<i>a</i>	
in	<i>b</i>	frac24_t value to be assigned

25.12.2.7 operator=

void operator=([m9p47_t](#) & a, [frac48_t](#) b)

Assignment of a [m9p47_t](#) type.

Location: sk5_cfx_dsp_assignment_operators.h:95

Parameters

Direction	Name	Description
	<i>a</i>	
in	<i>b</i>	frac48_t value to be assigned

25.12.2.8 operator=

void operator=([m9p47_t](#) & a, double b)

Assignment of a [m9p47_t](#) type.

Location: sk5_cfx_dsp_assignment_operators.h:105

Parameters

Direction	Name	Description
	<i>a</i>	
in	<i>b</i>	double value to be assigned

25.12.2.9 operator=

void operator=([m9p47_t](#) & a, long b)

Assignment of a [m9p47_t](#) type.

Location: sk5_cfx_dsp_assignment_operators.h:120

Parameters

Direction	Name	Description
	<i>a</i>	
in	<i>b</i>	long value to be assigned

25.12.2.10 operator=

```
void operator=(m8p16\_t & a, double b)
```

Assignment of a [m8p16_t](#) type.

Location: sk5_cfx_dsp_assignment_operators.h:130

Parameters

Direction	Name	Description
	<i>a</i>	
in	<i>b</i>	double value to be assigned

25.12.2.11 operator=

```
void operator=(m8p16\_t & a, int b)
```

Assignment of a [m8p16_t](#) type.

Location: sk5_cfx_dsp_assignment_operators.h:152

Parameters

Direction	Name	Description
	<i>a</i>	
in	<i>b</i>	int value to be assigned

25.12.2.12 operator=

```
void operator=(m8p40\_t & a, frac24\_t b)
```

Assignment of a [m8p40_t](#) type.

Location: sk5_cfx_dsp_assignment_operators.h:162

Parameters

Direction	Name	Description
	<i>a</i>	
in	<i>b</i>	frac24_t value to be assigned

25.12.2.13 operator=

```
void operator=(m8p40\_t & a, m8p16\_t b)
```

Assignment of a [m8p40_t](#) type.

Location: sk5_cfx_dsp_assignment_operators.h:172

Parameters

Direction	Name	Description
	<i>a</i>	
in	<i>b</i>	m8p16_t value to be assigned

25.12.2.14 operator=

```
void operator=(m8p40\_t & a, double b)
```

Assignment of a [m8p40 t](#) type.

Location: sk5_cfx_dsp_assignment_operators.h:182

Parameters

Direction	Name	Description
	<i>a</i>	
in	<i>b</i>	double value to be assigned

25.12.2.15 operator=

```
void operator=(m8p40 t & a, long b)
```

Assignment of a [m8p40 t](#) type.

Location: sk5_cfx_dsp_assignment_operators.h:204

Parameters

Direction	Name	Description
	<i>a</i>	
in	<i>b</i>	long value to be assigned

25.12.2.16 operator=

```
void operator=(m16p40 t & a, frac24 t b)
```

Assignment of a [m16p40 t](#) type.

Location: sk5_cfx_dsp_assignment_operators.h:214

Parameters

onsemi Confidential
Ezairo 8300 Firmware Reference

Direction	Name	Description
	<i>a</i>	
in	<i>b</i>	frac24_t value to be assigned

25.12.2.17 operator=

```
void operator=(m16p40\_t & a, m8p16\_t b)
```

Assignment of a [m16p40_t](#) type.

Location: sk5_cfx_dsp_assignment_operators.h:224

Parameters

Direction	Name	Description
	<i>a</i>	
in	<i>b</i>	m8p16_t value to be assigned

25.12.2.18 operator=

```
void operator=(m16p40\_t & a, m8p40\_t b)
```

Assignment of a [m16p40_t](#) type.

Location: sk5_cfx_dsp_assignment_operators.h:234

Parameters

Direction	Name	Description
	<i>a</i>	
in	<i>b</i>	m8p40_t value to be assigned

25.12.2.19 operator=

```
void operator=(m16p40\_t & a, double b)
```

Assignment of a [ml6p40 t](#) type.

Location: sk5_cfx_dsp_assignment_operators.h:244

Parameters

Direction	Name	Description
	<i>a</i>	
in	<i>b</i>	double value to be assigned

25.12.2.20 operator=

```
void operator=(ml6p40 t & a, long b)
```

Assignment of a [ml6p40 t](#) type.

Location: sk5_cfx_dsp_assignment_operators.h:259

Parameters

Direction	Name	Description
	<i>a</i>	
in	<i>b</i>	long value to be assigned

25.12.2.21 operator+=

```
void operator+=(frac24 t & a, frac24 t b)
```

In-place incrementation of a [frac24 t](#) type by another [frac24 t](#) type.

Location: sk5_cfx_dsp_assignment_operators.h:270

Parameters

Direction	Name	Description
	<i>a</i>	
in	<i>b</i>	frac24 t value to increment by

25.12.2.22 operator+=

```
void operator+=(frac48 t & a, frac48 t b)
```

In-place incrementation of a [frac48 t](#) type by another [frac48 t](#) type.

Location: sk5_cfx_dsp_assignment_operators.h:281

Parameters

Direction	Name	Description
	<i>a</i>	
in	<i>b</i>	frac48 t value to increment by

25.12.2.23 operator-=

```
void operator-=(frac24 t & a, frac24 t b)
```

In-place decrementation of a [frac24 t](#) type by another [frac24 t](#) type.

Location: sk5_cfx_dsp_assignment_operators.h:292

Parameters

Direction	Name	Description
	<i>a</i>	
in	<i>b</i>	frac24 t value to decrement by

25.12.2.24 operator-=

```
void operator-=(frac48 t & a, frac48 t b)
```

In-place decrementation of a [frac48_t](#) type by another [frac48_t](#) type.

Location: sk5_cfx_dsp_assignment_operators.h:303

Parameters

Direction	Name	Description
	<i>a</i>	
in	<i>b</i>	frac48_t value to decrement by

25.12.2.25 operator*=

```
void operator*=(frac24\_t & a, frac24\_t b)
```

In-place multiplication assignment of a [frac24_t](#) type by another [frac24_t](#) type.

Location: sk5_cfx_dsp_assignment_operators.h:314

Parameters

Direction	Name	Description
	<i>a</i>	
in	<i>b</i>	frac24_t value to multiply by

25.13 RELATIONAL OPERATORS

CFX DSP library relational operators.

25.13.1 Summary

Functions

- [operator>=](#) : Greater than or equal to operation comparing two values.
- [operator>=](#) : Greater than or equal to operation comparing two values.
- [operator>=](#) : Greater than or equal to operation comparing two values.
- [operator>=](#) : Greater than or equal to operation comparing two values.
- [operator>=](#) : Greater than or equal to operation comparing two values.

Ezairo 8300 Firmware Reference

- [operator>=](#) : Greater than or equal to operation comparing two values.
- [operator>](#) : Greater than operation comparing two values.
- [operator>](#) : Greater than operation comparing two values.
- [operator>](#) : Greater than operation comparing two values.
- [operator>](#) : Greater than operation comparing two values.
- [operator>](#) : Greater than operation comparing two values.
- [operator>](#) : Greater than operation comparing two values.
- [operator<=](#) : Less than or equal to operation comparing two values.
- [operator<=](#) : Less than or equal to operation comparing two values.
- [operator<=](#) : Less than or equal to operation comparing two values.
- [operator<=](#) : Less than or equal to operation comparing two values.
- [operator<=](#) : Less than or equal to operation comparing two values.
- [operator<=](#) : Less than or equal to operation comparing two values.
- [operator<](#) : Less than operation comparing two values.
- [operator<](#) : Less than operation comparing two values.
- [operator<](#) : Less than operation comparing two values.
- [operator<](#) : Less than operation comparing two values.
- [operator<](#) : Less than operation comparing two values.
- [operator<](#) : Less than operation comparing two values.
- [operator==](#) : Check if two values are equal to each other.
- [operator==](#) : Check if two values are equal to each other.
- [operator==](#) : Check if two values are equal to each other.
- [operator==](#) : Check if two values are equal to each other.
- [operator==](#) : Check if two values are equal to each other.
- [operator==](#) : Check if two values are equal to each other.
- [operator!=](#) : Check if two values are not equal to each other.
- [operator!=](#) : Check if two values are not equal to each other.
- [operator!=](#) : Check if two values are not equal to each other.
- [operator!=](#) : Check if two values are not equal to each other.
- [operator!=](#) : Check if two values are not equal to each other.
- [operator!=](#) : Check if two values are not equal to each other.

25.13.2 Relational Operators Function Documentation

25.13.2.1 operator>=

```
bool operator&gt;=(frac24 t a, frac24 t b)
```

Greater than or equal to operation comparing two values.

Location: sk5_cfx_dsp_relational_operators.h:37

Parameters

Direction	Name	Description
in	<i>a</i>	First value in frac24_t format
in	<i>b</i>	Second value in frac24_t format

Return

Return true if the first value is greater than or equal to the second value, false otherwise

25.13.2.2 operator>=

```
bool operator>=(frac48\_t a, frac48\_t b)
```

Greater than or equal to operation comparing two values.

Location: sk5_cfx_dsp_relational_operators.h:51

Parameters

Direction	Name	Description
in	<i>a</i>	First value in frac48_t format
in	<i>b</i>	Second value in frac48_t format

Return

Return true if the first value is greater than or equal to the second value, false otherwise

25.13.2.3 operator>=

```
bool operator>=(m9p47\_t a, m9p47\_t b)
```

Greater than or equal to operation comparing two values.

Location: sk5_cfx_dsp_relational_operators.h:65

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m9p47_t format
in	<i>b</i>	Second value in m9p47_t format

Return

Return true if the first value is greater than or equal to the second value, false otherwise

25.13.2.4 operator>=

```
bool operator>=(m8p16\_t a, m8p16\_t b)
```

Greater than or equal to operation comparing two values.

Location: sk5_cfx_dsp_relational_operators.h:79

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m8p16_t format
in	<i>b</i>	Second value in m8p16_t format

Return

Return true if the first value is greater than or equal to the second value, false otherwise

25.13.2.5 operator>=

```
bool operator>=(m8p40\_t a, m8p40\_t b)
```

Greater than or equal to operation comparing two values.

Location: sk5_cfx_dsp_relational_operators.h:93

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m8p40 t format
in	<i>b</i>	Second value in m8p40 t format

Return

Return true if the first value is greater than or equal to the second value, false otherwise

25.13.2.6 operator>=

```
bool operator>=(m16p40 t a, m16p40 t b)
```

Greater than or equal to operation comparing two values.

Location: sk5_cfx_dsp_relational_operators.h:107

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m16p40 t format
in	<i>b</i>	Second value in m16p40 t format

Return

Return true if the first value is greater than or equal to the second value, false otherwise

25.13.2.7 operator>

```
bool operator>(frac24\_t a, frac24\_t b)
```

Greater than operation comparing two values.

Location: sk5_cfx_dsp_relational_operators.h:121

Parameters

Direction	Name	Description
in	<i>a</i>	First value in frac24_t format
in	<i>b</i>	Second value in frac24_t format

Return

Return true if the first value is greater than the second value, false otherwise

25.13.2.8 operator>

```
bool operator>(frac48\_t a, frac48\_t b)
```

Greater than operation comparing two values.

Location: sk5_cfx_dsp_relational_operators.h:135

Parameters

Direction	Name	Description
in	<i>a</i>	First value in frac48_t format
in	<i>b</i>	Second value in frac48_t format

Return

Return true if the first value is greater than the second value, false otherwise

25.13.2.9 operator>

```
bool operator>(m9p47_t a, m9p47_t b)
```

Greater than operation comparing two values.

Location: sk5_cfx_dsp_relational_operators.h:149

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m9p47_t format
in	<i>b</i>	Second value in m9p47_t format

Return

Return true if the first value is greater than the second value, false otherwise

25.13.2.10 operator>

```
bool operator>(m8p16_t a, m8p16_t b)
```

Greater than operation comparing two values.

Location: sk5_cfx_dsp_relational_operators.h:163

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m8p16_t format
in	<i>b</i>	Second value in m8p16_t format

Return

Return true if the first value is greater than the second value, false otherwise

25.13.2.11 operator>;

```
bool operator>;(m8p40_t a, m8p40_t b)
```

Greater than operation comparing two values.

Location: sk5_cfx_dsp_relational_operators.h:177

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m8p40_t format
in	<i>b</i>	Second value in m8p40_t format

Return

Return true if the first value is greater than the second value, false otherwise

25.13.2.12 operator>;

```
bool operator>;(m16p40_t a, m16p40_t b)
```

Greater than operation comparing two values.

Location: sk5_cfx_dsp_relational_operators.h:191

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m16p40_t format
in	<i>b</i>	Second value in m16p40_t format

Return

Return true if the first value is greater than the second value, false otherwise

25.13.2.13 operator<=

```
bool operator<=(frac24\_t a, frac24\_t b)
```

Less than or equal to operation comparing two values.

Location: sk5_cfx_dsp_relational_operators.h:205

Parameters

Direction	Name	Description
in	<i>a</i>	First value in frac24_t format
in	<i>b</i>	Second value in frac24_t format

Return

Return true if the first value is less than or equal to the second value, false otherwise

25.13.2.14 operator<=

```
bool operator<=(frac48\_t a, frac48\_t b)
```

Less than or equal to operation comparing two values.

Location: sk5_cfx_dsp_relational_operators.h:219

Parameters

Direction	Name	Description
in	<i>a</i>	First value in frac48_t format
in	<i>b</i>	Second value in frac48_t format

Return

Return true if the first value is less than or equal to the second value, false otherwise

25.13.2.15 operator<=

```
bool operator<=(m9p47\_t a, m9p47\_t b)
```

Less than or equal to operation comparing two values.

Location: sk5_cfx_dsp_relational_operators.h:233

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m9p47_t format
in	<i>b</i>	Second value in m9p47_t format

Return

Return true if the first value is less than or equal to the second value, false otherwise

25.13.2.16 operator<=

```
bool operator<=(m8p16\_t a, m8p16\_t b)
```

Less than or equal to operation comparing two values.

Location: sk5_cfx_dsp_relational_operators.h:247

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m8p16_t format
in	<i>b</i>	Second value in m8p16_t format

Return

Return true if the first value is less than or equal to the second value, false otherwise

25.13.2.17 operator<=

```
bool operator<=(m8p40\_t a, m8p40\_t b)
```

Less than or equal to operation comparing two values.

Location: sk5_cfx_dsp_relational_operators.h:261

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m8p40_t format
in	<i>b</i>	Second value in m8p40_t format

Return

Return true if the first value is less than or equal to the second value, false otherwise

25.13.2.18 operator<=

```
bool operator<=(m16p40\_t a, m16p40\_t b)
```

Less than or equal to operation comparing two values.

Location: sk5_cfx_dsp_relational_operators.h:275

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m16p40_t format
in	<i>b</i>	Second value in m16p40_t format

Return

Return true if the first value is less than or equal to the second value, false otherwise

25.13.2.19 operator<

```
bool operator<(frac24\_t a, frac24\_t b)
```

Less than operation comparing two values.

Location: sk5_cfx_dsp_relational_operators.h:289

Parameters

Direction	Name	Description
in	<i>a</i>	First value in frac24_t format
in	<i>b</i>	Second value in frac24_t format

Return

Return true if the first value is less than the second value, false otherwise

25.13.2.20 operator<

```
bool operator<(frac48\_t a, frac48\_t b)
```

Less than operation comparing two values.

Location: sk5_cfx_dsp_relational_operators.h:303

Parameters

Direction	Name	Description
in	<i>a</i>	First value in frac48_t format
in	<i>b</i>	Second value in frac48_t format

Return

Return true if the first value is less than the second value, false otherwise

25.13.2.21 operator<

```
bool operator<(m9p47\_t a, m9p47\_t b)
```

Less than operation comparing two values.

Location: sk5_cfx_dsp_relational_operators.h:317

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m9p47_t format
in	<i>b</i>	Second value in m9p47_t format

Return

Return true if the first value is less than the second value, false otherwise

25.13.2.22 operator<

```
bool operator<(m8p16\_t a, m8p16\_t b)
```

Less than operation comparing two values.

Location: sk5_cfx_dsp_relational_operators.h:331

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m8p16_t format
in	<i>b</i>	Second value in m8p16_t format

Return

Return true if the first value is less than the second value, false otherwise

25.13.2.23 operator<

```
bool operator<(m8p40\_t a, m8p40\_t b)
```

Less than operation comparing two values.

Location: sk5_cfx_dsp_relational_operators.h:345

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m8p40_t format
in	<i>b</i>	Second value in m8p40_t format

Return

Return true if the first value is less than the second value, false otherwise

25.13.2.24 operator<

```
bool operator<(m16p40\_t a, m16p40\_t b)
```

Less than operation comparing two values.

Location: sk5_cfx_dsp_relational_operators.h:359

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m16p40_t format
in	<i>b</i>	Second value in m16p40_t format

Return

Return true if the first value is less than the second value, false otherwise

25.13.2.25 operator==

```
bool operator==(a, frac24\_t b)
```

Check if two values are equal to each other.

Location: sk5_cfx_dsp_relational_operators.h:373

Parameters

Direction	Name	Description
in	<i>a</i>	First value in frac24_t format
in	<i>b</i>	Second value in frac24_t format

Return

Return true if the first value is equal to the second value, false otherwise

25.13.2.26 operator==

```
bool operator==(frac48_t a, frac48_t b)
```

Check if two values are equal to each other.

Location: sk5_cfx_dsp_relational_operators.h:387

Parameters

Direction	Name	Description
in	<i>a</i>	First value in frac48_t format
in	<i>b</i>	Second value in frac48_t format

Return

Return true if the first value is equal to the second value, false otherwise

25.13.2.27 operator==

```
bool operator==(m9p47_t a, m9p47_t b)
```

Check if two values are equal to each other.

Location: sk5_cfx_dsp_relational_operators.h:401

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m9p47_t format
in	<i>b</i>	Second value in m9p47_t format

Return

Return true if the first value is equal to the second value, false otherwise

25.13.2.28 operator==

```
bool operator==(m8p16_t a, m8p16_t b)
```

Check if two values are equal to each other.

Location: sk5_cfx_dsp_relational_operators.h:415

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m8p16_t format
in	<i>b</i>	Second value in m8p16_t format

Return

Return true if the first value is equal to the second value, false otherwise

Ezairo 8300 Firmware Reference

25.13.2.29 operator==

```
bool operator==(m8p40_t a, m8p40_t b)
```

Check if two values are equal to each other.

Location: sk5_cfx_dsp_relational_operators.h:429

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m8p40_t format
in	<i>b</i>	Second value in m8p40_t format

Return

Return true if the first value is equal to the second value, false otherwise

25.13.2.30 operator==

```
bool operator==(m16p40_t a, m16p40_t b)
```

Check if two values are equal to each other.

Location: sk5_cfx_dsp_relational_operators.h:443

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m16p40_t format
in	<i>b</i>	Second value in m16p40_t format

Return

Ezairo 8300 Firmware Reference

Return true if the first value is equal to the second value, false otherwise

25.13.2.31 operator!=

```
bool operator!=(frac24\_t a, frac24\_t b)
```

Check if two values are not equal to each other.

Location: sk5_cfx_dsp_relational_operators.h:457

Parameters

Direction	Name	Description
in	<i>a</i>	First value in frac24_t format
in	<i>b</i>	Second value in frac24_t format

Return

Return true if the first value is not equal to the second value, false otherwise

25.13.2.32 operator!=

```
bool operator!=(frac48\_t a, frac48\_t b)
```

Check if two values are not equal to each other.

Location: sk5_cfx_dsp_relational_operators.h:471

Parameters

Direction	Name	Description
in	<i>a</i>	First value in frac48_t format
in	<i>b</i>	Second value in frac48_t format

Return

Return true if the first value is not equal to the second value, false otherwise

25.13.2.33 operator!=

```
bool operator!=(m9p47\_t a, m9p47\_t b)
```

Check if two values are not equal to each other.

Location: sk5_cfx_dsp_relational_operators.h:485

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m9p47_t format
in	<i>b</i>	Second value in m9p47_t format

Return

Return true if the first value is not equal to the second value, false otherwise

25.13.2.34 operator!=

```
bool operator!=(m8p16\_t a, m8p16\_t b)
```

Check if two values are not equal to each other.

Location: sk5_cfx_dsp_relational_operators.h:499

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m8p16_t format
in	<i>b</i>	Second value in m8p16_t format

Return

Return true if the first value is not equal to the second value, false otherwise

25.13.2.35 operator!=

```
bool operator!=(m8p40\_t a, m8p40\_t b)
```

Check if two values are not equal to each other.

Location: sk5_cfx_dsp_relational_operators.h:513

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m8p40_t format
in	<i>b</i>	Second value in m8p40_t format

Return

Return true if the first value is not equal to the second value, false otherwise

25.13.2.36 operator!=

```
bool operator!=(m16p40\_t a, m16p40\_t b)
```

Check if two values are not equal to each other.

Location: sk5_cfx_dsp_relational_operators.h:527

Parameters

Direction	Name	Description
in	<i>a</i>	First value in m16p40_t format
in	<i>b</i>	Second value in m16p40_t format

Return

Return true if the first value is not equal to the second value, false otherwise

25.14 SIGNAL PROCESSING

CFX DSP library signal processing functions.

25.14.1 Summary

25.15 FILTERING FUNCTIONS

CFX DSP library filtering functions.

25.15.1 Summary

Data Structures

- [AttackReleaseFilter](#) : Attack/release filter structure.
- [FIRFilter](#) : FIR filter structure.
- [Biquad](#) : Direct form I biquad filter structure.

Functions

- [cfx_attack_release_init](#) : Initialize [AttackReleaseFilter](#).
- [cfx_attack_release](#) : Calculate a first-order attack-release.
- [cfx_attack_release_vec](#) : Calculate a first-order attack-release on a vector input.
- [cfx_fir_init](#) : Initialize [FIRFilter](#).
- [cfx_fir](#) : FIR filter with an internally updated software FIFO.
- [cfx_fir](#) : FIR filter with an externally updated hardware FIFO.
- [cfx_biquad_init](#) : Initialize [Biquad](#).
- [cfx_biquad](#) : Direct form I biquad filter with internally updated filter history.

25.15.2 Filtering Functions Data Structures Type Documentation

25.15.2.1 AttackReleaseFilter

Location: sk5_cfx_dsp_signal_processing.h:49

Attack/release filter structure.

Used by [cfx_attack_release\(\)](#).

Data Fields

Type	Name	Description
frac24_t	<i>alpha</i>	Attack coefficient used when $x(n)$ is greater than or equal to $y(n-1)$
frac24_t	<i>beta</i>	Release coefficient used when $x(n)$ is less than $y(n-1)$
frac24_t	<i>y_n1</i>	Filter history $y(n-1)$

25.15.2.2 FIRFilter

Location: sk5_cfx_dsp_signal_processing.h:59

FIR filter structure.

Initialized by [cfx_fir_init\(\)](#) and used by [cfx_fir\(\)](#).

Data Fields

Type	Name	Description
frac24_t *	<i>bCoeffs</i>	Pointer to the filter coefficients.
frac24_t *	<i>state</i>	Pointer to the filter state buffer.

Ezairo 8300 Firmware Reference

		The input state buffer must be of length <code>block_size + num_taps - 1</code>
int	<i>numTaps</i>	Number of filter taps.
int	<i>blockSize</i>	Block size in samples.

25.15.2.3 Biquad

Location: `sk5_cfx_dsp_signal_processing.h:71`

Direct form I biquad filter structure.

Initialized by [cfx_biquad_init\(\)](#) and used by [cfx_biquad\(\)](#).

Data Fields

Type	Name	Description
frac24_t *	<i>bCoeffs</i>	Pointer to the forward-path filter coefficients [b0, b1, b2].
frac24_t *	<i>aCoeffs</i>	Pointer to the reverse-path filter coefficients [a1, a2].
frac24_t	<i>x_n1</i>	Filter history x(n-1)
frac24_t	<i>x_n2</i>	Filter history x(n-2)
frac24_t	<i>y_n1</i>	Filter history y(n-1)
frac24_t	<i>y_n2</i>	Filter history y(n-2)

25.15.3 Filtering Functions Function Documentation**25.15.3.1 cfx_attack_release_init**

```
void cfx_attack_release_init(AttackReleaseFilter * filter, frac24\_t alpha, frac24\_t beta)
```

Initialize [AttackReleaseFilter](#).

Location: `sk5_cfx_dsp_signal_processing.h:87`

Parameters

Ezairo 8300 Firmware Reference

Direction	Name	Description
	<i>filter</i>	
in	<i>alpha</i>	Attack coefficient used when $x(n)$ is greater than or equal to $y(n-1)$
in	<i>beta</i>	Release coefficient used when $x(n)$ is less than $y(n-1)$

Assumptions

None

25.15.3.2 cfx_attack_release

[frac24_t](#) cfx_attack_release([AttackReleaseFilter](#) * filter, [frac24_t](#) x_n)

Calculate a first-order attack-release.

The output of the attack-release filter is calculated as $\text{coeff} * x(n) + (1 - \text{coeff}) * y(n - 1)$.

Location: sk5_cfx_dsp_signal_processing.h:98

Parameters

Direction	Name	Description
	<i>filter</i>	
in	<i>x_n</i>	Input in frac24_t format

Return

Output in [frac24_t](#) format

Assumptions

Ezairo 8300 Firmware Reference

The filter has been initialized using [cfx_fir_init\(\)](#) prior to calling this function.

25.15.3.3 cfx_attack_release_vec

```
void cfx_attack_release_vec(AttackReleaseFilter * filter, frac24\_t * x_n, frac24\_t * y_n,
int size)
```

Calculate a first-order attack-release on a vector input.

See

Location: sk5_cfx_dsp_signal_processing.h:110

Parameters

Direction	Name	Description
	<i>filter</i>	
	<i>x_n</i>	
	<i>y_n</i>	
in	<i>size</i>	Number of values in the vector

[cfx_attack_release\(\)](#).

Assumptions

The filter has been initialized using [cfx_fir_init\(\)](#) prior to calling this function.

25.15.3.4 cfx_fir_init

```
void cfx_fir_init(FIRFilter * filter, frac24\_t * b_coeffs, volatile frac24\_t * state, const
int num_taps, const int block_size)
```

Initialize [FIRFilter](#).

Location: sk5_cfx_dsp_signal_processing.h:121

Parameters

Direction	Name	Description
	<i>filter</i>	
	<i>b_coeffs</i>	
	<i>state</i>	
in	<i>num_taps</i>	Number of filter taps
in	<i>block_size</i>	Block size in samples

25.15.3.5 cfx_fir

```
void cfx_fir()
```

FIR filter with an internally updated software FIFO.

Location: sk5_cfx_dsp_signal_processing.h:133

Assumptions

Input and output samples are ordered with the last sample in time at the start, and the first sample in time at the end. The filter has been initialized using [cfx_fir_init\(\)](#) prior to calling this function.

25.15.3.6 cfx_fir

```
void cfx_fir()
```

FIR filter with an externally updated hardware FIFO.

Location: sk5_cfx_dsp_signal_processing.h:146

Assumptions

Handling of input signal is completed externally prior to calling the FIR filter function. Input and output samples are ordered with the last sample in time at the start, and the first sample in time at the end. The filter has been initialized using [cfx_fir_init\(\)](#) prior to calling this function.

25.15.3.7 cfx_biquad_init

```
void cfx_biquad_init()
```

Initialize [Biquad](#).

Location: sk5_cfx_dsp_signal_processing.h:154

25.15.3.8 cfx_biquad

```
void cfx_biquad(Biquad * filter, frac24\_t * x, frac24\_t * y, int block_size)
```

Direct form I biquad filter with internally updated filter history.

Location: sk5_cfx_dsp_signal_processing.h:167

Parameters

Direction	Name	Description
	<i>filter</i>	
	<i>x</i>	
	<i>y</i>	
in	<i>block_size</i>	Block size in samples

Assumptions

Input and output samples are ordered with the last sample in time at the start, and the first sample in time at the end. The filter has been initialized using [cfx_biquad_init\(\)](#) prior to calling this function.

25.16 TRIGONOMETRIC FUNCTIONS

CFX DSP library trigonometric functions.

25.16.1 Summary

Functions

Ezairo 8300 Firmware Reference

- [cfx_sin](#) : Approximate sin().
- [cfx_sin_vec](#) : Approximate the sin() of each value in an array of inputs.
- [cfx_cos](#) : Approximate cos().
- [cfx_cos_vec](#) : Approximate the cos() of each value in an array of inputs.

25.16.2 Trigonometric Functions Function Documentation

25.16.2.1 cfx_sin

[frac24 t](#) cfx_sin([frac24 t](#) x)

Approximate sin().

Location: sk5_cfx_dsp_signal_processing.h:181

Parameters

Direction	Name	Description
in	x	Normalized input angle in the range [-1.0, 1.0) which maps to [-PI, PI)

Return

Output

25.16.2.2 cfx_sin_vec

void cfx_sin_vec([frac24 t](#) * x, [frac24 t](#) * y, int size)

Approximate the sin() of each value in an array of inputs.

Location: sk5_cfx_dsp_signal_processing.h:189

Parameters

Ezairo 8300 Firmware Reference

Direction	Name	Description
	<i>x</i>	
	<i>y</i>	
<i>in</i>	<i>size</i>	Number of values in the vector

25.16.2.3 cfx_cos

```
frac24\_t cfx_cos(frac24\_t x)
```

Approximate cos().

Location: sk5_cfx_dsp_signal_processing.h:196

Parameters

Direction	Name	Description
<i>in</i>	<i>x</i>	Normalized input angle in the range [-1.0, 1.0) which maps to [-PI, PI)

Return

Output

25.16.2.4 cfx_cos_vec

```
void cfx_cos_vec(frac24\_t * x, frac24\_t * y, int size)
```

Approximate the cos() of each value in an array of inputs.

Location: sk5_cfx_dsp_signal_processing.h:204

Parameters

Direction	Name	Description
	<i>x</i>	
	<i>y</i>	
in	<i>size</i>	Number of values in the vector

25.17 SIGNAL GENERATION FUNCTIONS

CFX DSP library signal generation functions.

25.17.1 Summary

Data Structures

- [ToneGenerator](#) : Tone generator structure.

Functions

- [cfx_tone_generator_init](#) : Initialize tone generator.
- [cfx_tone_generator](#) : Generates a tone at a desired amplitude and frequency.

25.17.2 Signal Generation Functions Data Structures Type Documentation

25.17.2.1 ToneGenerator

Location: sk5_cfx_dsp_signal_processing.h:217

Tone generator structure.

Used by [cfx_tone_generator\(\)](#).

Data Fields

Ezairo 8300 Firmware Reference

Type	Name	Description
frac24_t	<i>linearAmplitude</i>	Linear amplitude of the tone in frac24_t format. This can be modified between calls to cfx_tone_generator()
frac24_t	<i>normalizedFrequency</i>	Normalized frequency [frequency/(sample rate/2)]. This can be modified between calls to cfx_tone_generator()
m8p16_t	<i>currentPhase</i>	Current phase angle (maintained internally)

25.17.3 Signal Generation Functions Function Documentation

25.17.3.1 cfx_tone_generator_init

```
void cfx_tone_generator_init(ToneGenerator * signal_generator, m8p16\_t amplitude_dB,
frac24\_t normalized_frequency)
```

Initialize tone generator.

Location: sk5_cfx_dsp_signal_processing.h:230

Parameters

Direction	Name	Description
	<i>signal_generator</i>	
in	<i>amplitude_dB</i>	Amplitude in dB
in	<i>normalized_frequency</i>	Normalized frequency [frequency/(sample rate/2)]

Assumptions

Generated samples are ordered with the last sample

25.17.3.2 cfx_tone_generator

```
void cfx_tone_generator(ToneGenerator * signal_generator, frac24\_t * y, int size)
```

Generates a tone at a desired amplitude and frequency.

This function returns the next X samples every time it is called.

Location: sk5_cfx_dsp_signal_processing.h:243

Parameters

Direction	Name	Description
	<i>signal_generator</i>	
	<i>y</i>	
in	<i>size</i>	Number of values in the vector

Assumptions

Generated samples are ordered with the last sample in time at the start, and the first sample in time at the end. The generator has been initialized using [cfx_tone_generator_init\(\)](#) prior to calling this function.

25.18 INTERPOLATION FUNCTIONS

CFX DSP library interpolation functions.

25.18.1 Summary

Functions

- [cfx_linear_interpolation](#) : Linear interpolation.

25.18.2 Interpolation Functions Function Documentation

25.18.2.1 cfx_linear_interpolation

[frac24_t](#) cfx_linear_interpolation([frac24_t](#) x, [frac24_t](#) y0, [frac24_t](#) y1)

Linear interpolation.

The interpolated result is calculated as $y0 + x * (y1 - y0)$.

Location: sk5_cfx_dsp_signal_processing.h:260

Parameters

Direction	Name	Description
in	<i>x</i>	Index value
in	<i>y0</i>	Data point
in	<i>y1</i>	Data point

Return

Interpolation result

25.19 FIXED-POINT TYPE CONVERSION

CFX DSP library functions for converting between fixed-point data types.

25.19.1 Summary

Macros

- [to_frac28](#) : frac28 initialization macro for use with the Filter Engine.
- [to_m2p26](#) : m2p26 initialization macro for use with the Filter Engine.
- [to_frac24](#) : frac24 initialization macro for use with the Filter Engine.
- [to_m2p22](#) : m2p22 initialization macro for use with the Filter Engine.
- [to_m8p16](#) : m8p16 initialization macro for use with the Filter Engine.
- [to_m4p50](#) : m4p50 initialization macro for use with the Filter Engine.
- [to_m11p43](#) : m11p43 initialization macro for use with the Filter Engine.

Functions

- [cfx to_frac24](#) : Convert to [frac24 t](#) format.
- [cfx to_frac24](#) : Convert to [frac24 t](#) format.

Ezairo 8300 Firmware Reference

- [cfx to frac24](#) : Convert to [frac24 t](#) format.
- [cfx to frac24](#) : Convert to [frac24 t](#) format.
- [cfx to frac24](#) : Convert to [frac24 t](#) format.
- [cfx to frac24](#) : Convert to [frac24 t](#) format.
- [cfx to frac48](#) : Convert to [frac48 t](#) format.
- [cfx to frac48](#) : Convert to [frac48 t](#) format.
- [cfx to frac48](#) : Convert to [frac48 t](#) format.
- [cfx to frac48](#) : Convert to [frac48 t](#) format.
- [cfx to frac48](#) : Convert to [frac48 t](#) format.
- [cfx to frac48](#) : Convert to [frac48 t](#) format.
- [cfx to m9p47](#) : Convert to [m9p47 t](#) format.
- [cfx to m9p47](#) : Convert to [m9p47 t](#) format.
- [cfx to m9p47](#) : Convert to [m9p47 t](#) format.
- [cfx to m9p47](#) : Convert to [m9p47 t](#) format.
- [cfx to m9p47](#) : Convert to [m9p47 t](#) format.
- [cfx to m9p47](#) : Convert to [m9p47 t](#) format.
- [cfx to m9p47](#) : Convert to [m9p47 t](#) format.
- [cfx to m8p16](#) : Convert to [m8p16 t](#) format.
- [cfx to m8p16](#) : Convert to [m8p16 t](#) format.
- [cfx to m8p16](#) : Convert to [m8p16 t](#) format.
- [cfx to m8p16](#) : Convert to [m8p16 t](#) format.
- [cfx to m8p16](#) : Convert to [m8p16 t](#) format.
- [cfx to m8p16](#) : Convert to [m8p16 t](#) format.
- [cfx to m8p16](#) : Convert to [m8p16 t](#) format.
- [cfx to m8p40](#) : Convert to [m8p40 t](#) format.
- [cfx to m8p40](#) : Convert to [m8p40 t](#) format.
- [cfx to m8p40](#) : Convert to [m8p40 t](#) format.
- [cfx to m8p40](#) : Convert to [m8p40 t](#) format.
- [cfx to m8p40](#) : Convert to [m8p40 t](#) format.
- [cfx to m8p40](#) : Convert to [m8p40 t](#) format.
- [cfx to m16p40](#) : Convert to [m16p40 t](#) format.
- [cfx to m16p40](#) : Convert to [m16p40 t](#) format.
- [cfx to m16p40](#) : Convert to [m16p40 t](#) format.
- [cfx to m16p40](#) : Convert to [m16p40 t](#) format.
- [cfx to m16p40](#) : Convert to [m16p40 t](#) format.
- [cfx to m16p40](#) : Convert to [m16p40 t](#) format.
- [cfx as frac24](#) : Reinterpret int as [frac24 t](#).
- [cfx as frac48](#) : Reinterpret long as [frac48 t](#).
- [cfx as int](#) : Reinterpret [frac24 t](#) as int.
- [cfx as long](#) : Reinterpret [frac48 t](#) as long.

25.19.2 Detailed Description

A full set of fixed-point data type conversion functions are available. Some conversions can be done without losing any bits (i.e. [frac24 t](#) to [frac48 t](#)). Other conversions result in a loss of bits (i.e. [m8p16 t](#) to [frac24 t](#)). In the case where bits are lost, the binary point is aligned and data goes through the appropriate rounding, saturation, and selection (RSS) units. For example, an [m8p16 t](#) value of -6.5 that is converted to [frac24 t](#) using [cfx to frac24\(\)](#) will be saturated as -1.0, the largest negative value that can be encoded using a [frac24 t](#). A [frac24 t](#) value of 0.5 that is converted to [m8p16 t](#) using [cfx to m8p16\(\)](#) will still be 0.5 because it can be encoded in both types without loss of bits. It is recommended that 56 bit data types are used for intermediate calculations in order to minimize the chance of overflow where necessary, and that final calculation values are converted to the appropriate 24 or 48 bit data

Ezairo 8300 Firmware Reference

types using the Fixed-Point Type Conversion functions before being stored in memory. This will improve execution speed and minimize memory use.

Reinterpretation Functions

In some cases it is useful to be able to reinterpret an int/long as a fixed-point value, or a fixed-point value as an int/long. The underlying data is not changed. Several functions are provided to allow for this. For example: int a = 0x800000;

[frac24_t](#) b = cfx_as_frac24(a); // re-interpreted as -1.0 [frac48_t](#) c = 1.0;

long d = cfx_as_long(c); // re-interpreted as 0x7FFFFFFFFF;

25.19.3 Fixed-Point Type Conversion Macro Definition Documentation**25.19.3.1 to_frac28**

```
#define to_frac28 (int32_t) (to_long(lfix(f))>>20)
```

frac28 initialization macro for use with the Filter Engine.

Location: sk5_cfx_dsp_type_conversions.h:76

Parameters

Direction	Name	Description
in	<i>f</i>	Input decimal value

Return

Right-aligned s1m0p27 as an int32_t

25.19.3.2 to_m2p26

```
#define to_m2p26 (int32_t) (to_long((lfix)acc((f)/2.0))>>20)
```

m2p26 initialization macro for use with the Filter Engine.

Location: sk5_cfx_dsp_type_conversions.h:83

Parameters

Direction	Name	Description
in	<i>f</i>	Input decimal value

Return

Right-aligned s1m1p26 as an int32_t

25.19.3.3 to_frac24

```
#define to_frac24 (int32_t) (to_int(fix(f)))
```

frac24 initialization macro for use with the Filter Engine.

Location: sk5_cfx_dsp_type_conversions.h:90

Parameters

Direction	Name	Description
in	<i>f</i>	Input decimal value

Return

Right-aligned s1m0p23 as an int32_t

25.19.3.4 to_m2p22

```
#define to_m2p22 (int32_t) (to_int((fix)acc((f)/2.0)))
```

m2p22 initialization macro for use with the Filter Engine.

Location: sk5_cfx_dsp_type_conversions.h:97

Parameters

Direction	Name	Description
in	<i>f</i>	Input decimal value

Return

Right-aligned s1m1p22 as an int32_t

25.19.3.5 to_m8p16

```
#define to_m8p16 (int32_t) (to_int((fix)acc((f)/128.0)))
```

m8p16 initialization macro for use with the Filter Engine.

Location: sk5_cfx_dsp_type_conversions.h:104

Parameters

Direction	Name	Description
in	<i>f</i>	Input decimal value

Return

Right-aligned s1m7p16 as an int32_t

25.19.3.6 to_m4p50

```
#define to_m4p50 (int32_t) (to_long((lfix)acc((f)/8.0))>>21)
```

m4p50 initialization macro for use with the Filter Engine.

This only initializes the 27 most significant bits.

Location: sk5_cfx_dsp_type_conversions.h:112

Parameters

Direction	Name	Description
in	<i>f</i>	Input decimal value

Return

Right-aligned s1m3p23 as an int32_t.

25.19.3.7 to_m11p43

```
#define to_m11p43 (int32_t) (to_long((lfix)acc((f)/1024.0))>>21)
```

m11p43 initialization macro for use with the Filter Engine.

This only initializes the 27 most significant bits.

Location: sk5_cfx_dsp_type_conversions.h:120

Parameters

Direction	Name	Description
in	<i>f</i>	Input decimal value

Return

Right-aligned s1m10p16 as an int32_t.

25.19.4 Fixed-Point Type Conversion Function Documentation

25.19.4.1 cfx_to_frac24

[frac24_t](#) cfx_to_frac24([frac48_t](#) a)

Convert to [frac24_t](#) format.

Location: sk5_cfx_dsp_type_conversions.h:127

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in frac48_t format

Return

Converted value in [frac24_t](#) format

25.19.4.2 cfx_to_frac24

[frac24_t](#) cfx_to_frac24([m9p47_t](#) a)

Convert to [frac24_t](#) format.

Location: sk5_cfx_dsp_type_conversions.h:139

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in m9p47_t format

Return

Converted value in [frac24_t](#) format

25.19.4.3 cfx_to_frac24

[frac24_t](#) cfx_to_frac24([m8p16_t](#) a)

Convert to [frac24_t](#) format.

Location: sk5_cfx_dsp_type_conversions.h:151

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in m8p16_t format

Return

Converted value in [frac24_t](#) format

25.19.4.4 cfx_to_frac24

[frac24_t](#) cfx_to_frac24([m8p40_t](#) a)

Convert to [frac24_t](#) format.

Location: sk5_cfx_dsp_type_conversions.h:163

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in m8p40_t format

Return

Converted value in [frac24_t](#) format

25.19.4.5 cfx_to_frac24

[frac24_t](#) cfx_to_frac24([m16p40_t](#) a)

Convert to [frac24_t](#) format.

Location: sk5_cfx_dsp_type_conversions.h:175

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in m16p40_t format

Return

Converted value in [frac24_t](#) format

25.19.4.6 cfx_to_frac24

[frac24_t](#) cfx_to_frac24([m24p24_t](#) a)

Convert to [frac24_t](#) format.

Location: sk5_cfx_dsp_type_conversions.h:187

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in m24p24_t format

Return

Converted value in [frac24_t](#) format

25.19.4.7 cfx_to_frac48

[frac48_t](#) cfx_to_frac48([frac24_t](#) a)

Convert to [frac48_t](#) format.

Location: sk5_cfx_dsp_type_conversions.h:212

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in frac24_t format

Return

Converted value in [frac48_t](#) format

25.19.4.8 cfx_to_frac48

[frac48_t](#) cfx_to_frac48([m9p47_t](#) a)

Convert to [frac48_t](#) format.

Location: sk5_cfx_dsp_type_conversions.h:224

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in m9p47_t format

Return

Converted value in [frac48_t](#) format

25.19.4.9 cfx_to_frac48

[frac48_t](#) cfx_to_frac48([m8p16_t](#) a)

Convert to [frac48_t](#) format.

Location: sk5_cfx_dsp_type_conversions.h:236

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in m8p16_t format

Return

Converted value in [frac48_t](#) format

25.19.4.10 cfx_to_frac48

[frac48_t](#) cfx_to_frac48([m8p40_t](#) a)

Convert to [frac48_t](#) format.

Location: sk5_cfx_dsp_type_conversions.h:248

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in m8p40_t format

Return

Converted value in [frac48_t](#) format

25.19.4.11 cfx_to_frac48

[frac48_t](#) cfx_to_frac48([m16p40_t](#) a)

Convert to [frac48_t](#) format.

Location: sk5_cfx_dsp_type_conversions.h:260

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in m16p40_t format

Return

Converted value in [frac48_t](#) format

25.19.4.12 cfx_to_frac48

[frac48_t](#) cfx_to_frac48([m24p24_t](#) a)

Convert to [frac48_t](#) format.

Location: sk5_cfx_dsp_type_conversions.h:272

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in m24p24_t format

Return

Converted value in [frac48_t](#) format

25.19.4.13 cfx_to_m9p47

[m9p47 t](#) cfx_to_m9p47([frac24 t](#) a)

Convert to [m9p47 t](#) format.

Location: sk5_cfx_dsp_type_conversions.h:297

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in frac24 t format

Return

Converted value in [m9p47 t](#) format

25.19.4.14 cfx_to_m9p47

[m9p47 t](#) cfx_to_m9p47([frac48 t](#) a)

Convert to [m9p47 t](#) format.

Location: sk5_cfx_dsp_type_conversions.h:309

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in frac48 t format

Return

Converted value in [m9p47 t](#) format

25.19.4.15 cfx_to_m9p47

[m9p47_t](#) cfx_to_m9p47([m8p16_t](#) a)

Convert to [m9p47_t](#) format.

Location: sk5_cfx_dsp_type_conversions.h:321

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in m8p16_t format

Return

Converted value in [m9p47_t](#) format

25.19.4.16 cfx_to_m9p47

[m9p47_t](#) cfx_to_m9p47([m8p40_t](#) a)

Convert to [m9p47_t](#) format.

Location: sk5_cfx_dsp_type_conversions.h:333

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in m8p40_t format

Return

Converted value in [m9p47_t](#) format

25.19.4.17 cfx_to_m9p47

[m9p47 t](#) cfx_to_m9p47 ([m16p40 t](#) a)

Convert to [m9p47 t](#) format.

Location: sk5_cfx_dsp_type_conversions.h:345

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in m16p40 t format

Return

Converted value in [m9p47 t](#) format

25.19.4.18 cfx_to_m9p47

[m9p47 t](#) cfx_to_m9p47 ([m24p24 t](#) a)

Convert to [m9p47 t](#) format.

Location: sk5_cfx_dsp_type_conversions.h:357

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in m24p24 t format

Return

Converted value in [m9p47 t](#) format

25.19.4.19 cfx_to_m8p16

[m8p16 t](#) cfx_to_m8p16([frac24 t](#) a)

Convert to [m8p16 t](#) format.

Location: sk5_cfx_dsp_type_conversions.h:384

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in frac24 t format

Return

Converted value in [m8p16 t](#) format

25.19.4.20 cfx_to_m8p16

[m8p16 t](#) cfx_to_m8p16([frac48 t](#) a)

Convert to [m8p16 t](#) format.

Location: sk5_cfx_dsp_type_conversions.h:396

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in frac48 t format

Return

Converted value in [m8p16 t](#) format

25.19.4.21 cfx_to_m8p16

[m8p16 t](#) cfx_to_m8p16([m9p47 t](#) a)

Convert to [m8p16 t](#) format.

Location: sk5_cfx_dsp_type_conversions.h:408

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in m9p47 t format

Return

Converted value in [m8p16 t](#) format

25.19.4.22 cfx_to_m8p16

[m8p16 t](#) cfx_to_m8p16([m8p40 t](#) a)

Convert to [m8p16 t](#) format.

Location: sk5_cfx_dsp_type_conversions.h:420

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in m8p40 t format

Return

Converted value in [m8p16 t](#) format

25.19.4.23 cfx_to_m8p16

[m8p16 t](#) cfx_to_m8p16([m16p40 t](#) a)

Convert to [m8p16 t](#) format.

Location: sk5_cfx_dsp_type_conversions.h:432

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in m16p40 t format

Return

Converted value in [m8p16 t](#) format

25.19.4.24 cfx_to_m8p16

[m8p16 t](#) cfx_to_m8p16([m24p24 t](#) a)

Convert to [m8p16 t](#) format.

Location: sk5_cfx_dsp_type_conversions.h:444

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in m24p24 t format

Return

Converted value in [m8p16 t](#) format

25.19.4.25 cfx_to_m8p40

[m8p40_t](#) cfx_to_m8p40([frac24_t](#) a)

Convert to [m8p40_t](#) format.

Location: sk5_cfx_dsp_type_conversions.h:469

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in frac24_t format

Return

Converted value in [m8p40_t](#) format

25.19.4.26 cfx_to_m8p40

[m8p40_t](#) cfx_to_m8p40([frac48_t](#) a)

Convert to [m8p40_t](#) format.

Location: sk5_cfx_dsp_type_conversions.h:481

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in frac48_t format

Return

Converted value in [m8p40_t](#) format

25.19.4.27 cfx_to_m8p40

[m8p40_t](#) cfx_to_m8p40([m9p47_t](#) a)

Convert to [m8p40_t](#) format.

Location: sk5_cfx_dsp_type_conversions.h:493

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in m9p47_t format

Return

Converted value in [m8p40_t](#) format

25.19.4.28 cfx_to_m8p40

[m8p40_t](#) cfx_to_m8p40([m8p16_t](#) a)

Convert to [m8p40_t](#) format.

Location: sk5_cfx_dsp_type_conversions.h:505

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in m8p16_t format

Return

Converted value in [m8p40_t](#) format

25.19.4.29 cfx_to_m8p40

[m8p40_t](#) cfx_to_m8p40([m16p40_t](#) a)

Convert to [m8p40_t](#) format.

Location: sk5_cfx_dsp_type_conversions.h:517

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in m16p40_t format

Return

Converted value in [m8p40_t](#) format

25.19.4.30 cfx_to_m8p40

[m8p40_t](#) cfx_to_m8p40([m24p24_t](#) a)

Convert to [m8p40_t](#) format.

Location: sk5_cfx_dsp_type_conversions.h:529

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in m24p24_t format

Return

Converted value in [m8p40_t](#) format

25.19.4.31 cfx_to_m16p40

[m16p40_t](#) cfx_to_m16p40 ([frac24_t](#) a)

Convert to [m16p40_t](#) format.

Location: sk5_cfx_dsp_type_conversions.h:554

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in frac24_t format

Return

Converted value in [m16p40_t](#) format

25.19.4.32 cfx_to_m16p40

[m16p40_t](#) cfx_to_m16p40 ([frac48_t](#) a)

Convert to [m16p40_t](#) format.

Location: sk5_cfx_dsp_type_conversions.h:566

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in frac48_t format

Return

Converted value in [m16p40_t](#) format

25.19.4.33 cfx_to_m16p40

[m16p40_t](#) cfx_to_m16p40 ([m9p47_t](#) a)

Convert to [m16p40_t](#) format.

Location: sk5_cfx_dsp_type_conversions.h:578

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in m9p47_t format

Return

Converted value in [m16p40_t](#) format

25.19.4.34 cfx_to_m16p40

[m16p40_t](#) cfx_to_m16p40 ([m8p16_t](#) a)

Convert to [m16p40_t](#) format.

Location: sk5_cfx_dsp_type_conversions.h:590

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in m8p16_t format

Return

Converted value in [m16p40_t](#) format

25.19.4.35 cfx_to_m16p40

[m16p40_t](#) cfx_to_m16p40 ([m8p40_t](#) a)

Convert to [m16p40_t](#) format.

Location: sk5_cfx_dsp_type_conversions.h:602

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in m8p40_t format

Return

Converted value in [m16p40_t](#) format

25.19.4.36 cfx_to_m16p40

[m16p40_t](#) cfx_to_m16p40 ([m24p24_t](#) a)

Convert to [m16p40_t](#) format.

Location: sk5_cfx_dsp_type_conversions.h:614

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in m24p24_t format

Return

Converted value in [m16p40_t](#) format

25.19.4.37 cfx_as_frac24

[frac24_t](#) cfx_as_frac24(int a)

Reinterpret int as [frac24_t](#).

Location: sk5_cfx_dsp_type_conversions.h:641

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in int format

Return

Reinterpreted value in [frac24_t](#) format

25.19.4.38 cfx_as_frac48

[frac48_t](#) cfx_as_frac48(long a)

Reinterpret long as [frac48_t](#).

Location: sk5_cfx_dsp_type_conversions.h:653

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in long format

Return

Reinterpreted value in [frac48_t](#) format

25.19.4.39 cfx_as_int

```
int cfx_as_int(frac24\_t a)
```

Reinterpret [frac24_t](#) as int.

Location: sk5_cfx_dsp_type_conversions.h:665

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in frac24_t format

Return

Reinterpreted value as int

25.19.4.40 cfx_as_long

```
long cfx_as_long(frac48\_t a)
```

Reinterpret [frac48_t](#) as long.

Location: sk5_cfx_dsp_type_conversions.h:677

Parameters

Direction	Name	Description
in	<i>a</i>	Input value in frac48_t format

Return

Reinterpreted value as long

Ezairo 8300 Firmware Reference

25.20 FIXED-POINT TYPES

CFX DSP library fixed-point data types.

25.20.1 Summary

Data Structures

- [frac24_t](#) : Signed fixed-point data type with 1 integer bit and 23 fractional bits.
- [frac48_t](#) : Signed fixed-point data type with 1 integer bit and 47 fractional bits.
- [m9p47_t](#) : Signed fixed-point data type with 9 integer bit and 47 fractional bits.
- [m8p16_t](#) : Signed fixed-point data type with 8 integer bit and 16 fractional bits.
- [m8p40_t](#) : Signed fixed-point data type with 8 integer bit and 40 fractional bits.
- [m16p40_t](#) : Signed fixed-point data type with 16 integer bit and 40 fractional bits.
- [m24p24_t](#) : Signed fixed-point data type with 24 integer bit and 24 fractional bits.

25.20.2 Detailed Description

A fixed-point data type encodes a real number using a fixed number of bits before and after the decimal point. Every fixed-point data type provided is signed. Fixed-point data types can be initialized and used directly like any other basic type. It is recommended that 56 bit data types are used for intermediate calculations in order to minimize the chance of overflow where necessary, and that final calculation values are converted to the appropriate 24 or 48 bit data types using the Fixed-Point Type Conversion functions before being stored in memory. This will improve execution speed and minimize memory use.

Fixed-Point Data Types

The available fixed-point data types are summarized below:

Name	Bits	Range
frac24_t	24	$[-1, 1 - 2^{-23}]$
frac48_t	48	$[-1, 1 - 2^{-47}]$
m9p47_t	56	$[-2^8, 2^8 - 2^{-47}]$
m8p16_t	24	$[-2^7, 2^7 - 2^{-16}]$
m8p40_t	48	$[-2^7, 2^7 - 2^{-40}]$
m16p40_t ¹	56	$[-2^{15}, 2^{15} - 2^{-40}]$
m24p24_t ²	48	$[-2^{23}, 2^{23} - 2^{-24}]$

¹Note that [m16p40_t](#) initialization is limited to a range of $[-2^8, 2^8 - 2^{-40}]$.

²Note that [m24p24_t](#) is provided with only limited library support as the output data type of the [cfx_div_m24p24\(\)](#) function.

Fixed-Point Data Type Initialization

Fixed-point data types can be initialized using floating-point numbers and int/long literal values (where the value is stored directly). For example: [frac24_t](#) a = 0.5;

[frac48_t](#) b = 0x7FFFFFFFFFFFFF;

[m8p16_t](#) c = 12.5;

[frac48_t](#) d[] = { -1.0, 3, 0x1 }; // the first value is initialized as a floating-point number and the second and last

Ezairo 8300 Firmware Reference

values are reinterpreted as long literal values. Note that the compiler may raise an error if a floating-point number that does not fully encode into the number of available fractional bits is used for [m8p16_t](#), [m8p40_t](#), or [m16p40_t](#) data types when they are declared as static or global.

Memory Space and Address Selection

By default, all variables are stored in XMEM. The memory space of a variable can be specified explicitly using `_XMEM`, `_YMEM`, `_XYMEM`, or `_IOMEM`. Note that only 48 bit data types can be stored using `_XYMEM`, and that this allows for a more efficient read/write of 48 bit data. For example: [frac24_t](#) `_XMEM a = 0.5;`
[frac48_t](#) `_XYMEM b = 0x7FFFFFFFFF;`
[m8p16_t](#) `_YMEM c = 12.5;`
[m8p40_t](#) `_XYMEM d[] = { -6.25, 3, 0x1 };` // the first value is initialized as a floating-point number and the second and last values are reinterpreted as long literal values. If pointers are used to any memory space other than XMEM, take care to declare them appropriately. For example: [frac48_t](#) `_XYMEM *ptr;` // pointer to [frac48_t](#) stored in XMEM. It is also possible to specify the memory location of a variable using `_XADDR`, `_YADDR`, `_XYADDR`, or `_IOMEM`. 24 bit data types can be stored at any address. 56 and 48 bit data types can only be stored starting at an even address. For example: [frac24_t](#) `_XADDR(25) a = 0.5;` // store a in address 25 of XMEM. Global and static variables are pre-calculated and stored in memory. Stack variables are initialized at run-time.

25.20.3 Fixed-Point Types Data Structures Type Documentation**25.20.3.1 frac24_t**

Location: `sk5_cfx_dsp_types.h:125`

Signed fixed-point data type with 1 integer bit and 23 fractional bits.

[frac24_t](#) variables can be initialized using floating-point numbers in the range $[-1, 1 - 2^{-23}]$, and int literal values (where the value is stored directly). An initial value of 1.0 is accepted for convenience but will be converted to $1 - 2^{-23}$ because this is outside of the mathematical range of this fixed-point format.

Example Code for frac24_t

```
frac24_t a = 0.5;
frac24_t b = 0x7FFFFFFF;
frac24_t c[] = { 0.25, -0.5, 1.0, 0x1 };
```

Data Fields

Type	Name	Description
------	------	-------------

25.20.3.2 frac48_t

Location: sk5_cfx_dsp_types.h:159

Signed fixed-point data type with 1 integer bit and 47 fractional bits.

[frac48_t](#) variables can be initialized using floating-point numbers in the range $[-1, 1 - 2^{-47}]$, and long literal values (where the value is stored directly). An initial value of 1.0 is accepted for convenience but will be converted to $1 - 2^{-47}$ because this is outside of the mathematical range of this fixed-point format.

Example Code for frac48_t

```
frac48_t a = 0.5;  
frac48_t b = 0x7FFFFFFFFFFFFF;  
frac48_t c[] = { 0.25, -0.5, 1.0, 0x1 };
```

Data Fields

Type	Name	Description
------	------	-------------

25.20.3.3 m9p47_t

Location: sk5_cfx_dsp_types.h:204

Signed fixed-point data type with 9 integer bit and 47 fractional bits.

[m9p47_t](#) variables can be initialized using floating-point numbers in the range $[-2^8, 2^8 - 2^{-47}]$, and long literal values (where the value is stored directly). An initial value of 256.0 is accepted for convenience but will be converted to $2^8 - 2^{-47}$ because this is outside of the mathematical range of this fixed-point format. This type is useful for intermediate calculations.

Example Code for m9p47_t

```
m9p47_t a = 6.5;  
m9p47_t b = 0x7FFFFFFFFFFFFF;  
m9p47_t c[] = { 6.25, -0.5, 1.0, 0x1 };
```

Data Fields

Type	Name	Description
------	------	-------------

25.20.3.4 m8p16_t

Location: sk5_cfx_dsp_types.h:253

Signed fixed-point data type with 8 integer bit and 16 fractional bits.

[m8p16_t](#) variables can be initialized using floating-point numbers in the range $[-2^7, 2^7 - 2^{-16}]$, and int literal values (where the value is stored directly). An initial value of 128.0 is accepted for convenience but will be converted to $2^7 - 2^{-16}$ because this is outside of the mathematical range of this fixed-point format.

Example Code for m8p16_t

```
m8p16_t a = 6.5  
m8p16_t b = 0x7FFFFFFF  
m8p16_t c[] = { 6.25, -0.5, 1.0, 0x1 };
```

Data Fields

Type	Name	Description
------	------	-------------

25.20.3.5 m8p40_t

Location: sk5_cfx_dsp_types.h:299

Signed fixed-point data type with 8 integer bit and 40 fractional bits.

[m8p40_t](#) variables can be initialized using floating-point numbers in the range $[-2^7, 2^7 - 2^{-40}]$, and long literal values (where the value is stored directly). An initial value of 128.0 is accepted for convenience but will be converted to $2^7 - 2^{-40}$ because this is outside of the mathematical range of this fixed-point format.

Example Code for m8p40_t

```
m8p40_t a = 6.5
m8p40_t b = 0x7FFFFFFFFFFFFF
m8p40_t c[] = { 6.25, -0.5, 1.0, 0x1 };
```

Data Fields

Type	Name	Description
------	------	-------------

25.20.3.6 m16p40_t

Location: sk5_cfx_dsp_types.h:356

Signed fixed-point data type with 16 integer bit and 40 fractional bits.

[m16p40_t](#) variables can be initialized using floating-point numbers in the range $[-2^8, 2^8 - 2^{-40}]$, and long literal values (where the value is stored directly). [m16p40_t](#) initialization is limited to a range of $[-2^8, 2^8 - 2^{-40}]$. An initial value of 256.0 is accepted for convenience but will be converted to $2^8 - 2^{-40}$. This type is useful for intermediate calculations.

Example Code for m16p40_t

```
m16p40_t a = 6.5
m16p40_t b = 0x7FFFFFFFFFFFFF
m16p40_t c[] = { 6.25, -0.5, 1.0, 0x1 };
```

Data Fields

Type	Name	Description
------	------	-------------

25.20.3.7 m24p24_t

Location: sk5_cfx_dsp_types.h:406

Signed fixed-point data type with 24 integer bit and 24 fractional bits.

m24p42_t variables can be initialized using floating-point numbers in the range $[-2^8, 2^8 - 2^{-24}]$, and long literal values (where the value is stored directly). An initial value of 256.0 is accepted for convenience but will be converted to $2^8 - 2^{-24}$. Note that this type is provided with only limited support as the output of the [cfx_div_m24p24\(\)](#) function. It is recommended that you convert this to the appropriate 24 or 48 bit data type using the Fixed-Point Type Conversion functions.

Example Code for m24p24_t

```
m24p42_t a = 6.5  
m24p42_t b = 0x7FFFFFFFFFFFFF  
m24p42_t c[] = { 6.25, -0.5, 1.0, 0x1 };
```

Data Fields

Type	Name	Description
------	------	-------------

APPENDIX A

Glossary

The following abbreviations and terms are used in this group of topics:

<i>ACL</i>	asynchronous connection-oriented logical transport
<i>ADC</i>	analog-to-digital converter
<i>Arm Cortex-M3 processor</i>	processing core that can be used to configure the overall system after initial boot, and is typically used to coordinate the system interaction with external components
<i>CFX Digital Signal Processor</i>	processing core that is used to configure the system at initial boot; this core can also be used to configure the overall system, and coordinate the flow of signal data progressing through the system
<i>CMSIS</i>	Cortex Microcontroller Software Interface Standard
<i>CRC</i>	cyclic redundancy check
<i>DAC</i>	digital-to-analog converter
<i>DIO</i>	digital input/output
<i>DMA</i>	direct memory access
<i>DSP</i>	digital signal processor
<i>GPIO</i>	general-purpose input/output
<i>I²C</i>	inter-IC communication protocol
<i>JTAG</i>	joint test action group (developer of IEEE standard 1149.1-1990)
<i>LSB</i>	least significant bit
<i>MSB</i>	

	most significant bit
<i>MUX</i>	multiplexer, selector of one signal from many
<i>NVIC</i>	nested vectored interrupt controller
<i>NVM</i>	non-volatile memory
<i>PCM</i>	pulse code modulation
<i>RAM</i>	random-access memory
<i>ROM</i>	read-only memory
<i>SCL</i>	serial clock (part of an I ² C or I ³ C bus)
<i>SDA</i>	serial data (part of an I ² C or I ³ C bus)
<i>SPI</i>	Serial Peripheral Interface
<i>SWJ-DP</i>	serial wire and JTAG debug port
<i>UART</i>	universal asynchronous receiver-transmitter
<i>VDDA</i>	supply voltage for the non-RF analog blocks and flash memory
<i>VDDC</i>	supply voltage for the digital logic
<i>VDDO*</i>	the four input supplies for the digital I/O pads, including the debug ports (I ² C, SWJ-DP)
<i>VDD_XTAL</i>	crystal voltage domain
<i>VDDIF</i>	supply voltage supporting external components and their interfaces

<i>VDDM</i>	supply voltage for the memories
<i>VDDOD</i>	supply voltage for the output drivers
<i>VMIC</i>	supply voltage for microphone units
<i>WDF</i>	wave digital filter
<i>XMEM</i>	X memory, one of two banks of 24-bit memory attached to the CFX DSP
<i>YMEM</i>	Y memory, one of two banks of 24-bit memory attached to the CFX DSP

onsemi Confidential
Ezairo 8300 Firmware Reference

Ezairo and SignaKlara are either trademarks or registered trademarks of SCILLC. All other brand names and product names appearing in this document are trademarks of their respective holders.

onsemi and the onsemi logo are trademarks of Semiconductor Components Industries, LLC dba onsemi or its subsidiaries in the United States and/or other countries. onsemi owns the rights to a number of patents, trademarks, copyrights, trade secrets, and other intellectual property. A listing of onsemi's product/patent coverage may be accessed at www.onsemi.com/site/pdf/Patent-Marking.pdf. onsemi reserves the right to make changes without further notice to any products herein. onsemi makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does onsemi assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation special, consequential or incidental damages. Buyer is responsible for its products and applications using onsemi products, including compliance with all laws, regulations and safety requirements or standards, regardless of any support or applications information provided by onsemi. "Typical" parameters which may be provided in onsemi data sheets and/or specifications can and do vary in different applications and actual performance may vary over time. All operating parameters, including "Typicals" must be validated for each customer application by customer's technical experts. onsemi does not convey any license under its patent rights nor the rights of others. onsemi products are not designed, intended, or authorized for use as a critical component in life support systems or any FDA Class 3 medical devices or medical devices with a same or similar classification in a foreign jurisdiction or any devices intended for implantation in the human body. Should Buyer purchase or use onsemi products for any such unintended or unauthorized application, Buyer shall indemnify and hold onsemi and its officers, employees, subsidiaries, affiliates, and distributors harmless against all claims, costs, damages, and expenses, and reasonable attorney fees arising out of, directly or indirectly, any claim of personal injury or death associated with such unintended or unauthorized use, even if such claim alleges that onsemi was negligent regarding the design or manufacture of the part. onsemi is an Equal Opportunity/Affirmative Action Employer. This literature is subject to all applicable copyright laws and is not for resale in any manner.

PUBLICATION ORDERING INFORMATION

LITERATURE FULFILLMENT:

Literature Distribution Center for onsemi

19521 E. 32nd Pkwy, Aurora, Colorado 80011 USA

Phone: 303-675-2175 or 800-344-3860 Toll Free USA/Canada

Fax: 303-675-2176 or 800-344-3867 Toll Free USA/Canada

Email: orderlit@onsemi.com

N. American Technical Support:

800-282-9855 Toll Free USA/Canada

Europe, Middle East and Africa Technical

Support: Phone: 421 33 790 2910

onsemi Website: www.onsemi.com

Order Literature: <http://www.onsemi.com/orderlit>

For additional information, please contact your local Sales Representative

M-20845-018